

CODE FOR PUBLIC HEALTH UK

# SQL & R for UK Public Health

*from zero to reproducible pipelines*

105 lessons · 10 modules · 210 quick-check quizzes · 105 exercises with solutions

This is the book edition of the interactive course. Quizzes here show the correct answer (marked ✓) with its explanation, and every exercise is followed by its worked solution — attempt each one before reading on. Blocks labelled “SQL · try it in the playground” refer to the live database built into the companion HTML edition; the queries are printed in full here so you can run them in any SQLite tool. All data mentioned in this course is fictional and computer-generated.

July 2026 · companion to the interactive edition

# Contents

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>Module 1 — Foundations &amp; setup</b>                                       | <b>6</b>  |
| 1. Why analysts code: your job in UK public health . . . . .                    | 7         |
| 2. Files, folders and paths on Windows, Mac and Linux . . . . .                 | 8         |
| 3. Meet the command line (PowerShell, Terminal, bash) . . . . .                 | 10        |
| 4. Install your SQL toolkit: SQLite + DB Browser . . . . .                      | 12        |
| 5. Install your analysis toolkit: R + RStudio . . . . .                         | 14        |
| 6. How data flows through UK health services . . . . .                          | 15        |
| 7. Information governance: the rules you must never break . . . . .             | 17        |
| 8. Thinking like an analyst: from vague question to defensible answer . . . . . | 18        |
| 9. Why databases beat spreadsheets (and what a database is) . . . . .           | 20        |
| 10. Anatomy of a table: rows, columns, types and keys . . . . .                 | 22        |
| 11. Your first query — and how to use the playground . . . . .                  | 24        |
| <b>Module 2 — SQL core skills</b>                                               | <b>25</b> |
| 12. SELECT and FROM: choosing your columns . . . . .                            | 27        |
| 13. WHERE: filtering rows with conditions . . . . .                             | 29        |
| 14. ORDER BY and LIMIT: sorting and topping . . . . .                           | 30        |
| 15. DISTINCT: seeing and counting unique values . . . . .                       | 31        |
| 16. Working with text: LIKE, case, trimming and joining strings . . . . .       | 33        |
| 17. Numbers: arithmetic, rounding and the integer-division trap . . . . .       | 34        |
| 18. NULL: the meaning of missing . . . . .                                      | 36        |
| 19. CASE WHEN: categorising on the fly . . . . .                                | 37        |
| 20. Dates part 1: storing, comparing and extracting . . . . .                   | 38        |
| 21. Dates part 2: ages, financial years and month spines . . . . .              | 40        |
| 22. Aggregates: COUNT, SUM, AVG, MIN, MAX . . . . .                             | 42        |
| 23. GROUP BY: summaries for every category . . . . .                            | 43        |
| 24. HAVING: filtering the groups themselves . . . . .                           | 45        |
| 25. JOIN part 1: INNER JOIN, keys and aliases . . . . .                         | 46        |
| 26. JOIN part 2: LEFT JOIN and finding what's missing . . . . .                 | 49        |
| 27. JOIN part 3: chains, fan-out and the duplication trap . . . . .             | 50        |
| 28. UNION and friends: stacking result sets . . . . .                           | 52        |
| 29. Subqueries: queries inside queries . . . . .                                | 53        |
| <b>Module 3 — SQL for health data</b>                                           | <b>55</b> |
| 30. CTEs: readable, professional query pipelines . . . . .                      | 56        |
| 31. Window functions 1: ROW_NUMBER and "latest record per patient" . . . . .    | 57        |

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| 32. Window functions 2: running totals, moving averages and date spines . . . . . | 59 |
| 33. Window functions 3: LAG, LEAD and gaps between events . . . . .               | 61 |
| 34. Deduplication: deciding what "one record" means . . . . .                     | 63 |
| 35. Cohorts: defining exactly who is in your analysis . . . . .                   | 64 |
| 36. Length of stay, spells and pathway thinking . . . . .                         | 66 |
| 37. Rates: joining numerators to populations . . . . .                            | 68 |
| 38. The classic metric: emergency readmissions within 30 days . . . . .           | 69 |
| 39. Data quality checks: trust nothing, verify everything . . . . .               | 71 |
| 40. The T-SQL phrasebook: SQL Server for NHS analysts . . . . .                   | 73 |
| 41. Views, temp tables and saving results . . . . .                               | 75 |

## **Module 4 — R fundamentals 76**

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| 42. Welcome to R: console, scripts and projects . . . . .                | 78  |
| 43. Objects and vectors: R's building blocks . . . . .                   | 79  |
| 44. Vectorised thinking, logical tests and NA . . . . .                  | 81  |
| 45. Data frames and tibbles: tables arrive in R . . . . .                | 82  |
| 46. Packages: installing R's superpowers (and the tidyverse) . . . . .   | 84  |
| 47. Importing data: read_csv and friends . . . . .                       | 86  |
| 48. dplyr part 1: filter rows, arrange order . . . . .                   | 88  |
| 49. dplyr part 2: select, mutate and rename columns . . . . .            | 89  |
| 50. The pipe, group_by and summarise: the analysis engine . . . . .      | 91  |
| 51. Joins in R: left_join, inner_join and the mighty anti_join . . . . . | 92  |
| 52. Tidy data: pivot_longer and pivot_wider . . . . .                    | 94  |
| 53. Strings with stringr: cleaning the text you're given . . . . .       | 95  |
| 54. Dates with lubridate: UK dates tamed . . . . .                       | 97  |
| 55. Factors with forcats: categories in the right order . . . . .        | 98  |
| 56. Writing your own functions . . . . .                                 | 99  |
| 57. Iteration: for loops and purrr's map . . . . .                       | 102 |

## **Module 5 — Visualisation & reporting in R 103**

|                                                                        |     |
|------------------------------------------------------------------------|-----|
| 58. ggplot2: the grammar of graphics . . . . .                         | 104 |
| 59. Time series and epi curves . . . . .                               | 105 |
| 60. Charts people can actually read: style and accessibility . . . . . | 106 |
| 61. Small multiples: facets . . . . .                                  | 108 |
| 62. Distributions: histograms, boxplots and honest summaries . . . . . | 110 |
| 63. Maps: small-area geography with sf . . . . .                       | 111 |
| 64. Tables for reports: gt and kable . . . . .                         | 113 |
| 65. Quarto: reports that rebuild themselves . . . . .                  | 114 |

|                                                                                   |            |
|-----------------------------------------------------------------------------------|------------|
| 66. Parameterised reports: one template, forty outputs . . . . .                  | 116        |
| 67. Safe outputs: disclosure control before anything leaves . . . . .             | 118        |
| <b>Module 6 — Statistics &amp; epidemiology methods</b>                           | <b>119</b> |
| 68. Describing data honestly: the right summary for the shape . . . . .           | 121        |
| 69. Rates in R: crude rates, person-time and per-100,000 thinking . . . . .       | 122        |
| 70. Age-standardisation: comparing places fairly . . . . .                        | 126        |
| 71. Uncertainty: confidence intervals for counts and rates . . . . .              | 127        |
| 72. Comparing proportions: 2x2 tables, chi-square and risk ratios . . . . .       | 129        |
| 73. Comparing means: t-tests, Mann-Whitney and what "significant" means . . . . . | 131        |
| 74. Correlation and the ecological fallacy . . . . .                              | 132        |
| 75. Linear regression: modelling a continuous outcome . . . . .                   | 134        |
| 76. Logistic regression and odds ratios . . . . .                                 | 136        |
| 77. SPC charts: Making Data Count . . . . .                                       | 137        |
| 78. Funnel plots: comparing organisations fairly . . . . .                        | 139        |
| 79. Screening and tests: sensitivity, specificity and the PPV trap . . . . .      | 141        |
| <b>Module 7 — Pipelines, Git &amp; reproducibility</b>                            | <b>143</b> |
| 80. Connecting R to databases: DBI and odbc . . . . .                             | 145        |
| 81. dbplyr: dplyr that writes SQL for you . . . . .                               | 147        |
| 82. Parameterised queries: the injection lesson . . . . .                         | 149        |
| 83. Project structure and {here}: work that travels . . . . .                     | 151        |
| 84. Git: version control for analysts . . . . .                                   | 153        |
| 85. GitHub: sharing, review and the audit trail . . . . .                         | 155        |
| 86. RAP: Reproducible Analytical Pipelines . . . . .                              | 157        |
| 87. Scheduling and testing: analysis that runs itself (safely) . . . . .          | 159        |
| <b>Module 8 — Working with AI</b>                                                 | <b>161</b> |
| 88. How LLMs actually work — and why they confidently invent things . . . . .     | 163        |
| 89. Prompting for SQL: context is everything . . . . .                            | 164        |
| 90. Prompting for R and debugging with AI . . . . .                               | 166        |
| 91. AI code review: the adversarial second pair of eyes . . . . .                 | 168        |
| 92. Governance: what never goes in the box . . . . .                              | 170        |
| 93. AI for documentation, explanation and learning . . . . .                      | 172        |
| 94. Failure modes: a field guide to confident wrongness . . . . .                 | 174        |
| 95. Judgement: the analyst's half of the partnership . . . . .                    | 176        |
| <b>Module 9 — Career &amp; professional practice</b>                              | <b>178</b> |
| 96. The NHS data landscape: what flows where . . . . .                            | 180        |

|                                                                               |     |
|-------------------------------------------------------------------------------|-----|
| 97. Open data: ONS, UKHSA and the art of the free denominator . . . . .       | 181 |
| 98. Communities, competencies and the shape of the profession . . . . .       | 183 |
| 99. Building a portfolio that proves it . . . . .                             | 185 |
| 100. Getting hired: applications, interviews and the technical test . . . . . | 186 |
| 101. Your 12-week plan: from course to career . . . . .                       | 188 |

## **Module 10 — Capstone projects 191**

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| 102. Capstone A: the readmissions pipeline (SQL) . . . . .               | 192 |
| 103. Capstone B: vaccination equity (SQL extract + R analysis) . . . . . | 194 |
| 104. Capstone C: the weekly surveillance report (Quarto) . . . . .       | 196 |
| 105. Capstone D: the mini-RAP, both hands (and the rubric) . . . . .     | 199 |

## MODULE 1

# Module 1 — Foundations & setup

## In this module

- 1 Why analysts code: your job in UK public health
- 2 Files, folders and paths on Windows, Mac and Linux
- 3 Meet the command line (PowerShell, Terminal, bash)
- 4 Install your SQL toolkit: SQLite + DB Browser
- 5 Install your analysis toolkit: R + RStudio
- 6 How data flows through UK health services
- 7 Information governance: the rules you must never break
- 8 Thinking like an analyst: from vague question to defensible answer
- 9 Why databases beat spreadsheets (and what a database is)
- 10 Anatomy of a table: rows, columns, types and keys
- 11 Your first query — and how to use the playground

## MODULE 1 • FOUNDATIONS &amp; SETUP • LESSON 1 OF 105

# 1. Why analysts code: your job in UK public health

Welcome. This course takes you from **zero programming knowledge** to being able to do the two things every UK public health analyst is paid to do: **pull data out of databases with SQL**, and **analyse and present it with R**. Those are the tools you will meet in NHS trusts, Integrated Care Boards (ICBs), the UK Health Security Agency (UKHSA), and local authority public health teams.

Why code at all, when Excel exists? Three reasons that come up in real UK health jobs:

- **Scale.** A single year of hospital admissions for one region can be millions of rows. Excel stops at 1,048,576 and slows down long before that. Databases don't.
- **Reproducibility.** A script is a written record of exactly what you did. When your director asks "how did you get this number?", you can show them. When next month's data arrives, you re-run the script instead of re-doing forty clicks.
- **Safety.** Manual copy-paste is where errors are born. A famous UK example: in October 2020, 15,841 positive COVID-19 test results were lost because an old Excel file format ran out of columns. Code with proper tools avoids whole categories of that mistake.

## Where you might work

This course blends three settings, because most graduate analysts move between them:

- **NHS trust / ICB analytics** — waiting lists, A&E performance, readmissions, service planning.
- **UKHSA / health protection** — outbreak surveillance, vaccination coverage, epidemiology.
- **Local authority public health** — population health needs, deprivation, prevention services, the Joint Strategic Needs Assessment (JSNA).

All three use SQL to query databases and, overwhelmingly, **R** for analysis — UKHSA and many NHS teams have standardised on R, and there is a thriving NHS-R community. That's why this course pairs SQL with R.

### HOW THIS COURSE WORKS

Every lesson has a short guide, real worked examples, at least one quiz, and an exercise with a hidden solution. Lessons 1–11 need no software at all. From lesson 11 you get a **live SQL playground** built into this page, with a synthetic health database — no installation needed. Everything here uses fictional, computer-generated data only.

### QUICK CHECK

**Roughly how many rows can one Excel worksheet hold before it simply cannot accept more?**

- ☐ About 65 thousand
- ✓ **About 1 million**
- ☐ About 100 million
- ☐ There is no limit

**Why:** Excel's hard ceiling is 1,048,576 rows per sheet. Hospital activity data for even a modest region exceeds this quickly, which is one reason analysts use databases.

**QUICK CHECK**

**Which of these is the strongest argument for scripting an analysis rather than doing it by hand in a spreadsheet?**

- ☐ Scripts always run faster than Excel
- ☐ Scripts look more professional in meetings
- ✓ **A script is a reproducible, checkable record of every step**
- ☐ Spreadsheets cannot make charts

**Why:** Speed varies and charts exist in both. The decisive advantage is reproducibility: anyone (including future you) can re-run and audit the exact steps.

**EXERCISE**

No code yet. Write down, in one sentence each, three questions a public health team in your area might ask of data (for example: "Is flu vaccination uptake lower in more deprived neighbourhoods?"). Keep them — you will be able to answer questions like these by the end of the course.

**SOLUTION · TEXT**

Example answers:

1. Which GP practices have the lowest childhood MMR uptake, and is it linked to deprivation?
2. Are emergency readmissions within 30 days rising in older age groups?
3. Did A&E attendances for respiratory illness spike after the cold snap in January?

Any specific, answerable question naming a measure, a population and (ideally) a time period is a good answer.

**KEY POINTS**

- UK public health analysis runs on SQL (to query databases) and R (to analyse and report).
- Code beats spreadsheets on scale, reproducibility and error-safety.
- You'll practise in a built-in SQL playground on synthetic data — nothing to install for the SQL half.

**MODULE 1 · FOUNDATIONS & SETUP · LESSON 2 OF 105**

## 2. Files, folders and paths on Windows, Mac and Linux

Before any code: you must be able to say **where a file lives**. Every file has a *\*path\** — its address on the computer. Analysts constantly read data from paths and save results to paths, and the format differs by operating system.

| System  | Example path                                | Separator       | Home folder shorthand |
|---------|---------------------------------------------|-----------------|-----------------------|
| Windows | C:\Users\asha\Documents\data\admissions.csv | Backslash \     | %USERPROFILE%         |
| macOS   | /Users/asha/Documents/data/admissions.csv   | Forward slash / | ~                     |
| Linux   | /home/asha/documents/data/admissions.csv    | Forward slash / | ~                     |

Two kinds of path:

- An **absolute path** starts from the root of the drive (C:\ or /) and works from anywhere.
- A **relative path** starts from wherever you currently are. `data/admissions.csv` means "the admissions.csv inside the data folder, right here". Relative paths make projects portable — they still work when a colleague copies your folder.



**File extensions** (the bit after the dot) tell you the format: `.csv` is plain-text comma-separated data, `.xlsx` is Excel, `.R` is an R script, `.sql` is a SQL script, `.zip` is a compressed bundle. Windows and macOS hide extensions by default — turn that off (Windows: File Explorer ► View ► File name extensions; macOS: Finder ► Settings ► Advanced ► Show all filename extensions). Analysts need to see exactly what a file is.

## CSV: the analyst's workhorse

A CSV is just text. Open one in Notepad/TextEdit and you'll see the first row is usually **column headers**, then one row per record, values separated by commas:

### ADMISSIONS.CSV (FIRST 3 LINES)

```
patient_id,admission_date,specialty
1042,2025-11-03,Respiratory
1187,2025-11-03,Cardiology
```

Because CSV is plain text, every tool — SQL, R, Python, Excel — can read it. One trap for UK analysts: Excel loves to "help" by reformatting things. It turns `01/02/2025` into an ambiguous date and famously converted gene names like `MARCH1` into dates in genetics research. Keep master copies as CSV and do the analysis in code.

### QUICK CHECK

Which of these is a **\*relative\*** path?

- ☐ C:\Users\sam\project\data\pop.csv
- ☐ /home/sam/project/data/pop.csv
- ✓ **data/pop.csv**
- ☐ D:\pop.csv

**Why:** It doesn't start from a drive root, so it's resolved relative to your current folder — which is exactly what makes shared projects portable.

### QUICK CHECK

Your colleague on a Mac sends you a path `/Users/jo/analysis/output.csv`. What is the Windows-style equivalent structure?

- ☐ `/Users/jo/analysis/output.csv` works unchanged on Windows
- ✓ **C:\Users\jo\analysis\output.csv**
- ☐ `C:/jo/output.csv`
- ☐ Windows does not use paths

**Why:** Windows paths use a drive letter and backslashes. (Inside R you can actually use forward slashes on Windows too — a handy trick coming in Module 4.)

### EXERCISE

On your own machine, create a folder for this course containing two sub-folders, `data` and `outputs`. Then write out the absolute path to the `data` folder as your operating system would express it.

### SOLUTION - TEXT

Examples of correct answers:

Windows: `C:\Users\<yourname>\Documents\ph-course\data`

macOS: `/Users/<yourname>/Documents/ph-course/data`

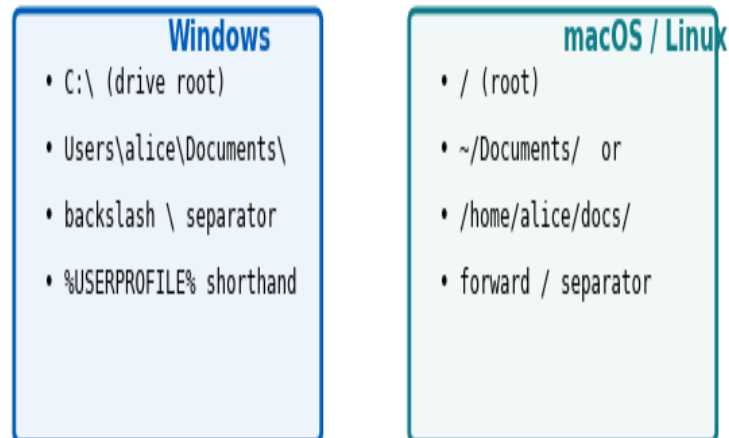
Linux: `/home/<yourname>/ph-course/data`

The exact wording varies — what matters is: correct separator, starts at the root/drive, ends at the `data` folder.

**KEY POINTS**

- A path is a file's address; Windows uses \ and a drive letter, Mac/Linux use /.
- Relative paths make projects portable; absolute paths pin an exact location.
- Show file extensions, and treat CSV as your safe, universal data format.

## File system: Windows vs. Unix



*Lesson: Always use forward slashes in R/Python; they work on all systems.*

**MODULE 1 • FOUNDATIONS & SETUP • LESSON 3 OF 105**

## 3. Meet the command line (PowerShell, Terminal, bash)

The command line is a text window where you type instructions instead of clicking. You don't need to become a wizard, but every analyst needs the basics: installers, Git, and servers all speak command line. Each OS has one built in:

- **Windows:** press Start, type `PowerShell`, open **Windows PowerShell** (or Windows Terminal).
- **macOS:** press Cmd+Space, type `Terminal`.
- **Linux:** Ctrl+Alt+T opens a **bash** shell in most distributions.

The window shows a **prompt** and waits. You type a command, press Enter, it runs, and the prompt returns. Here is the same 4-step session — see where you are, list files, make a course folder, move into it — on each system:

**WINDOWS**

```
# Windows PowerShell
pwd                # print working directory (where am I?)
ls                 # list files here (alias of Get-ChildItem)
mkdir ph-course    # make a new folder
cd ph-course       # change directory into it
```

**MACOS**

```
# macOS Terminal (zsh)
pwd                # where am I?
ls                 # list files here
mkdir ph-course    # make a new folder
cd ph-course       # move into it
```

**LINUX**

```
# Linux (bash)
pwd                # where am I?
ls -l              # list files with details
mkdir ph-course    # make a new folder
cd ph-course       # move into it
```

Happily, PowerShell accepts the classic Unix names (`ls`, `pwd`, `cd`, `mkdir`), so the core vocabulary transfers across all three systems. A few more you'll use: `cd ..` goes \*up\* one level; `cp a.csv b.csv` copies a file (`copy` also works on Windows); `rm file` deletes — **carefully**, there is no recycle bin on the command line. Pressing **Tab** auto-completes names; pressing the **up arrow** recalls previous commands.

**IN NHS AND GOVERNMENT WORKPLACES**

On managed work laptops you often can't install software yourself and some terminals are restricted. That's normal — IT will install approved tools (this is why we practise on your own machine, and why the SQL playground in this page needs nothing installed).

**QUICK CHECK**

**You are in `/home/asha/ph-course` and type `cd ..` — where are you now?**

- ☐ `/home/asha/ph-course/..`
- ✓ ☒ **`/home/asha`**
- ☐ `/` (the root)
- ☐ Nowhere; it's an error

**Why:** Two dots mean "the parent folder", so you move up one level to `/home/asha`.

**QUICK CHECK**

**Which command creates a new folder on all three systems as shown?**

- ☐ `newfolder`
- ☐ `create`
- ✓ ☒ **`mkdir`**
- ☐ `folder -n`

**Why:** `mkdir` ("make directory") works in PowerShell, zsh and bash.

**EXERCISE**

In your command line: create a folder called `ph-course` with sub-folders `data` and `outputs` inside it, then list the contents of `ph-course` to confirm. Write the commands you used.

**SOLUTION · SHELL**

```
mkdir ph-course
cd ph-course
mkdir data
mkdir outputs
ls          # should show: data  outputs
# (One-liner on Mac/Linux: mkdir -p ph-course/{data,outputs})
```

**KEY POINTS**

- PowerShell (Windows), Terminal (macOS) and bash (Linux) share core commands: `pwd`, `ls`, `cd`, `mkdir`, `cp`, `rm`.
- Tab completes, up-arrow repeats, `cd ..` goes up.
- Command-line deletion is permanent — pause before `rm`.

**MODULE 1 · FOUNDATIONS & SETUP · LESSON 4 OF 105**

## 4. Install your SQL toolkit: SQLite + DB Browser

Real NHS databases mostly run **Microsoft SQL Server** (with some Oracle and, increasingly, cloud platforms). You can't install those casually — but you don't need to, because SQL the *\*language\** is what matters, and a tiny free engine called **SQLite** speaks it. SQLite stores a whole database in one file and runs on anything. Pair it with **DB Browser for SQLite**, a free point-and-click app for opening databases and running queries.

This page's built-in playground means you can do every SQL lesson **without installing anything**. But installing the real thing on your own computer is good practice for the workplace:

**WINDOWS**

```
# Option A (no command line): download the installer from
# https://sqlitebrowser.org -> "DB Browser for SQLite - Standard installer for 64-bit Windows"
# Run it, accept defaults.
# Option B (PowerShell, if you have winget):
winget install -e --id DBBrowserForSQLite.DBBrowserForSQLite
# Verify: open "DB Browser for SQLite" from the Start menu
```

**MACOS**

```
# Option A: download the .dmg from https://sqlitebrowser.org
# and drag "DB Browser for SQLite" into Applications.
# Option B (Terminal, if you use Homebrew):
brew install --cask db-browser-for-sqlite
# macOS also ships the sqlite3 command-line tool already:
sqlite3 --version
```

**LINUX**

```
# Debian/Ubuntu:
sudo apt update
sudo apt install sqlitebrowser sqlite3
# Fedora:
sudo dnf install sqlitebrowser sqlite
# Verify:
```

**LINUX**

```
sqlite3 --version
```

Once installed, open DB Browser and click **New Database** ► save as `practice.db` in your ph-course folder ► Cancel the table wizard ► open the **Execute SQL** tab. That tab is where you'll paste queries from this course. The playground here and DB Browser both run SQLite, so everything transfers exactly.

**DIALECT HEADS-UP**

SQL has small dialect differences, like British vs American spelling. This course teaches standard SQL that runs in SQLite, and Lesson 40 gives you a translation guide to SQL Server (T-SQL), which you'll meet in the NHS.

**QUICK CHECK**

**Why is SQLite a sensible learning tool for someone heading into NHS analytics?**

- ☐ The NHS runs entirely on SQLite
- ✓ **The SQL language skills transfer, and SQLite needs no server or licence**
- ☐ SQLite is faster than SQL Server for national datasets
- ☐ It is the only database that runs on Windows

***Why:** NHS platforms are mostly SQL Server, but ~95% of the SQL you write is identical. SQLite gives you that language in a single free file.*

**QUICK CHECK**

**In DB Browser for SQLite, where do you type and run queries?**

- ☐ The Browse Data tab
- ☐ The Database Structure tab
- ✓ **The Execute SQL tab**
- ☐ The Edit Pragmas tab

***Why:** Execute SQL is the query window; Browse Data is for eyeballing tables.*

**EXERCISE**

Install DB Browser for SQLite (or confirm `sqlite3` runs in your terminal). Create `practice.db` inside your ph-course folder. Then answer: what is the full path to that file on your machine?

**SOLUTION · TEXT**

Example answers:

Windows: `C:\Users\<you>\Documents\ph-course\practice.db`

macOS: `/Users/<you>/Documents/ph-course/practice.db`

Linux: `/home/<you>/ph-course/practice.db`

If installation is blocked (e.g. a locked-down machine), that's fine – the in-page playground covers every SQL lesson.

**KEY POINTS**

- SQLite = the SQL language in one free file; DB Browser = the friendly app around it.
- NHS reality is mostly SQL Server; the language transfers with minor dialect tweaks (Lesson 40).
- The in-page playground means no install is ever required to progress.

## MODULE 1 • FOUNDATIONS &amp; SETUP • LESSON 5 OF 105

## 5. Install your analysis toolkit: R + RStudio

**R** is the language; **RStudio Desktop** is the free app you'll write it in (made by Posit). Install R first, then RStudio. Both are free and run on all three systems.

### WINDOWS

```
# 1) R: go to https://cran.r-project.org -> "Download R for Windows"
#    -> "base" -> "Download R 4.x.x for Windows". Run installer, accept defaults.
# 2) RStudio: https://posit.co/download/rstudio-desktop -> Windows installer.
# 3) Later, some packages need build tools; if prompted, install "Rtools"
#    from CRAN (same site, "Rtools" link).
# Or with winget in PowerShell:
winget install -e --id RProject.R
winget install -e --id Posit.RStudio
```

### MACOS

```
# 1) R from https://cran.r-project.org -> "Download R for macOS"
#    (pick the .pkg matching your chip: arm64 for Apple Silicon, x86_64 for Intel).
# 2) RStudio from https://posit.co/download/rstudio-desktop (.dmg).
# 3) If a package ever asks for developer tools, run:
xcode-select --install
# Or with Homebrew:
brew install --cask r rstudio
```

### LINUX

```
# Debian/Ubuntu (CRAN's build is more current than the distro's):
sudo apt update
sudo apt install --no-install-recommends r-base r-base-dev
# RStudio: download the .deb from https://posit.co/download/rstudio-desktop
sudo apt install ./rstudio-*-amd64.deb
# r-base-dev matters: many R packages compile from source on Linux.
```

Open RStudio. You'll see four panes: the **Console** (bottom-left) runs code immediately; the **Source** editor (top-left) holds scripts you save; **Environment** (top-right) lists your data objects; and **Files/Plots/Help** (bottom-right). Type this in the Console and press Enter:

### YOUR FIRST R COMMANDS

```
2 + 2
round(37.68492, 1)      # a function with two arguments
"Public health"         # text is written in quotes
```

If you see 4, 37.7, and the text echoed back, everything works. One setting to change immediately: **Tools ► Global Options ► General ► untick "Restore .RData at startup" and set "Save workspace" to Never**. This forces every session to start clean, which is exactly what reproducible analysis needs.

### CAN'T INSTALL AT WORK?

Many NHS organisations provide R/RStudio through an internal software centre, or host **Posit Workbench** in a browser. If your own laptop is the blocker, webR (R in the browser) exists for small experiments — but for this course a normal install is strongly recommended for the R modules.

**QUICK CHECK****What is the correct install order and relationship?**

- ☐ RStudio first; R is included inside it
- ✓ **R first (the language), then RStudio (the editor that uses it)**
- ☐ Either alone is enough; they are alternatives
- ☐ R is only for Linux; RStudio is only for Windows

**Why:** RStudio is an IDE — a workbench. It needs the R engine installed underneath.

**QUICK CHECK****Why switch off "Restore .RData at startup" in RStudio?**

- ☐ It makes R run faster on old laptops
- ✓ **Clean sessions make results depend only on your script, not leftover objects**
- ☐ It is required by NHS information governance
- ☐ It enables dark mode

**Why:** Reproducibility: if your script works in a fresh session, it will work for colleagues and for future you.

**EXERCISE**

Install R and RStudio. In the Console, calculate how many days there are in 52 weeks, and use `round()` to round  $365.25 / 7$  to two decimal places. Paste both commands.

**SOLUTION · R**

```
52 * 7           # 364
round(365.25 / 7, 2) # 52.18
```

**KEY POINTS**

- Install R (engine) then RStudio (workbench); Linux users also want `r-base-dev`.
- Console = run now; Source = saved scripts; keep sessions clean (never save workspace).
- Functions look like `name(argument1, argument2)` — you just used `round()`.

**MODULE 1 · FOUNDATIONS & SETUP · LESSON 6 OF 105**

## 6. How data flows through UK health services

Analysts add value by knowing **where numbers come from**. A simplified map of the UK (mainly English) health data landscape:

- **Primary care (GP practices):** every consultation, prescription and vaccination is coded (using **SNOMED CT** terms) in systems like EMIS and SystmOne. Extracts flow to national programmes and local shared-care records.
- **Secondary care (hospitals):** each admission generates a coded record — diagnoses in **ICD-10**, procedures in **OPCS-4**. Trusts submit these to the national **Secondary Uses Service (SUS)**, and cleaned versions become **Hospital Episode Statistics (HES)**. A&E flows via the **Emergency Care Data Set (ECDS)**.
- **NHS England** holds national collections (SUS/HES, waiting lists, workforce) — it absorbed NHS Digital in 2023.
- **UKHSA** runs infectious disease surveillance: lab notifications via its **Second Generation Surveillance System (SGSS)**, outbreak data, immunisation coverage.

- **ONS** (Office for National Statistics) provides the denominators — population estimates, life expectancy — plus births, deaths and the Census.
- **Local authorities** hold public health services data (health visiting, sexual health, drug & alcohol treatment) and use all the above for JSNAs.

The glue between these is a set of identifiers: the **NHS number** (a 10-digit unique patient identifier), **ODS codes** for organisations (every GP practice and trust has one), and geography codes such as **LSOA** (Lower layer Super Output Area — small neighbourhoods of ~1,500 people) which link health records to ONS population and deprivation data.

| Dataset                | What it covers                    | Typical analyst use                      |
|------------------------|-----------------------------------|------------------------------------------|
| SUS / HES              | Hospital admissions & outpatients | Readmissions, activity trends, case-mix  |
| ECDS                   | A&E attendances                   | Winter pressures, attendance patterns    |
| SGSS (UKHSA)           | Lab-confirmed infections          | Outbreak detection, surveillance reports |
| GP extracts            | Primary care events               | Vaccination uptake, disease registers    |
| ONS mid-year estimates | Population by age/sex/area        | Denominators for every rate you compute  |
| IMD                    | Deprivation rank for each LSOA    | Health inequality analysis               |

Public-facing versions of much of this live on UKHSA/DHSC's **Fingertips** platform ([fingertips.phe.org.uk](http://fingertips.phe.org.uk)) — hundreds of indicators by local authority. Later you'll pull Fingertips data straight into R with one function call.

#### QUICK CHECK

**A hospital diagnosis of pneumonia will be recorded in an admissions dataset using which coding system?**

- ☐ SNOMED CT
- ✓ **ICD-10**
- ☐ OPCS-4
- ☐ ODS

**Why:** Hospital diagnoses use ICD-10 (pneumonia is J18.9 when unspecified); OPCS-4 codes procedures; SNOMED CT dominates primary care; ODS codes identify organisations.

#### QUICK CHECK

**You need a denominator to turn a count of flu admissions into a rate per 100,000 residents. Which source provides it?**

- ☐ SUS
- ☐ ECDS
- ✓ **ONS mid-year population estimates**
- ☐ SGSS

**Why:** Health datasets give numerators (events). ONS population estimates by age, sex and area are the standard denominator.

#### EXERCISE

Match each question to the most likely primary data source (SUS/HES, ECDS, SGSS, GP extract, ONS): (a) How many measles cases were lab-confirmed last quarter? (b) What proportion of 5-year-olds have had two MMR doses? (c) Did emergency admissions for COPD rise last winter? (d) How many people aged 80+ live in each LSOA?



**SOLUTION · TEXT**

- (a) SGSS — lab-confirmed infections sit in UKHSA surveillance.
- (b) GP extract — childhood vaccinations are recorded in primary care.
- (c) SUS/HES — emergency admissions are hospital activity.
- (d) ONS — small-area population estimates.

**KEY POINTS**

- Numerators come from health systems (SUS/HES, ECDS, SGSS, GP data); denominators come from ONS.
- ICD-10 = hospital diagnoses; OPCS-4 = procedures; SNOMED CT = primary care; NHS number links a patient across systems.
- LSOAs link health data to population and deprivation — the backbone of inequalities analysis.

**MODULE 1 · FOUNDATIONS & SETUP · LESSON 7 OF 105**

## 7. Information governance: the rules you must never break

Health data is among the most sensitive information the state holds, and the UK regulates it hard. Getting this wrong isn't a style point — it can end careers and harm patients' trust. The legal frame: **UK GDPR** and the **Data Protection Act 2018**, plus the **common law duty of confidentiality**. In practice, four working concepts:

- **Personal data / PID** (person-identifiable data): anything that can identify a living person — name, address, date of birth, NHS number, or combinations that single someone out.
- **Pseudonymised data**: identifiers replaced by codes (e.g. NHS number swapped for a study ID). Safer, but legally still personal data, because someone holds the key.
- **Anonymised / aggregate data**: counts and rates that can't identify anyone. This is what leaves the building in reports.
- **The Caldicott Principles**: eight rules (justify the purpose; use the minimum necessary; access on a need-to-know basis...; the eighth: inform patients how data is used). Every NHS organisation has a **Caldicott Guardian** who adjudicates.

### Small numbers: the analyst's daily judgement

Even aggregate tables can identify people. If your table says "2 cases of HIV in Little Bexton ward", locals might work out who. Standard UK practice is **statistical disclosure control**: suppress small counts (commonly anything from 1 to 7 is shown as \* or "<5" — thresholds vary by organisation; HES conventionally suppresses 1–7), and sometimes round others so the suppressed cell can't be back-calculated from totals. When in doubt: suppress, and ask your information governance (IG) team.

**THREE PERSONAL RULES TO ADOPT TODAY**

1) Never move patient-level data off approved systems — no emailing extracts, no USB sticks, no personal laptops, and (as Module 8 covers) **never pasting it into public AI tools**. 2) Never look up anyone you know — access is audited. 3) Publish aggregates, not people: apply small-number suppression before anything leaves your team.

**QUICK CHECK**

**A dataset where NHS numbers have been replaced with random study IDs (and a key exists elsewhere) is best described as...**

- ☐ Anonymised — GDPR no longer applies
- ✓ **Pseudonymised — still personal data under UK GDPR**
- ☐ Synthetic
- ☐ Public data

**Why:** Because re-identification is possible via the key, UK GDPR still treats it as personal data. Only truly irreversible aggregation/anonymisation escapes that.

**QUICK CHECK**

**Your output table includes a cell showing 3 teenage pregnancies in one small village. Best practice is to...**

- ☐ Publish it; 3 is a real number
- ☐ Delete the whole table
- ✓ **Suppress the small count (e.g. show "<5") and check totals can't reveal it**
- ☐ Round it up to 10

**Why:** Small-number suppression protects individuals in small areas. Also check row/column totals don't let readers reconstruct the suppressed value.

**EXERCISE**

You've produced a ward-level table of drug-related deaths for a council report. Counts range from 0 to 14; four wards have counts of 1 or 2. List the steps you'd take before this table leaves your team.

**SOLUTION · TEXT**

1. Apply the organisation's suppression rule (e.g. replace 1–4, or 1–7, with "<5"/"\*").
2. Check totals and complementary cells can't be used to back-calculate suppressed values (secondary suppression if needed).
3. Confirm the output contains no names, dates of birth, or other identifiers.
4. Follow local sign-off: senior analyst review and, if unsure, the IG team/Caldicott Guardian.
5. Record what you did so the method is reproducible next time.

**KEY POINTS**

- UK GDPR + DPA 2018 + confidentiality law govern all health data; Caldicott Principles operationalise them.
- Pseudonymised ≠ anonymised; only irreversible aggregation truly de-identifies.
- Suppress small numbers before publishing; never move PID outside approved systems.

**MODULE 1 · FOUNDATIONS & SETUP · LESSON 8 OF 105**

## 8. Thinking like an analyst: from vague question to defensible answer

The hardest part of analysis isn't code — it's turning "Are we doing OK on childhood jabs?" into something precise. Professionals use a repeatable chain: **Question → Data → Method → Output → Check**.

- 1. Sharpen the question.** Force out the measure, population, place, time and comparison. Vague: "How's MMR uptake?" Sharp: "What percentage of children turning 5 in 2025/26 had received two MMR doses, by GP practice in our ICB, compared with the 95% WHO target?" A useful skeleton for indicators: numerator, denominator, time period, breakdown.
- 2. Find the data — and its caveats.** Which system records it? How complete is it? Coding changes, service disruption (COVID years!), and lag all shape what you can claim.
- 3. Choose the method.** Often just counting and dividing — carefully. Sometimes age-standardisation, confidence intervals, or a model (Module 6).
- 4. Produce the output for the audience.** A director wants one chart and three sentences; a fellow analyst wants your code.
- 5. Check.** Sanity-check magnitudes (does 40,000 A&E attendances a month sound right for your trust's size?), reconcile against published figures, and get a second pair of eyes. In NHS analytics this is often formalised as **QA by a colleague** before release.

#### FINANCIAL YEARS — A VERY BRITISH TRAP

The NHS and most UK public services run on **financial years: 1 April to 31 March**, written 2025/26. School-age cohorts, GP payments and most performance data follow it. Always confirm whether "the year" means calendar or financial — mixing them is one of the commonest real-world errors, and you'll write code for it in Lessons 21 and 54.

#### QUICK CHECK

**Which restatement of "Are A&E waits getting worse?" is analysis-ready?**

- ☐ Is A&E slower than it used to be?
- ✓ **What % of A&E attendances at our trust were admitted, transferred or discharged within 4 hours, each month for the last 3 financial years?**
- ☐ How do patients feel about A&E waits?
- ☐ Is the NHS in crisis?

**Why:** It names the measure (4-hour standard %), the population (our trust's attendances), the time frame and the comparison (monthly trend).

#### QUICK CHECK

**A report says "uptake in 2024" without qualification. As a UK analyst your first clarifying question is...**

- ☐ Which font was used
- ✓ **Calendar year 2024 or financial year 2024/25?**
- ☐ Was it Excel or R
- ☐ Who is the Caldicott Guardian

**Why:** Calendar vs financial year changes the cohort and the answer. Establish it before comparing anything.

#### EXERCISE

Sharpen this request from a councillor into an analysis-ready question: "Is smoking still a problem in poor areas?" Identify measure, population, geography, time and comparison.

#### SOLUTION · TEXT

One good version:

"What was the smoking prevalence (% of adults 18+ who currently smoke, from the Annual Population Survey) in 2024/25, for each electoral ward / LSOA-derived area

SOLUTION · TEXT

in the borough, compared between the most and least deprived IMD quintiles, and how has the gap changed over the last five years?"  
Measure: adult smoking prevalence. Population: adults 18+. Geography: wards or IMD quintiles within the borough. Time: 2024/25 plus 5-year trend.  
Comparison: most vs least deprived.

KEY POINTS

- Question → Data → Method → Output → Check. Write the sharp question down before touching code.
- Every indicator = numerator + denominator + period + breakdown.
- UK health data runs April-March; always state which "year" you mean.

MODULE 1 · FOUNDATIONS & SETUP · LESSON 9 OF 105

9. Why databases beat spreadsheets (and what a database is)

A **relational database** stores data in **tables** (rows and columns), with defined **relationships** between them. Instead of one giant sheet where the patient's name is repeated on every admission row, a database keeps a `patients` table (one row per person) and an `admissions` table (one row per admission) joined by `patient_id`. This design — **normalisation** — means each fact is stored once, so it can't drift out of sync.

| Concept    | Spreadsheet world      | Database world                            |
|------------|------------------------|-------------------------------------------|
| A dataset  | Worksheet              | Table                                     |
| One record | Row                    | Row                                       |
| One field  | Column (A, B, C...)    | Named, typed column                       |
| Identity   | Whatever's in column A | Primary key (unique per row)              |
| Linking    | VLOOKUP per cell       | JOIN on keys, once, declaratively         |
| Rules      | Hope                   | Constraints (types, NOT NULL, uniqueness) |

- Three database superpowers matter for health data:
- **Types and constraints.** A column declared as DATE can't quietly hold "N/A" or "see notes"; a primary key can't duplicate. The database refuses bad data at the door.
  - **Concurrency and audit.** Fifty analysts can query the warehouse at once without emailing `final_v7_REAL.xlsx` around.
  - **Scale.** Millions of rows are routine.

The language for asking questions of relational databases is **SQL — Structured Query Language** (say "sequel" or spell it out; both are fine). It's *\*declarative\**: you describe **what** you want ("emergency admissions for over-65s in March, counted by specialty") and the engine works out how to fetch it. SQL has been an ISO standard since 1987, which is why skills you learn on SQLite transfer to the NHS's SQL Server, to Oracle, and to modern cloud warehouses.

**QUICK CHECK**

**In a well-designed health database, a patient's date of birth is stored...**

- ☐ On every admission row, for convenience
- ✓ **Once, in the patients table, linked to admissions by patient\_id**
- ☐ In a separate file per year
- ☐ In the admissions table only

**Why:** *One fact, one place. Repeating it per admission invites contradictions (three different DOBs for one person is a classic data-quality find).*

**QUICK CHECK**

**SQL is described as "declarative". That means...**

- ☐ You must specify the exact loop the computer runs
- ✓ **You state what result you want; the engine decides how to compute it**
- ☐ It only works on Microsoft products
- ☐ Queries must be written in capital letters

**Why:** *You declare the result's shape; the query optimiser picks the retrieval strategy. (Capitals for keywords are a style convention, not a rule.)*

**EXERCISE**

A team tracks flu vaccinations in one spreadsheet with columns: patient name, DOB, GP practice, practice phone number, vaccination date, batch. Sketch a better two- or three-table database design and say which columns go where.

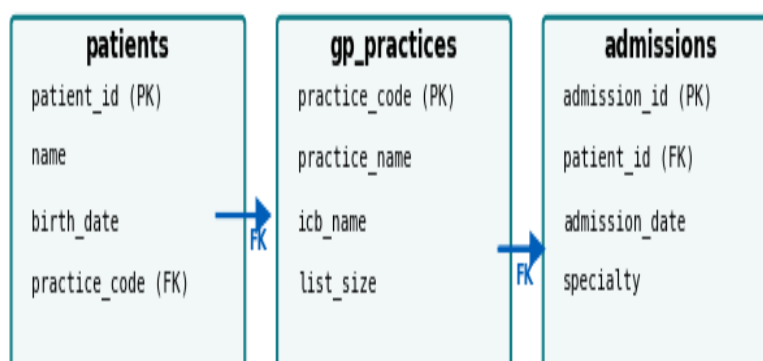
**SOLUTION - TEXT**

```
patients:      patient_id (primary key), name, dob, practice_code
gp_practices:  practice_code (primary key), practice_name, phone
vaccinations:  vaccination_id (primary key), patient_id, vaccination_date, batch
Links: vaccinations.patient_id -> patients.patient_id;
      patients.practice_code -> gp_practices.practice_code.
Each fact now lives once: change a practice phone number in one row, not thousands.
```

**KEY POINTS**

- Relational databases = typed tables linked by keys; each fact stored once.
- Constraints stop bad data at entry; spreadsheets only hope.
- SQL is the standard, declarative language for querying them — and it transfers everywhere.

## Relational database: tables & keys



- PK = Primary Key (unique row identifier)
- FK = Foreign Key (reference to another table)
- Joins connect tables via FK→PK relationships

### MODULE 1 • FOUNDATIONS & SETUP • LESSON 10 OF 105

## 10. Anatomy of a table: rows, columns, types and keys

Time to read a real table definition. Throughout this course you'll use a synthetic database for the fictional county of **Barsetshire** — three local authorities (Barchester, Millford, Kingsbay), twelve GP practices, ~600 patients. Here is its `patients` table:

#### THE PATIENTS TABLE (DEFINITION)

```

CREATE TABLE patients (
  patient_id    INTEGER PRIMARY KEY,  -- unique per person
  sex           TEXT,                 -- 'F' or 'M'
  birth_date    TEXT,                 -- ISO format 'YYYY-MM-DD'
  ethnicity     TEXT,                 -- ONS high-level groups
  lsoa_code     TEXT,                 -- links to lsoa_imd
  practice_code TEXT                  -- links to gp_practices
);
  
```

Reading it: each **column** has a name and a **type**. SQLite keeps types simple — `INTEGER` (whole numbers), `REAL` (decimals), `TEXT` (strings, including dates stored as ISO `YYYY-MM-DD`). Bigger systems

add richer types (`DATE`, `DECIMAL(10,2)`, `VARCHAR(50)`), but the idea is identical. The **PRIMARY KEY** is the column guaranteed unique and non-empty — the row's identity. Columns that point at another table's key (`lsoa_code`, `practice_code`) are **foreign keys**: the joins you'll write in Lesson 25 run along them.

One row of this table might be: (1042, 'F', '1958-03-14', 'White', 'E01000123', 'P81004') — a woman born March 1958, living in LSOA E01000123, registered at practice P81004. Notice what's *not* here: no name, no address. This mirrors how analysts usually receive data — pseudonymised, with `patient_id` standing in for identity.

#### WHY STORE DATES AS YYYY-MM-DD?

ISO 8601 dates sort correctly as plain text ('2025-03-01' < '2025-11-30'), are unambiguous internationally (no 03/04 confusion), and every tool parses them. UK analysts constantly convert *\*from\* DD/MM/YYYY \*into\* ISO* — you'll learn the code for that in both SQL and R.

#### QUICK CHECK

**Which column in patients is a foreign key?**

- ☐ patient\_id
- ☐ birth\_date
- ✓ ☒ practice\_code
- ☐ sex

**Why:** *practice\_code* points to the primary key of *gp\_practices* — that's what makes it foreign. *patient\_id* is this table's own primary key.

#### QUICK CHECK

**Why would '02/03/2025' be a risky way to store a date?**

- ✓ ☒ **It's ambiguous — 2 March or 3 February depending on convention — and sorts wrongly as text**
- ☐ Slashes are illegal in databases
- ☐ It uses too much storage
- ☐ Dates cannot be stored as text

**Why:** *DD/MM* vs *MM/DD* ambiguity has caused real reporting errors; and as text, '02/03/2025' sorts before '01/04/2024'. ISO *YYYY-MM-DD* fixes both.

#### EXERCISE

Design a `CREATE TABLE` statement for `lab_results` holding: a unique result id, the patient it belongs to, the test name, a numeric value, the units, and the date taken. Choose sensible types and mark the keys.

#### SOLUTION · SQL

```
CREATE TABLE lab_results (
  result_id  INTEGER PRIMARY KEY,
  patient_id INTEGER,          -- foreign key -> patients.patient_id
  test       TEXT,
  value      REAL,
  units      TEXT,
  date_taken TEXT             -- ISO 'YYYY-MM-DD'
);
-- (This is, in fact, almost exactly the table in your playground.)
```

**KEY POINTS**

- A table = named, typed columns; a primary key gives each row identity; foreign keys create relationships.
- SQLite types: INTEGER, REAL, TEXT — dates as ISO text.
- Analyst-facing data is usually pseudonymised: IDs, not names.

**MODULE 1 • FOUNDATIONS & SETUP • LESSON 11 OF 105**

## 11. Your first query — and how to use the playground

Everything is in place. This page contains a **live SQL engine** with the Barsetshire database loaded: eight tables of synthetic patients, admissions, vaccinations, lab results, deaths, GP practices, deprivation scores and population. Open it any time from the menu (► **SQL playground**) or with the **Run** ► button on SQL examples like the one below.

The playground has: a query box, a **Run** button, table-name chips that show each table's columns, and a **Reset database** button if you ever break something (you genuinely can't break anything important — it rebuilds in a second). Your first query:

**RUN YOUR FIRST QUERY**

```
SELECT patient_id, sex, birth_date
FROM patients
LIMIT 5;
```

Read it aloud: *"Select these three columns from the patients table; give me just 5 rows."* Press Run and five synthetic patients appear. Three habits to start now:

- **Keywords in CAPITALS** (`SELECT`, `FROM`, `LIMIT`) and names in lower case — pure convention, but it makes queries readable, and readability is a professional skill.
- **End statements with a semicolon.** Optional for single statements in some tools, required in scripts.
- **LIMIT while exploring.** Real tables have millions of rows; asking for all of them by accident is the classic new-analyst move. Peek first.

Also try the chips: tap `admissions` to see its columns, then adapt the query yourself (`SELECT * FROM admissions LIMIT 5;` — the `*` means "all columns").

**STUDY RHYTHM FOR THIS COURSE**

Each lesson takes 20–40 minutes. A sustainable pace is one or two lessons a day — the course is a marathon of 105 lessons, and skills compound only if you type the code yourself. Answer every quiz, attempt every exercise *\*before\** revealing the solution, and re-run yesterday's queries from memory as a warm-up.

**QUICK CHECK**

**In `SELECT patient_id, sex FROM patients LIMIT 5;` what does `LIMIT 5` do?**

- ☐ Shows only 5 columns
- ✓ **Returns at most 5 rows**
- ☐ Deletes all but 5 rows
- ☐ Sorts the first 5 values

**Why:** `LIMIT` caps the number of rows returned. It never changes the stored data — `SELECT` is read-only.



**QUICK CHECK**

**What does the asterisk mean in `SELECT * FROM admissions`?**

- ☐ All rows
- ✓ **All columns**
- ☐ A multiplication
- ☐ The primary key only

**Why:** \* = every column. Handy for a quick look; in production queries you'll name columns explicitly.

**EXERCISE**

In the playground: return the first 10 rows of the `gp_practices` table, all columns. Then modify it to show only `practice_name` and `la_name`. Write both queries.

**SOLUTION · SQL**

```
SELECT * FROM gp_practices LIMIT 10;
SELECT practice_name, la_name
FROM gp_practices
LIMIT 10;
```

**KEY POINTS**

- `SELECT columns FROM table LIMIT n` — your exploration pattern for every new table.
- `SELECT` never modifies data; you cannot break the playground (and there's a Reset anyway).
- Type everything yourself; the muscle memory is the point.

**MODULE 2**

# Module 2 — SQL core skills

**In this module**

- 12** `SELECT` and `FROM`: choosing your columns
- 13** `WHERE`: filtering rows with conditions
- 14** `ORDER BY` and `LIMIT`: sorting and topping
- 15** `DISTINCT`: seeing and counting unique values
- 16** Working with text: `LIKE`, case, trimming and joining strings
- 17** Numbers: arithmetic, rounding and the integer-division trap
- 18** `NULL`: the meaning of missing
- 19** `CASE WHEN`: categorising on the fly
- 20** Dates part 1: storing, comparing and extracting
- 21** Dates part 2: ages, financial years and month spines
- 22** Aggregates: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
- 23** `GROUP BY`: summaries for every category
- 24** `HAVING`: filtering the groups themselves
- 25** `JOIN` part 1: `INNER JOIN`, keys and aliases
- 26** `JOIN` part 2: `LEFT JOIN` and finding what's missing

- 27** JOIN part 3: chains, fan-out and the duplication trap
- 28** UNION and friends: stacking result sets
- 29** Subqueries: queries inside queries

## MODULE 2 · SQL CORE SKILLS · LESSON 12 OF 105

## 12. SELECT and FROM: choosing your columns

Every query you will ever write starts the same way: `SELECT` (which columns) `FROM` (which table). The database returns a result set — itself a little table — and changes nothing on disk. You can also \*compute\* new columns on the fly and rename things with `AS`:

### SELECTING, COMPUTING, RENAMING

```
-- Comments start with two dashes: the engine ignores them.
SELECT
  patient_id,
  birth_date,
  sex          AS patient_sex,      -- rename a column
  'Barsetshire' AS county          -- a constant column
FROM patients
LIMIT 8;
```

Points to absorb:

- **Column order in your `SELECT` controls output order.** The table's own order is irrelevant.
- **`AS` creates an alias** — the result column's display name. If an alias needs a space, wrap it in double quotes: `AS "Patient sex"` (works, but analysts usually stick to snake\_case names and prettify later in R).
- **Comments** (`-- like this`) are how professionals leave notes for colleagues and future selves. A query without comments is a query someone will misread.
- Layout is free-form: SQL ignores line breaks and extra spaces. One clause per line, indented, is the readable house style you'll see in NHS code.

Try editing the query in the playground: add `ethnicity`, remove `sex`, change the alias. Errors are welcome — read the message, fix, re-run. A typo like `SELCT` produces near `"SELCT": syntax error`, which tells you exactly where to look.

### QUICK CHECK

**What does `SELECT birth_date AS dob FROM patients` do to the stored table?**

- ☐ Renames the column permanently
- ☐ Deletes the other columns
- ✓ **Nothing — it only labels the output column `dob` for this query**
- ☐ Creates a new table called `dob`

**Why:** `SELECT` is read-only. `AS` affects only the result set you're looking at.

### QUICK CHECK

**Which line is a valid SQL comment?**

- ☐ `// fetch patients`
- ☐ `# fetch patients`
- ✓ **`-- fetch patients`**
- ☐ `<!-- fetch patients -->`

**Why:** Standard SQL comments are two dashes (SQLite and SQL Server also accept `/* block comments */`). The others belong to different languages.

**EXERCISE**

Write a query returning, from admissions: the admission\_id renamed to spell\_id, the specialty, and a constant column data\_source containing the text 'SUS'. Limit to 6 rows.

**SOLUTION · SQL**

```
SELECT
  admission_id AS spell_id,
  specialty,
  'SUS'        AS data_source
FROM admissions
LIMIT 6;
```

**KEY POINTS**

- SELECT columns FROM table; output order follows your SELECT list.
- AS = alias for readability; -- = comment; layout is for humans.
- SELECT never alters data — experiment freely.

## SQL SELECT statement anatomy

**SELECT**      *Which columns?*

**FROM**        *Which table?*

**WHERE**        *Which rows?*

**ORDER BY**    *Sort how?*

**Example result:**

| patient_id | admission_date | specialty   |
|------------|----------------|-------------|
| 1042       | 2025-12-15     | Emergency   |
| 1056       | 2025-12-10     | Respiratory |

## MODULE 2 · SQL CORE SKILLS · LESSON 13 OF 105

## 13. WHERE: filtering rows with conditions

`WHERE` keeps only the rows that satisfy a condition — it is how "all admissions" becomes "emergency admissions to Respiratory since April". The comparison operators: `=`, `<>` (not equal; `!=` also works), `<`, `<=`, `>`, `>=`. Text values go in **single quotes** and, in most engines, are case-sensitive — `'emergency'` will not match `'Emergency'`.

**FILTERING ADMISSIONS**

```
SELECT admission_id, patient_id, admission_date, specialty
FROM admissions
WHERE admission_method = 'Emergency'
      AND admission_date >= '2025-04-01'
      AND specialty      IN ('Respiratory', 'General Medicine')
LIMIT 10;
```

Combine conditions with `AND` (both must hold), `OR` (either), `NOT` (negate). Two power tools:

- `IN (list)` — cleaner than chained ORs, exactly as above.
- `BETWEEN a AND b` — inclusive range: `value BETWEEN 48 AND 58` includes both ends. For dates, `admission_date BETWEEN '2025-04-01' AND '2026-03-31'` covers financial year 2025/26 (thanks to ISO date text sorting correctly).

**The precedence trap:** `AND` binds tighter than `OR`. `WHERE specialty = 'Cardiology' OR specialty = 'Respiratory' AND admission_method = 'Emergency'` means Cardiology-anything OR emergency-Respiratory — probably not what you meant. **When mixing `AND` with `OR`, always add parentheses:**

**PARENTHESES MAKE INTENT EXPLICIT**

```
WHERE (specialty = 'Cardiology' OR specialty = 'Respiratory')
      AND admission_method = 'Emergency'
```

**QUICK CHECK**

**Which `WHERE` clause returns admissions in financial year 2024/25 only?**

- ☐ `WHERE admission_date BETWEEN '2024-01-01' AND '2024-12-31'`
- ✓ **`WHERE admission_date BETWEEN '2024-04-01' AND '2025-03-31'`**
- ☐ `WHERE admission_date = 2024/25`
- ☐ `WHERE year = '2024/25'`

**Why:** UK financial years run 1 April to 31 March. ISO-formatted text dates make `BETWEEN` safe here.

**QUICK CHECK**

**`WHERE sex = 'F' OR sex = 'M' AND imd_decile = 1` — without parentheses this returns...**

- ☐ Only decile-1 patients of either sex
- ✓ **All females, plus decile-1 males**
- ☐ All males, plus decile-1 females
- ☐ A syntax error

**Why:** `AND` evaluates first, so the condition is: `sex='F' OR (sex='M' AND decile=1)`. Parentheses would fix the ambiguity.

**EXERCISE**

From `lab_results`, return rows where the test is 'HbA1c' and the value is 48 or above (the diabetes diagnostic threshold in mmol/mol), taken on or after 1 April 2025. Show `patient_id`, `value` and `date_taken`, limited to 10 rows.

**SOLUTION · SQL**

```
SELECT patient_id, value, date_taken
FROM lab_results
WHERE test = 'HbA1c'
      AND value >= 48
      AND date_taken >= '2025-04-01'
LIMIT 10;
```

**KEY POINTS**

- WHERE filters rows; text in 'single quotes'; `<>` means not-equal.
- IN and BETWEEN (inclusive) tidy up common patterns.
- Mixing AND/OR without parentheses is the classic silent logic bug.

**MODULE 2 · SQL CORE SKILLS · LESSON 14 OF 105**

## 14. ORDER BY and LIMIT: sorting and topping

Result sets have **no guaranteed order** unless you ask for one — an engine may return rows in any sequence it finds convenient, and that sequence can change between runs. If order matters (it usually does in reports), say so with `ORDER BY`:

**LARGEST PRACTICES FIRST**

```
SELECT practice_name, la_name, list_size
FROM gp_practices
ORDER BY list_size DESC,      -- biggest first
        practice_name ASC    -- tie-break alphabetically
LIMIT 5;
```

- `ASC` (ascending) is the default; `DESC` flips it.
- Multiple sort keys act as tie-breakers, left to right.
- You can order by a column you didn't select, or by an alias you created in the `SELECT`.
- `ORDER BY + LIMIT = "top N"`: the query above is *\*the five largest practices\**. Add `OFFSET 5` to skip the first five (rows 6–10) — occasionally useful for paging.

**Dialect note you'll need in the NHS:** SQL Server has no `LIMIT`. Its idiom is `SELECT TOP 5 ... ORDER BY ...`, or the standard `OFFSET 0 ROWS FETCH NEXT 5 ROWS ONLY` after `ORDER BY`. Same idea, different spelling — Lesson 40 collects all of these.

Sorting text follows character order: for ISO dates that's chronological (the whole reason we store them that way), but beware numbers stored as text: `'10'` sorts before `'2'`. If a "numeric" column sorts oddly, its type is text — a data-quality clue.

**QUICK CHECK****Without an ORDER BY, the row order of a SELECT is...**

- ☐ Always the order rows were inserted
- ☐ Always primary-key order
- ✓ **Unspecified — the engine may return any order**
- ☐ Alphabetical by the first column

**Why:** Order is only guaranteed when you specify it. Never rely on "it came out sorted last time".**QUICK CHECK****Which query gives the 3 most recent admissions?**

- ☐ SELECT \* FROM admissions LIMIT 3;
- ✓ **SELECT \* FROM admissions ORDER BY admission\_date DESC LIMIT 3;**
- ☐ SELECT TOP admissions BY date;
- ☐ SELECT \* FROM admissions ORDER BY admission\_date ASC LIMIT 3;

**Why:** Sort newest-first (DESC), then cap at 3. ASC would give the three oldest; LIMIT alone gives three arbitrary rows.**EXERCISE**

List the 10 oldest patients: patient\_id, sex, birth\_date, sorted so the earliest birth date comes first, with ties broken by patient\_id.

**SOLUTION · SQL**

```
SELECT patient_id, sex, birth_date
FROM patients
ORDER BY birth_date ASC, patient_id ASC
LIMIT 10;
```

**KEY POINTS**

- No ORDER BY = no promised order; state your sort in anything that gets reported.
- ORDER BY col DESC LIMIT n = "top n"; SQL Server spells it TOP n.
- Text-sorting quirks (('10' < '2')) reveal columns with the wrong type.

**MODULE 2 · SQL CORE SKILLS · LESSON 15 OF 105****15. DISTINCT: seeing and counting unique values**

"What specialties do we actually have in this data?" is a first-day-with-a-new-dataset question, and **DISTINCT** answers it — it collapses duplicate rows in the output:

**WHAT VALUES EXIST?**

```
SELECT DISTINCT specialty
FROM admissions
ORDER BY specialty;
```

Used on **several columns**, **DISTINCT** returns unique \*combinations\*: `SELECT DISTINCT specialty, admission_method FROM admissions` lists each specialty-method pairing that occurs. This is your profiling tool: run it on any code-like column (ethnicity, vaccine, admission\_method) the moment you meet a new table, because it exposes the messy truth — stray values like 'emergency ' with a trailing space, 'RESP' vs 'Respiratory', or mysterious codes needing a lookup.

DISTINCT also pairs with COUNT (formally taught in Lesson 22) for the single most common analyst question — \*how many people, not how many rows\*:

#### ROWS VS PEOPLE

```
SELECT
  COUNT(*)                AS admission_rows,
  COUNT(DISTINCT patient_id) AS distinct_patients
FROM admissions;
```

Those two numbers differ because some patients are admitted more than once. Confusing them is one of the most consequential real-world errors in health analysis: "10,000 admissions" is activity; "10,000 patients" is people. Commissioners fund the first; epidemiology counts the second.

#### QUICK CHECK

**A vaccination table has 3 rows for one child (two MMR doses and a flu jab). SELECT DISTINCT patient\_id FROM vaccinations returns how many rows for this child?**

- ☒ 1
- ☐ 2
- ☐ 3
- ☐ 0

**Why:** *DISTINCT collapses duplicates: one unique patient\_id, however many events.*

#### QUICK CHECK

**Your trust reports "42,000 A&E attendances" and a colleague says "42,000 patients used A&E". Why might the colleague be wrong?**

- ☐ Attendances are always fewer than patients
- ☒ **Some people attend more than once, so distinct patients ≤ attendances**
- ☐ A&E does not record patients
- ☐ The two numbers are always equal

**Why:** *Frequent attenders mean the same person appears in many rows. COUNT(\*) counts events; COUNT(DISTINCT patient\_id) counts people.*

#### EXERCISE

Profile the vaccinations table: (1) list the distinct vaccine names; (2) list the distinct vaccine + dose\_number combinations, ordered by vaccine then dose.

#### SOLUTION · SQL

```
SELECT DISTINCT vaccine
FROM vaccinations
ORDER BY vaccine;

SELECT DISTINCT vaccine, dose_number
FROM vaccinations
ORDER BY vaccine, dose_number;
```

#### KEY POINTS

- DISTINCT reveals the actual values (and value combinations) a column contains — run it on every new categorical column.
- COUNT(\*) = rows/events; COUNT(DISTINCT id) = people. Know which your question needs.
- Odd values surfaced by DISTINCT are data-quality leads, not annoyances.



## MODULE 2 • SQL CORE SKILLS • LESSON 16 OF 105

## 16. Working with text: LIKE, case, trimming and joining strings

Health data is full of text — codes, names, free-ish labels — and text is where mess lives. Core tools:

**Pattern matching with LIKE.** % matches any run of characters, \_ matches exactly one:

### ICD-10 CHAPTER HUNTING

```
-- All respiratory-chapter diagnoses (ICD-10 codes starting with J)
SELECT DISTINCT primary_diagnosis
FROM admissions
WHERE primary_diagnosis LIKE 'J%'
ORDER BY primary_diagnosis;
```

LIKE 'J1\_' would match J10–J19 style three-character codes only. In SQLite, LIKE ignores case for plain letters; in SQL Server it depends on the database's collation — so professionals often normalise case explicitly rather than rely on settings:

**Case and whitespace.** UPPER(x) / LOWER(x) standardise case; TRIM(x) strips leading/trailing spaces (LTRIM/RTRIM for one side). The defensive comparison pattern worth memorising: WHERE UPPER(TRIM(specialty)) = 'RESPIRATORY' — it survives ' respiratory ' and 'RESPIRATORY' alike.

**Slicing and joining.** SUBSTR(text, start, length) extracts characters (start counts from 1): SUBSTR(primary\_diagnosis, 1, 1) gives the ICD-10 chapter letter. Concatenation is || in standard SQL/SQLite: 'Practice ' || practice\_code. (SQL Server uses + or CONCAT(); another one for the Lesson 40 phrasebook.) LENGTH(x) returns character count — great for validation, e.g. NHS numbers must be 10 digits.

### BUILDING LABELS

```
SELECT
  practice_code,
  UPPER(la_name) AS la_upper,
  practice_name || ' (' || practice_code || ')' AS label
FROM gp_practices
LIMIT 6;
```

### QUICK CHECK

**WHERE lsoa\_code LIKE 'E01%' matches...**

- ☐ Codes ending in E01
- ✓ **Codes beginning E01, of any length**
- ☐ Exactly the code E01
- ☐ Codes containing exactly one E

**Why:** % is a wildcard for any (possibly empty) sequence of characters after E01 — this pattern selects English LSOA codes.

**QUICK CHECK**

Which expression reliably matches 'Elective', 'ELECTIVE' and ' elective ' all at once?

- ☐ admission\_method = 'Elective'
- ☐ admission\_method LIKE 'Elective'
- ✓ **UPPER(TRIM(admission\_method)) = 'ELECTIVE'**
- ☐ TRIM(admission\_method)

**Why:** Normalise both case and whitespace before comparing. It's the standard defence against inconsistently entered categories.

**EXERCISE**

From admissions, extract the ICD-10 chapter letter of each primary\_diagnosis as a column called chapter, and count rows per chapter using SELECT chapter, COUNT(\*) ... GROUP BY chapter (a preview of Lesson 23). Order by the count, largest first.

**SOLUTION · SQL**

```
SELECT
  SUBSTR(primary_diagnosis, 1, 1) AS chapter,
  COUNT(*)                      AS n
FROM admissions
GROUP BY SUBSTR(primary_diagnosis, 1, 1)
ORDER BY n DESC;
```

**KEY POINTS**

- LIKE with % and \_ finds patterns; leading-letter patterns navigate ICD-10 chapters.
- UPPER/LOWER + TRIM = defensive comparisons against messy text.
- SUBSTR slices, || concatenates (SQL Server: + / CONCAT), LENGTH validates.

**MODULE 2 · SQL CORE SKILLS · LESSON 17 OF 105**

## 17. Numbers: arithmetic, rounding and the integer-division trap

SQL does arithmetic in the SELECT list with + - \* / and % (remainder in many engines). You'll use it constantly to build rates and percentages. But there is one trap that has produced wrong numbers in real dashboards: **integer division**. In many databases (SQL Server prominently; SQLite included), dividing an integer by an integer discards the remainder:

**THE TRAP, DEMONSTRATED**

```
SELECT
  7 / 2                      AS int_division,    -- 3, not 3.5!
  7 / 2.0                    AS with_decimal,    -- 3.5
  CAST(7 AS REAL) / 2        AS with_cast,      -- 3.5
  100.0 * 37 / 120           AS pct_correct,    -- 30.83...
  ROUND(100.0 * 37 / 120, 1) AS pct_rounded     -- 30.8
FROM patients LIMIT 1;  -- (any one-row source works for a demo)
```

The professional habit: when computing any proportion, **make one operand a decimal** — either write 100.0 / multiply by 1.0, or CAST(numerator AS REAL). CAST(x AS type) is the general conversion tool (REAL, INTEGER, TEXT in SQLite; FLOAT, INT, DECIMAL(10,2), VARCHAR in SQL Server).

`ROUND(x, n)` rounds to *n* decimal places — apply it **only at the final display step**, never mid-calculation, or rounding errors accumulate. Also useful: `ABS(x)` for absolute values (e.g. difference between two readings). One more subtlety for later modules: rounding rules at .5 differ slightly between engines and R (R uses "round half to even"), which is why reconciliations sometimes differ by a penny — knowing this saves you an afternoon someday.

**QUICK CHECK**

In SQLite/SQL Server, `SELECT 45 / 60` returns...

- ☒ 0
- ☐ 0.75
- ☐ 75
- ☐ An error

**Why:** *Integer ÷ integer = integer: the .75 is truncated to 0. Write `45 / 60.0` or `CAST` one side to `REAL` to get 0.75.*

**QUICK CHECK**

Which expression correctly computes a percentage of emergency admissions as a decimal like 61.3?

- ☐ `emergency / total * 100`
- ☐ `ROUND(emergency / total, 1) * 100`
- ☒ `ROUND(100.0 * emergency / total, 1)`
- ☐ `CAST(emergency AS TEXT) / total`

**Why:** *The 100.0 forces decimal arithmetic before the division; ROUND then trims for display. The first two fall into integer division if both columns are integers.*

**EXERCISE**

Each GP practice has a `list_size`. Write a query showing `practice_name`, `list_size`, and `list_size` as a percentage of 100000 (a notional ICB population), rounded to 2 decimal places, ordered largest first, top 5.

**SOLUTION · SQL**

```
SELECT
  practice_name,
  list_size,
  ROUND(100.0 * list_size / 100000, 2) AS pct_of_icb
FROM gp_practices
ORDER BY pct_of_icb DESC
LIMIT 5;
```

**KEY POINTS**

- Integer ÷ integer truncates: force decimals with `100.0`, `1.0 *`, or `CAST(... AS REAL)`.
- `ROUND(x, n)` at the display step only; `ABS` for magnitudes; `CAST` converts types.
- Every rate and percentage you ever publish passes through this lesson.

## MODULE 2 • SQL CORE SKILLS • LESSON 18 OF 105

## 18. NULL: the meaning of missing

NULL is SQL's marker for **no value** — not zero, not an empty string, but \*unknown/absent\*. In health data NULLs are everywhere and they \*mean\* things: a NULL `discharge_date` can mean the patient is still in a bed; a NULL `date_of_death` means (as far as this system knows) the person is alive. Treating NULL casually produces silently wrong counts, so learn its three rules:

**1. NULL compares as unknown.** `discharge_date = NULL` is never true — not even for NULL rows — because nothing equals unknown. The correct tests are `IS NULL` and `IS NOT NULL`:

**WHO IS STILL IN HOSPITAL?**

```
SELECT admission_id, patient_id, admission_date, specialty
FROM admissions
WHERE discharge_date IS NULL
ORDER BY admission_date;
```

**2. NULL propagates through calculations.** `value + NULL` is NULL; `'text' || NULL` is NULL. One missing component poisons the expression.

**3. NULL is invisible to both sides of a filter.** `WHERE ethnicity <> 'White'` excludes NULL ethnicities too — rows that are neither equal nor not-equal simply drop out. If you mean "non-White \*or unknown\*", say `WHERE ethnicity <> 'White' OR ethnicity IS NULL`.

The repair tool is `COALESCE(a, b, c, ...)` — it returns the first non-NULL argument. `COALESCE(ethnicity, 'Not recorded')` gives reports a tidy label; `COALESCE(discharge_date, date('now'))` treats current inpatients as "discharged today" for a length-of-stay-so-far calculation. (SQL Server also offers `ISNULL(a,b)`; `COALESCE` is the standard and takes any number of arguments.)

**QUICK CHECK**

**Which query correctly finds patients with no recorded ethnicity?**

- ☐ `WHERE ethnicity = NULL`
- ☐ `WHERE ethnicity = 'NULL'`
- ✓ **`WHERE ethnicity IS NULL`**
- ☐ `WHERE ethnicity == none`

**Why:** Equality with NULL is never true; `IS NULL` is the dedicated test. 'NULL' in quotes is just a four-letter word.

**QUICK CHECK**

**A table has 100 rows: 60 with ethnicity 'White', 30 'Asian', 10 NULL. How many rows does `WHERE ethnicity <> 'White'` return?**

- ☐ 40
- ✓ **30**
- ☐ 10
- ☐ 100

**Why:** The 10 NULL rows satisfy neither `=` nor `<>` — they vanish from both filters. This silent exclusion is the classic NULL bug.

**EXERCISE**

Write a query listing `admission_id`, `admission_date`, and a column `discharge_shown` that displays the `discharge_date`, or the text 'Still in hospital' where it is missing. Hint: `COALESCE` mixes types happily in SQLite.

**SOLUTION · SQL**

```
SELECT
  admission_id,
  admission_date,
  COALESCE(discharge_date, 'Still in hospital') AS discharge_shown
FROM admissions
ORDER BY admission_date DESC
LIMIT 12;
```

**KEY POINTS**

- NULL = unknown: test with IS NULL / IS NOT NULL, never = NULL.
- NULL poisons arithmetic and escapes both sides of <> filters — count your NULLs before filtering.
- COALESCE substitutes a fallback; in health data, first ask what the NULL means clinically.

**MODULE 2 · SQL CORE SKILLS · LESSON 19 OF 105**

## 19. CASE WHEN: categorising on the fly

CASE WHEN is SQL's if-then-else, and it is the tool behind every age band, flag and RAG rating you'll ever build. Conditions are checked **top to bottom**; the first match wins; ELSE catches everything left (omit it and non-matches become NULL).

**FLAGGING AND BANDING**

```
SELECT
  patient_id,
  test,
  value,
  CASE
    WHEN value >= 48          THEN 'Diabetes range'
    WHEN value BETWEEN 42 AND 47 THEN 'Pre-diabetes'
    WHEN value IS NULL        THEN 'No result'
    ELSE 'Normal'
  END AS hba1c_band
FROM lab_results
WHERE test = 'HbA1c'
LIMIT 12;
```

Because evaluation stops at the first true branch, **order your conditions from most to least specific** — a WHEN value >= 0 placed first would swallow everything. CASE expressions can live anywhere an expression can: in ORDER BY (custom sort orders), inside aggregates (Lesson 22's conditional counting trick), and in GROUP BY.

The other daily use is banding ages. You don't have an age column yet (that's Lesson 21), but the pattern is universal:

**THE AGE-BAND PATTERN (MEMORISE THE SHAPE)**

```
CASE
  WHEN age < 5          THEN '0-4'
  WHEN age BETWEEN 5 AND 17 THEN '5-17'
  WHEN age BETWEEN 18 AND 39 THEN '18-39'
  WHEN age BETWEEN 40 AND 64 THEN '40-64'
  WHEN age BETWEEN 65 AND 79 THEN '65-79'
  ELSE '80+'
END AS age_band
```

These bands match the `la_population` table in your playground — a deliberate design so numerators and denominators can meet (the essence of rate calculation, coming in Lesson 37).

**QUICK CHECK**

**In a CASE expression, what happens if two WHEN conditions are both true for a row?**

- ☐ The engine raises an error
- ☐ Both labels are returned
- ✓ **The first true condition (top-down) wins**
- ☐ The last true condition wins

**Why:** Evaluation is strictly top to bottom and stops at the first match — which is why condition order matters.

**QUICK CHECK**

**A CASE with no ELSE, where no WHEN matches, returns...**

- ☐ 0
- ☐ An empty string
- ✓ **NULL**
- ☐ The first WHEN's value

**Why:** No match + no ELSE = NULL. Adding an explicit ELSE 'Other' (or similar) is usually kinder to whoever reads the output.

**EXERCISE**

From admissions, create a column `los_flag`: 'Same day' when `discharge_date` equals `admission_date`, 'Still in' when `discharge_date` IS NULL, otherwise 'Overnight+'. Show `admission_id`, both dates and the flag for 12 rows.

**SOLUTION · SQL**

```
SELECT
  admission_id,
  admission_date,
  discharge_date,
  CASE
    WHEN discharge_date IS NULL      THEN 'Still in'
    WHEN discharge_date = admission_date THEN 'Same day'
    ELSE 'Overnight+'
  END AS los_flag
FROM admissions
LIMIT 12;
```

**KEY POINTS**

- CASE WHEN ... THEN ... ELSE ... END: first true branch wins; missing ELSE yields NULL.
- Order conditions most-specific-first; put IS NULL checks in explicitly.
- Age banding via CASE is the analyst's bread and butter — align bands with your population table.

**MODULE 2 · SQL CORE SKILLS · LESSON 20 OF 105**

## 20. Dates part 1: storing, comparing and extracting

Dates power almost every health question — trends, waits, ages, financial years — and they're where SQL dialects differ most. The universal foundation: store and write dates as **ISO 8601 text**, YYYY-MM-DD. Comparisons and BETWEEN then just work, as you've already seen.

SQLite's date toolkit (used by your playground) is three functions:

- `date('now')` — today's date. `datetime('now')` adds the time (UTC).
- `date(d, modifier...)` — date arithmetic via text modifiers: `date('2026-03-31', '-1 year')`, `date(admission_date, '+30 day')`, `date('now', 'start of month')`.
- `strftime(format, d)` — extract parts: `'%Y'` year, `'%m'` month, `'%d'` day, `'%Y-%m'` a month key for grouping.

**EXTRACTING AND SHIFTING**

```
SELECT
  admission_date,
  strftime('%Y', admission_date) AS yr,
  strftime('%Y-%m', admission_date) AS yr_month,
  date(admission_date, '+30 day') AS follow_up_due
FROM admissions
LIMIT 6;
```

Subtracting dates: `julianday(a) - julianday(b)` gives the difference in days — the basis of length of stay:

**LENGTH OF STAY IN DAYS**

```
SELECT
  admission_id,
  admission_date,
  discharge_date,
  CAST(julianday(discharge_date) - julianday(admission_date) AS INTEGER) AS los_days
FROM admissions
WHERE discharge_date IS NOT NULL
ORDER BY los_days DESC
LIMIT 8;
```

**Dialect phrasebook (preview):** SQL Server writes today as `GETDATE()` or `CAST(GETDATE() AS DATE)`, differences as `DATEDIFF(DAY, a, b)`, shifts as `DATEADD(DAY, 30, a)`, and parts as `YEAR(d) / MONTH(d) / FORMAT(d, 'yyyy-MM')`. Same concepts, different names — the full table waits in Lesson 40.

**QUICK CHECK**

**In SQLite, which expression gives the number of days between admission and discharge?**

- ☐ `discharge_date - admission_date`
- ☐ `DATEDIFF(discharge_date, admission_date)`
- ✓ **`julianday(discharge_date) - julianday(admission_date)`**
- ☐ `AGE(discharge_date, admission_date)`

**Why:** Subtracting ISO text directly doesn't work in SQLite; `julianday` converts each date to a day-number first. `DATEDIFF` is SQL Server's spelling.

**QUICK CHECK**

**`strftime('%Y-%m', '2025-12-07')` returns...**

- ☐ 2025
- ☐ 12-2025
- ✓ **2025-12**
- ☐ December 2025

**Why:** `%Y-%m` formats year and month — the standard "month key" you'll `GROUP BY` for monthly trend charts.

**EXERCISE**

Return the 10 most recent lab results with: `date_taken`, the year-month key, and the date exactly one year after `date_taken` (call it `review_due`).

**SOLUTION · SQL**

```
SELECT
  date_taken,
  strftime('%Y-%m', date_taken) AS yr_month,
  date(date_taken, '+1 year') AS review_due
FROM lab_results
ORDER BY date_taken DESC
LIMIT 10;
```

**KEY POINTS**

- ISO YYYY-MM-DD text makes comparisons, BETWEEN and sorting safe everywhere.
- SQLite: `date()/datetime()` shift, `strftime()` extracts, `julianday` differences count days.
- SQL Server says GETDATE, DATEDIFF, DATEADD, YEAR/MONTH — same ideas (Lesson 40 has the full phrasebook).

**MODULE 2 · SQL CORE SKILLS · LESSON 21 OF 105**

## 21. Dates part 2: ages, financial years and month spines

Three date calculations come up so often in UK health work that they deserve their own lesson.

**1. Age at an event.** Age is the person's completed years at a reference date. A precise, portable formula: take the year difference, then subtract 1 if the birthday hasn't happened yet that year. In SQLite a compact trick compares the MM-DD parts as text:

**AGE AT ADMISSION, EXACTLY**

```
SELECT
  a.admission_id,
  p.birth_date,
  a.admission_date,
  (strftime('%Y', a.admission_date) - strftime('%Y', p.birth_date))
  - (strftime('%m-%d', a.admission_date) < strftime('%m-%d', p.birth_date))
  AS age_at_admission
FROM admissions a
JOIN patients p ON p.patient_id = a.patient_id -- joins arrive properly in Lesson 25
LIMIT 8;
```

That `(condition)` evaluates to 1 when true, 0 when false — subtracting it handles the not-yet-birthday case. An approximation like `(julianday(d1)-julianday(d0))/365.25` is fine for grouping but can misplace people near band boundaries; use the exact form for anything published.

**2. Financial year.** April–March, labelled 2025/26. The rule: if the month is January–March, the FY started the \*previous\* calendar year.

**DERIVING FINANCIAL YEAR**

```
SELECT
  admission_date,
  CASE
    WHEN CAST(strftime('%m', admission_date) AS INTEGER) >= 4
      THEN strftime('%Y', admission_date) || '/' ||
        SUBSTR(CAST(strftime('%Y', admission_date) + 1 AS TEXT), 3, 2)
    ELSE (strftime('%Y', admission_date) - 1) || '/' ||
```



**DERIVING FINANCIAL YEAR**

```

        SUBSTR(strftime('%Y', admission_date), 3, 2)
    END AS financial_year
FROM admissions
LIMIT 8;

```

**3. Month keys for trends.** `strftime('%Y-%m', d)` gives a groupable, sortable month. One caution for real reports: months with zero events simply *\*don't appear\** in grouped output — a gap, not a zero. The fix (a "date spine" you join onto) is built in Lesson 32.

**QUICK CHECK**

**A patient born 1960-07-15 is admitted on 2025-07-01. Their age at admission is...**

- ☐ 65
- ✓ **64**
- ☐ 64.96
- ☐ 66

**Why:** The 65th birthday (15 July) hasn't happened yet on 1 July, so completed age is 64. This is exactly the case the exact formula handles and the /365.25 shortcut can fumble.

**QUICK CHECK**

**An admission on 2026-02-10 belongs to which financial year?**

- ☐ 2026/27
- ✓ **2025/26**
- ☐ 2025
- ☐ 2026

**Why:** February is before April, so it falls in the FY that began the previous April: 2025/26.

**EXERCISE**

Write a query showing each vaccination's `date_given` and its financial year, then count vaccinations per financial year (GROUP BY sneak preview) ordered by year.

**SOLUTION · SQL**

```

SELECT
  CASE
    WHEN CAST(strftime('%m', date_given) AS INTEGER) >= 4
      THEN strftime('%Y', date_given) || '/' ||
            SUBSTR(CAST(strftime('%Y', date_given) + 1 AS TEXT), 3, 2)
    ELSE (strftime('%Y', date_given) - 1) || '/' ||
          SUBSTR(strftime('%Y', date_given), 3, 2)
  END   AS financial_year,
  COUNT(*) AS vaccinations
FROM vaccinations
GROUP BY financial_year
ORDER BY financial_year;

```

**KEY POINTS**

- Exact age = year difference minus (birthday not yet reached); publish nothing with the 365.25 shortcut.
- FY rule: months Jan–Mar belong to the FY that began the previous April.
- Month keys via `strftime('%Y-%m')`; remember grouped output omits zero months.

## MODULE 2 • SQL CORE SKILLS • LESSON 22 OF 105

## 22. Aggregates: COUNT, SUM, AVG, MIN, MAX

Aggregate functions collapse many rows into one summary value — the moment SQL turns from *\*fetching\** data into *\*analysing\** it.

**ONE-ROW SUMMARY OF ADMISSIONS**

```
SELECT
  COUNT(*)                               AS total_admissions,
  COUNT(DISTINCT patient_id)             AS patients_admitted,
  MIN(admission_date)                    AS first_date,
  MAX(admission_date)                    AS last_date,
  ROUND(AVG(julianday(discharge_date)
            - julianday(admission_date)), 1) AS mean_los_days
FROM admissions;
```

Rules with teeth:

- **COUNT(\*)** counts **rows**. **COUNT(column)** counts **non-NULL values** in that column — so **COUNT(discharge\_date)** is completed discharges, and **COUNT(\*) - COUNT(discharge\_date)** is current inpatients. That NULL-skipping applies to all aggregates: **AVG** averages only the values that exist.
- **SUM** adds; on a column of 1/0 flags it becomes a conditional counter.
- The **conditional aggregation** idiom — CASE inside an aggregate — builds several filtered counts in one pass:

**SEVERAL COUNTS IN ONE QUERY**

```
SELECT
  COUNT(*)                               AS all_adm,
  SUM(CASE WHEN admission_method = 'Emergency' THEN 1 ELSE 0 END) AS emergency,
  SUM(CASE WHEN admission_method = 'Elective' THEN 1 ELSE 0 END) AS elective,
  ROUND(100.0 * SUM(CASE WHEN admission_method = 'Emergency' THEN 1 ELSE 0 END)
        / COUNT(*), 1)                  AS pct_emergency
FROM admissions;
```

That pattern — numerator built with CASE, denominator with COUNT(\*), joined by 100.0 \* — is the skeleton of half the performance indicators in the NHS.

**QUICK CHECK**

**In a table of 500 admissions where 40 have NULL discharge\_date, COUNT(discharge\_date) returns...**

- ☐ 500  
☐ 40  
☒ **460**  
☐ NULL

**Why:** *COUNT(column)* skips NULLs:  $500 - 40 = 460$  completed discharges. *COUNT(\*)* would give 500.

**QUICK CHECK****AVG(value) over the values 10, 20, NULL is...**

- ☐ 10
- ✓ **15**
- ☐ 7.5 because NULL counts as 0
- ☐ NULL

**Why:** Aggregates ignore NULLs entirely:  $(10+20)/2 = 15$ . If you wanted missing to count as zero, you'd `COALESCE(value,0)` first — a deliberate analytical choice.

**EXERCISE**

For `lab_results` where `test = 'Systolic BP'`: in one query return the number of readings, number of distinct patients, mean value (1 dp), minimum and maximum, and the count of readings of 140 or higher (call it `high_bp_readings`).

**SOLUTION · SQL**

```
SELECT
  COUNT(*)                AS readings,
  COUNT(DISTINCT patient_id) AS patients,
  ROUND(AVG(value), 1)     AS mean_sbp,
  MIN(value)              AS min_sbp,
  MAX(value)              AS max_sbp,
  SUM(CASE WHEN value >= 140 THEN 1 ELSE 0 END) AS high_bp_readings
FROM lab_results
WHERE test = 'Systolic BP';
```

**KEY POINTS**

- `COUNT(*)` = rows; `COUNT(col)` = non-NULLs; all aggregates skip NULLs.
- `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` = filtered counting inside one query.
- $\text{Numerator} \div \text{denominator} \times 100.0$  — the indicator skeleton you'll reuse forever.

**MODULE 2 · SQL CORE SKILLS · LESSON 23 OF 105**

## 23. GROUP BY: summaries for every category

`GROUP BY` splits rows into groups and runs your aggregates **once per group** — turning "how many admissions?" into "how many admissions \*per specialty, per month, per practice\*?". It is the single most-used clause in health analytics.

**ADMISSIONS BY SPECIALTY AND METHOD**

```
SELECT
  specialty,
  admission_method,
  COUNT(*)                AS admissions,
  ROUND(AVG(julianday(discharge_date)
            - julianday(admission_date)), 1) AS mean_los
FROM admissions
GROUP BY specialty, admission_method
ORDER BY admissions DESC;
```

The iron rule: **every column in your SELECT must be either inside an aggregate or listed in GROUP BY**. SQLite is permissive and will quietly pick an arbitrary row's value for an unaggregated column — a silent wrong answer. SQL Server refuses to run such a query at all, which is friendlier.

Write to the strict rule always.

Monthly trend tables fall straight out by grouping on an expression:

#### THE MONTHLY TREND PATTERN

```
SELECT
  strftime('%Y-%m', admission_date) AS yr_month,
  COUNT(*)                          AS admissions,
  COUNT(DISTINCT patient_id)        AS patients
FROM admissions
GROUP BY strftime('%Y-%m', admission_date)
ORDER BY yr_month;
```

Reading grouped output critically is a skill: small groups (a specialty with 3 admissions) produce unstable averages; and remember from Lesson 21 that missing months are absent, not zero. Both caveats belong in your commentary when this becomes a report.

#### QUICK CHECK

**Which query is correctly formed?**

- ☐ SELECT specialty, admission\_date, COUNT(\*) FROM admissions GROUP BY specialty;
- ✓ **SELECT specialty, COUNT(\*) FROM admissions GROUP BY specialty;**
- ☐ SELECT specialty, COUNT(\*) FROM admissions;
- ☐ SELECT COUNT(\*) GROUP BY specialty;

**Why:** Every selected column must be grouped or aggregated. Option 1 selects admission\_date without either; option 3 mixes an aggregate with a bare column and no GROUP BY.

#### QUICK CHECK

**GROUP BY strftime('%Y-%m', admission\_date) produces one output row per...**

- ☐ Admission
- ☐ Year
- ✓ **Calendar month that has at least one admission**
- ☐ Patient

**Why:** One row per distinct month key present in the data — which is why zero-activity months vanish (date spines fix that later).

#### EXERCISE

Produce a table of vaccinations given, by vaccine and dose\_number: count of doses and count of distinct children, ordered by vaccine then dose\_number.

#### SOLUTION · SQL

```
SELECT
  vaccine,
  dose_number,
  COUNT(*)                AS doses_given,
  COUNT(DISTINCT patient_id) AS children
FROM vaccinations
GROUP BY vaccine, dose_number
ORDER BY vaccine, dose_number;
```

**KEY POINTS**

- GROUP BY = one summary row per category (or category combination).
- Strict rule: select only grouped columns or aggregates — SQLite won't warn you, SQL Server will.
- Group on expressions (month keys) for trends; distrust tiny groups.

**MODULE 2 · SQL CORE SKILLS · LESSON 24 OF 105****24. HAVING: filtering the groups themselves**

WHERE filters **rows before grouping**; HAVING filters **groups after aggregation**. You need HAVING the moment your condition involves an aggregate — you cannot write `WHERE COUNT(*) > 5`, because when WHERE runs, no counting has happened yet.

**FREQUENT ATTENDERS: 3+ EMERGENCY ADMISSIONS**

```
SELECT
  patient_id,
  COUNT(*) AS emergency_admissions
FROM admissions
WHERE admission_method = 'Emergency'  -- row filter first
GROUP BY patient_id
HAVING COUNT(*) >= 3                  -- then group filter
ORDER BY emergency_admissions DESC;
```

That query is a real public-health workhorse: identifying **frequent A&E attenders / repeat emergency admissions** for case-management programmes. Note the division of labour — WHERE cut the data to emergencies (efficient: less to group), HAVING kept only the patients whose \*aggregate\* crossed the threshold.

The full logical order of a query is worth memorising, because it explains most beginner errors:

| Step | Clause         | What happens             |
|------|----------------|--------------------------|
| 1    | FROM (+ JOINS) | Assemble the source rows |
| 2    | WHERE          | Filter rows              |
| 3    | GROUP BY       | Form groups              |
| 4    | HAVING         | Filter groups            |
| 5    | SELECT         | Compute output columns   |
| 6    | ORDER BY       | Sort results             |
| 7    | LIMIT          | Cap the output           |

Because SELECT runs *after* WHERE, a column alias made in SELECT can't be used in WHERE (engines vary on HAVING/ORDER BY; SQLite allows aliases in both). When in doubt, repeat the expression.

**QUICK CHECK**

"Practices with more than 200 registered flu vaccinations" needs which construction?

- ☐ WHERE COUNT(\*) > 200
- ✓ **GROUP BY practice ... HAVING COUNT(\*) > 200**
- ☐ HAVING before GROUP BY
- ☐ ORDER BY COUNT(\*) > 200

**Why:** The condition tests an aggregate, so it must run after grouping — that's precisely HAVING's job.

**QUICK CHECK**

Why keep admission\_method = 'Emergency' in WHERE rather than HAVING?

- ☐ HAVING cannot reference text columns
- ☐ It changes the answer
- ✓ **It's a row-level condition, so filtering early is correct and more efficient**
- ☐ WHERE is deprecated

**Why:** Row conditions belong in WHERE (smaller groups, faster, clearer intent). HAVING is reserved for conditions on aggregates. Putting a row condition in HAVING sometimes runs but muddles meaning.

**EXERCISE**

Find LSOAs (lsoa\_code in patients) with at least 15 registered patients: return lsoa\_code and the patient count, largest first.

**SOLUTION · SQL**

```
SELECT
  lsoa_code,
  COUNT(*) AS patients
FROM patients
GROUP BY lsoa_code
HAVING COUNT(*) >= 15
ORDER BY patients DESC;
```

**KEY POINTS**

- WHERE = rows before grouping; HAVING = groups after aggregation.
- Frequent-attender lists are GROUP BY patient + HAVING COUNT ≥ threshold.
- Learn the logical clause order — it predicts what the engine will accept.

**MODULE 2 · SQL CORE SKILLS · LESSON 25 OF 105****25. JOIN part 1: INNER JOIN, keys and aliases**

So far every query has used one table, but the questions that matter span several: \*admissions\* by \*deprivation decile\* needs admissions → patients → lsoa\_imd. JOIN stitches tables together by matching key values. An **INNER JOIN** keeps only rows that find a match on both sides.

**ADMISSIONS WITH PATIENT DETAILS ATTACHED**

```
SELECT
  a.admission_id,
  a.admission_date,
  a.specialty,
  p.sex,
```

**ADMISSIONS WITH PATIENT DETAILS ATTACHED**

```
p.birth_date,
p.practice_code
FROM admissions a
INNER JOIN patients p
    ON p.patient_id = a.patient_id
LIMIT 8;
```

Anatomy:

- `admissions a` gives the table a short **alias**; thereafter every column is written `alias.column`. With multiple tables this isn't optional politeness — `patient_id` exists in both, and an unqualified reference is ambiguous.
- `ON p.patient_id = a.patient_id` is the **join condition**: match rows where the keys are equal. Joining on the wrong columns, or forgetting `ON` entirely (which produces every-row-with-every-row chaos), are the classic failures.
- Conceptually the engine finds, for each admission, the patient row with the same id, and glues the columns side by side.

Joined results feed straight into everything from earlier lessons — filter them, band them, group them:

**EMERGENCY ADMISSIONS BY SEX**

```
SELECT
    p.sex,
    COUNT(*) AS emergency_admissions
FROM admissions a
JOIN patients p ON p.patient_id = a.patient_id    -- INNER is the default
WHERE a.admission_method = 'Emergency'
GROUP BY p.sex;
```

**QUICK CHECK**

In `FROM admissions a JOIN patients p ON p.patient_id = a.patient_id`, what are `a` and `p`?

- ☐ Column names
- ☐ Temporary copies of the tables
- ✓ **Aliases — short local names for each table**
- ☐ Keywords required by SQL

**Why:** Aliases keep multi-table queries readable and make column references unambiguous. Any short names work; initials are conventional.

**QUICK CHECK**

An **INNER JOIN** between `admissions` and `patients` drops which rows?

- ☐ Admissions with NULL discharge dates
- ✓ **Any admission whose `patient_id` has no match in patients (and any patient with no admissions)**
- ☐ Duplicate admissions
- ☐ Nothing; INNER JOIN keeps everything

**Why:** INNER keeps matched pairs only. Unmatched rows on either side silently vanish — sometimes fine, sometimes a data-quality alarm (Lesson 26 handles keeping them).

**EXERCISE**

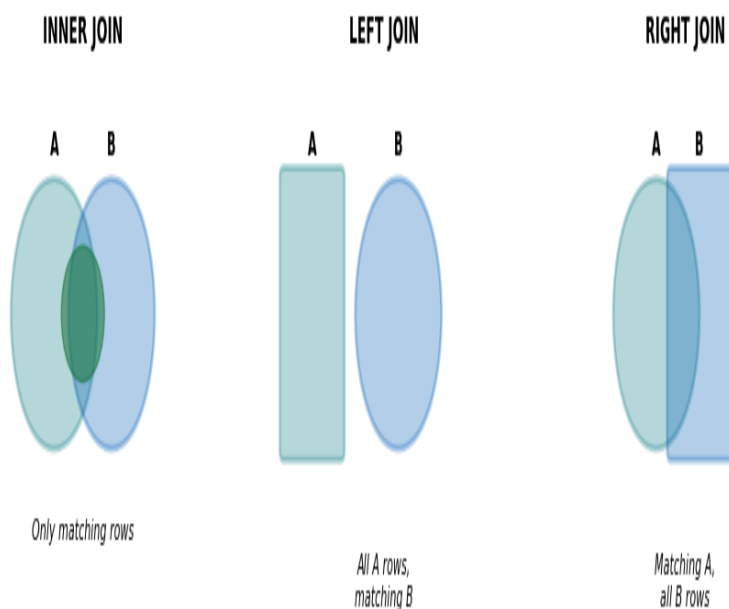
Join `lab_results` to `patients` and count HbA1c tests by patient sex. Return sex and the test count.

**SOLUTION · SQL**

```
SELECT
  p.sex,
  COUNT(*) AS hba1c_tests
FROM lab_results l
JOIN patients p ON p.patient_id = l.patient_id
WHERE l.test = 'HbA1c'
GROUP BY p.sex;
```

**KEY POINTS**

- INNER JOIN ... ON key = key: keeps matching pairs, drops non-matches on both sides.
- Alias every table; qualify every column as alias.column.
- Join first, then filter/aggregate/group exactly as before — the clauses all still apply.

**SQL JOIN types**



## MODULE 2 • SQL CORE SKILLS • LESSON 26 OF 105

## 26. JOIN part 2: LEFT JOIN and finding what's missing

INNER JOIN answers "who matches?" — but public health constantly asks the opposite: **who's missing?** Which children have *\*no\** MMR record? Which practices had *\*no\** admissions? A **LEFT JOIN** keeps **every row of the left table**, matched or not; where the right side has no match, its columns come back as NULL. That NULL is the fingerprint of absence, and filtering on it is the standard "find the missing" pattern:

**PATIENTS WITH NO VACCINATION RECORD AT ALL**

```
SELECT
  p.patient_id,
  p.birth_date,
  v.vaccination_id      -- NULL when no match exists
FROM patients p
LEFT JOIN vaccinations v
  ON v.patient_id = p.patient_id
WHERE v.vaccination_id IS NULL
LIMIT 10;
```

Read it as: keep all patients; attach vaccinations where they exist; then keep only the rows where nothing attached. (This is called an **anti-join**, and R has a function named exactly that — Lesson 51.)

A second essential use: **preserving zeros in grouped counts**. Count admissions per practice with an INNER JOIN and practices with none disappear; LEFT JOIN keeps them, and counting a right-side column (which is NULL for non-matches) turns absence into 0:

**EVERY PRACTICE, INCLUDING THE QUIET ONES**

```
SELECT
  g.practice_code,
  g.practice_name,
  COUNT(a.admission_id) AS admissions      -- COUNT(col) skips NULLs -> 0
FROM gp_practices g
LEFT JOIN patients p ON p.practice_code = g.practice_code
LEFT JOIN admissions a ON a.patient_id = p.patient_id
GROUP BY g.practice_code, g.practice_name
ORDER BY admissions ASC;
```

Note `COUNT(a.admission_id)`, not `COUNT(*)`: the latter would count the practice's placeholder row itself and report 1 instead of 0. A **RIGHT JOIN** exists (keep all right-side rows) but analysts almost always just reorder the tables and use **LEFT**; **FULL OUTER JOIN** (keep both sides) is rarer and absent from SQLite.

**QUICK CHECK**

**After patients LEFT JOIN vaccinations, a patient with no vaccinations appears...**

- ☐ Not at all
- ☐ Once per vaccine type with zeros
- ✓ **Once, with the vaccination columns NULL**
- ☐ As an error

**Why:** *LEFT* keeps every left row; unmatched right columns are NULL — which is exactly what *WHERE ... IS NULL* then detects.

**QUICK CHECK**

**In the zero-preserving count, why COUNT(a.admission\_id) instead of COUNT(\*)?**

- ☐ They are identical
- ☐ COUNT(\*) is slower
- ✓ **COUNT(\*) counts the unmatched placeholder row as 1; COUNT(a.admission\_id) skips its NULL and yields a true 0**
- ☐ admission\_id is the primary key

**Why:** The NULL-skipping behaviour of COUNT(column) — from Lesson 22 — is what turns "no match" into the honest zero.

**EXERCISE**

List patients who have a death record's underlying\_cause of 'C34' available — no, better: list all patients aged in the data who have NO row in deaths (i.e. presumed alive), returning patient\_id and birth\_date, 10 rows.

**SOLUTION · SQL**

```
SELECT
  p.patient_id,
  p.birth_date
FROM patients p
LEFT JOIN deaths d ON d.patient_id = p.patient_id
WHERE d.patient_id IS NULL
LIMIT 10;
```

**KEY POINTS**

- LEFT JOIN keeps all left rows; unmatched right columns are NULL.
- WHERE right\_key IS NULL = the anti-join: who's missing a record.
- LEFT JOIN + COUNT(right\_column) preserves true zeros in grouped counts.

**MODULE 2 · SQL CORE SKILLS · LESSON 27 OF 105****27. JOIN part 3: chains, fan-out and the duplication trap**

Real questions chain several joins. "Emergency admissions per IMD decile" walks admissions → patients → lsoa\_imd:

**ADMISSIONS BY DEPRIVATION DECILE**

```
SELECT
  d.imd_decile,
  COUNT(*) AS emergency_admissions
FROM admissions a
JOIN patients p ON p.patient_id = a.patient_id
JOIN lsoa_imd d ON d.lsoa_code = p.lsoa_code
WHERE a.admission_method = 'Emergency'
GROUP BY d.imd_decile
ORDER BY d.imd_decile;
```

Deciles run 1 (most deprived) to 10 (least). If a health-service dataset is behaving as the literature predicts, emergency activity tilts toward low deciles — you now have the tool to check.

**The trap: join fan-out.** Joins match *every* pair of matching rows. Join patients (1 row each) to admissions (many rows each) and each patient row is **duplicated once per admission** — correct

for counting admissions, catastrophic if you then count "patients" with `COUNT(*)` or `SUM` a per-patient value like list size. Symptoms: totals mysteriously bigger than the source table; the fix: know each relationship's shape —

| Relationship | Example                              | Effect of joining                                   |
|--------------|--------------------------------------|-----------------------------------------------------|
| One-to-one   | patients ↔ deaths                    | Row count unchanged                                 |
| One-to-many  | patients → admissions                | Left rows repeat per match                          |
| Many-to-many | admissions ↔ lab_results via patient | Multiplicative blow-up — almost never join directly |

Defensive habits: check `COUNT(*)` before and after each join; count people with `COUNT(DISTINCT patient_id)` after any one-to-many join; and never join two "many" tables to each other on `patient_id` — aggregate one side first (subqueries and CTEs, next lessons, exist for exactly this).

#### QUICK CHECK

**patients has 600 rows; admissions has 900. `SELECT COUNT(*) FROM patients p JOIN admissions a ON a.patient_id = p.patient_id` returns at most...**

- ☐ 600
- ☒ **900**
- ☐ 1500
- ☐ 540000

**Why:** One output row per matching admission: patients fan out to their admissions. (Exactly 900 if every admission's patient exists — worth verifying!)

#### QUICK CHECK

**After joining patients to admissions, the safe way to report "number of patients admitted" is...**

- ☐ `COUNT(*)`
- ☐ `COUNT(admission_id)`
- ☒ **`COUNT(DISTINCT p.patient_id)`**
- ☐ `SUM(1)`

**Why:** Fan-out duplicates patients across their admissions; `DISTINCT` collapses them back to people.

#### EXERCISE

Produce emergency admissions per local authority: chain admissions → patients → lsoa\_imd, group by `la_name`, and report both admissions and distinct patients.

#### SOLUTION · SQL

```
SELECT
  d.la_name,
  COUNT(*) AS emergency_admissions,
  COUNT(DISTINCT p.patient_id) AS patients
FROM admissions a
JOIN patients p ON p.patient_id = a.patient_id
JOIN lsoa_imd d ON d.lsoa_code = p.lsoa_code
WHERE a.admission_method = 'Emergency'
GROUP BY d.la_name
ORDER BY emergency_admissions DESC;
```

**KEY POINTS**

- Chain joins to walk relationships: event → person → place is the health-data spine.
- One-to-many joins duplicate the "one" side — count people with COUNT(DISTINCT id).
- Never join two many-sided tables directly on patient\_id; aggregate one first.

**MODULE 2 • SQL CORE SKILLS • LESSON 28 OF 105**

## 28. UNION and friends: stacking result sets

Joins combine tables **side by side**; set operators combine them **top to bottom**. You'll reach for them when similar data arrives split — this year's file and last year's, or two systems recording the same kind of event.

- **UNION ALL** stacks two SELECTs, keeping everything (fast, honest about duplicates).
- **UNION** stacks and **removes duplicate rows** (slower; a deduplication step in disguise).
- **INTERSECT** keeps rows appearing in both results; **EXCEPT** keeps rows in the first but not the second. (SQL Server spells EXCEPT the same; Oracle calls it MINUS.)

Rules: each SELECT must have the **same number of columns with compatible types**; column names come from the first SELECT; a single ORDER BY applies at the very end.

**ONE EVENT LIST FROM TWO TABLES**

```
SELECT patient_id, date_given AS event_date, 'Vaccination: ' || vaccine AS event
FROM vaccinations
UNION ALL
SELECT patient_id, date_taken AS event_date, 'Lab test: ' || test AS event
FROM lab_results
ORDER BY patient_id, event_date
LIMIT 15;
```

That "long event list" shape — one row per event, a type label column — is a genuinely useful analytical structure (R calls it *\*tidy data\**; Lesson 52).

INTERSECT and EXCEPT shine at cohort logic and QA:

**WHO HAD COVID-19 VACCINATION BUT NO FLU JAB?**

```
SELECT patient_id FROM vaccinations WHERE vaccine = 'COVID-19'
EXCEPT
SELECT patient_id FROM vaccinations WHERE vaccine = 'Flu';
```

**QUICK CHECK**

**The difference between UNION and UNION ALL is...**

- ☐ UNION ALL requires identical column names
- ✓ **UNION removes duplicate rows; UNION ALL keeps every row**
- ☐ UNION works across databases; UNION ALL doesn't
- ☐ No difference

**Why:** *ALL* means "all rows, duplicates included". Default *UNION* deduplicates — occasionally what you want, often a surprise and always slower.

**QUICK CHECK**

**You stack admissions from two hospital systems with UNION (not ALL) and the total is less than the two counts added. Most likely explanation?**

- ☐ The database lost rows
- ✓ **Identical rows existed in both sources and were deduplicated**
- ☐ UNION always halves the data
- ☐ A join fanned out

**Why:** UNION's dedupe removed rows appearing (identically) in both inputs — which itself is a finding: are those true duplicates or coincidences?

**EXERCISE**

Using INTERSECT, find patient\_ids that appear in both the admissions table and the deaths table (people who were admitted at some point and later have a death record).

**SOLUTION · SQL**

```
SELECT patient_id FROM admissions
INTERSECT
SELECT patient_id FROM deaths;
```

**KEY POINTS**

- UNION ALL stacks; UNION stacks + dedupes; INTERSECT = in both; EXCEPT = in first only.
- Column counts/types must align; the first SELECT names the output.
- Stacked "event list" tables are an analytical asset, not a compromise.

**MODULE 2 · SQL CORE SKILLS · LESSON 29 OF 105**

## 29. Subqueries: queries inside queries

A subquery is a SELECT nested inside another statement, in brackets. Three flavours cover nearly all use:

**1. Scalar subquery** — returns one value, used like a number:

**COMPARE EACH PRACTICE TO THE AVERAGE**

```
SELECT
  practice_name,
  list_size,
  list_size - (SELECT AVG(list_size) FROM gp_practices) AS diff_from_mean
FROM gp_practices
ORDER BY diff_from_mean DESC;
```

**2. IN-list subquery** — returns one column, used as a filter set. "Admissions of patients living in the most deprived decile":

**FILTER BY MEMBERSHIP OF A COMPUTED SET**

```
SELECT a.admission_id, a.admission_date, a.specialty
FROM admissions a
WHERE a.patient_id IN (
  SELECT p.patient_id
  FROM patients p
  JOIN lsoa_imd d ON d.lsoa_code = p.lsoa_code
  WHERE d.imd_decile = 1)
```

**FILTER BY MEMBERSHIP OF A COMPUTED SET**

```
)
LIMIT 10;
```

`NOT IN` inverts it — but carries a nasty edge: if the subquery returns any `NULL`, `NOT IN` matches \*nothing at all\* (NULL logic from Lesson 18 striking again). The `LEFT JOIN ... IS NULL` anti-join, or `NOT EXISTS`, avoids this and is what experienced analysts write.

**3. Subquery in FROM (a derived table)** — treat a query's result as a table and query \*that\*.  
"Average number of admissions per admitted patient":

**AGGREGATE OF AN AGGREGATE**

```
SELECT ROUND(AVG(n), 2) AS mean_admissions_per_patient
FROM (
  SELECT patient_id, COUNT(*) AS n
  FROM admissions
  GROUP BY patient_id
) AS per_patient;
```

Two levels of aggregation need two steps — inner counts per patient, outer averages the counts. Nested brackets get unreadable fast, though; the next lesson's CTEs express the same idea legibly, and are what you should default to.

**QUICK CHECK****A scalar subquery must return...**

- ☐ One column, any rows
- ✓ **Exactly one value (one row, one column)**
- ☐ At least two columns
- ☐ Only integers

**Why:** It stands where a single value would: one row, one column. More rows = an error (or, in permissive engines, silent surprise).

**QUICK CHECK****Why do professionals avoid NOT IN (SELECT patient\_id FROM ...) when the subquery column can be NULL?**

- ☐ It is slower than everything else
- ✓ **A single NULL in the list makes NOT IN return no rows at all**
- ☐ NOT IN is not valid SQL
- ☐ It deletes rows

**Why:** `x NOT IN (... , NULL)` evaluates to unknown for every `x`, so the filter passes nothing. `NOT EXISTS` or an anti-join are NULL-safe.

**EXERCISE**

Using an `IN` subquery, list the vaccination records (`patient_id`, `vaccine`, `dose_number`, `date_given`) of patients registered at the single largest practice by `list_size`. (Hint: the inner query finds that `practice_code`; you may nest two levels or use `LIMIT 1`.)

**SOLUTION · SQL**

```
SELECT v.patient_id, v.vaccine, v.dose_number, v.date_given
FROM vaccinations v
WHERE v.patient_id IN (
  SELECT p.patient_id
```

**SOLUTION · SQL**

```
FROM patients p
WHERE p.practice_code = (
  SELECT practice_code
  FROM gp_practices
  ORDER BY list_size DESC
  LIMIT 1
)
)
ORDER BY v.patient_id, v.date_given;
```

**KEY POINTS**

- Scalar (one value), IN-list (filter set), derived table (query a query) — the three subquery shapes.
- NOT IN + NULLs = empty results; prefer NOT EXISTS or anti-joins.
- Aggregating an aggregate requires nesting — which CTEs (next) make readable.

**MODULE 3**

# Module 3 — SQL for health data

**In this module**

- 30** CTEs: readable, professional query pipelines
- 31** Window functions 1: ROW\_NUMBER and "latest record per patient"
- 32** Window functions 2: running totals, moving averages and date spines
- 33** Window functions 3: LAG, LEAD and gaps between events
- 34** Deduplication: deciding what "one record" means
- 35** Cohorts: defining exactly who is in your analysis
- 36** Length of stay, spells and pathway thinking
- 37** Rates: joining numerators to populations
- 38** The classic metric: emergency readmissions within 30 days
- 39** Data quality checks: trust nothing, verify everything
- 40** The T-SQL phrasebook: SQL Server for NHS analysts
- 41** Views, temp tables and saving results

## MODULE 3 • SQL FOR HEALTH DATA • LESSON 30 OF 105

## 30. CTEs: readable, professional query pipelines

A **Common Table Expression** (CTE) names a subquery up front with `WITH`, so the main query reads like a pipeline of steps. Everything a derived table can do, a CTE does more legibly — and legible SQL is what NHS teams QA, share and maintain. The Lesson 29 example, rewritten:

**SAME LOGIC, READABLE SHAPE**

```
WITH per_patient AS (
  SELECT patient_id, COUNT(*) AS n
  FROM admissions
  GROUP BY patient_id
)
SELECT ROUND(AVG(n), 2) AS mean_admissions_per_patient
FROM per_patient;
```

Chain several with commas — each step can use the ones before it. A realistic mini-pipeline: emergency admissions, add patient deprivation, then summarise:

**A THREE-STEP PIPELINE**

```
WITH emerg AS (
  SELECT patient_id, admission_date
  FROM admissions
  WHERE admission_method = 'Emergency'
),
with_imd AS (
  SELECT e.patient_id, e.admission_date, d.imd_decile
  FROM emerg e
  JOIN patients p ON p.patient_id = e.patient_id
  JOIN lsoa_imd d ON d.lsoa_code = p.lsoa_code
)
SELECT
  CASE WHEN imd_decile <= 2 THEN 'Most deprived 20%'
        ELSE 'Rest' END          AS grp,
  COUNT(*)                      AS admissions,
  COUNT(DISTINCT patient_id)    AS patients
FROM with_imd
GROUP BY grp;
```

Style points that mark out professional code: one CTE per logical step; name CTEs after what they \*contain\* (`emerg`, `with_imd`, `latest_hbabc`), not `t1`, `t2`; keep each step small enough to test alone — you can highlight just the inner `SELECT` and run it. CTEs exist in every serious engine (SQL Server, PostgreSQL, SQLite alike), so this habit transfers wholesale into NHS work.

**QUICK CHECK**

**What is the main practical advantage of a CTE over a nested subquery?**

- ☐ CTEs run faster in every engine
- ✓ **The query reads as named, testable steps instead of inside-out nesting**
- ☐ CTEs can modify data; subqueries cannot
- ☐ Subqueries are deprecated

**Why:** Performance is usually similar; the win is readability and QA-ability. In audit-heavy NHS work, that is worth a lot.



**QUICK CHECK**

In **WITH a AS (...), b AS (...)** **SELECT ...** — which is true?

- ☐ b cannot reference a
- ✓ **b may reference a, and the final SELECT may use both**
- ☐ Only one CTE is allowed per query
- ☐ CTEs persist in the database after the query ends

**Why:** CTEs chain top-to-bottom within one statement and vanish when it finishes (persisting results is Lesson 41).

**EXERCISE**

Using two CTEs, find the mean HbA1c per patient (first CTE: each patient's average value across their HbA1c tests), then return how many patients have a personal mean of 48+ versus below 48 (second step: CASE + GROUP BY).

**SOLUTION · SQL**

```
WITH patient_means AS (
  SELECT patient_id, AVG(value) AS mean_hba1c
  FROM lab_results
  WHERE test = 'HbA1c'
  GROUP BY patient_id
)
SELECT
  CASE WHEN mean_hba1c >= 48 THEN '48 and over' ELSE 'Under 48' END AS band,
  COUNT(*) AS patients
FROM patient_means
GROUP BY band;
```

**KEY POINTS**

- WITH name AS (SELECT ...) turns nesting into named steps — the professional default.
- Chain CTEs with commas; later steps can use earlier ones.
- Name steps for their content; test steps in isolation.

**MODULE 3 · SQL FOR HEALTH DATA · LESSON 31 OF 105**

## 31. Window functions 1: ROW\_NUMBER and "latest record per patient"

Aggregates collapse rows; **window functions** compute across related rows \*while keeping every row\*. The syntax is `function() OVER (PARTITION BY ... ORDER BY ...)`: PARTITION BY says who is compared with whom; ORDER BY says in what order.

The single most useful pattern in health data: **the latest record per patient**. Patients have many HbA1c results; clinical questions usually want the most recent one each:

**LATEST HBA1C PER PATIENT**

```
WITH ranked AS (
  SELECT
    patient_id,
    value,
    date_taken,
    ROW_NUMBER() OVER (
      PARTITION BY patient_id
```

**LATEST HbA1c PER PATIENT**

```

        ORDER BY date_taken DESC
      ) AS rn
    FROM lab_results
    WHERE test = 'HbA1c'
  )
  SELECT patient_id, value, date_taken
  FROM ranked
  WHERE rn = 1
  ORDER BY value DESC
  LIMIT 10;

```

Read it: within each patient (PARTITION), sort their tests newest-first (ORDER BY ... DESC), number them 1, 2, 3... (ROW\_NUMBER), keep number 1. Because window results can't sit directly in WHERE, the CTE-then-filter shape is standard.

Cousins to know: `RANK()` gives ties the same number and skips (1,1,3); `DENSE_RANK()` gives ties the same number without skipping (1,1,2). For "latest per patient", `ROW_NUMBER` with a decisive ORDER BY (add a tiebreak like `result_id DESC`) is what you want — `RANK` could return two "latest" rows on a tied date.

**QUICK CHECK****PARTITION BY patient\_id in a window function means...**

- ☐ Group rows and collapse each patient to one row
- ✓ **Restart the calculation for each patient, keeping all rows**
- ☐ Split the table into physical files
- ☐ Sort by patient\_id

**Why:** Windows never collapse rows: each patient's rows form a separate frame for the calculation, and every input row survives to the output.

**QUICK CHECK****Two tests share a patient's latest date. Which choice guarantees exactly one "latest" row?**

- ☐ `RANK()` ... WHERE `rnk = 1`
- ☐ `DENSE_RANK()` ... WHERE `rnk = 1`
- ✓ **`ROW_NUMBER()` with a tiebreaker in ORDER BY ... WHERE `rn = 1`**
- ☐ `MAX(date_taken)`

**Why:** `RANK/DENSE_RANK` both label ties 1, returning two rows. `ROW_NUMBER` always numbers uniquely — make the order deterministic with a tiebreak column.

**EXERCISE**

Find each patient's FIRST ever admission (earliest admission\_date; tiebreak on admission\_id), returning patient\_id, admission\_date and specialty, 10 rows.

**SOLUTION · SQL**

```

WITH ranked AS (
  SELECT
    patient_id, admission_date, specialty,
    ROW_NUMBER() OVER (
      PARTITION BY patient_id
      ORDER BY admission_date ASC, admission_id ASC
    ) AS rn

```

**SOLUTION · SQL**

```
FROM admissions
)
SELECT patient_id, admission_date, specialty
FROM ranked
WHERE rn = 1
LIMIT 10;
```

**KEY POINTS**

- function() OVER (PARTITION BY who ORDER BY sequence) — compute per group, keep all rows.
- ROW\_NUMBER + WHERE rn = 1 = latest/first record per patient — memorise it.
- Add a tiebreaker to ORDER BY; use RANK/DENSE\_RANK only when ties should share.

**MODULE 3 · SQL FOR HEALTH DATA · LESSON 32 OF 105**

## 32. Window functions 2: running totals, moving averages and date spines

Surveillance charts live on two calculations: **cumulative totals** and **moving averages**. Both are windows with an ORDER BY and (for the average) a *\*frame\** saying how many rows to include.

**MONTHLY COUNTS, CUMULATIVE AND SMOOTHED**

```
WITH monthly AS (
  SELECT strftime('%Y-%m', admission_date) AS yr_month,
         COUNT(*) AS n
  FROM admissions
  WHERE admission_method = 'Emergency'
  GROUP BY yr_month
)
SELECT
  yr_month,
  n,
  SUM(n) OVER (ORDER BY yr_month) AS cumulative,
  ROUND(AVG(n) OVER (ORDER BY yr_month
                     ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1) AS moving_avg_3m
FROM monthly
ORDER BY yr_month;
```

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW = this row plus the two before: a 3-month moving average. During COVID, the 7-day moving average of daily cases used exactly this shape with days instead of months.

**But there's a flaw:** if a month had zero emergencies, it's missing from `monthly`, and the "3-month" average silently spans 4 calendar months. The professional fix is a **date spine** — generate every period, then LEFT JOIN the counts on:

**A MONTH SPINE WITH ZEROS PRESERVED**

```
WITH RECURSIVE spine(month_start) AS (
  SELECT '2024-04-01'
  UNION ALL
  SELECT date(month_start, '+1 month') FROM spine
  WHERE month_start < '2026-03-01'
),
monthly AS (
  SELECT strftime('%Y-%m', admission_date) AS yr_month, COUNT(*) AS n
  FROM admissions
```

**A MONTH SPINE WITH ZEROS PRESERVED**

```

GROUP BY yr_month
)
SELECT strftime('%Y-%m', s.month_start) AS yr_month,
       COALESCE(m.n, 0) AS admissions
FROM spine s
LEFT JOIN monthly m ON m.yr_month = strftime('%Y-%m', s.month_start)
ORDER BY yr_month;

```

The recursive CTE builds rows by repeatedly adding a month until the stop condition. (SQL Server: same idea, or join to a pre-built calendar/[Tally](#) table, which most NHS warehouses keep.) Spine first, then windows, and your trends are honest.

**QUICK CHECK**

**ROWS BETWEEN 6 PRECEDING AND CURRENT ROW** computes, on daily data ordered by date...

- ☐ A 6-day average excluding today
- ✓ **A 7-day window: today plus the previous six rows**
- ☐ The whole series to date
- ☐ Six days into the future

**Why:** Current row + 6 preceding = 7 rows — the classic 7-day moving average, provided no days are missing (hence the spine).

**QUICK CHECK**

**Why does a moving average over grouped month counts need a date spine?**

- ☐ Window functions require one
- ✓ **Missing zero-count months make the frame span the wrong calendar period**
- ☐ SQLite cannot average otherwise
- ☐ To make the query faster

**Why:** ROWS frames count rows, not calendar time. Absent months shift the window silently; a spine restores one-row-per-month with zeros.

**EXERCISE**

Build a monthly count of 'Flu' vaccinations with a cumulative total column, using a CTE for the monthly counts (spine optional here). Order by month.

**SOLUTION · SQL**

```

WITH monthly AS (
  SELECT strftime('%Y-%m', date_given) AS yr_month, COUNT(*) AS n
  FROM vaccinations
  WHERE vaccine = 'Flu'
  GROUP BY yr_month
)
SELECT yr_month, n,
       SUM(n) OVER (ORDER BY yr_month) AS cumulative
FROM monthly
ORDER BY yr_month;

```

**KEY POINTS**

- SUM(...) OVER (ORDER BY t) = running total; AVG with a ROWS frame = moving average.
- Frames count rows, so zero periods must exist: build a date spine (recursive CTE or calendar table).
- Spine + LEFT JOIN + COALESCE(n,0) is the honest-trend pattern.

**MODULE 3 • SQL FOR HEALTH DATA • LESSON 33 OF 105**

## 33. Window functions 3: LAG, LEAD and gaps between events

LAG(col) fetches a value from the **previous row** in the window; LEAD(col) from the next. Partition by patient, order by date, and you can measure the gap between one event and the last — the mechanic behind readmission intervals, repeat-test intervals and dose spacing.

**DAYS SINCE THE PATIENT'S PREVIOUS ADMISSION**

```
SELECT
  patient_id,
  admission_id,
  admission_date,
  LAG(discharge_date) OVER (
    PARTITION BY patient_id
    ORDER BY admission_date, admission_id
  ) AS prev_discharge,
  CAST(julianday(admission_date) -
    julianday(LAG(discharge_date) OVER (
      PARTITION BY patient_id
      ORDER BY admission_date, admission_id)) AS INTEGER) AS days_since_discharge
FROM admissions
ORDER BY patient_id, admission_date
LIMIT 15;
```

Notes: the first admission per patient has no previous row, so LAG returns **NULL** — correct and meaningful (nothing to compare). LAG(col, 2) looks two rows back; LAG(col, 1, 0) supplies a default instead of NULL. Repeating the window is legal but noisy — in engines that allow it you can name a window (WINDOW w AS (...)), or compute LAG in one CTE and difference it in the next, which is the portable, readable route:

**CLEANER: LAG IN ONE STEP, ARITHMETIC IN THE NEXT**

```
WITH ordered AS (
  SELECT patient_id, admission_id, admission_date, discharge_date,
    LAG(discharge_date) OVER (
      PARTITION BY patient_id
      ORDER BY admission_date, admission_id) AS prev_discharge
  FROM admissions
)
SELECT *,
  CAST(julianday(admission_date) - julianday(prev_discharge) AS INTEGER)
  AS days_since_discharge
FROM ordered;
```

Lesson 38 turns exactly this into the NHS's emergency-readmissions-within-30-days indicator.

**QUICK CHECK**

**For each patient's first admission, LAG(discharge\_date) OVER (PARTITION BY patient\_id ORDER BY admission\_date) returns...**

- ☐ The last admission's discharge date
- ☐ 0
- ✓ **NULL — there is no previous row in the partition**
- ☐ An error

**Why:** No prior row means no value: NULL, which downstream logic should treat as "not a readmission candidate".

**QUICK CHECK**

**You want each admission paired with the NEXT admission's date for the same patient. Which function?**

- ☐ LAG(admission\_date)
- ✓ **LEAD(admission\_date)**
- ☐ ROW\_NUMBER()
- ☐ MAX(admission\_date)

**Why:** LEAD looks forward in the ordered partition; LAG looks back.

**EXERCISE**

For lab\_results where test = 'HbA1c', show each patient's tests with the days since their previous HbA1c (NULL for the first). Return patient\_id, date\_taken, value, days\_since\_prev; order by patient then date; 15 rows.

**SOLUTION · SQL**

```
WITH ordered AS (
  SELECT patient_id, date_taken, value,
         LAG(date_taken) OVER (
           PARTITION BY patient_id
           ORDER BY date_taken) AS prev_date
  FROM lab_results
  WHERE test = 'HbA1c'
)
SELECT patient_id, date_taken, value,
       CAST(julianday(date_taken) - julianday(prev_date) AS INTEGER) AS days_since_prev
FROM ordered
ORDER BY patient_id, date_taken
LIMIT 15;
```

**KEY POINTS**

- LAG/LEAD read neighbouring rows within a partition's order — gaps and intervals fall out by subtraction.
- First-row LAG is NULL by design; LAG(col, n, default) customises.
- Compute the window in a CTE, do arithmetic in the next step: portable and readable.

## MODULE 3 • SQL FOR HEALTH DATA • LESSON 34 OF 105

## 34. Deduplication: deciding what "one record" means

Real extracts contain duplicates: interface hiccups double-send rows, patients register twice, the same test posts from two systems. Before counting anything, an analyst asks: **what should be unique here?** One row per admission? Per patient per day? Per patient overall? That decision is analytical, not technical — then SQL enforces it.

### Step 1 — measure the problem:

#### ARE THERE EXACT DUPLICATE VACCINATION EVENTS?

```
SELECT patient_id, vaccine, dose_number, date_given, COUNT(*) AS copies
FROM vaccinations
GROUP BY patient_id, vaccine, dose_number, date_given
HAVING COUNT(*) > 1;
```

**Step 2 — deduplicate by rule.** For *\*exact\** duplicates, `SELECT DISTINCT` is enough. For *\*near\**-duplicates (same event, different rows — e.g. two rows for one admission with different discharge dates), you need a **keep rule**: usually "the most recent / most complete record wins", which is Lesson 31's `ROW_NUMBER` pattern with the rule encoded in `ORDER BY`:

#### KEEP ONE ROW PER PATIENT-VACCINE-DOSE: LATEST DATE WINS

```
WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY patient_id, vaccine, dose_number
      ORDER BY date_given DESC, vaccination_id DESC
    ) AS rn
  FROM vaccinations
)
SELECT * FROM ranked WHERE rn = 1;
```

**Step 3 — document it.** "Deduplicated to one row per patient/vaccine/dose, keeping the latest date\_given" is a sentence that belongs in your methods note; different keep rules give different uptake figures, and a QA reviewer must be able to see which you chose. Golden rule: dedupe in queries and views, **never** by deleting from source tables — sources stay raw; rules live in code.

#### QUICK CHECK

**"One row per patient per financial year" as a uniqueness rule means PARTITION BY...**

- ☐ patient\_id
- ✓ **patient\_id and the derived financial year**
- ☐ financial year only
- ☐ admission\_id

**Why:** The partition is exactly the combination that should be unique — the `ROW_NUMBER` then ranks candidates within it.

**QUICK CHECK**

**Why prefer deduplicating in a query/view rather than deleting duplicate rows from the source table?**

- ☐ DELETE is not real SQL
- ✓ **The raw source stays intact and the rule is visible, reversible and re-runnable**
- ☐ Views are faster than tables
- ☐ Duplicates are always meaningful

**Why:** Raw data is evidence. Encoding the rule in code keeps it auditable and lets you change your mind without loss.

**EXERCISE**

Some patients have multiple Systolic BP results on the same date. Produce one row per patient per date\_taken, keeping the HIGHEST value (a "worst reading" rule). Return patient\_id, date\_taken, value; 12 rows.

**SOLUTION · SQL**

```
WITH ranked AS (
  SELECT patient_id, date_taken, value,
         ROW_NUMBER() OVER (
           PARTITION BY patient_id, date_taken
           ORDER BY value DESC, result_id DESC
         ) AS rn
  FROM lab_results
  WHERE test = 'Systolic BP'
)
SELECT patient_id, date_taken, value
FROM ranked
WHERE rn = 1
ORDER BY patient_id, date_taken
LIMIT 12;
```

**KEY POINTS**

- First define what should be unique; GROUP BY ... HAVING COUNT(\*)>1 measures violations.
- DISTINCT for exact copies; ROW\_NUMBER with a keep rule for near-duplicates.
- Dedupe in code (views/CTEs), never by deleting source rows — and write the rule down.

**MODULE 3 · SQL FOR HEALTH DATA · LESSON 35 OF 105**

## 35. Cohorts: defining exactly who is in your analysis

A **cohort** is the precisely defined group your analysis is about: "adults 65+ registered in Barchester on 1 April 2025", "children turning 5 in 2025/26", "patients with an emergency respiratory admission during winter". Every serious piece of health analysis starts by building the cohort as its own step — usually a CTE ending in **one row per person** — and everything downstream joins to it.

Two distinctions to get right:

- **Prevalent vs incident.** Prevalent = everyone \*with\* the condition/state in a window (all diabetics being managed). Incident = \*new\* occurrences in the window (first-ever diagnoses). Mixing them muddles trends: prevalence can rise while incidence falls.



- **The index date.** Each cohort member gets an anchor date (first qualifying event, or a fixed census date) from which "within 30 days", "age at index" and follow-up are measured.

#### COHORT: PATIENTS WITH ANY HbA1c $\geq$ 48, WITH INDEX DATE

```
WITH qualifying AS (
  SELECT patient_id, date_taken, value
  FROM lab_results
  WHERE test = 'HbA1c' AND value >= 48
),
cohort AS (
  SELECT patient_id,
         MIN(date_taken) AS index_date,      -- first qualifying result
         COUNT(*)        AS qualifying_results
  FROM qualifying
  GROUP BY patient_id
)
SELECT c.patient_id, c.index_date, c.qualifying_results,
       p.sex, p.birth_date, p.practice_code
FROM cohort c
JOIN patients p ON p.patient_id = c.patient_id
ORDER BY c.index_date
LIMIT 12;
```

Checklist for any cohort definition (write the answers down before coding): inclusion criteria; exclusion criteria (e.g. died before index? not registered locally?); the time window; the index date rule; and whether a person can appear once or many times. In real work these sentences go in the methods section — analysts who skip them produce numbers nobody can reproduce.

#### QUICK CHECK

**"New type 2 diabetes diagnoses in 2025/26" is a question about...**

- ☐ Prevalence
- ✓ **Incidence**
- ☐ Mortality
- ☐ Screening coverage

**Why:** New occurrences in a period = incidence. "How many people currently have diabetes" would be prevalence.

#### QUICK CHECK

**In the example, MIN(date\_taken) per patient implements which cohort concept?**

- ☐ Exclusion criteria
- ☐ Prevalence
- ✓ **The index date — the first qualifying event anchoring follow-up**
- ☐ Deduplication of exact copies

**Why:** Choosing the first qualifying event as the anchor is the standard incident-cohort index rule.

#### EXERCISE

Build a cohort of frequent emergency attenders: patients with 3 or more Emergency admissions overall. One row per patient with patient\_id, first emergency date (index\_date), emergency count; joined to patients for sex. Order by count descending.

#### SOLUTION · SQL

```
WITH emerg AS (
```

**SOLUTION · SQL**

```

SELECT patient_id, admission_date
FROM admissions
WHERE admission_method = 'Emergency'
),
cohort AS (
  SELECT patient_id,
         MIN(admission_date) AS index_date,
         COUNT(*)           AS emergency_count
  FROM emerg
  GROUP BY patient_id
  HAVING COUNT(*) >= 3
)
SELECT c.*, p.sex
FROM cohort c
JOIN patients p ON p.patient_id = c.patient_id
ORDER BY c.emergency_count DESC;

```

**KEY POINTS**

- A cohort CTE (one row per person, with an index date) anchors every serious analysis.
- Distinguish incident (new) from prevalent (existing); state inclusions, exclusions, window.
- Downstream steps join to the cohort — never re-derive membership ad hoc.

**MODULE 3 · SQL FOR HEALTH DATA · LESSON 36 OF 105**

## 36. Length of stay, spells and pathway thinking

Hospital data has structure inside it. In SUS/HES, a **provider spell** is one continuous stay in one hospital; inside it, **consultant episodes** track handovers between teams; transfers between hospitals chain spells into a **continuous inpatient (CIP) spell**. Your playground's `admissions` table is one row per spell — the common analyst-facing shape — but knowing the layers matters when numbers from different systems disagree ("their 'admissions' are episodes, ours are spells").

**Length of stay (LOS)** = discharge – admission in days; same-day cases are 0. LOS is the workhorse of capacity analysis, and it is famously **skewed**: most stays are short, a few are very long, so mean > median and the mean swings with a handful of outliers. Report the median (and percentiles) alongside any mean.

**LOS BY SPECIALTY: MEAN VS MEDIAN-ISH VIEW**

```

WITH los AS (
  SELECT specialty,
         CAST(julianday(discharge_date) - julianday(admission_date) AS INTEGER) AS days
  FROM admissions
  WHERE discharge_date IS NOT NULL
)
SELECT specialty,
       COUNT(*)           AS spells,
       ROUND(AVG(days), 1) AS mean_los,
       MAX(days)          AS longest,
       SUM(CASE WHEN days = 0 THEN 1 ELSE 0 END) AS same_day,
       SUM(CASE WHEN days >= 7 THEN 1 ELSE 0 END) AS week_plus
FROM los
GROUP BY specialty
ORDER BY mean_los DESC;

```

(SQLite lacks a built-in MEDIAN; you can get it with window tricks, and in R it's one function — a good example of using each tool where it's strong.) Two data-quality realities to check whenever LOS looks odd: **negative LOS** (discharge before admission — data error) and **NULL discharges** (still in, or never closed — Lesson 39 makes these routine checks). Pathway thinking generalises: A&E attendance → admission → discharge → readmission is a sequence of linked events per patient, and the window functions you now own (ROW\_NUMBER, LAG/LEAD) are precisely the tools that walk it.

**QUICK CHECK**

**Mean LOS is 4.1 days, median is 1 day. The best interpretation is...**

- ☐ The data is wrong
- ✓ **Stays are right-skewed: most short, a few very long pulling the mean up**
- ☐ Median must be recalculated
- ☐ Patients stay 4 days typically

**Why:** Classic skew. The median says a typical stay is short; the mean is dragged by long-stay outliers. Report both.

**QUICK CHECK**

**In SUS/HES terms, one continuous stay in one hospital that contains three consultant handovers is...**

- ☐ Three spells
- ✓ **One provider spell containing three episodes**
- ☐ One episode containing three spells
- ☐ A CIP spell only

**Why:** Episodes nest inside spells; chained inter-hospital spells form a CIP spell. Counting episodes as admissions triples the number.

**EXERCISE**

Find suspicious records: any admissions where discharge\_date is BEFORE admission\_date (negative LOS), and separately count how many spells are still open (NULL discharge). Two short queries.

**SOLUTION · SQL**

```
SELECT admission_id, patient_id, admission_date, discharge_date
FROM admissions
WHERE discharge_date IS NOT NULL
      AND discharge_date < admission_date;
SELECT COUNT(*) AS open_spells
FROM admissions
WHERE discharge_date IS NULL;
```

**KEY POINTS**

- Episodes nest in spells nest in CIP spells — know which layer a count refers to.
- LOS is right-skewed: medians and percentiles beside means, always.
- Negative LOS and unclosed spells are standing data-quality checks.

## MODULE 3 • SQL FOR HEALTH DATA • LESSON 37 OF 105

## 37. Rates: joining numerators to populations

Counts mislead: 120 emergency admissions in Barchester vs 80 in Kingsbay says little until you know how many people live in each. A **crude rate** = events ÷ population, scaled per 1,000 or 100,000. Your playground carries `la_population` (population by LA, sex and age band) precisely so numerators can meet denominators:

**CRUDE EMERGENCY ADMISSION RATE PER 1,000, BY LA**

```
WITH numerator AS (
  SELECT d.la_name, COUNT(*) AS admissions
  FROM admissions a
  JOIN patients p ON p.patient_id = a.patient_id
  JOIN lsoa_imd d ON d.lsoa_code = p.lsoa_code
  WHERE a.admission_method = 'Emergency'
  GROUP BY d.la_name
),
denominator AS (
  SELECT la_name, SUM(population) AS pop
  FROM la_population
  GROUP BY la_name
)
SELECT n.la_name,
       n.admissions,
       d.pop,
       ROUND(1000.0 * n.admissions / d.pop, 1) AS rate_per_1000
FROM numerator n
JOIN denominator d ON d.la_name = n.la_name
ORDER BY rate_per_1000 DESC;
```

The pattern — numerator CTE, denominator CTE, join, `1000.0 *` (decimal!, Lesson 17's trap) — is the same for vaccination coverage, GP consultation rates, mortality rates, everything. Three professional cautions:

- **Match the definitions.** Numerator "residents of Barchester" must divide by \*residents\* of Barchester, not patients \*treated\* there — resident-based vs provider-based counting is a real and common mismatch.
- **Match the period.** Two years of admissions over a one-year population doubles your rate; divide by person-years or use one year of events.
- **Crude rates hide age.** An older LA will "look worse" on almost anything. Comparing places fairly needs **age-standardisation** — the method is Lesson 70, in R, where it's far more pleasant.

**QUICK CHECK**

**Barchester: 120 emergency admissions, population 40,000. The crude rate per 1,000 is...**

- ☐ 0.3
- ✓ **3.0**
- ☐ 30
- ☐ 333

**Why:**  $1000 \times 120 / 40000 = 3.0$  per 1,000. (Per 100,000 it would be 300.)

**QUICK CHECK**

**An LA with a much older population shows a higher crude COPD admission rate than a young city. Before concluding its services are worse, you should...**

- ☐ Re-run the query
- ☐ Suppress the number
- ✓ **Age-standardise, since age structure alone can explain crude differences**
- ☐ Use counts instead of rates

**Why:** Crude rates confound age with everything else. Standardisation (Lesson 70) compares like with like.

**EXERCISE**

Compute the death rate per 1,000 population by LA: numerator from deaths joined via patients to lsoa\_imd; denominator from la\_population. Round to 2 dp.

**SOLUTION · SQL**

```
WITH numerator AS (
  SELECT l.la_name, COUNT(*) AS deaths
  FROM deaths d
  JOIN patients p ON p.patient_id = d.patient_id
  JOIN lsoa_imd l ON l.lsoa_code = p.lsoa_code
  GROUP BY l.la_name
),
denominator AS (
  SELECT la_name, SUM(population) AS pop
  FROM la_population
  GROUP BY la_name
)
SELECT n.la_name, n.deaths, d.pop,
       ROUND(1000.0 * n.deaths / d.pop, 2) AS death_rate_per_1000
FROM numerator n
JOIN denominator d ON d.la_name = n.la_name
ORDER BY death_rate_per_1000 DESC;
```

**KEY POINTS**

- Rate = numerator ÷ matching denominator × scale; build each as a CTE and join.
- Match residence vs provider, and period vs period — mismatches are silent errors.
- Crude rates confound age: standardise before comparing areas (Lesson 70).

**MODULE 3 · SQL FOR HEALTH DATA · LESSON 38 OF 105**

## 38. The classic metric: emergency readmissions within 30 days

"Emergency readmissions within 30 days of discharge" is a national NHS indicator — a marker of care quality and system pressure — and it exercises everything you've learned: ordering events per patient, LAG, date arithmetic, cohort discipline. The logic: for each admission, find the patient's **previous discharge**; if this admission is an **Emergency** and the gap is **0-30 days**, it counts as a readmission.

**30-DAY EMERGENCY READMISSIONS**

```
WITH ordered AS (
  SELECT
```

**30-DAY EMERGENCY READMISSIONS**

```

patient_id,
admission_id,
admission_date,
discharge_date,
admission_method,
LAG(discharge_date) OVER (
  PARTITION BY patient_id
  ORDER BY admission_date, admission_id
) AS prev_discharge
FROM admissions
),
flagged AS (
  SELECT *,
  CASE
    WHEN admission_method = 'Emergency'
    AND prev_discharge IS NOT NULL
    AND julianday(admission_date) - julianday(prev_discharge)
    BETWEEN 0 AND 30
    THEN 1 ELSE 0
  END AS readmission_30d
  FROM ordered
)
SELECT
  COUNT(*) AS admissions,
  SUM(readmission_30d) AS readmissions_30d,
  ROUND(100.0 * SUM(readmission_30d) / COUNT(*), 1) AS pct
FROM flagged;
```

Definition details separate a defensible figure from a wrong one — the published indicator makes choices you should recognise: gaps measured **discharge → next admission**; day 0 (same-day bounce-back) counts; the *\*index\** denominator excludes stays ending in death (you can LEFT JOIN *deaths* to exclude); planned/elective returns don't count as events (our Emergency filter handles this); and specialties like obstetrics are often excluded. When your number differs from a published one, the explanation is almost always one of these choices — which is why you **write the definition down**.

**QUICK CHECK**

**Why measure the gap from the previous DISCHARGE rather than the previous admission?**

- ☐ Admission dates are unreliable
- ✓ **The indicator asks how soon after leaving hospital the patient bounced back**
- ☐ Discharge dates are always NULL
- ☐ It makes the SQL shorter

**Why:** "Within 30 days of discharge" is the clinical concept: the risk window starts when the patient leaves.

**QUICK CHECK**

**A patient dies during their index admission. For the standard indicator, that admission should be...**

- ☐ Counted as a readmission
- ☐ Counted in the denominator
- ✓ **Excluded from the denominator — they cannot be readmitted**
- ☐ Duplicated

**Why:** Deaths in hospital can't generate a readmission, so including them deflates the rate. Exclude via a join to the deaths table.

**EXERCISE**

Extend the query: report the 30-day emergency readmission percentage BY SPECIALTY of the index (previous) admission — hint: also LAG the specialty. Order by percentage descending.

**SOLUTION · SQL**

```
WITH ordered AS (
  SELECT patient_id, admission_id, admission_date, admission_method,
         LAG(discharge_date) OVER (PARTITION BY patient_id
                                   ORDER BY admission_date, admission_id) AS prev_discharge,
         LAG(specialty)       OVER (PARTITION BY patient_id
                                   ORDER BY admission_date, admission_id) AS prev_specialty
  FROM admissions
),
flagged AS (
  SELECT prev_specialty,
         CASE WHEN admission_method = 'Emergency'
              AND prev_discharge IS NOT NULL
              AND julianday(admission_date) - julianday(prev_discharge) BETWEEN 0 AND 30
              THEN 1 ELSE 0 END AS readmit
  FROM ordered
  WHERE prev_discharge IS NOT NULL
)
SELECT prev_specialty AS index_specialty,
       COUNT(*)       AS discharges_followed,
       SUM(readmit)    AS readmissions_30d,
       ROUND(100.0 * SUM(readmit) / COUNT(*), 1) AS pct
FROM flagged
GROUP BY prev_specialty
ORDER BY pct DESC;
```

**KEY POINTS**

- Readmissions = LAG(discharge) per patient + Emergency filter + 0–30 day gap.
- The definition's small choices (day 0, deaths, exclusions) move the number — record them.
- This one query pattern demonstrates cohorts, windows and date logic together: interview gold.

**MODULE 3 · SQL FOR HEALTH DATA · LESSON 39 OF 105**

## 39. Data quality checks: trust nothing, verify everything

Professional analysts run a **DQ pass** on every new extract before analysing it. Not because data teams are careless — because pipelines break, coding practices drift, and *\*your\** conclusions inherit every flaw. A reusable six-check battery, each a small query:

**1. Row counts & date coverage** — is the volume plausible, and does the data span the period you think?

#### COVERAGE CHECK

```
SELECT COUNT(*) AS rows_,
       MIN(admission_date) AS earliest,
       MAX(admission_date) AS latest
FROM admissions;
```

**2. Completeness** — how NULL is each key column?

#### MISSINGNESS PROFILE

```
SELECT
  COUNT(*) AS rows_,
  SUM(CASE WHEN discharge_date IS NULL THEN 1 ELSE 0 END) AS null_discharge,
  SUM(CASE WHEN ethnicity IS NULL THEN 1 ELSE 0 END) AS null_ethnicity_via_join
FROM admissions a
LEFT JOIN patients p ON p.patient_id = a.patient_id;
```

**3. Validity** — values in legal ranges (ages 0-110, HbA1c not 999, dates not in the future). **4.**

**Consistency** — cross-field logic: discharge  $\geq$  admission; death date after last discharge. **5.**

**Referential integrity** — orphans: events whose patient doesn't exist (the anti-join from Lesson 26):

#### ORPHANED ADMISSIONS

```
SELECT COUNT(*) AS orphan_admissions
FROM admissions a
LEFT JOIN patients p ON p.patient_id = a.patient_id
WHERE p.patient_id IS NULL;
```

**6. Distribution sniff** — GROUP BY each categorical and eyeball: unexpected categories, one category at 95%, sudden monthly cliffs (a feed that died). Save your battery as a script and re-run it every refresh; in Module 7 you'll automate it. The habit to internalise: **a DQ finding is a finding** — "ethnicity is 38% missing and worse in deprived LSOAs" changes what your inequality analysis can claim.

#### QUICK CHECK

**An admissions extract shows max(admission\_date) three months before today. The most likely story is...**

- ☐ Admissions stopped happening
- ✓ **The feed lags or broke — investigate coverage before analysing trends**
- ☐ SQL cannot read recent dates
- ☐ Someone deleted rows

**Why:** A data cliff is a pipeline symptom. Recent "declines" in activity are the classic artefact of an incomplete latest period.



**QUICK CHECK**

**Orphaned admissions (patient\_id with no patients row) most directly threaten which analyses?**

- ☐ Row counts of admissions
- ✓ **Anything joining to patient attributes — those events silently drop out of INNER JOINS**
- ☐ ORDER BY queries
- ☐ Nothing; orphans are harmless

**Why:** Every demographic breakdown loses the orphans without warning. Measure them, then decide how to handle.

**EXERCISE**

Write two consistency checks: (a) lab\_results with implausible Systolic BP (below 60 or above 260); (b) deaths whose date\_of\_death is earlier than the patient's latest admission\_date (event after death).

**SOLUTION · SQL**

```
SELECT * FROM lab_results
WHERE test = 'Systolic BP'
      AND (value < 60 OR value > 260);
SELECT d.patient_id, d.date_of_death, MAX(a.admission_date) AS latest_admission
FROM deaths d
JOIN admissions a ON a.patient_id = d.patient_id
GROUP BY d.patient_id, d.date_of_death
HAVING MAX(a.admission_date) > d.date_of_death;
```

**KEY POINTS**

- Six checks every extract: volume/coverage, completeness, validity, consistency, orphans, distributions.
- Save the battery; re-run on every refresh; automate later.
- DQ findings are analytical findings — they bound what you can claim.

**MODULE 3 · SQL FOR HEALTH DATA · LESSON 40 OF 105**

## 40. The T-SQL phrasebook: SQL Server for NHS analysts

Walk into an NHS analytics team and the database is very likely **Microsoft SQL Server**, queried through **SSMS** (SQL Server Management Studio) or Azure Data Studio, in its dialect **T-SQL**. Everything you've learned transfers; here is the translation table for the parts that differ:

| Task              | SQLite (this course)        | SQL Server (T-SQL)                  |
|-------------------|-----------------------------|-------------------------------------|
| First n rows      | LIMIT 10                    | SELECT TOP 10 ... (or OFFSET/FETCH) |
| Today             | date('now')                 | CAST(GETDATE() AS DATE)             |
| Date difference   | julianday(a) – julianday(b) | DATEDIFF(DAY, b, a)                 |
| Date shift        | date(d, '+30 day')          | DATEADD(DAY, 30, d)                 |
| Year / month part | strftime('%Y', d)           | YEAR(d) / MONTH(d)                  |
| Month key         | strftime('%Y-%m', d)        | FORMAT(d, 'yyyy-MM')                |
| Concatenate       | a    b                      | a + b or CONCAT(a, b)               |

| Task            | SQLite (this course) | SQL Server (T-SQL)         |
|-----------------|----------------------|----------------------------|
| Integer to text | CAST(x AS TEXT)      | CAST(x AS VARCHAR(20))     |
| NULL fallback   | COALESCE(a, b)       | COALESCE(a, b) (or ISNULL) |
| Temp result     | CTE / temp table     | CTE / #temp_table          |

Everything else — SELECT/WHERE/GROUP BY/HAVING, JOINS, CASE, CTEs, ROW\_NUMBER/LAG/LEAD, UNION — is identical or near-identical. A few workplace-specific notes:

- **Schemas & brackets:** tables are addressed as `database.schema.table` (e.g. `Warehouse.dbo.Admissions`); square brackets `[Like This]` quote awkward names.
- **TOP needs ORDER BY** to be meaningful, exactly like LIMIT.
- **Case sensitivity** of text comparisons depends on collation (often insensitive) — keep the defensive `UPPER(TRIM(...))` habit.
- **Permissions:** you'll usually have read-only access to warehouse schemas and write access to a sandbox — which is the correct shape; analysts create in sandboxes, never in source.
- **GETDATE() vs GETUTCDATE():** local vs UTC time — matters for overnight jobs.

#### THE READMISSIONS CORE, TRANSLATED TO T-SQL

```
WITH ordered AS (
  SELECT patient_id, admission_id, admission_date, admission_method,
         LAG(discharge_date) OVER (PARTITION BY patient_id
                                   ORDER BY admission_date, admission_id) AS prev_discharge
  FROM Warehouse.dbo.Admissions
)
SELECT COUNT(*) AS admissions,
       SUM(CASE WHEN admission_method = 'Emergency'
                AND DATEDIFF(DAY, prev_discharge, admission_date)
                BETWEEN 0 AND 30
                THEN 1 ELSE 0 END) AS readmissions_30d
FROM ordered;
```

#### QUICK CHECK

The T-SQL equivalent of SQLite's LIMIT 20 is...

- ☐ LIMIT 20
- ☐ FIRST 20
- ✓ **SELECT TOP 20 ... (with ORDER BY for a meaningful "top")**
- ☐ ROWNUM 20

**Why:** TOP is the T-SQL idiom (OFFSET/FETCH is the standard alternative). ROWNUM is Oracle's.

#### QUICK CHECK

DATEDIFF(DAY, prev\_discharge, admission\_date) corresponds to which SQLite expression?

- ☐ julianday(prev\_discharge) – julianday(admission\_date)
- ✓ **julianday(admission\_date) – julianday(prev\_discharge)**
- ☐ date(admission\_date, '-30 day')
- ☐ strftime('%d', admission\_date)

**Why:** DATEDIFF(unit, start, end) = end – start. Argument order trips people crossing dialects — check signs with a known example.

**EXERCISE**

Translate to T-SQL: `SELECT patient_id, date(date_given, '+1 year') AS due, strftime('%Y-%m', date_given) AS ym FROM vaccinations LIMIT 5;`

**SOLUTION · SQL**

```
SELECT TOP 5
    patient_id,
    DATEADD(YEAR, 1, date_given) AS due,
    FORMAT(date_given, 'yyyy-MM') AS ym
FROM vaccinations;
```

**KEY POINTS**

- T-SQL differences are vocabulary, not concepts: TOP, GETDATE, DATEDIFF, DATEADD, FORMAT, +/CONCAT.
- Address tables as database.schema.table; expect read-only warehouse + writable sandbox.
- Keep one known-answer example to sanity-check argument order when switching dialects.

**MODULE 3 · SQL FOR HEALTH DATA · LESSON 41 OF 105**

## 41. Views, temp tables and saving results

So far your queries vanish on completion. Three ways to keep logic or results around — each with a proper use:

**Views** store the *\*query\**, not the data. Query a view like a table and it re-runs its definition live — one blessed definition of "emergency admissions, cleaned" that the whole team shares, always current:

**CREATE AND USE A VIEW**

```
CREATE VIEW IF NOT EXISTS v_emergency AS
SELECT a.admission_id, a.patient_id, a.admission_date,
    a.specialty, p.sex, p.lsoa_code
FROM admissions a
JOIN patients p ON p.patient_id = a.patient_id
WHERE a.admission_method = 'Emergency';
SELECT specialty, COUNT(*) AS n
FROM v_emergency
GROUP BY specialty
ORDER BY n DESC;
```

**Tables from queries** store the *\*result\** — a frozen snapshot. `CREATE TABLE cohort_2025 AS SELECT ...` (T-SQL spelling: `SELECT ... INTO cohort_2025 FROM ...`). Use for expensive results you'll reuse, or cohorts that must not shift while you analyse them. Snapshots go **stale by design** — name them with dates (`cohort_20250401`) and record the creation query.

**Temp tables** live only for your session — `CREATE TEMP TABLE t AS SELECT ...` (SQLite), `#t` (SQL Server) — ideal for multi-step working that's too big for one statement but not worth keeping.

Rules of the road in shared systems: create views/tables **only in your sandbox schema**; prefix or folder them by project; never `DROP` anything you didn't create; and remember `DROP TABLE / DELETE` are immediate and irreversible — in the playground you can experiment freely (Reset database restores everything), which is precisely what it's for.

**QUICK CHECK**

**Analysis code queries v\_emergency every morning. Overnight, new admissions arrive. The view...**

✓ **Reflects them automatically — it re-runs its definition each time**

- ☐ Shows yesterday's data until refreshed
- ☐ Must be dropped and recreated daily
- ☐ Locks the source table

**Why:** Views store logic, not rows: always-current by construction. A CREATE TABLE AS snapshot is the one that freezes.

**QUICK CHECK**

**You need this quarter's cohort to stay IDENTICAL for six weeks of analysis while source data keeps flowing. Best tool?**

✓ **A snapshot table (CREATE TABLE ... AS), dated and documented**

- ☐ A view
- ☐ A temp table
- ☐ Re-running the cohort query each time

**Why:** Stability requires freezing rows. Views drift with the source; temp tables die with the session; re-running invites silent cohort drift.

**EXERCISE**

In the playground: create a view v\_loS containing admission\_id, specialty and length of stay in days (completed spells only); then query it for mean LOS by specialty. Finally, tidy up with DROP VIEW v\_loS;

**SOLUTION · SQL**

```
CREATE VIEW v_loS AS
SELECT admission_id, specialty,
       CAST(julianday(discharge_date) - julianday(admission_date) AS INTEGER) AS los_days
FROM admissions
WHERE discharge_date IS NOT NULL;
SELECT specialty, ROUND(AVG(los_days), 1) AS mean_loS
FROM v_loS
GROUP BY specialty;
DROP VIEW v_loS;
```

**KEY POINTS**

- Views = stored logic, always current; snapshot tables = frozen results, dated; temp tables = session scratch.
- Choose by question: shared definition → view; stable cohort → snapshot; working steps → temp.
- Create only in your sandbox; drop only what you made.

## MODULE 4

# Module 4 — R fundamentals

## In this module

- 42** Welcome to R: console, scripts and projects
- 43** Objects and vectors: R's building blocks
- 44** Vectorised thinking, logical tests and NA
- 45** Data frames and tibbles: tables arrive in R
- 46** Packages: installing R's superpowers (and the tidyverse)
- 47** Importing data: `read_csv` and friends
- 48** dplyr part 1: filter rows, arrange order
- 49** dplyr part 2: select, mutate and rename columns
- 50** The pipe, `group_by` and summarise: the analysis engine
- 51** Joins in R: `left_join`, `inner_join` and the mighty `anti_join`
- 52** Tidy data: `pivot_longer` and `pivot_wider`
- 53** Strings with stringr: cleaning the text you're given
- 54** Dates with lubridate: UK dates tamed
- 55** Factors with forcats: categories in the right order
- 56** Writing your own functions
- 57** Iteration: for loops and purrr's map

## MODULE 4 • R FUNDAMENTALS • LESSON 42 OF 105

## 42. Welcome to R: console, scripts and projects

SQL fetched and shaped data \*inside\* the database. **R** is where analysis lives: statistics, charts, reports — and it's the de facto standard across UKHSA and a large slice of NHS analytics (the NHS-R community exists for a reason). Open RStudio (installed in Lesson 5) and get oriented:

- **Console** (bottom-left): type, Enter, instant result. For experiments.
- **Source** (top-left): script files (.R) you save and run — your real work. Run the current line with **Ctrl+Enter** (Cmd+Enter on Mac); this shortcut is 90% of your keystrokes.
- **Environment** (top-right): the objects (datasets, values) currently in memory.
- **Files / Plots / Packages / Help** (bottom-right): `?mean` opens help for any function.

**TYPE THESE IN THE CONSOLE**

```
1200 / 30                # arithmetic
#> [1] 40
sqrt(144)                # functions: name(arguments)
#> [1] 12
round(3.14159, digits = 2) # named arguments read better
#> [1] 3.14
# Anything after a hash is a comment - R ignores it
```

Lines beginning `#>` show what R prints back — a convention this course uses throughout so you can check yourself.

**Work in Projects from day one.** File ► New Project ► New Directory ► name it `ph-course-r`. A project pins R's working directory to that folder, so relative paths like `data/admissions.csv` behave identically on your machine and a colleague's — the R answer to the portability problem from Lesson 2. Inside it, create `data/`, `R/` (scripts) and `outputs/` folders. And begin the habit that defines professionals: **everything in a script**. The console is for trying; the script is the record. If it only happened in the console, it didn't happen.

**ONE TOOL, EVERY OS**

Unlike installation, the R language itself is identical on Windows, macOS and Linux — every lesson from here runs unchanged on all three. The rare OS-specific moments (paths, database drivers, scheduling) are flagged when they arrive in Module 7.

**QUICK CHECK**

**Ctrl+Enter (Cmd+Enter) in the Source pane does what?**

- ☐ Saves the file
- ☒ **Runs the current line or selection in the Console**
- ☐ Opens a new script
- ☐ Comments the line

**Why:** It sends code from your script to the console — the core loop of writing R: edit line, run line, look, repeat.

**QUICK CHECK****Why start every piece of work as an RStudio Project?**

- ☐ Projects run faster
- ✓ **The working directory is pinned, so relative paths work for anyone who opens the project**
- ☐ R requires it
- ☐ Projects enable colour themes

**Why:** Portability and reproducibility: `data/file.csv` resolves the same way on every machine that opens the `.Rproj`.

**EXERCISE**

Create the `ph-course-r` project with `data/`, `R/` and `outputs/` folders. Make a script `R/lesson42.R` containing three commands: `52 * 7`; `round(365.25/12, 1)`; and a comment with today's date. Run each with `Ctrl+Enter`.

**SOLUTION · R**

```
# lesson42.R - first script, 2026-XX-XX
52 * 7
#> [1] 364
round(365.25 / 12, 1)
#> [1] 30.4
```

**KEY POINTS**

- Console experiments; Source scripts record; `Ctrl/Cmd+Enter` runs the current line.
- Always work inside an RStudio Project with `data/`, `R/`, `outputs/` folders.
- `#>` lines in this course show expected output — verify as you go.

**MODULE 4 · R FUNDAMENTALS · LESSON 43 OF 105**

## 43. Objects and vectors: R's building blocks

R stores things in named **objects** using the assignment arrow `<-` (type it with `Alt+minus` in RStudio; `=` mostly works but `<-` is universal R style):

**ASSIGNMENT AND VECTORS**

```
la_name <- "Barchester"
patients <- 640
rate <- 1000 * 120 / 40000
ages <- c(67, 45, 81, 23, 59) # c() combines values into a vector
ages
#> [1] 67 45 81 23 59
length(ages)
#> [1] 5
ages[1] # square brackets extract - R counts from 1
#> [1] 67
ages[c(2, 4)]
#> [1] 45 23
```

The **vector** — an ordered collection of values, all the same type — is R's atom; even `patients` above is a vector of length 1. The types you'll meet daily:

- **numeric** — `c(67, 45, 3.2)`
- **character** — text in quotes: `c("Emergency", "Elective")`

- **logical** — TRUE / FALSE (capitals, unquoted)
- **Date** — coming in Lesson 54; and **factor** (categories) in Lesson 55.

`class(x)` reveals a vector's type, and type explains behaviour: "7" + 1 is an error because "7" is text. A whole column of a health extract arriving as character when it should be numeric (thanks to one stray "N/A") is a rite of passage — now you'll recognise it.

Naming style: **snake\_case**, descriptive, lowercase — `mean_lo`, `emergency_admissions`, `imd_decile`. R is case-sensitive: `Ages` and `ages` are different objects, and the error object 'Ages' not found is R telling you exactly that.

#### QUICK CHECK

After `x <- c(10, 20, 30)`, what does `x[2]` return?

- ☐ 10
- ✓ **20**
- ☐ `c(10, 20)`
- ☐ An error

**Why:** R indexes from 1 (unlike Python's 0). Position 2 holds 20.

#### QUICK CHECK

`class(c("48", "51", "N/A"))` is...

- ☐ numeric
- ✓ **character**
- ☐ logical
- ☐ factor

**Why:** Quotes make text, and one non-number would force the whole vector to character anyway — the classic reason an imported column won't do arithmetic.

#### EXERCISE

Create a vector `monthly_admissions` with values 112, 98, 130, 121, 145 (May-September). Extract: (a) the third value; (b) the first and last by position; (c) the vector's length; (d) its class.

#### SOLUTION · R

```
monthly_admissions <- c(112, 98, 130, 121, 145)
monthly_admissions[3]
#> [1] 130
monthly_admissions[c(1, 5)]
#> [1] 112 145
length(monthly_admissions)
#> [1] 5
class(monthly_admissions)
#> [1] "numeric"
```

#### KEY POINTS

- `name <- value` assigns; `c()` builds vectors; `[ ]` extracts, counting from 1.
- Types (numeric, character, logical) drive behaviour; `class()` diagnoses.
- `snake_case` names; R is case-sensitive.



## MODULE 4 • R FUNDAMENTALS • LESSON 44 OF 105

## 44. Vectorised thinking, logical tests and NA

The idea that makes R feel magical after SQL: operations apply to **whole vectors at once** — no loops required:

**VECTORISED ARITHMETIC**

```
admissions <- c(120, 80, 95)
population <- c(40000, 26000, 31000)
rate_per_1000 <- 1000 * admissions / population # element by element
round(rate_per_1000, 1)
#> [1] 3.0 3.1 3.1
```

Comparisons are vectorised too, yielding **logical vectors** — and logicals have a superpower: `sum()` treats TRUE as 1, so counting-things-meeting-a-condition is one line:

**LOGICAL VECTORS DO THE COUNTING**

```
hba1c <- c(41, 52, 48, 39, 66, 47)
hba1c >= 48
#> [1] FALSE TRUE TRUE FALSE TRUE FALSE
sum(hba1c >= 48) # how many at/over threshold
#> [1] 3
mean(hba1c >= 48) # what proportion
#> [1] 0.5
hba1c[hba1c >= 48] # filter: keep where TRUE
#> [1] 52 48 66
```

Combine conditions with `&` (and), `|` (or), `!` (not) — single symbols, unlike SQL's words.

**NA** is R's NULL-cousin: missing. Like SQL's NULL it poisons calculations — `mean(c(41, NA, 52))` is NA — but R makes you opt in to skipping: `mean(x, na.rm = TRUE)`. Test for missingness with `is.na(x)` (never `x == NA`, which — exactly like SQL — returns NA). `sum(is.na(x))` counts the missing, your first data-quality reflex in R.

**HANDLING NA DELIBERATELY**

```
values <- c(41, NA, 52, 48, NA)
mean(values)
#> [1] NA
mean(values, na.rm = TRUE)
#> [1] 47
sum(is.na(values))
#> [1] 2
```

**QUICK CHECK**

**sum(los\_days > 7) computes...**

- ☐ Total days over a week
- ☒ **How many stays exceeded 7 days**
- ☐ An error: sum needs numbers
- ☐ The longest stay

**Why:** The comparison makes TRUE/FALSE per stay; sum counts the TRUES. mean() of the same gives the proportion.

**QUICK CHECK****mean(c(10, 20, NA)) returns NA because...**

- ☐ R cannot average three values
- ☐ NA equals zero
- ✓ **Missing propagates: the true mean is unknowable unless you explicitly exclude NA**
- ☐ The vector is character

**Why:** Same philosophy as SQL NULL, but R demands the explicit choice: `na.rm = TRUE` says "ignore missing, and I know I'm doing it".

**EXERCISE**

Given `sbp <- c(128, 152, NA, 141, 119, 160, NA, 135)`: (a) count missing readings; (b) count readings of 140+ (excluding NA - hint: `na.rm` works in `sum` too); (c) proportion of non-missing readings that are 140+; (d) the mean of readings under 140.

**SOLUTION · R**

```
sbp <- c(128, 152, NA, 141, 119, 160, NA, 135)
sum(is.na(sbp))
#> [1] 2
sum(sbp >= 140, na.rm = TRUE)
#> [1] 3
mean(sbp >= 140, na.rm = TRUE)
#> [1] 0.5
mean(sbp[sbp < 140], na.rm = TRUE)
#> [1] 127.3333
```

**KEY POINTS**

- Operations and comparisons act on whole vectors; `sum()`/`mean()` of logicals count and proportion.
- `&` | `!` combine conditions; [logical] filters.
- NA propagates until you opt out with `na.rm = TRUE`; `is.na()` tests; never `== NA`.

**MODULE 4 · R FUNDAMENTALS · LESSON 45 OF 105**

## 45. Data frames and tibbles: tables arrive in R

A **data frame** is R's table: columns are vectors (each one type), rows are records — your SQL result sets reborn. The tidyverse's refined version is the **tibble**; we'll use tibbles throughout (they print politely and complain early about mistakes). Build one by hand with `tibble()`, or row-wise with `tribble()`:

**YOUR FIRST TIBBLE**

```
library(tibble) # after Lesson 46 you'll load the whole tidyverse
practices <- tribble(
  ~practice_code, ~practice_name, ~la_name, ~list_size,
  "P81001", "Riverside Surgery", "Barchester", 8200,
  "P81002", "High Street Practice", "Barchester", 5400,
  "P81003", "Millford Health Ctr", "Millford", 11250,
  "P81004", "Bay Medical Group", "Kingsbay", 9100
)
practices
#> # A tibble: 4 x 4
#>   practice_code practice_name la_name list_size
#>   <chr>         <chr>         <chr>    <dbl>
```

**YOUR FIRST TIBBLE**

```
#> 1 P81001      Riverside Surgery      Barchester      8200
#> ...
```

The printout is information-dense: dimensions (4 x 4), column types (<chr> character, <dbl> numeric/double), then rows. First-look tools for any table:

**INSPECTING A DATA FRAME**

```
nrow(practices); ncol(practices)
names(practices)
dplyr::glimpse(practices)      # every column, type, first values - use constantly
summary(practices$list_size) # $ pulls one column out as a vector
#>   Min. 1st Qu. Median   Mean 3rd Qu.  Max.
#>  5400   7500   8650   8488   9638 11250
```

`glimpse()` is the R twin of your SQL habit of `DISTINCT`-and-`LIMIT` on new tables: run it on everything you import, every time. The `$` operator extracts a single column as a vector — so all of Lesson 44 applies: `sum(practices$list_size)`, `practices$list_size >= 9000`. (Base R's `practices[rows, cols]` bracket indexing exists and you'll read it in older code, but `dplyr` — next lessons — is how modern health teams write.)

**QUICK CHECK**

**In a tibble printout, <chr> and <dbl> under the column names tell you...**

- ☐ Column widths
- ✓ **Each column's type: character and double (numeric)**
- ☐ Compression settings
- ☐ Primary keys

**Why:** *Types at a glance — the fastest way to spot the "numbers arrived as text" problem before it bites.*

**QUICK CHECK**

**`practices$list_size` returns...**

- ☐ A one-column tibble
- ✓ **A plain numeric vector of the list sizes**
- ☐ The first row
- ☐ An error

**Why:** *`$` extracts the column as a vector, ready for `sum()`, `mean()`, comparisons — the bridge between tables and Lesson 44's vector skills.*

**EXERCISE**

Build a tibble `las` with columns `la_name` (Barchester, Millford, Kingsbay) and `population` (40000, 26000, 31000). Then: `glimpse` it; extract the population column; compute total population; and a logical showing which LAs exceed 30,000.

**SOLUTION · R**

```
library(tibble)
las <- tribble(
  ~la_name,      ~population,
  "Barchester",  40000,
  "Millford",    26000,
  "Kingsbay",    31000
)
dplyr::glimpse(las)
```

**SOLUTION · R**

```
las$population
#> [1] 40000 26000 31000
sum(las$population)
#> [1] 97000
las$population > 30000
#> [1] TRUE FALSE TRUE
```

**KEY POINTS**

- Tibbles = typed columns (vectors) + rows; the printout shows dimensions and types.
- `glimpse()` every new table; `$` extracts a column as a vector.
- `tribble()` builds small tables readably — handy for lookups and examples.

**MODULE 4 · R FUNDAMENTALS · LESSON 46 OF 105**

## 46. Packages: installing R's superpowers (and the tidyverse)

Base R is capable; **packages** make it formidable. A package is a bundle of functions others wrote, published on **CRAN** (the quality-controlled central archive). Two verbs, one crucial difference:

**INSTALL ONCE, LOAD EVERY SESSION**

```
install.packages("tidyverse") # downloads to your library - ONCE per machine
library(tidyverse)           # loads it - EVERY session/script that uses it
```

The **tidyverse** is the modern-R toolkit this course uses — `dplyr` (manipulation), `ggplot2` (charts), `readr` (import), `tidyr`, `stringr`, `forcats`, `lubridate` and friends, sharing one coherent grammar. NHS-R and UKHSA code is overwhelmingly tidyverse-shaped. Installation is the one moment R touches your operating system, so:

**WINDOWS**

```
# Windows: install.packages("tidyverse") usually "just works"
# (CRAN ships pre-built binaries).
# If a package ever says it needs compilation "from source",
# install Rtools from https://cran.r-project.org/bin/windows/Rtools/
# then restart RStudio and retry.
install.packages("tidyverse")
```

**MACOS**

```
# macOS: binaries also provided; occasionally compilation tools are wanted.
# If prompted about developer tools, run once in Terminal:
#   xcode-select --install
install.packages("tidyverse")
```

**LINUX**

```
# Linux compiles most packages from source, so system libraries matter.
# Debian/Ubuntu, before installing heavy packages:
#   sudo apt install r-base-dev libcurl4-openssl-dev libssl-dev libxml2-dev
install.packages("tidyverse") # then this compiles happily (takes a while)
```

Etiquette and gotchas: `library()` calls sit **at the top of every script**, so a reader instantly sees dependencies; never bury an `install.packages()` inside a script that others run (it changes *their* machine — leave it commented, or use dependency tooling later). If two packages export the same function name (`dplyr` and `base` both have `filter`), R uses the most recently loaded — write `dplyr::filter()` to be explicit when it matters. `packageVersion("dplyr")` tells you what you're running;

versions matter for reproducibility, a Module 7 theme. On locked-down NHS machines, packages may come from an internal mirror or need an IT request — the incantations are identical.

**QUICK CHECK****install.packages() vs library() — the right mental model is...**

- ☐ Both must run every session
- ✓ **Install downloads once per machine; library loads once per session**
- ☐ library installs; install.packages loads
- ☐ Only one is ever needed

**Why:** Installing puts the package on disk; loading attaches it to the running session. Scripts assume installed, and declare loads at the top.

**QUICK CHECK****A script for colleagues should contain install.packages("tidyverse")...**

- ☐ At the top, always
- ☐ After library()
- ✓ **Not as live code — installing changes other people's machines without asking**
- ☐ Inside a loop

**Why:** Declare dependencies with library() and document installs (comment/README); don't silently modify a colleague's library.

**EXERCISE**

Install the tidyverse (watch your OS note above), then write a script header that loads it, prints your dplyr version, and confirms filter comes from dplyr by calling dplyr::filter on a tiny tibble.

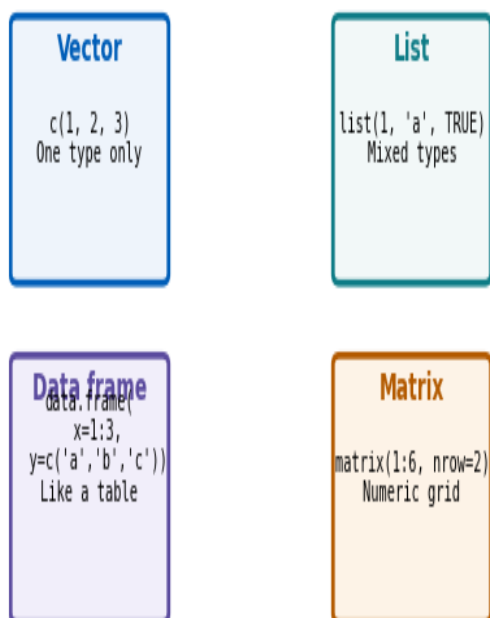
**SOLUTION · R**

```
library(tidyverse)
packageVersion("dplyr")
#> [1] '1.1.x' (any recent version is fine)
dplyr::filter(tibble(x = 1:5), x > 3)
#> # A tibble: 2 x 1
#>       x
#> 1     4
#> 2     5
```

**KEY POINTS**

- CRAN packages: install.packages() once per machine, library() per session, declared at the top.
- The tidyverse is the UK-health-analytics dialect of R; OS build tools (Rtools/xcode/dev libs) fix "from source" errors.
- package::function() disambiguates name clashes; packageVersion() records what ran.

## R data structures



### MODULE 4 • R FUNDAMENTALS • LESSON 47 OF 105

## 47. Importing data: read\_csv and friends

Analysis begins when data enters R. The tidyverse importer is `readr::read_csv()` — fast, predictable, and honest about what it did:

### IMPORTING A CSV

```
library(tidyverse)
admissions <- read_csv("data/admissions.csv")
#> Rows: 900 Columns: 7
#> -- Column specification -----
#> chr  (4): discharge_date, admission_method, specialty, primary_diagnosis
#> dbl  (2): admission_id, patient_id
#> date (1): admission_date
glimpse(admissions)
```

**Get a real file to practise on:** open this page's SQL playground, run `SELECT * FROM admissions;`, and press **Download CSV** — save it into your project's `data/` folder. (Any query's results export the same way, so you can manufacture practice files all course long.)

Read the import message like a QA report: row count plausible? Types right? A date column arriving as `chr` means the format wasn't ISO — the **UK trap**, since British DD/MM/YYYY files confuse default parsers. Fixes:

#### UK DATES AND MESSY VALUES

```
# Tell readr the format explicitly:
admissions <- read_csv("data/admissions.csv",
                      col_types = cols(admission_date = col_date("%d/%m/%Y")))

# Declare the strings that mean "missing":
labs <- read_csv("data/labs.csv", na = c("", "NA", "N/A", "NULL", "Unknown"))
```

Other arrivals you'll meet: **Excel** via `readxl::read_excel("file.xlsx", sheet = "Data", skip = 2)` (skip letterhead rows); **semicolon-separated European CSVs** via `read_csv2()`; **fixed messy headers** — `janitor::clean_names()` converts "Admission Date " into `admission_date` and is worth running on almost every import. Two habits that mark professionals: **never edit the raw file** (fix in code, so the fix reruns next month), and **glimpse + summary immediately after every import** — Lesson 39's DQ instinct, now in R.

#### QUICK CHECK

**read\_csv reports a date-of-birth column as <chr>. Most likely cause and fix?**

- ☐ The file is corrupt; re-download
- ✓ **Non-ISO (e.g. DD/MM/YYYY) dates; specify `col_date("%d/%m/%Y")` or parse with `lubridate`**
- ☐ Dates cannot be imported
- ☐ Use Excel instead

**Why:** Character-typed dates almost always mean an unrecognised format. Declare it at import (or `dmy()` later - Lesson 54).

#### QUICK CHECK

**Why fix a broken header in code (`clean_names()`) rather than editing the CSV in Excel?**

- ☐ Excel cannot edit CSVs
- ✓ **The raw file stays pristine and the fix re-applies automatically to next month's file**
- ☐ `clean_names` is faster to type
- ☐ Headers do not matter

**Why:** Reproducibility again: raw data immutable, transformations scripted. Next refresh costs zero clicks.

#### EXERCISE

Export patients from the playground (`SELECT * FROM patients;`) to `data/patients.csv`. Import it with `read_csv`, glimpse it, and report: rows, columns, the type of `birth_date`, and how many NAs ethnicity has (`sum(is.na(...))`).

#### SOLUTION · R

```
library(tidyverse)
patients <- read_csv("data/patients.csv")
glimpse(patients)
#> Rows: ~600, Columns: 6
#> birth_date imports as <date> (ISO format - one benefit of Lesson 10's advice)
sum(is.na(patients$ethnicity))
#> [1] (a smallish number - some ethnicities are unrecorded, as in real data)
```

**KEY POINTS**

- `read_csv` + immediate glimpse/summary; read the import message as a QA report.
- UK DD/MM/YYYY needs `col_date("%d/%m/%Y")` (or `lubridate::dmy`); declare NA strings.
- `readxl` for Excel, `clean_names()` for headers; never hand-edit raw files.

**MODULE 4 • R FUNDAMENTALS • LESSON 48 OF 105****48. dplyr part 1: filter rows, arrange order**

**dplyr** gives five verbs that cover most manipulation; each takes a data frame first and returns a new one, original untouched. First pair — and their SQL twins:

**filter()** = **WHERE**. Conditions use Lesson 44's comparisons; commas mean AND:

**FILTER IN ACTION**

```
library(tidyverse)
admissions <- read_csv("data/admissions.csv") # from Lesson 47
emergency_resp <- filter(admissions,
                        admission_method == "Emergency",
                        specialty %in% c("Respiratory", "General Medicine"),
                        admission_date >= as.Date("2025-04-01"))
nrow(emergency_resp)
```

Watch the details: `==` **for equality** (single `=` inside filter is an argument-assignment error you'll make exactly once); `%in%` is SQL's IN; OR is `|` with parentheses to taste; and **NA behaves like SQL NULL** — `filter(x > 5)` silently drops NA rows, so the "keep unknowns too" version is `filter(x > 5 | is.na(x))`.

**arrange()** = **ORDER BY**. Ascending default; `desc()` flips:

**ARRANGE IN ACTION**

```
arrange(admissions, desc(admission_date), patient_id) # newest first
```

The "top n" idiom gets its own verb: `slice_max(admissions, order_by = admission_date, n = 5)` (and `slice_min`, `slice_head`, `slice_sample` for a random peek). Everything composes: filter, then arrange, then slice — though chaining reads far better with the pipe, arriving in Lesson 50.

**QUICK CHECK**

**filter(labs, value >= 48) on a column containing NAs will...**

- ☐ Keep the NA rows
- ✓ **Drop the NA rows silently, exactly like SQL's WHERE**
- ☐ Error
- ☐ Convert NA to 0

**Why:** Comparisons with NA are NA, and filter keeps only TRUE. Add `| is.na(value)` when unknowns should stay — a deliberate choice either way.



**QUICK CHECK**

The dplyr equivalent of SQL's WHERE la IN ('Barchester','Millford') is...

- ☐ filter(df, la == c("Barchester","Millford"))
- ✓ **filter(df, la %in% c("Barchester","Millford"))**
- ☐ filter(df, la = "Barchester" OR "Millford")
- ☐ arrange(df, la)

**Why:** %in% tests membership. == against a vector recycles element-by-element - a subtle wrong answer, not an error, which makes it worse.

**EXERCISE**

From admissions: keep Elective admissions in financial year 2024/25 (dates 2024-04-01 to 2025-03-31 inclusive), arrange by specialty then newest admission\_date first, and show the top 5 longest... no - keep it to filter/arrange: show the first 5 rows via slice\_head(n = 5).

**SOLUTION · R**

```
fy2425_elective <- filter(admissions,
                          admission_method == "Elective",
                          admission_date >= as.Date("2024-04-01"),
                          admission_date <= as.Date("2025-03-31"))
result <- arrange(fy2425_elective, specialty, desc(admission_date))
slice_head(result, n = 5)
```

**KEY POINTS**

- filter() = WHERE (== , %in%, & | !, commas = AND); NA rows drop unless invited.
- arrange() = ORDER BY, with desc(); slice\_max/min = top-n.
- Verbs return new data frames; originals are never modified in place.

**MODULE 4 · R FUNDAMENTALS · LESSON 49 OF 105**

## 49. dplyr part 2: select, mutate and rename columns

**select()** chooses columns — SQL's SELECT list, with power tools:

**SELECTING COLUMNS**

```
select(admissions, admission_id, admission_date, specialty) # name them
select(admissions, -primary_diagnosis)                     # everything except
select(admissions, patient_id, starts_with("admission"))   # helpers:
# starts_with, ends_with, contains, where(is.numeric)...
```

**mutate()** creates or transforms columns — SQL's computed expressions, vectorised over the whole table:

**CREATING COLUMNS**

```
admissions2 <- mutate(admissions,
  los_days = as.numeric(discharge_date - admission_date), # Date maths just works
  is_emerg = admission_method == "Emergency",             # logical flag
  los_band = case_when(
    is.na(discharge_date) ~ "Still in",
    los_days == 0         ~ "Same day",
    los_days <= 7         ~ "1-7 days",
    .default              = "Over a week"
  )
)
```

## CREATING COLUMNS

)

`case_when()` is CASE WHEN reborn: `condition ~ value` pairs, evaluated top-down, `.default` as the ELSE. (In older code you'll see `TRUE ~ "Over a week"` for the default — same thing.) Simpler two-way splits: `if_else(condition, yes, no)`.

**`rename(new = old)`** fixes names — `rename(admissions, method = admission_method)` — and pairs with `janitor::clean_names()` from Lesson 47. Column order for reports: `relocate(los_days, .after = discharge_date)`. One discipline: mutate freely during analysis, but *\*derive\**, don't overwrite — `los_days_capped = pmin(los_days, 30)` beats destroying the original, because you will want it back.

## QUICK CHECK

**In `case_when()`, a row matching two conditions gets...**

- ☐ Both values pasted
- ☐ The last match
- ✓ **The first matching condition's value, top-down — same rule as SQL CASE**
- ☐ NA

**Why:** Order conditions most-specific first, exactly as you learned in Lesson 19.

## QUICK CHECK

**`select(df, -patient_id)` does what?**

- ☐ Deletes `patient_id` from the source file
- ✓ **Returns a copy without the `patient_id` column**
- ☐ Sorts by `patient_id` descending
- ☐ Renames `patient_id`

**Why:** Minus excludes. The source object (and file) are untouched — *dplyr* always returns new data.

## EXERCISE

On `lab_results` (export from the playground if needed): keep only `patient_id`, `test`, `value`, `date_taken`; create `flag_high = TRUE` for `HbA1c >= 48` or `Systolic BP >= 140` (FALSE otherwise, using `case_when` with `.default = FALSE`); rename `date_taken` to `test_date`.

## SOLUTION · R

```
labs2 <- lab_results |> ... # pipe comes next lesson; without it:
labs_small <- select(lab_results, patient_id, test, value, date_taken)
labs_flagged <- mutate(labs_small,
  flag_high = case_when(
    test == "HbA1c"      & value >= 48 ~ TRUE,
    test == "Systolic BP" & value >= 140 ~ TRUE,
    .default = FALSE
  )
)
labs_final <- rename(labs_flagged, test_date = date_taken)
glimpse(labs_final)
```

**KEY POINTS**

- `select()` picks/drops columns (helpers: `starts_with`, `contains`); `rename(new = old)`.
- `mutate()` computes vectorised columns; `case_when()` is your CASE WHEN, `if_else()` the quick binary.
- Derive new columns rather than overwrite; keep raw values recoverable.

**MODULE 4 • R FUNDAMENTALS • LESSON 50 OF 105**

## 50. The pipe, `group_by` and summarise: the analysis engine

Nesting verbs inside-out reads horribly. The **pipe** `|>` (say "then") passes the left side in as the right side's first argument, so pipelines read top-to-bottom like a recipe:

**THE SAME LOGIC, THREE WAYS**

```
# Nested (hard to read):
arrange(filter(admissions, admission_method == "Emergency"), desc(admission_date))

# Piped (professional style):
admissions |>
  filter(admission_method == "Emergency") |>
  arrange(desc(admission_date))
```

(Older code uses magrittr's `%>%` — read them as identical; write the native `|>`. RStudio inserts it with Ctrl+Shift+M once you set Tools ► Global Options ► Code ► "Use native pipe".)

Now the payoff pair. **`group_by()` + `summarise()`** is GROUP BY + aggregates, and together with the pipe it is *the* shape of descriptive health analysis:

**ADMISSIONS SUMMARISED BY SPECIALTY**

```
admissions |>
  mutate(los_days = as.numeric(discharge_date - admission_date)) |>
  group_by(specialty, admission_method) |>
  summarise(
    n          = n(),                # COUNT(*)
    patients   = n_distinct(patient_id), # COUNT(DISTINCT ...)
    mean_los   = mean(los_days, na.rm = TRUE),
    median_los = median(los_days, na.rm = TRUE), # trivial in R!
    .groups    = "drop"              # ungroup afterwards
  ) |>
  arrange(desc(n))
```

Everything from SQL transfers — and improves: `median()`, `quantile()`, `sd()` are one word each. `count(df, specialty, sort = TRUE)` shortcuts the commonest case. Grouped **`mutate`** (no collapsing — SQL's window functions!) also works: `group_by(patient_id) |> mutate(test_number = row_number())`. Include `.groups = "drop"` (or `ungroup()`) after summarising: forgotten groups silently haunt later steps — the R equivalent of SQLite's quiet GROUP BY traps.

**QUICK CHECK**

**`df |> filter(x > 5) |> summarise(m = mean(x))` — the pipe's job here is...**

- ☐ To run both functions in parallel
- ✓ **To pass each result as the first argument of the next verb, top to bottom**
- ☐ To save `df` to disk
- ☐ Decoration

**Why:** Left-hand result flows into the next call's data slot. Recipes, not nesting.

**QUICK CHECK**

After `group_by(la_name)`, the difference between `summarise(n = n())` and `mutate(n = n())` is...

- ☐ None
- ✓ **summarise collapses to one row per LA; mutate keeps all rows, attaching each LA's n**
- ☐ mutate errors on grouped data
- ☐ summarise needs ungroup first

**Why:** Collapse vs annotate — the same distinction as SQL aggregates vs window functions, in two verbs.

**EXERCISE**

Pipeline on `lab_results`: keep HbA1c tests; group by `patient_id`; summarise tests = `n()`, latest = `max(date_taken)`, mean\_value = `mean(value)`; then keep patients with 2+ tests and arrange by mean\_value descending, top 10 via `slice_head`.

**SOLUTION · R**

```
lab_results |>
  filter(test == "HbA1c") |>
  group_by(patient_id) |>
  summarise(tests = n(),
            latest = max(date_taken),
            mean_value = mean(value),
            .groups = "drop") |>
  filter(tests >= 2) |>           # HAVING, in R clothing
  arrange(desc(mean_value)) |>
  slice_head(n = 10)
```

**KEY POINTS**

- `|>` pipes data through verbs: read "then"; write pipelines, not nests.
- `group_by` + `summarise` = GROUP BY; `n()`, `n_distinct()`, `median()` included; drop groups after.
- `group_by` + `mutate` = window functions (annotate without collapsing).

**MODULE 4 · R FUNDAMENTALS · LESSON 51 OF 105**

## 51. Joins in R: left\_join, inner\_join and the mighty anti\_join

dplyr joins mirror SQL exactly — same concepts, same traps, friendlier syntax:

**JOINING PATIENTS TO THEIR DEPRIVATION DECILE**

```
patients_imd <- patients |>
  left_join(lsoa_imd, by = "lsoa_code")
# Different key names on each side:
adm_full <- admissions |>
  left_join(patients, by = join_by(patient_id)) |>
  left_join(lsoa_imd, by = join_by(lsoa_code))
```

- `inner_join()` keeps matches only; `left_join()` keeps all left rows (your default, as in SQL); `full_join()` keeps both sides.
- **Fan-out still applies** (Lesson 27!): joining patients→admissions duplicates patients. Modern dplyr even warns on unexpected many-to-many — read that warning, don't silence it.
- Suffixes: shared non-key columns become `la_name.x` / `la_name.y`; better to select/rename before joining.

Two joins with no SQL keyword of their own earn their keep in QA:

#### FILTERING JOINS

```
# semi_join: keep patients WHO HAVE at least one emergency admission
frequent_flyers <- patients |>
  semi_join(filter(admissions, admission_method == "Emergency"),
            by = "patient_id")
# anti_join: patients with NO vaccination record at all (Lesson 26's pattern, one verb)
never_vaccinated <- patients |>
  anti_join(vaccinations, by = "patient_id")
nrow(never_vaccinated)
```

`semi_join` filters by existence (no fan-out, no new columns); `anti_join` by absence. The professional reconciliation habit: after any important join, `anti_join` both directions and \*look at what didn't match\* — unmatched rows are where data quality stories live.

#### QUICK CHECK

**`anti_join(children, mmr_records, by = "patient_id")` returns...**

- ☐ Children with exactly one MMR record
- ☐ All rows from both tables
- ✓ **Children with no MMR record — the unvaccinated list**
- ☐ MMR records with no child

**Why:** *Anti = present left, absent right. Reverse the table order for orphan MMR records (the other direction of QA).*

#### QUICK CHECK

**`semi_join` differs from `inner_join` how?**

- ☐ It is slower
- ✓ **It filters the left table by existence of a match — no columns added, no row duplication**
- ☐ It keeps unmatched rows
- ☐ No difference

**Why:** *semi\_join answers "which left rows have a match?" without inheriting fan-out — often exactly what a cohort filter wants.*

#### EXERCISE

Using `anti_join` twice on admissions and patients: (a) count admissions whose `patient_id` has no patients row (orphans); (b) count patients with no admissions at all. Then use `semi_join` to build a tibble of patients who HAVE at least one admission.

#### SOLUTION · R

```
admissions |> anti_join(patients, by = "patient_id") |> nrow() # (a) orphans
patients   |> anti_join(admissions, by = "patient_id") |> nrow() # (b) never admitted
ever_admitted <- patients |> semi_join(admissions, by = "patient_id")
```

#### KEY POINTS

- `left_join` is the default; `inner` drops non-matches; fan-out and its warnings still apply.
- `semi_join` = filter by existence; `anti_join` = filter by absence (the anti-join, named).
- Reconcile every important join by `anti_joining` both directions and inspecting.

## MODULE 4 • R FUNDAMENTALS • LESSON 52 OF 105

## 52. Tidy data: `pivot_longer` and `pivot_wider`

**Tidy data** = one variable per column, one observation per row, one value per cell. Analysis tools (dplyr, ggplot2, models) want tidy; humans and Excel exports love *\*wide\** — months spread across columns, one row per practice. Reshaping between them is two functions.

**`pivot_longer()`: wide → tidy.** The classic import from a performance spreadsheet:

### MELTING MONTH COLUMNS INTO ROWS

```
wide <- tribble(
  ~practice, ~`2025-04`, ~`2025-05`, ~`2025-06`,
  "P81001",   42,       51,       47,
  "P81002",   30,       28,       35
)

long <- wide |>
  pivot_longer(cols = -practice,          # everything except practice
               names_to = "month",
               values_to = "admissions")

long
#> # A tibble: 6 x 3
#>   practice month   admissions
#>   <chr>    <chr>         <dbl>
#> 1 P81001  2025-04             42
#> 2 P81001  2025-05             51
#> 3 P81001  2025-06             47
#> 4 P81002  2025-04             30
#> 5 P81002  2025-05             28
#> 6 P81002  2025-06             35
```

Now `group_by(month)`, joins, and charts all work — three columns, however many months arrive next quarter. **`pivot_wider()`: tidy → presentation**, for the report table humans expect:

### CASTING BACK TO WIDE

```
long |>
  pivot_wider(names_from = month, values_from = admissions)
```

`values_fill = 0` supplies zeros where combinations are absent (the date-spine idea in reshape form); `values_fn = sum` resolves duplicates if several rows map to one cell — and if you *\*need\** `values_fn` unexpectedly, that's a hint your data has duplicates to investigate first (Lesson 34 instincts). Rule of thumb: **store and analyse long; present wide** — `pivot_wider` is usually the last step before a table lands in a report.

### QUICK CHECK

**A CSV has columns: `la_name`, `m_2404`, `m_2405`, ... `m_2603` (24 month columns of counts). To chart a monthly trend you should first...**

- ☐ `pivot_wider`
- ✓ **`pivot_longer` the month columns into month + count**
- ☐ Delete the months
- ☐ Transpose in Excel

**Why:** Charts and grouped summaries want one row per LA-month. Longer first; the month names become data you can parse into dates.

**QUICK CHECK**

**pivot\_wider needs values\_fn = sum to work. The professional reaction is...**

- ☐ Add it and move on
- ✓ **Pause: multiple rows per cell means duplicates or a missing grouping variable — investigate before aggregating**
- ☐ Use values\_fill instead
- ☐ Switch to pivot\_longer

**Why:** values\_fn silently aggregates collisions. Sometimes correct — but only after you know why collisions exist.

**EXERCISE**

Given long data la\_name, month, deaths for 3 LAs x 4 months (make it with tribble or expand from imagination): produce the wide report table (months as columns, values\_fill = 0), then pivot it back to long and confirm you recover 12 rows.

**SOLUTION · R**

```
long <- tribble(
  ~la_name,    ~month,    ~deaths,
  "Barchester", "2025-04", 8,
  "Barchester", "2025-05", 11,
  "Millford",   "2025-04", 5,
  "Kingsbay",   "2025-06", 7
) # (any similar rows are fine)
wide <- long |>
  pivot_wider(names_from = month, values_from = deaths, values_fill = 0)
back <- wide |>
  pivot_longer(-la_name, names_to = "month", values_to = "deaths")
nrow(back) # LAs x months present in wide
```

**KEY POINTS**

- Tidy (long) for analysis: one variable per column, one observation per row.
- pivot\_longer melts wide imports; pivot\_wider casts presentation tables (values\_fill for zeros).
- Needing values\_fn is a duplicate-data smell — investigate, then aggregate.

**MODULE 4 · R FUNDAMENTALS · LESSON 53 OF 105**

## 53. Strings with stringr: cleaning the text you're given

Health extracts arrive with trailing spaces, inconsistent case, codes embedded in text. **stringr** (loaded with the tidyverse) gives consistent str\_\* functions — vectorised, so they slot straight into mutate:

**THE DAILY STRING TOOLKIT**

```
codes <- c(" e10.9", "J18.9 ", "i21.0", "J45.9")
str_trim(codes)           # strip outer whitespace
str_to_upper(codes)       # case
str_sub(codes, 1, 1)      # slice: first character
str_detect(codes, "^J")   # regex test: starts with J
#> [1] FALSE  TRUE  FALSE  TRUE
str_starts(str_trim(codes), "J") # friendlier for simple prefixes
str_replace(codes, "\\.", "")   # first match; str_replace_all for all
str_length(str_trim(codes))
```

**THE DAILY STRING TOOLKIT**

```
str_c("ICD:", str_trim(codes))      # concatenate (SQL's ||)
```

That "**^J**" is a **regular expression** — a pattern mini-language. You need only a starter kit: **^** start, **\$** end, **.** any character (so a literal dot is **\\.**), **[0-9]** a digit, **+** one-or-more. "**^J[0-9]+**" = starts with J then digits: respiratory-chapter ICD-10 codes. Anything fancier, look it up when needed.

The standardise-then-compare pattern, now in R (Lesson 16's twin):

**DEFENSIVE CLEANING IN A PIPELINE**

```
admissions |>
  mutate(method_clean = str_to_title(str_trim(admission_method))) |>
  count(method_clean)
# "emergency ", "EMERGENCY", "Emergency" -> all "Emergency"
```

And the extraction move that saves hours: `str_extract(text, "[0-9]{6}")` pulls the first 6-digit run (a practice code buried in a free-text field, say). `separate_wider_delim()` splits "Barchester - P81001" style columns into two.

**QUICK CHECK**

**str\_detect(diagnosis, "^J")** returns...

- ☐ The J codes themselves
- ✓ **TRUE/FALSE per element: does it start with J?**
- ☐ The position of J
- ☐ An error on NA

**Why:** `detect` = logical test (perfect inside filter). `str_extract` returns the matched text; `str_which` the positions.

**QUICK CHECK**

**To match a literal full stop in a regex you write...**

- ☐ .
- ☐ 'dot'
- ✓ **\\. (backslash-escaped, doubled in R strings)**
- ☐ [.]. is impossible

**Why:** Unescaped . means "any character". In R source you type two backslashes to send one to the regex engine. ([.] also works.)

**EXERCISE**

Clean a messy specialty vector: `x <- c(" cardiology", "RESPIRATORY ", "Gen Med", "respiratory")` — produce Title Case, trimmed values; count how many are Respiratory; and build labels like "Specialty: Cardiology" with `str_c`.

**SOLUTION · R**

```
x <- c(" cardiology", "RESPIRATORY ", "Gen Med", "respiratory")
clean <- str_to_title(str_trim(x))
clean
#> [1] "Cardiology" "Respiratory" "Gen Med" "Respiratory"
sum(clean == "Respiratory")
#> [1] 2
str_c("Specialty: ", clean)
```



**KEY POINTS**

- `str_trim` + `str_to_upper`/title before comparing; `str_detect` for pattern filters.
- Regex starter kit: `^ $ . [0-9] +` and `\\. for a literal dot` — learn more only as needed.
- `str_extract/separate` pull structure out of free-ish text.

**MODULE 4 • R FUNDAMENTALS • LESSON 54 OF 105**

## 54. Dates with lubridate: UK dates tamed

**lubridate** (in the tidyverse) makes dates almost pleasant. Parsing is named after the order the human wrote: British dates are `dmy()`, ISO are `ymd()`, and each has a `_hm/_hms` sibling for times:

**PARSING EVERY UK FORMAT YOU'LL MEET**

```
dmy("03/11/2025"); dmy("3 November 2025"); dmy("03-11-25")
#> all -> "2025-11-03"
ymd("2025-11-03")           # ISO
dmy_hm("03/11/2025 14:30")  # date-times
```

Once parsed, extraction and arithmetic are civilised:

**WORKING WITH REAL DATES**

```
d <- ymd("2025-11-03")
year(d); month(d); day(d); wday(d, label = TRUE) # components
floor_date(d, "month")                          # 2025-11-01: the month key!
d + days(30); d - months(6)                     # shifting
as.numeric(ymd("2025-12-01") - ymd("2025-11-03")) # gap in days: 28
# Exact age (handles leap years and un-had birthdays properly):
age <- floor(interval(ymd("1958-03-14"), ymd("2025-11-03")) / years(1))
#> 67
```

`floor_date(x, "month")` replaces SQL's `strftime` month keys — group by it for monthly trends. And here is the **financial year function** you'll reuse forever (Lesson 21's logic, R-shaped):

**FY(): FINANCIAL YEAR LABELLER**

```
fy <- function(date) {
  y <- if_else(month(date) >= 4, year(date), year(date) - 1)
  str_c(y, "/", str_sub(y + 1, 3, 4))
}
fy(ymd(c("2025-03-31", "2025-04-01", "2026-02-10")))
#> [1] "2024/25" "2025/26" "2025/26"
```

(You're reading your first home-made function — Lesson 56 teaches the mechanics.) One caution: `Sys.Date()` is today, but pinning "today" into saved outputs makes them irreproducible — prefer explicit snapshot dates in anything published.

**QUICK CHECK**

**A UK column of "31/03/2025" strings becomes real Dates via...**

- ☐ `ymd()`
- ✓ **`dmy()`**
- ☐ `mdy()`
- ☐ `as.numeric()`

**Why:** Day-month-year order = `dmy()`. `mdy()` would happily misread 03/04 dates and corrupt a third of your rows silently — the ambiguity Lesson 10 warned about.

**QUICK CHECK**

**floor\_date(admission\_date, "month") is the R equivalent of which SQL idiom?**

- ☐ julianday differences
- ✓ **strftime('%Y-%m', admission\_date) month keys for grouping**
- ☐ date('now')
- ☐ BETWEEN

**Why:** Both collapse dates to their month for trend grouping — floor\_date returns a proper Date (the 1st), which charts even better.

**EXERCISE**

Parse mixed <- c("14/03/1958", "01/04/2025", "29/02/2024") with dmy; compute each date's financial year with fy() above; and calculate exact age on 2026-03-31 for the first date.

**SOLUTION · R**

```
mixed <- dmy(c("14/03/1958", "01/04/2025", "29/02/2024"))
fy(mixed)
#> [1] "1957/58" "2025/26" "2023/24"
floor(interval(mixed[1], ymd("2026-03-31"))) / years(1)
#> [1] 68
```

**KEY POINTS**

- dmy() for British, ymd() for ISO; parse once, then components/arithmetic are easy.
- floor\_date(x, "month") = month keys; interval()/years(1) = exact ages.
- Keep fy() in your toolkit; every UK analysis wants it eventually.

**MODULE 4 · R FUNDAMENTALS · LESSON 55 OF 105**

## 55. Factors with forcats: categories in the right order

Character vectors sort alphabetically — which is why naive charts order age bands "18-39", "40-64", "5-17", ... and month names Apr, Aug, Dec.... **Factors** are R's categorical type: values restricted to defined **levels**, in an order you control. **forcats** (fct\_\*) manages them:

**AGE BANDS THAT BEHAVE**

```
bands <- c("65-79", "0-4", "18-39", "80+", "5-17", "40-64", "18-39")
band_levels <- c("0-4", "5-17", "18-39", "40-64", "65-79", "80+")
bands_f <- factor(bands, levels = band_levels)
sort(bands_f)          # now sorts by meaning, not alphabet
table(bands_f)         # counts, in order, including zero-count levels
```

Where factors earn their keep:

- **Ordering chart axes.** ggplot (next module) plots factor levels in order — set levels once, every chart obeys.
- **Ordering by value:** fct\_reorder(specialty, n) sorts a bar chart by height; fct\_rev() flips.
- **Lumping small groups:** fct\_lump\_n(ethnicity, n = 4) keeps the 4 commonest and pools the rest into "Other" — tidy tables and a first step toward small-number safety.
- **Recoding labels:** fct\_recode(sex, Female = "F", Male = "M") (new = old).
- **Surfacing the unexpected:** a typo'd "0-4 " becomes its own level and shows up instantly in table() — data quality made visible.

The classic factor trap: a *\*numeric\** column accidentally read as factor, where `as.numeric()` returns the level *\*codes\** (1,2,3...) not the values. The safe incantation is `as.numeric(as.character(x))` — you'll meet this in older datasets and interview questions alike.

**QUICK CHECK**

**Your bar chart shows age bands ordered 0-4, 18-39, 40-64, 5-17, 65-79, 80+. The fix is...**

- ☐ Sort the data frame with `arrange`
- ✓ **Make `age_band` a factor with levels in the correct order**
- ☐ Rename 5-17 to 05-17
- ☐ Convert the bands to numbers

**Why:** Alphabetical "5" > "4" > "1" ordering is the giveaway of an unfactored category. (Renaming to 05-17 also works but changes your labels; factors fix order without touching text.)

**QUICK CHECK**

**x is a factor with levels "3","7","12"; the value "7" under `as.numeric(x)` may return...**

- ☐ 7
- ✓ **2 — the internal level code, not the value**
- ☐ NA always
- ☐ "7"

**Why:** Factors store codes pointing at level labels. `as.numeric(as.character(x))` recovers true values — a classic gotcha.

**EXERCISE**

Given `ethnicity <- c("White","Asian","White","Black","Mixed","White","Other","Asian","Chinese heritage","White")`: make it a factor, lump to the top 3 plus "Other" with `fct_lump_n`, and produce an ordered count table with `fct_infreq` (most common first).

**SOLUTION · R**

```
ethnicity <- c("White","Asian","White","Black","Mixed","White",
              "Other","Asian","Chinese heritage","White")
eth_f <- ethnicity |>
  factor() |>
  fct_lump_n(n = 3) |>
  fct_infreq()
table(eth_f)
#> White (4), Asian (2), then remaining pooled into Other
```

**KEY POINTS**

- Factors = categories with controlled, ordered levels; charts and tables obey them.
- `fct_reorder` (by value), `fct_infreq` (by frequency), `fct_lump_n` (pool small groups), `fct_recode` (relabel).
- Numeric-as-factor needs `as.numeric(as.character(x))` — the level-code trap.

**MODULE 4 · R FUNDAMENTALS · LESSON 56 OF 105**

## 56. Writing your own functions

When you copy-paste a block the third time, stop: **write a function**. Functions remove repetition, concentrate bugs in one fixable place, and turn your hard-won logic (`fy()!`) into one-word tools.

**ANATOMY OF A FUNCTION**

```
rate_per_1000 <- function(events, population, dp = 1) {
  round(1000 * events / population, dp)
}
rate_per_1000(120, 40000)
#> [1] 3
rate_per_1000(120, 40000, dp = 2)
#> [1] 3
rate_per_1000(c(120, 80, 95), c(40000, 26000, 31000)) # vectorised for free
#> [1] 3.0 3.1 3.1
```

Pieces: a name (verb-ish, snake\_case), arguments (with defaults where sensible, like `dp = 1`), a body, and the **last expression is the return value** (explicit `return()` exists but idiomatic R rarely needs it). Because the body is vector code, your function inherits vectorisation.

Add guard rails for functions others will use — fail loudly and early:

**DEFENSIVE VERSION**

```
rate_per_1000 <- function(events, population, dp = 1) {
  if (any(population <= 0, na.rm = TRUE)) {
    stop("population must be positive - check your denominator join")
  }
  round(1000 * events / population, dp)
}
```

`if () { } else { }` here is **scalar** control flow — it decides what the \*function\* does once, unlike vectorised `if_else()/case_when()` which compute per element. New writers mixing those up is the second-commonest R confusion (after NA). Keep reusable functions in their own file — `R/functions.R` — and pull them into any script with `source("R/functions.R")`: your personal package in embryo.

**QUICK CHECK****In R, a function returns...**

- ☐ Whatever `return()` names, which is mandatory
- ✓ **The value of the last evaluated expression (`return()` optional)**
- ☐ Nothing unless printed
- ☐ Only single numbers

**Why:** Last expression wins. Explicit `return()` is for early exits; everyday functions just end with their answer.

**QUICK CHECK****Inside a function deciding whether to `stop()` on bad input, you use `if/else` rather than `if_else` because...**

- ☐ They are identical
- ✓ **`if/else` makes one scalar decision about flow; `if_else` is a vectorised value-chooser per element**
- ☐ `if_else` is deprecated
- ☐ `stop()` only works with `if_else`

**Why:** Control flow (do this or that, once) vs data transformation (a value for every row) — different tools for different jobs.

**EXERCISE**

Write `suppress_small(n, threshold = 5)` returning "<5" (as character) when  $0 < n < \text{threshold}$ , otherwise the number as character; make it vectorised with `if_else/case_when`; test on `c(0, 3, 5, 17)`. (This is a real disclosure-control helper — Lesson 67 uses the idea in anger.)

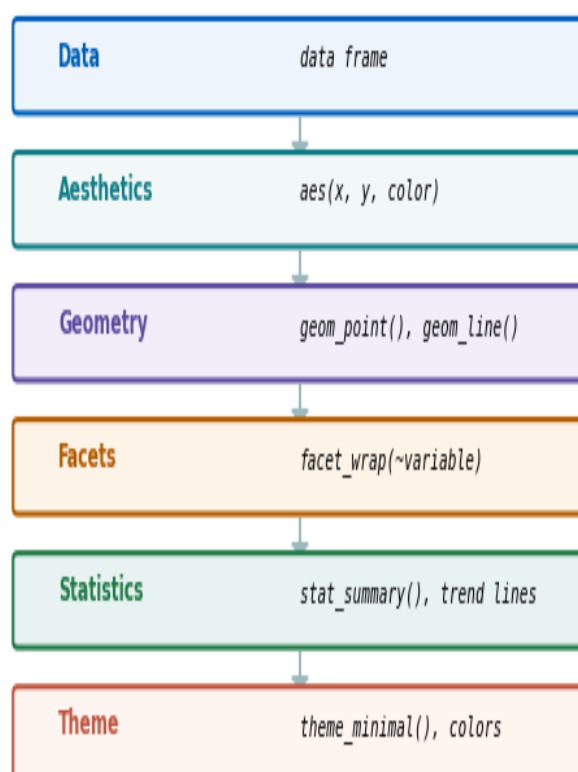
**SOLUTION · R**

```
suppress_small <- function(n, threshold = 5) {
  case_when(
    n > 0 & n < threshold ~ "<5",
    .default = as.character(n)
  )
}
suppress_small(c(0, 3, 5, 17))
#> [1] "0" "<5" "5" "17"
```

**KEY POINTS**

- Third copy-paste = write a function; defaults make it friendly, vector bodies make it fast.
- Last expression returns; `stop()` early on bad input with a helpful message.
- Scalar if/else steers functions; `if_else/case_when` transform columns. Keep functions in `R/functions.R` and `source()` them.

## ggplot2: layered grammar of graphics



## MODULE 4 • R FUNDAMENTALS • LESSON 57 OF 105

## 57. Iteration: for loops and purrr's map

Vectorisation and grouped dplyr cover most repetition — reach for explicit iteration when the unit of work is bigger than a row: *\*per file\**, *\*per organisation\**, *\*per model\**. R offers two styles.

**The for loop** — universal, readable:

**READING A FOLDER OF MONTHLY CSVS**

```
files <- list.files("data/monthly", pattern = "\\*.csv$", full.names = TRUE)
results <- list()
for (f in files) {
  results[[f]] <- read_csv(f, show_col_types = FALSE)
}
all_months <- bind_rows(results, .id = "source_file")
```

**purrr::map()** — the tidyverse's functional style: apply a function to each element, collect the results. The same task in three lines:

**MAP: APPLY AND COMBINE**

```
all_months <- map(files, read_csv, show_col_types = FALSE) |>
  bind_rows()
# Typed variants promise the output shape:
map_dbl(list(a = 1:5, b = 10:12), mean) # always numbers back
#>      a      b
#> 3.0 11.0
# Anonymous functions for custom steps:
map_dbl(files, \(f) nrow(read_csv(f, show_col_types = FALSE)))
```

`\(x)` is shorthand for `function(x)`. Which style? **Genuinely equivalent** for most analyst work — loops read plainly and debug easily; map is compact and composes with pipes (and `map_dfr`-style patterns dominate NHS-R code you'll inherit). Learn to *\*read\** both, write whichever is clearer. Two disciplines either way: pre-decide how results combine (`bind_rows`, a named list), and make each iteration **independent** — no step relying on the previous loop's leftovers, which is what makes Module 7's parallel/automated pipelines possible.

**QUICK CHECK**

**map\_dbl(x, mean) differs from map(x, mean) how?**

- ☐ It is faster
- ☒ **It guarantees a numeric vector back (erroring if not), instead of a list**
- ☐ It removes NAs
- ☐ It sorts the output

**Why:** Typed `map_*` variants are contracts about output shape — they fail loudly instead of returning a surprise list.

**EXERCISE**

You have three LA names: `las <- c("Barchester", "Millford", "Kingsbay")`. Using `map` (or a `for` loop), produce a named list where each element is a one-row tibble: `la_name` plus a placeholder `report_date = Sys.Date()`; then `bind_rows` it. (Structure practice for Lesson 66's parameterised reports.)

**SOLUTION • R**

```
las <- c("Barchester", "Millford", "Kingsbay")
```

**SOLUTION · R**

```
reports <- map(las, \(la) tibble(la_name = la, report_date = Sys.Date())) |>
  set_names(las) |>
  bind_rows(.id = "id")
reports
#> 3 rows, one per LA
```

**QUICK CHECK**

**The best reason each loop iteration should be independent is...**

- ☐ Loops error otherwise
- ✓ **Independent steps can be re-run, re-ordered, parallelised and automated safely**
- ☐ It uses less memory
- ☐ purrr requires it

**Why:** Hidden state between iterations breaks re-runs and scheduling. Independence is what lets Module 7 automate your work.

**KEY POINTS**

- Vectorise first; iterate when the unit is a file/org/model.
- for loops and purrr::map are equally professional — read both, write the clearer one.
- Typed map\_\* variants enforce output shape; keep iterations independent and combine deliberately.

**MODULE 5**

# Module 5 — Visualisation & reporting in R

**In this module**

- 58** ggplot2: the grammar of graphics
- 59** Time series and epi curves
- 60** Charts people can actually read: style and accessibility
- 61** Small multiples: facets
- 62** Distributions: histograms, boxplots and honest summaries
- 63** Maps: small-area geography with sf
- 64** Tables for reports: gt and kable
- 65** Quarto: reports that rebuild themselves
- 66** Parameterised reports: one template, forty outputs
- 67** Safe outputs: disclosure control before anything leaves

## MODULE 5 • VISUALISATION &amp; REPORTING IN R • LESSON 58 OF 105

## 58. ggplot2: the grammar of graphics

**ggplot2** builds charts from three declared parts: **data**, **aesthetics** (`aes()` — which columns map to x, y, colour...), and **geoms** (the marks: bars, lines, points), added with `+`. Once the grammar clicks, every chart is the same sentence with different words.

### FIRST CHART: ADMISSIONS BY SPECIALTY

```
library(tidyverse)
spec_counts <- admissions |>
  count(specialty, name = "n")
ggplot(spec_counts, aes(x = n, y = fct_reorder(specialty, n))) +
  geom_col()
```

Read it: *\*take spec\_counts; map count to x and specialty (ordered by count — Lesson 55 paying off) to y; draw columns.\** Horizontal bars because specialty names are long — a small choice that makes NHS charts instantly more readable.

Two column-geoms to keep straight: `geom_col()` draws values you've already computed (paired with `count()/summarise` — the analyst default, because you can check the numbers first); `geom_bar()` counts raw rows itself. Add mappings and the chart grows:

### MAPPING A THIRD VARIABLE

```
admissions |>
  count(specialty, admission_method) |>
  ggplot(aes(x = n, y = fct_reorder(specialty, n), fill = admission_method)) +
  geom_col(position = "dodge")      # side-by-side; "stack" is the default
```

Inside `aes()` = mapped to data (`fill = admission_method` gives a legend); outside = fixed (`geom_col(fill = "steelblue")` paints everything). Confusing those two is the classic first-week ggplot error. Save any plot with `ggsave("outputs/specialty.png", width = 8, height = 5, dpi = 300)` — real sizes for real reports.

#### QUICK CHECK

**geom\_col(fill = "red") vs aes(fill = method): the difference is...**

- ☐ No difference
- ✓ **Outside aes = one fixed colour; inside aes = colour varies by data, with a legend**
- ☐ aes is for bars only
- ☐ fill must always be in aes

**Why:** Mapping (data-driven, legended) vs setting (constant). If you "map" to a constant string inside aes you get a mysterious one-item legend — the tell-tale of mixing them up.

#### QUICK CHECK

**You've summarised admissions per month into a tibble with columns month, n. Which geom draws the bars?**

- ☐ `geom_bar()`, which will count rows
- ✓ **`geom_col()`, which draws the n values you computed**
- ☐ `geom_point()`
- ☐ `geom_histogram()`

**Why:** Pre-computed values -> `geom_col`. `geom_bar` on this data would count one row per month and draw all bars height 1.



**EXERCISE**

Chart vaccinations by vaccine: count rows per vaccine, then draw horizontal bars ordered by count, filled a single fixed colour of your choice, and save it to `outputs/vaccine_counts.png` at 7x4 inches.

**SOLUTION · R**

```
vaccinations |>
  count(vaccine) |>
  ggplot(aes(x = n, y = fct_reorder(vaccine, n))) +
  geom_col(fill = "#005EB8") +
  labs(x = "Doses recorded", y = NULL)
ggsave("outputs/vaccine_counts.png", width = 7, height = 4, dpi = 300)
```

**KEY POINTS**

- `ggplot(data, aes(mappings)) + geom_*()` — data, mappings, marks.
- `geom_col` for computed values (analyst default); horizontal bars for long labels; `fct_reorder` for order.
- Inside `aes` = varies with data; outside = fixed. `ggsave` with explicit sizes.

**MODULE 5 · VISUALISATION & REPORTING IN R · LESSON 59 OF 105**

## 59. Time series and epi curves

Public health lives on time: monthly activity, weekly cases, winter pressure. The pattern — summarise to one row per period (a real Date, via `floor_date`), then `geom_line()`:

**MONTHLY EMERGENCY ADMISSIONS**

```
monthly <- admissions |>
  filter(admission_method == "Emergency") |>
  mutate(month = floor_date(admission_date, "month")) |>
  count(month)
ggplot(monthly, aes(month, n)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1.5) +
  scale_x_date(date_breaks = "3 months", date_labels = "%b %Y") +
  labs(x = NULL, y = "Emergency admissions")
```

Because `month` is a Date, `scale_x_date()` gives sensible axis labels (`%b %Y` → "Apr 2025"). Points on top of the line make individual months readable. **The zero-month rule from Lesson 32 applies here too** — R's spine idiom is `complete()`:

**FILLING MISSING PERIODS WITH TRUE ZEROS**

```
monthly_full <- monthly |>
  complete(month = seq(min(month), max(month), by = "month"),
    fill = list(n = 0))
```

An **epi curve** — the outbreak's signature chart — is the same idea with bars per day/week of symptom onset: `count(onset_week) |> ggplot(aes(onset_week, n)) + geom_col(width = 6)`. Bars (not a line) because each period's count is a discrete tally, and gaps should look like gaps. Add a smoothed layer for noisy dailies with the moving average from Lesson 32's logic (`zoo::rollmean` or a 7-day summarise) rather than `geom_smooth()`'s statistical fit — surveillance wants the actual smoothed data, not a model's opinion. Finally: **start count axes at zero** (`expand_limits(y = 0)`); truncated axes exaggerate wiggles and will be (rightly) challenged in QA.

**QUICK CHECK**

**For an outbreak's epidemic curve, bars beat a line because...**

- ☐ Bars are prettier
- ✓ **Each period's case count is a discrete tally, and missing periods should show as gaps, not be bridged**
- ☐ Lines cannot show dates
- ☐ ggplot has no geom\_line

**Why:** A line implies continuity and interpolates across gaps; an epi curve is a histogram in time.

**QUICK CHECK**

**Your monthly trend line dips sharply in the final month. Before narrating a decline, check...**

- ☐ The chart colours
- ✓ **Whether the last period is incomplete (data lag) — the Lesson 39 cliff, now in chart form**
- ☐ The legend position
- ☐ The y-axis label

**Why:** Partial final periods create fake drops. Standard practice: exclude or visually flag the incomplete period.

**EXERCISE**

Draw a monthly line chart of Flu vaccinations: floor\_date month, count, complete() missing months to zero, line + points, y-axis from zero, labelled "Flu vaccinations per month".

**SOLUTION · R**

```
flu_monthly <- vaccinations |>
  filter(vaccine == "Flu") |>
  mutate(month = floor_date(date_given, "month")) |>
  count(month) |>
  complete(month = seq(min(month), max(month), by = "month"),
           fill = list(n = 0))
ggplot(flu_monthly, aes(month, n)) +
  geom_line() +
  geom_point() +
  expand_limits(y = 0) +
  scale_x_date(date_breaks = "3 months", date_labels = "%b %y") +
  labs(x = NULL, y = "Doses", title = "Flu vaccinations per month")
```

**KEY POINTS**

- floor\_date -> count -> geom\_line/geom\_col is the trend pattern; scale\_x\_date for labels.
- complete() supplies true zeros; flag or drop incomplete final periods.
- Epi curves are bars; smooth with moving averages, not model fits; count axes start at zero.

**MODULE 5 · VISUALISATION & REPORTING IN R · LESSON 60 OF 105**

## 60. Charts people can actually read: style and accessibility

A chart in an NHS board paper or UKHSA report has one job: transmit a finding accurately to a busy, mixed-ability audience. House rules that survive contact with communications teams:

**Say the finding.** Titles state the message ("Emergency admissions rose 12% over winter"), not the dataset ("Admissions by month"). `labs()` carries title, subtitle (the how/what), caption (source + date), and axis names in plain English with units.

**Declutter.** `theme_minimal()` strips the grey; then remove what remains unearned — heavy gridlines, redundant legends (`show.legend = FALSE` when labels sit on the chart). Direct-label lines where possible; legends force eye-travel.

**Colour is information, and 1 in 12 men are colour-vision deficient.** Red-vs-green encodings fail exactly the audience RAG ratings target. Use colour-blind-safe palettes: `scale_fill_viridis_d()` ships with ggplot; the Okabe-Ito palette is the accessibility standard; and check anything important with `colorblindr` or a simulator.

#### A BOARD-READY MAKE-OVER

```
monthly |>
  ggplot(aes(month, n)) +
  geom_line(colour = "#005EB8", linewidth = 0.9) +
  expand_limits(y = 0) +
  labs(
    title = "Emergency admissions climbed through winter 2025/26",
    subtitle = "Monthly emergency admissions, Barsetshire residents",
    caption = "Source: synthetic SUS-style extract | Produced 2026-04-02",
    x = NULL, y = "Admissions"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"))
```

Consistency scales: define your theme + palette once (a function! Lesson 56) and apply everywhere — that's how UKHSA/NHS teams get their recognisable look, and packages like `NHSRtheme` exist to hand you NHS-identity colours. Finally, **alt text**: anywhere a chart is published digitally, one sentence describing the finding for screen-reader users (Quarto: `fig-alt`). Accessibility in UK public bodies isn't optional — public sector websites are legally required to meet accessibility standards.

#### QUICK CHECK

**The strongest chart title for a board paper is...**

- ☐ Figure 3
- ☐ Admissions data 2024-2026
- ☒ **Emergency admissions rose 12% over winter, driven by respiratory illness**
- ☐ Chart produced in R

**Why:** Titles carry the finding; subtitles carry the description. Busy readers may only read titles — make them work.

#### QUICK CHECK

**Why avoid a plain red/green "worse/better" encoding?**

- ☐ Red is alarming
- ☒ **Red-green confusion is the most common colour-vision deficiency — the encoding fails ~8% of men**
- ☐ Green prints badly
- ☐ ggplot lacks red

**Why:** Pair colour with another cue (shape, label, position) and choose CVD-safe palettes (viridis, Okabe-Ito).

**EXERCISE**

Take your Lesson 59 flu chart and make it publication-ready: finding-led title, subtitle naming population and period, caption with source and date, theme\_minimal, NHS blue line, y from zero. Write the full code.

**SOLUTION · R**

```
ggplot(flu_monthly, aes(month, n)) +
  geom_line(colour = "#005EB8", linewidth = 0.9) +
  geom_point(colour = "#005EB8", size = 1.4) +
  expand_limits(y = 0) +
  scale_x_date(date_breaks = "3 months", date_labels = "%b %y") +
  labs(
    title = "Flu vaccination activity peaks each autumn",
    subtitle = "Monthly recorded flu doses, Bassetshire synthetic data",
    caption = "Source: course playground extract | Produced 2026",
    x = NULL, y = "Doses"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"))
```

**KEY POINTS**

- Titles state findings; subtitles describe; captions cite source and date.
- theme\_minimal + restraint; direct labels over legends where possible.
- Colour-blind-safe palettes (viridis/Okabe-Ito), redundant cues, alt text: accessibility is a legal duty in UK public bodies.

**MODULE 5 · VISUALISATION & REPORTING IN R · LESSON 61 OF 105**

## 61. Small multiples: facets

"Show me that, but for every LA / practice / age band" — the request that defines organisational analytics. Cramming ten lines onto one chart makes spaghetti; **facets** repeat a small clear chart per group on shared axes:

**ONE PANEL PER LOCAL AUTHORITY**

```
adm_geo <- admissions |>
  left_join(patients, by = "patient_id") |>
  left_join(lsoa_imd, by = "lsoa_code") |>
  mutate(month = floor_date(admission_date, "month")) |>
  count(la_name, month)
ggplot(adm_geo, aes(month, n)) +
  geom_line(colour = "#0E7C86") +
  facet_wrap(~ la_name) +
  expand_limits(y = 0) +
  labs(x = NULL, y = "Admissions")
```

facet\_wrap(~ group) wraps panels into a grid (ncol = to control); facet\_grid(rows ~ cols) crosses two dimensions (age band × sex — the demographer's grid). The decision that changes the message: **scales**.

- Shared axes (default) compare *sizes* — Barchester dwarfs Kingsbay.
- scales = "free\_y" compares *shapes* — every panel's own trend fills its frame, but sizes are no longer comparable, and readers WILL misread it unless the subtitle says so.

Ordering panels is a factor job (`fct_reorder(la_name, n, sum)` puts busiest first); tiny-count panels wobble randomly (Lesson 23's small-groups caution goes visual — consider lumping or annotating). Facets are also the honest alternative to dual-axis charts: two measures, two panels, no scale trickery.

**QUICK CHECK**

**`facet_wrap(~ la_name, scales = "free_y")` lets each panel set its own y-axis. The cost is...**

- ☐ Slower rendering
- ✓ **Panel heights are no longer comparable — readers can't judge relative size across areas**
- ☐ Facets lose their titles
- ☐ Lines become bars

**Why:** Free scales trade comparability for per-panel detail. Fine for shape-of-trend questions; label it clearly.

**QUICK CHECK**

**Sex (2 levels) by age band (6 levels) as a grid of panels calls for...**

- ☐ `facet_wrap(~ sex)`
- ✓ **`facet_grid(sex ~ age_band)`**
- ☐ Twelve separate scripts
- ☐ A dual-axis chart

**Why:** `facet_grid` crosses two dimensions into rows x columns — 12 aligned panels from one line of code.

**EXERCISE**

Facet monthly admissions by specialty (one panel each), shared y-axis, panels ordered by total volume (hint: `mutate(specialty = fct_reorder(specialty, n, .fun = sum, .desc = TRUE))` before plotting).

**SOLUTION · R**

```
adm_spec <- admissions |>
  mutate(month = floor_date(admission_date, "month")) |>
  count(specialty, month) |>
  mutate(specialty = fct_reorder(specialty, n, .fun = sum, .desc = TRUE))
ggplot(adm_spec, aes(month, n)) +
  geom_line() +
  facet_wrap(~ specialty) +
  expand_limits(y = 0) +
  labs(x = NULL, y = "Admissions")
```

**KEY POINTS**

- `facet_wrap(~ g)` for one grouping; `facet_grid(a ~ b)` for crossed panels.
- Shared axes compare size; free scales compare shape — and must be labelled.
- Order panels with factors; distrust and annotate tiny-count panels.

## MODULE 5 • VISUALISATION &amp; REPORTING IN R • LESSON 62 OF 105

## 62. Distributions: histograms, boxplots and honest summaries

Before summarising a numeric column, **look at its shape** — health data is riotously non-normal, and the shape decides which summary is honest. Length of stay (Lesson 36) is the canonical example:

### SEEING THE LOS DISTRIBUTION

```
los <- admissions |>
  filter(!is.na(discharge_date)) |>
  mutate(los_days = as.numeric(discharge_date - admission_date))
ggplot(los, aes(los_days)) +
  geom_histogram(binwidth = 1, fill = "#0E7C86", colour = "white") +
  labs(x = "Length of stay (days)", y = "Spells")
```

A tall spike at 0–2 days and a long right tail: **right-skewed**. Consequences: the mean sits above the median (dragged by the tail); "average stay: 4.1 days" describes almost nobody; the tail *is* the capacity story. `binwidth` is an analytical choice — try 1 and 7; different bins, different visible stories.

**Boxplots compare distributions across groups** — median line, interquartile box, whiskers, outlier dots:

### LOS BY SPECIALTY AT A GLANCE

```
ggplot(los, aes(x = los_days, y = fct_reorder(specialty, los_days, median))) +
  geom_boxplot(fill = "#E4F0F1") +
  labs(x = "Length of stay (days)", y = NULL)
```

Ordering by median makes the comparison instant. For small groups where a box over-summarises, show the actual points (`geom_jitter(alpha = .3)`) or a violin. Reporting rule that follows: **skewed measures get median + IQR** (and percentiles like the 95th for tails); reserve mean  $\pm$  SD for genuinely symmetric things. When a director asks why your "average" differs from someone's dashboard, the answer is usually mean-vs-median on a skewed measure — now you can draw the picture that explains it.

#### QUICK CHECK

**For right-skewed LOS data, the mean is typically...**

- ☐ Below the median
- ☐ Equal to the median
- ☒ **Above the median, pulled by the long tail of long stays**
- ☐ Undefined

**Why:** Tail values drag the mean upward. Median = the typical patient; mean = accounting for total bed-days. Different questions, different statistics.

#### QUICK CHECK

**A boxplot's box spans...**

- ☐ Minimum to maximum
- ☒ **The interquartile range: 25th to 75th percentile, with the median line inside**
- ☐ Mean  $\pm$  SD
- ☐ The 95% confidence interval

**Why:** The middle half of the data. Whiskers extend further; dots beyond are flagged as outliers worth a look.

**EXERCISE**

Explore Systolic BP values: histogram with binwidth 5; then boxplots of value by patient sex (join to patients), ordered by median. In one sentence: is mean or median the better headline summary here, and why?

**SOLUTION · R**

```
sbp <- lab_results |>
  filter(test == "Systolic BP") |>
  left_join(patients, by = "patient_id")
ggplot(sbp, aes(value)) +
  geom_histogram(binwidth = 5, fill = "#0E7C86", colour = "white")
ggplot(sbp, aes(value, sex)) +
  geom_boxplot()
# BP is roughly symmetric with mild right tail: mean and median will be close,
# so either is defensible - but check the picture first; that's the lesson.
```

**KEY POINTS**

- Histogram first: shape decides the summary. Skew is the norm in health data.
- Skewed -> median + IQR/percentiles; symmetric -> mean ± SD.
- Boxplots (ordered by median) compare groups; jitter/violin when boxes hide too much.

**MODULE 5 · VISUALISATION & REPORTING IN R · LESSON 63 OF 105**

## 63. Maps: small-area geography with sf

Public health findings are spatial — deprivation clusters, uptake gaps, outbreak spread — and the UK has superb open geography. The R standard is **sf** (simple features): boundary files become data frames with a geometry column, so every dplyr verb still works, and `geom_sf()` draws them.

The workflow: get boundaries from the **ONS Open Geography Portal** ([geoportal.statistics.gov.uk](https://geoportal.statistics.gov.uk) — LSOA, MSOA, LA, ICB boundaries, open licence); join your indicator by area code; map:

**THE CHOROPLETH PATTERN**

```
library(sf)
# 1. Boundaries (downloaded GeoJSON/shapefile from ONS Geoportal)
lsoa_shapes <- st_read("data/lsoa_boundaries.geojson")
# 2. Your indicator by LSOA code (e.g. emergency admission rate)
rates <- admissions |>
  left_join(patients, by = "patient_id") |>
  count(lsoa_code, name = "admissions")
# 3. Join and map
lsoa_shapes |>
  left_join(rates, by = c("LSOA21CD" = "lsoa_code")) |>
  ggplot() +
  geom_sf(aes(fill = admissions), colour = NA) +
  scale_fill_viridis_c(na.value = "grey90") +
  theme_void() +
  labs(fill = "Admissions")
```

(Your playground's synthetic LSOAs have no real boundaries, so this lesson is pattern-study; the code runs verbatim on real ONS files.) Craft notes: **join direction matters** — shapes on the left keeps every area, so data gaps show as grey rather than holes; **viridis** for continuous fills (CVD-safe, prints well); **counts mislead on maps** even more than usual, because area population varies — map `*rates*`, or use quantile bins (`cut_number(rate, 5)`) so one extreme area doesn't flatten the palette;

**theme\_void()** drops axes (lat/long ticks help nobody). Interactive maps for exploration: `leaflet` or `tmap` — but static + clear beats interactive + cluttered in most reports.

**QUICK CHECK**

**For a fair map of emergency admissions across LSOAs, you should map...**

- ☐ Raw counts, coloured red
- ✓ **Rates (or quantile-binned rates), since LSOA populations differ**
- ☐ Alphabetical area codes
- ☐ Only the largest LSOA

**Why:** *Counts track population size; rates reveal the pattern you actually care about. (Age-standardised rates - Lesson 70 - are fairer still.)*

**QUICK CHECK**

**shapes |> left\_join(data) rather than data |> left\_join(shapes) is preferred because...**

- ☐ It's faster
- ✓ **Every boundary is kept, so areas with missing data appear grey instead of vanishing from the map**
- ☐ Joins only work that way
- ☐ It sorts the legend

**Why:** *A map with silent holes misleads; grey "no data" areas tell the truth.*

**EXERCISE**

No live boundaries here, so write the code you WOULD run: map crude admission rates per 1,000 by LA using an ONS LA boundary file (`lad_boundaries.geojson`, key `LAD24NM` matching `la_name`), quantile-binned into 5 groups with `viridis`.

**SOLUTION · R**

```
library(sf)
la_rates <- admissions |>
  left_join(patients, by = "patient_id") |>
  left_join(lsoa_imd, by = "lsoa_code") |>
  count(la_name, name = "admissions") |>
  left_join(la_population |> summarise(pop = sum(population), .by = la_name),
    by = "la_name") |>
  mutate(rate = 1000 * admissions / pop,
    rate_band = cut_number(rate, 5))
st_read("data/lad_boundaries.geojson") |>
  left_join(la_rates, by = c("LAD24NM" = "la_name")) |>
  ggplot() +
  geom_sf(aes(fill = rate_band), colour = "white", linewidth = 0.2) +
  scale_fill_viridis_d(na.value = "grey90") +
  theme_void() +
  labs(fill = "Admissions per 1,000")
```

**KEY POINTS**

- `sf` makes boundaries data frames; ONS Open Geography Portal supplies them freely.
- Shapes-left joins keep every area; map rates (often binned), never bare counts.
- `viridis` fills, `theme_void`, grey for no-data: the credible-choropleth kit.



## MODULE 5 • VISUALISATION &amp; REPORTING IN R • LESSON 64 OF 105

## 64. Tables for reports: gt and kable

Half your outputs will be tables, and a raw data-frame printout is not a deliverable. Two tools cover the territory: `knitr::kable()` for quick clean tables inside reports, and **gt** for publication-grade ones.

### FROM SUMMARY TO FORMATTED TABLE

```
library(gt)
summary_tbl <- admissions |>
  mutate(los = as.numeric(discharge_date - admission_date)) |>
  summarise(
    Spells      = n(),
    Patients    = n_distinct(patient_id),
    `Mean LOS`  = mean(los, na.rm = TRUE),
    .by = specialty
  ) |>
  arrange(desc(Spells))
summary_tbl |>
  gt() |>
  tab_header(
    title      = "Inpatient activity by specialty",
    subtitle   = "Barsetshire synthetic data, FY 2024/25-2025/26"
  ) |>
  fmt_number(columns = `Mean LOS`, decimals = 1) |>
  fmt_number(columns = c(Spells, Patients), decimals = 0, sep_mark = ",") |>
  tab_source_note("Source: course playground extract")
```

The pipeline mirrors ggplot's philosophy: data first, then layered presentation — headers, number formats (thousands separators! decimals!), source notes. More gt verbs as needed: `cols_label()` to rename, `sub_missing()` to print em-dashes for NA, `tab_style()` to bold a total row, `gtsave("outputs/table1.png")` (or `.docx/.html`) to export.

Table craft, independent of tool: right-align numbers, consistent decimals per column (rates to 1 dp, counts to 0), order rows meaningfully (by size or an agreed sequence, never accidental), keep to the columns the reader needs, and put the units in the header not every cell. And the health-specific one: **this is where suppression happens** — your Lesson 56 `suppress_small()` runs on the data *\*before\** it reaches gt, so the pretty table is also a safe table (Lesson 67 completes that story).

#### QUICK CHECK

**`fmt_number(columns = Spells, sep_mark = ",", decimals = 0)` exists because...**

- ☐ gt requires it
- ✓ **12345 reads as 12,345 — formatting is part of accurate communication**
- ☐ Numbers must be text in tables
- ☐ It sorts the column

**Why:** *Thousands separators, sensible decimals and alignment are the difference between a printout and a professional table.*

**QUICK CHECK****Small-number suppression belongs...**

- ☐ In gt's styling functions
- ✓ **In the data step before the table is built (e.g. a `suppress_small` mutate)**
- ☐ In the reader's judgement
- ☐ Nowhere in tables

**Why:** *Suppress in data, then format: the safe value flows to every output. Styling can't be trusted to hide what the data still contains.*

**EXERCISE**

Build a gt table of vaccinations: doses and distinct children by vaccine and dose\_number, with a header, comma-separated counts, and a source note. (Optional: gtsave it.)

**SOLUTION · R**

```
vaccinations |>
  summarise(Doses = n(),
            Children = n_distinct(patient_id),
            .by = c(vaccine, dose_number)) |>
  arrange(vaccine, dose_number) |>
  gt() |>
  tab_header(title = "Recorded vaccinations by product and dose") |>
  fmt_number(columns = c(Doses, Children), decimals = 0, sep_mark = ",") |>
  tab_source_note("Source: synthetic course data")
```

**KEY POINTS**

- kable for quick report tables; gt for polished ones — data, then layered formatting.
- Right-aligned numbers, fixed decimals, separators, ordered rows, units in headers.
- Suppression is a data step before formatting — safe first, pretty second.

**MODULE 5 · VISUALISATION & REPORTING IN R · LESSON 65 OF 105**

## 65. Quarto: reports that rebuild themselves

The reproducibility endgame: a document where narrative, code, results and charts live together, and one command regenerates everything when data updates. That's **Quarto** (`.qmd` — successor to R Markdown `.Rmd`; both ship with RStudio, and you'll meet plenty of `.Rmd` in the wild).

A `.qmd` has three ingredients: a **YAML header** (metadata + output format), **markdown prose**, and **code chunks**:

**REPORT.QMD - A MINIMAL SKELETON**

```
---
title: "Monthly activity report"
author: "Analytics team"
date: 2026-04-02
format: html
execute:
  echo: false      # readers see results, not code
  warning: false
---
## Emergency admissions
Admissions data refreshed to end of March.
```

## REPORT.QMD - A MINIMAL SKELETON

```

``{r}
#| label: setup
library(tidyverse)
admissions <- read_csv("data/admissions.csv")
``

``{r}
#| label: fig-trend
#| fig-cap: "Monthly emergency admissions"
#| fig-alt: "Line chart showing emergency admissions rising through winter"
admissions |>
  filter(admission_method == "Emergency") |>
  count(month = floor_date(admission_date, "month")) |>
  ggplot(aes(month, n)) + geom_line() + expand_limits(y = 0)
``

Total admissions this period: `r nrow(admissions)`.
```

Press **Render** and Quarto runs every chunk top-to-bottom **in a fresh session** and produces the output — `format`: `html` for rich shareable pages, `docx` for the Word documents NHS workflows still expect, `pdf`, or `revealjs` slides. The fresh-session rule is a feature: if it renders, it's reproducible; hidden console state can't sneak in.

The inline ``r nrow(admissions)`` is the quiet superpower — numbers in your prose that update with the data, ending the "text says 412, table says 418" class of error forever. Chunk options (`#|` lines) control each block: `echo` (show code?), `fig-cap`, `fig-alt` (Lesson 60's accessibility), `label`` for cross-references. Next month's report = drop in new data, Render, proof-read. Minutes, not days.

## QUICK CHECK

## Quarto renders in a fresh R session so that...

- ☐ Rendering is faster
- ✓ **The document depends only on its own code and files — guaranteeing reproducibility**
- ☐ Plots look sharper
- ☐ YAML is optional

**Why:** Anything relying on leftover console objects fails at render — surfacing irreproducibility immediately instead of on a colleague's machine.

## QUICK CHECK

Inline code like `r round(rate, 1)` in your prose means...

- ☐ The reader sees the code
- ✓ **The computed value is printed in the sentence and updates whenever the report re-renders**
- ☐ An error in Word output
- ☐ A footnote

**Why:** Narrative numbers bound to data: the end of copy-paste drift between text and tables.

## EXERCISE

Create `monthly_report.qmd`: YAML with title and format `html` + `echo: false`; a setup chunk loading `tidyverse` and one playground CSV; one finding-titled chart chunk with `fig-cap` and `fig-alt`; and a sentence using inline code for the row count. Render it.

**SOLUTION · TEXT**

Success looks like: an HTML file opens with your chart, no visible code, a caption, and a sentence such as "This report covers 900 admissions." where 900 was computed, not typed. If render fails, read the error's chunk label - it names exactly where to look.

**KEY POINTS**

- Quarto = YAML + prose + chunks; Render runs clean and outputs html/docx/pdf.
- echo: false for readers; fig-cap/fig-alt for accessibility; labels for cross-refs.
- Inline `r` values keep narrative numbers honest — the update-proof report.

**MODULE 5 · VISUALISATION & REPORTING IN R · LESSON 66 OF 105**

## 66. Parameterised reports: one template, forty outputs

The request that breaks copy-paste workflows: "lovely report — now one per LA / practice / ICB, monthly." **Parameters** solve it: the report declares inputs in YAML, code reads them, and a loop renders every variant from one template.

### 1. Declare parameters and use them:

**IN LA\_REPORT.QMD**

```
---
title: "Public health summary"
format: html
params:
  la: "Barchester"
---
``{r}
la_data <- admissions_geo |>
  filter(la_name == params$la)
``
## `r params$la`
This report covers `r nrow(la_data)` admissions for `r params$la` residents.
```

### 2. Render every variant from a script:

**RENDER\_ALL.R**

```
library(quarto)
las <- c("Barchester", "Millford", "Kingsbay")
for (la in las) {
  # purrr::walk works too (Lesson 57!)
  quarto_render(
    "la_report.qmd",
    execute_params = list(la = la),
    output_file = paste0("outputs/report_", janitor::make_clean_names(la), ".html")
  )
}
```

Three files, three tailored reports, identical methodology — and when the definition of an indicator changes, you fix **one template** and re-render, instead of editing forty documents inconsistently. Design habits that keep templates healthy: make *everything* flow from params (titles, filters, filenames); test with the smallest and the messiest unit (the tiny LA with suppressible counts is where templates break); include a rendered-date and data-version line in the footer; and keep a **combined** param path (`la: "All"`) for the overview report. This pattern — template + parameter loop — is the beating heart of RAP-grade reporting in Module 7.

**QUICK CHECK**

**A methodology change must reach all 42 ICB reports. With parameterised reporting you...**

- ☐ Edit 42 files carefully
- ✓ **Edit the single template and re-render the batch**
- ☐ Ask each ICB to edit theirs
- ☐ Rewrite in Excel

**Why:** *One source of truth, many outputs: consistency by construction, not diligence.*

**QUICK CHECK**

**Why test the template on the smallest area first?**

- ☐ Small areas render faster
- ✓ **Edge cases — zero counts, suppression, empty groups — live there and break naive templates**
- ☐ Big areas are unimportant
- ☐ Quarto requires it

**Why:** *Templates fail at the margins. The area with 3 events finds your bugs before the director does.*

**EXERCISE**

Sketch the two artefacts for practice-level reports: the params block + first filter chunk of `practice_report.qmd`, and the render loop over three practice codes writing to `outputs/`.

**SOLUTION · R**

```
# --- practice_report.qmd (top) ---
# ---
# title: "Practice profile"
# format: html
# params:
#   practice: "P81001"
# ---
# ```{r}
# pr_data <- admissions_full |> filter(practice_code == params$practice)
# ```
# --- render_all.R ---
library(quarto)
for (p in c("P81001", "P81002", "P81003")) {
  quarto_render("practice_report.qmd",
    execute_params = list(practice = p),
    output_file = paste0("outputs/profile_", p, ".html"))
}
```

**KEY POINTS**

- params in YAML + `params$x` in code = a report template; a loop renders the batch.
- Fix once, re-render everywhere; date-stamp outputs.
- Test on the smallest, messiest unit — that's where suppression and empty-group bugs hide.

## MODULE 5 • VISUALISATION &amp; REPORTING IN R • LESSON 67 OF 105

## 67. Safe outputs: disclosure control before anything leaves

Module 1 gave the law and principles; now the operational habit. **Everything that leaves your team — tables, charts, CSVs, the numbers inside sentences — passes disclosure control first.** In R, that means suppression is a *\*pipeline step\**, not a manual edit.

### A REUSABLE SUPPRESSION STEP

```
suppress_counts <- function(df, count_cols, threshold = 5) {
  df |>
    mutate(across(
      all_of(count_cols),
      \(n) if_else(n > 0 & n < threshold, "<5", format(n, big.mark = ","))
    ))
}
ward_deaths |>
  suppress_counts("deaths") |>
  gt()
```

The full checklist, run in order:

- **Primary suppression:** hide small non-zero counts (threshold per your organisation's policy — 5 is common; HES-derived outputs traditionally suppress 1–7). Zeros are usually safe to show; check local rules.
- **Secondary suppression:** if a row/column total lets a reader back-calculate one hidden cell, hide a second cell (or round/remove the total). One suppressed cell alongside an exact total is no suppression at all.
- **Derived values leak too:** a rate of 2.1 per 1,000 on a published denominator of 950 reveals the count of 2. Suppress the rate when its numerator is suppressed; consider rounding denominators.
- **Charts count as outputs:** a bar of height 2 discloses exactly like a table cell; aggregate small categories (fct\_lump!) or annotate "<5".
- **Row-level exports are the sharpest edge:** a "harmless" CSV of rare-condition events with LSOA + age + sex can single people out by combination. Aggregate before export; if record-level must move, it goes through IG-approved channels, never email.

Codify it: your team's thresholds live in one sourced function; every report template calls it; QA checks it. When genuinely unsure whether something identifies, the answer is your IG team or Caldicott Guardian *\*before\** release — asking first is a strength signal in this profession, never a weakness.

### QUICK CHECK

**A table shows Ward A: <5, Ward B: 12, Total: 15. The problem is...**

- ☐ Nothing; primary suppression applied
- ✓ **The total back-calculates Ward A as 3 — secondary suppression (or total removal/rounding) is needed**
- ☐ Totals are never allowed
- ☐ Ward B should also be hidden

**Why:** Complementary disclosure: exact margins + one hidden cell = arithmetic, not protection.

**QUICK CHECK**

**Publishing "rate: 4.2 per 1,000 (population 480)" for a rare event risks...**

- ☐ Nothing; rates are aggregate
- ✓ **Revealing the numerator (2) — derived statistics inherit their inputs' disclosure risk**
- ☐ Confusing readers with decimals
- ☐ Breaching copyright

**Why:**  $4.2/1000 \times 480 \approx 2$ . Suppress or coarsen derived values whose suppressed ingredients they expose.

**EXERCISE**

Extend `suppress_counts` to also blank an accompanying rate column when its count is suppressed (arguments: `count_col`, `rate_col`). Show it on a toy tibble of area, deaths, rate.

**SOLUTION · R**

```
suppress_with_rate <- function(df, count_col, rate_col, threshold = 5) {
  df |>
    mutate(
      hide = .data[[count_col]] > 0 & .data[[count_col]] < threshold,
      "{rate_col}" := if_else(hide, "*", format(round(.data[[rate_col]], 1))),
      "{count_col}" := if_else(hide, "<5", format(.data[[count_col]])),
    ) |>
    select(-hide)
}

tibble(area = c("A", "B", "C"), deaths = c(2, 14, 0), rate = c(4.2, 9.8, 0)) |>
  suppress_with_rate("deaths", "rate")
#> A: <5 with rate *, B: 14 / 9.8, C: 0 / 0
```

**KEY POINTS**

- Suppression is a coded pipeline step applied to every leaving output — tables, charts, CSVs, sentences.
- Primary, then secondary (totals!), then derived values (rates leak numerators).
- Record-level exports need IG-approved routes; when unsure, ask before release.

**MODULE 6**

# Module 6 — Statistics & epidemiology methods

**In this module**

- 68** Describing data honestly: the right summary for the shape
- 69** Rates in R: crude rates, person-time and per-100,000 thinking
- 70** Age-standardisation: comparing places fairly
- 71** Uncertainty: confidence intervals for counts and rates
- 72** Comparing proportions: 2x2 tables, chi-square and risk ratios
- 73** Comparing means: t-tests, Mann-Whitney and what "significant" means

- 74** Correlation and the ecological fallacy
- 75** Linear regression: modelling a continuous outcome
- 76** Logistic regression and odds ratios
- 77** SPC charts: Making Data Count
- 78** Funnel plots: comparing organisations fairly
- 79** Screening and tests: sensitivity, specificity and the PPV trap



## MODULE 6 · STATISTICS &amp; EPIDEMIOLOGY METHODS · LESSON 68 OF 105

## 68. Describing data honestly: the right summary for the shape

Statistics starts with description, and description starts with **shape** (Lesson 62). The decision tree an analyst runs on every numeric variable:

- Roughly symmetric? → **mean ± standard deviation (SD)**.
- Skewed / outlier-prone (LOS, waiting times, costs — most health data)? → **median + IQR**, plus percentiles for tails (90th/95th).
- Categorical? → **counts + percentages**, with the denominator stated.

### A DESCRIBE-EVERYTHING BLOCK

```
los_summary <- los |>
  summarise(
    n      = n(),
    mean   = mean(los_days),
    sd     = sd(los_days),
    median = median(los_days),
    q25    = quantile(los_days, 0.25),
    q75    = quantile(los_days, 0.75),
    p95    = quantile(los_days, 0.95),
    max    = max(los_days)
  )
```

Interpretation beats computation. **SD** describes spread of \*individuals\* (mean 4.1, SD 6.2 says individuals vary hugely — and SD > mean on a non-negative measure screams skew). Percentages need their denominator in the same breath: "62% (of the 34 patients tested)" and "62% (of 12,000 admissions)" are different classes of claim. Small denominators make wild percentages — one event in a practice of 3 eligible children is "33% uptake"; flag or suppress tiny denominators rather than headline them.

Two more honesty rules. **Report missingness alongside every summary** — "median HbA1c 49 (n = 412; 38 missing)" — because who's missing is often the story. And **round for the reader**: rates to 1 dp, percentages to whole numbers usually; spurious precision ("uptake was 62.3847%") signals a beginner.

### QUICK CHECK

#### Mean LOS 4.1 days with SD 6.2 days tells you...

- ☐ Data entry errors
- ✓ **Massive individual variation and (given LOS can't be negative) right skew — median needed**
- ☐ Nothing useful
- ☐ The mean is wrong

**Why:** SD near or above the mean of a non-negative variable is a skew signature. The picture (histogram) and the median tell the honest story.

**QUICK CHECK****"Screening uptake: 100%" is least trustworthy when...**

- ☐ The programme is national
- ✓ **The denominator is tiny (e.g. 2 of 2 eligible) — small denominators produce unstable, headline-grabbing percentages**
- ☐ It comes from SQL
- ☐ Uptake is measured monthly

**Why:** Always ask "percent of how many?" Percentages inherit the fragility of their denominators.**EXERCISE**

Write a reusable `describe_num(df, col)` function returning `n`, `n_missing`, `mean`, `sd`, `median`, `q25`, `q75` for any numeric column (hint: use `{{ col }}` for tidy evaluation). Run it on `lab_results` value for `HbA1c` only.

**SOLUTION · R**

```
describe_num <- function(df, col) {
  df |>
    summarise(
      n      = sum(!is.na({{ col }})),
      n_missing = sum(is.na({{ col }})),
      mean   = mean({{ col }}, na.rm = TRUE),
      sd     = sd({{ col }}, na.rm = TRUE),
      median = median({{ col }}, na.rm = TRUE),
      q25    = quantile({{ col }}, 0.25, na.rm = TRUE),
      q75    = quantile({{ col }}, 0.75, na.rm = TRUE)
    )
}
lab_results |>
  filter(test == "HbA1c") |>
  describe_num(value)
```

**KEY POINTS**

- Shape chooses the summary: symmetric → mean/SD; skewed → median/IQR/percentiles; categorical → n and % with denominator.
- Report missingness with every summary; distrust percentages on tiny denominators.
- Round for humans; precision beyond the data is noise.

**MODULE 6 · STATISTICS & EPIDEMIOLOGY METHODS · LESSON 69 OF 105****69. Rates in R: crude rates, person-time and per-100,000 thinking**

Lesson 37 built rates in SQL; R is where they meet inference. Rebuild the pattern tidily — numerators and denominators as data frames, joined, scaled:

**CRUDE EMERGENCY ADMISSION RATES BY LA**

```
numerator <- admissions |>
  filter(admission_method == "Emergency") |>
  left_join(patients, by = "patient_id") |>
  left_join(lsoa_imd, by = "lsoa_code") |>
  count(la_name, name = "events")
```

**CRUDE EMERGENCY ADMISSION RATES BY LA**

```
denominator <- la_population |>
  summarise(pop = sum(population), .by = la_name)
rates <- numerator |>
  left_join(denominator, by = "la_name") |>
  mutate(rate_per_100k = 100000 * events / pop)
```

Concepts that stop rate mistakes:

- **Choose the scale for the audience:** per 1,000 for common events (GP consultations), per 100,000 for rarer ones (mortality, incidence) — pick whatever makes numbers land between ~1 and a few hundred, and say the scale everywhere.
- **Person-time:** when people enter and leave observation (registrations churn, cohorts age in and out), the honest denominator is **person-years at risk**, not a headcount — 500 people followed for 6 months = 250 person-years. Rate = events / person-years. Mid-year population estimates are the standard approximation for whole-area annual rates.
- **Events vs people:** 300 admissions from 190 patients gives you two different rates — an *\*admission\** rate and a *\*patient\** rate. Decide which the question wants; say which you used.
- **Period matching** (Lesson 37's rule): a 2-year numerator needs 2 years of person-time.

**QUICK CHECK**

**500 residents observed for 6 months each contribute how much person-time?**

- ☐ 500 person-years
- ✓ **250 person-years**
- ☐ 3,000 person-years
- ☐ 6 person-years

**Why:**  $500 \times 0.5$  years. Rates divide events by time genuinely at risk — headcounts overstate the denominator when follow-up is partial.

**QUICK CHECK**

**A town has 84 emergency COPD admissions from 51 patients in a year. "The COPD admission rate" should...**

- ☐ Use 84; patients don't matter
- ☐ Use 51; admissions don't matter
- ✓ **Be stated explicitly as either admissions-based (84) or patient-based (51) — they answer different questions**
- ☐ Average the two

**Why:** Activity questions (bed pressure) want events; population-health questions (how many people affected) want people. Ambiguity here causes real report disputes.

**EXERCISE**

Compute both rates for each LA: emergency admissions per 100,000 AND emergency patients (distinct) per 100,000, in one summarise (n() and n\_distinct), joined to population. Which LA's two rates differ most (in ratio terms)?

**SOLUTION · R**

```
per_la <- admissions |>
  filter(admission_method == "Emergency") |>
  left_join(patients, by = "patient_id") |>
  left_join(lsoa_imd, by = "lsoa_code") |>
```

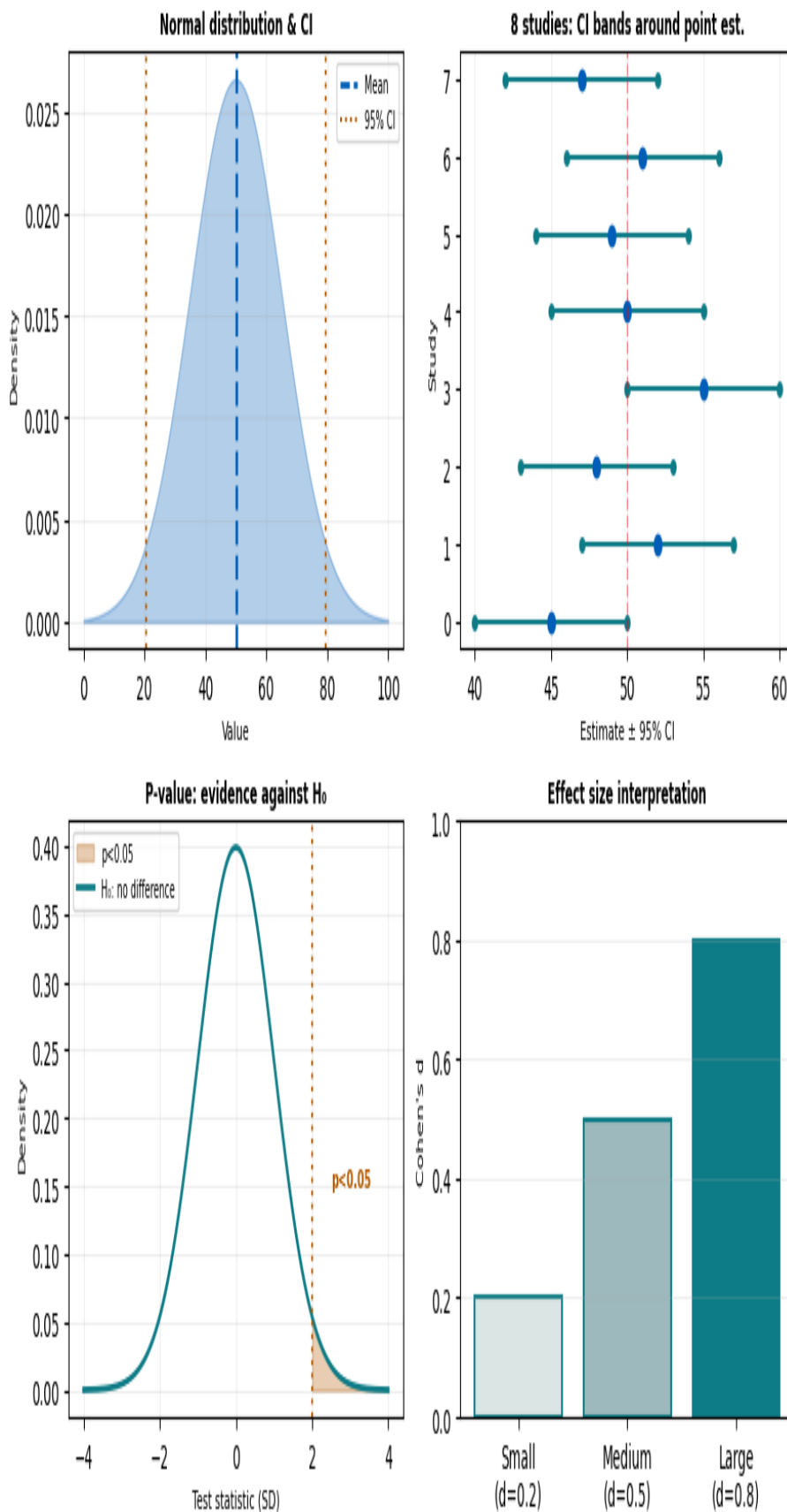
**SOLUTION · R**

```
summarise(events = n(),
           patients = n_distinct(patient_id),
           .by = la_name) |>
left_join(denominator, by = "la_name") |>
mutate(event_rate = 100000 * events / pop,
       patient_rate = 100000 * patients / pop,
       ratio = event_rate / patient_rate) |>
arrange(desc(ratio))
per_la
# The highest ratio = most repeat admissions per patient - itself a finding.
```

**KEY POINTS**

- Rate = events / population-or-person-time x scale; choose scale for readability and state it.
- Person-years for churning cohorts; mid-year estimates for annual area rates.
- Decide events vs people explicitly — the ratio between them is the readmission story.

## Key statistical concepts



## MODULE 6 • STATISTICS &amp; EPIDEMIOLOGY METHODS • LESSON 70 OF 105

## 70. Age-standardisation: comparing places fairly

Crude rates confound age (Lesson 37's warning). **Direct age-standardisation** removes it: compute age-specific rates for each area, then apply every area's rates to the *same* reference population — the **European Standard Population 2013 (ESP2013)** in UK practice. The result, a **directly standardised rate (DSR)**, answers: *what would this area's rate be if it had the standard age structure?*

**DSR BY HAND (THE LOGIC LAID BARE)**

```
esp <- tribble(
  ~age_band, ~std_pop,
  "0-39",    45000,
  "40-64",    33000,
  "65+",     22000
)

dsr <- age_specific |>          # columns: la_name, age_band, events, pop
  mutate(rate = events / pop) |>
  left_join(esp, by = "age_band") |>
  summarise(
    dsr_per_100k = 100000 * sum(rate * std_pop) / sum(std_pop),
    .by = la_name
  )
```

Read the core line: each band's *local rate* × the *standard* band size, summed, divided by the standard total — a weighted average with everyone forced to share weights. In production you don't hand-roll it: the **PHEindicatormethods** package (built for exactly this by UK public health statisticians) does DSRs with confidence intervals in one call — `phe_dsr(data, x = events, n = pop, stdpop = std_pop)` — using the full 5-year ESP2013 bands.

Judgement notes: DSRs are for **comparison**, not for stating the real burden (the crude rate is what actually happens to actual residents — report both); areas with tiny band counts give unstable DSRs (CIs, next lesson, are mandatory); and standardisation only removes *age* (and sex if you standardise by it) — deprivation and everything else still differ. **Indirect standardisation** (SMRs — expected deaths from standard rates applied to local structure) is the sibling used when local band counts are too small; recognise the term.

**QUICK CHECK****A DSR answers which question?**

- ☐ What is the actual admission rate here?
- ✓ **What would the rate be if this area had the standard (ESP2013) age structure?**
- ☐ How many events occurred?
- ☐ What is the median age?

**Why:** *It's a fair-comparison construct. For "what is actually happening to residents", the crude rate remains the truthful number — report both.*

**QUICK CHECK**

**Area A (young) and Area B (old) have equal crude mortality rates. Their DSRs will likely show...**

- ☐ Equality
- ✓ **Area A worse — matching an old area's crude rate despite a young population implies higher age-specific rates**
- ☐ Area B worse
- ☐ Nothing; DSRs can't compare

**Why:** Standardisation reveals what age structure concealed: a young population "should" have a lower crude rate; parity means A's underlying rates are higher.

**EXERCISE**

Using the 3-band ESP above and made-up age\_specific data for two LAs (you choose plausible events/pop per band, making one LA older), compute crude rates and DSRs, and show a case where the crude ranking and DSR ranking flip.

**SOLUTION · R**

```
age_specific <- tribble(
  ~la_name, ~age_band, ~events, ~pop,
  "Oldshire", "0-39", 10, 8000,
  "Oldshire", "40-64", 40, 12000,
  "Oldshire", "65+", 150, 20000,
  "Youngton", "0-39", 30, 25000,
  "Youngton", "40-64", 35, 10000,
  "Youngton", "65+", 40, 5000
)

crude <- age_specific |>
  summarise(crude = 100000 * sum(events) / sum(pop), .by = la_name)

dsr <- age_specific |>
  mutate(rate = events / pop) |>
  left_join(esp, by = "age_band") |>
  summarise(dsr = 100000 * sum(rate * std_pop) / sum(std_pop), .by = la_name)

left_join(crude, dsr, by = "la_name")
#> Oldshire has the higher crude rate (old population),
#> but Youngton's age-specific rates are worse - its DSR comes out higher.
```

**KEY POINTS**

- Direct standardisation: local age-specific rates x ESP2013 weights = comparable DSRs.
- Use `PHEindicatormethods::phe_dsr` in real work (CIs included); report crude alongside.
- Standardisation removes age only; small bands need CIs; SMR = the indirect sibling.

**MODULE 6 · STATISTICS & EPIDEMIOLOGY METHODS · LESSON 71 OF 105**

## 71. Uncertainty: confidence intervals for counts and rates

Sixteen deaths this year, twelve last year — real change or chance? Counts of events wobble naturally (statisticians model them as **Poisson**), and small counts wobble \*proportionally more\*. A **95% confidence interval** expresses that wobble: the range of underlying rates reasonably compatible with what you observed.

The UK public health workhorse is **Byar's method** — an excellent Poisson approximation you can compute directly, and the one `PHEindicatormethods` implements:

**BYAR'S CI FOR A COUNT, THEN A RATE**

```
byars_ci <- function(x, conf = 0.95) {           # x = observed count
  z <- qnorm(1 - (1 - conf) / 2)
  lower <- x * (1 - 1/(9*x) - z / (3 * sqrt(x)))^3
  upper <- (x + 1) * (1 - 1/(9*(x+1)) + z / (3 * sqrt(x + 1)))^3
  tibble(lower, upper)
}
byars_ci(16)           # observed 16 deaths
#> lower ~9.1, upper ~26.0
# As a rate per 100,000 on population 40,000:
100000 * c(9.1, 16, 26.0) / 40000
#> ~22.9  40.0  64.9 per 100,000
```

Sixteen deaths is compatible with an underlying count anywhere from ~9 to ~26 — so twelve-vs-sixteen is comfortably within chance. Reading rules that keep you honest:

- **Overlapping CIs → don't claim a difference** (non-overlap suggests one; the strict test comes later, but this heuristic prevents most over-claiming).
- **Small counts, huge intervals:** 3 events → CI roughly 0.6–8.8. This is the statistical twin of suppression: small numbers can't support strong claims.
- CIs quantify **random variation only** — no interval fixes a biased numerator or wrong denominator.
- In production: `PHEindicatormethods::phe_rate()` and `phe_dsr()` return rates with Byar's-based CIs ready for publication; error bars in ggplot via `geom_errorbar(aes(ymin = lower, ymax = upper))`.

**QUICK CHECK**

**Ward A rate 58 (95% CI 40-82); Ward B rate 71 (95% CI 52-95), per 100,000. The defensible statement is...**

- ☐ B is significantly worse
- ✓ **The intervals overlap substantially — the data are consistent with no real difference**
- ☐ A is improving
- ☐ Both are zero

**Why:** Wide, overlapping intervals = chance can explain the gap. Claiming otherwise is the commonest statistical error in local reports.

**QUICK CHECK**

**Why do 3 observed events produce an enormous relative CI?**

- ☐ Byar's formula is inaccurate
- ✓ **Poisson variation is proportionally huge at small counts — the data barely constrain the underlying rate**
- ☐ Rates cannot be computed under 10 events
- ☐ The population must be wrong

**Why:** Small-count instability is fundamental, not fixable by formula choice — which is why small numbers get suppressed and never headlined.

**EXERCISE**

Compute Byar's 95% CIs for observed counts 3, 16 and 160 with the function above, express each as (count ± interval) and note how the RELATIVE width (upper/lower ratio) shrinks as counts grow.



**SOLUTION · R**

```
map_dfr(c(3, 16, 160), \(x) byars_ci(x) |> mutate(x = x, .before = 1)) |>
  mutate(rel_width = upper / lower)
#> x=3:    ~0.6-8.8    (ratio ~14 - almost no information)
#> x=16:   ~9.1-26    (ratio ~2.9)
#> x=160: ~136-187    (ratio ~1.4 - now quite precise)
```

**KEY POINTS**

- Event counts are Poisson-wobbly; Byar's method gives the standard UK CI for counts, rates and DSRs.
- Overlapping CIs = no claimed difference; small counts = wide intervals = weak claims.
- CIs cover chance only — bias needs fixing at source. Use `PHEindicatormethods` in production.

**MODULE 6 · STATISTICS & EPIDEMIOLOGY METHODS · LESSON 72 OF 105**

## 72. Comparing proportions: 2x2 tables, chi-square and risk ratios

"Is uptake lower in deprived areas?" is a two-proportion question, and its home is the **2×2 table**: exposure (deprived: yes/no) by outcome (vaccinated: yes/no).

**BUILD THE 2X2 AND THE MEASURES**

```
tab <- imms |>                                # child-level: imd_group, vaccinated
  count(imd_group, vaccinated) |>
  pivot_wider(names_from = vaccinated, values_from = n) # yes / no columns
# Proportions with CIs, per group:
imms |>
  summarise(n = n(), vacc = sum(vaccinated), .by = imd_group) |>
  mutate(p = vacc / n,
         se = sqrt(p * (1 - p) / n),
         lower = p - 1.96 * se,
         upper = p + 1.96 * se)
```

Two complementary tools:

- **The risk ratio (RR)** — the *size* of the difference:  $p_1/p_2$ . "Uptake 84% vs 92%: RR 0.91 — deprived-area children are 9% less likely to be vaccinated." Effect sizes carry the public health meaning; a risk *difference* (92% – 84% = 8 percentage points) translates directly into children missed.
- **The chi-square test** — the *chance* question: could a gap this size arise if uptake were truly equal? `chisq.test(table(imms$imd_group, imms$vaccinated))` returns a p-value: the probability of a gap at least this large under "no real difference". Small p (< 0.05 by convention) → chance is an unlikely explanation. For small cell counts (expected < 5), use `fisher.test()` instead.

The professional framing: **report the effect size with its CI, use the test as support, never headline a bare p-value**. "p = 0.03" says almost nothing; "8 percentage points lower (95% CI 3–13), p = 0.03" says everything. And with large denominators, trivially small gaps become "significant" — statistical significance ≠ public-health importance; 0.2 percentage points can be p < 0.05 and still not worth a meeting.

**QUICK CHECK**

**Uptake gap of 0.3 percentage points,  $p = 0.001$ ,  $n = 400,000$  per group. Best interpretation?**

- ☐ A major inequality demanding action
- ✓ **A real but tiny difference — huge samples make trivial gaps "significant"; judge importance by effect size**
- ☐ The test failed
- ☐ Uptake is equal

**Why:** *p-values measure detectability, not importance. With  $n$  that large, almost any true difference is detectable.*

**QUICK CHECK**

**A 2x2 has an expected cell count of 2. You should...**

- ☐ Use `chisq.test` anyway
- ✓ **Use `fisher.test`, designed for small counts**
- ☐ Double the data
- ☐ Skip the analysis

**Why:** *Chi-square's approximation breaks with sparse cells; Fisher's exact test is the standard fallback (and R will warn you).*

**EXERCISE**

From the playground, export a child-level MMR table (or construct a toy one): 480 deprived children with 396 vaccinated; 510 non-deprived with 466. Compute both proportions, the risk difference, the RR, and run `chisq.test` on the 2x2 counts.

**SOLUTION · R**

```
tab <- matrix(c(396, 480-396,
               466, 510-466),
             nrow = 2, byrow = TRUE,
             dimnames = list(c("Deprived", "Not deprived"),
                             c("Vaccinated", "Not")))

prop1 <- 396/480; prop2 <- 466/510
prop1; prop2
#> 0.825, 0.914

prop2 - prop1      # risk difference ~0.089 (8.9 points)
prop1 / prop2      # RR ~0.90
chisq.test(tab)
#> p-value well under 0.05 - chance is an unlikely explanation
```

**KEY POINTS**

- 2x2 tables → risk difference and risk ratio (the meaning) + chi-square/Fisher (the chance check).
- Report effect size with CI; p-values support, never headline.
- Large  $n$  makes trivia "significant"; small expected cells demand `fisher.test`.

## MODULE 6 • STATISTICS &amp; EPIDEMIOLOGY METHODS • LESSON 73 OF 105

## 73. Comparing means: t-tests, Mann-Whitney and what "significant" means

"Is mean HbA1c different between two groups?" — the two-group numeric comparison. Two standard tools, chosen (again) by shape:

- **Welch's t-test** — compares means; R's default `t.test(value ~ group, data = d)` (unequal variances assumed — the sensible default). Wants roughly symmetric data or decent sample sizes.
- **Mann-Whitney / Wilcoxon rank-sum** — `wilcox.test(value ~ group)` — compares distributions via ranks; the go-to for skewed measures like LOS, where means mislead anyway.

### BOTH TESTS ON LAB DATA

```
hba <- lab_results |>
  filter(test == "HbA1c") |>
  left_join(patients, by = "patient_id")
t.test(value ~ sex, data = hba)
#> t = ..., p-value = ...
#> 95 percent CI of the difference: [lo, hi]
#> sample means: F ..., M ...
wilcox.test(value ~ sex, data = hba)
```

Read the t-test output in this order: **the two means** (the finding), **the CI of the difference** (plausible size range — crossing 0 means "no difference" is compatible), then the p-value (support). A p-value's actual meaning — worth memorising because it's an interview classic — is: \*the probability of a difference at least this large if the groups were truly identical\*. It is **not** the probability the finding is true, and 0.05 is a convention, not physics.

Practical wisdom: paired designs (same patients before/after) need `paired = TRUE` — pairing is powerful and forgetting it wastes it; check the shape first (Lesson 62's histograms) rather than running normality tests ritually; with many outcome comparisons, some  $p < 0.05$  results appear by chance alone (multiple testing — flag it, or adjust); and always ask whether the difference \*matters clinically\* — 0.4 mmol/mol of HbA1c can be "significant" and clinically irrelevant.

### QUICK CHECK

**The 95% CI for a difference in means is (-0.8, +3.2). This tells you...**

- ☐ The difference is exactly 1.2
- ✓ **Values from slightly negative to moderately positive are all compatible — including zero, so no confident direction**
- ☐ The test is broken
- ☐ Both groups are identical

**Why:** A CI crossing zero is the interval version of  $p > 0.05$  — but it says more: how big the difference could plausibly be, either way.

**QUICK CHECK**

**For length-of-stay comparisons between two wards, the usual better choice is...**

- ☐ t-test, because means matter
- ✓ **Mann-Whitney (wilcox.test), because LOS is heavily skewed and rank-based tests aren't distorted by the tail**
- ☐ Chi-square
- ☐ No test exists

**Why:** Skew wrecks mean-based inference at modest  $n$ . Rank tests (or comparing medians with bootstrap CIs) fit the data's reality.

**EXERCISE**

Compare Systolic BP by sex: histogram to check shape, then run the appropriate test, and write ONE sentence reporting group summaries, the difference (with CI if t-test), and the p-value - in that order.

**SOLUTION · R**

```
sbp <- lab_results |>
  filter(test == "Systolic BP") |>
  left_join(patients, by = "patient_id")
ggplot(sbp, aes(value)) + geom_histogram(binwidth = 5) + facet_wrap(~sex)
# Roughly symmetric -> t-test is fine:
t.test(value ~ sex, data = sbp)
# Reporting template:
# "Mean SBP was X in females and Y in males (difference Z, 95% CI a-b, p = c)."
```

**KEY POINTS**

- t.test for means (Welch default), wilcox.test for skewed data; paired = TRUE when it's the same people.
- Read: means → CI of difference → p-value; CI crossing zero = direction unproven.
- Significance is not importance; multiple comparisons inflate false positives.

**MODULE 6 · STATISTICS & EPIDEMIOLOGY METHODS · LESSON 74 OF 105**

## 74. Correlation and the ecological fallacy

"Deprivation and emergency admissions look related — how strongly?" **Correlation** puts a number on the straight-line association between two continuous variables:  $r$ , from  $-1$  (perfect negative) through 0 (none) to  $+1$  (perfect positive).

**CORRELATION IN R**

```
library(tidyverse)
set.seed(3)
lsoa <- tibble(
  imd_score = runif(60, 5, 55),
  adm_rate = 60 + 1.9 * imd_score + rnorm(60, 0, 14)
)
cor(lsoa$imd_score, lsoa$adm_rate)           # Pearson r, ~0.8
cor(lsoa$imd_score, lsoa$adm_rate,
    method = "spearman")                     # rank-based
cor.test(lsoa$imd_score, lsoa$adm_rate)      # r + CI + p-value
```

**Pearson** measures *\*linear\** association and is disturbed by outliers and skew; **Spearman** correlates the *\*ranks\**, so it captures any consistently increasing/decreasing relationship and shrugs at outliers — often the safer default for skewed health data. Rough reading of  $|r|$ : 0.1 weak, 0.3 moderate, 0.5+ strong — but context rules; in noisy population data,  $r = 0.4$  can be a headline.

Non-negotiable habit: **plot before you correlate**. Anscombe's quartet (type `anscombe` into R) is four datasets with identical  $r \approx 0.82$  — one linear, one curved, one outlier-driven, one vertical-line-with-a-fluke. The number cannot distinguish them; your eyes can in half a second. `ggplot(lsoa, aes(imd_score, adm_rate)) + geom_point()` costs one line.

Two famous traps, both career-relevant:

**Correlation is not causation.** Deprived areas have higher admission rates — does deprivation *\*cause\** admissions, do admissions-relevant factors (age structure, chronic disease, access) cluster in deprived areas, or does something upstream drive both?  $r$  cannot say. Language discipline follows: "associated with", never "leads to", unless the study design earns it.

**The ecological fallacy.** Your correlation used *\*areas\** as the unit. It tells you deprived *\*LSOAs\** have higher rates — it does **not** tell you deprived *\*individuals\** within them are the ones being admitted. Area-level and individual-level associations can differ in size and even direction (a classic: areas with more immigrants once showed higher literacy — because immigrants settled in literate cities, not because immigrants were more literate). Public health lives on area data, so this fallacy is an occupational hazard: keep conclusions at the level of the data.

#### QUICK CHECK

**An LSOA-level analysis finds  $r = 0.7$  between IMD score and smoking-related admissions. Which claim is justified?**

- ☐ Deprived individuals smoke more
- ☐ Deprivation causes smoking admissions
- ✓ **More deprived areas tend to have higher smoking-related admission rates**
- ☐ 70% of admissions are due to deprivation

**Why:** Area-level association, area-level claim — anything about individuals commits the ecological fallacy, and "causes" needs far more than  $r$ . (And  $r$  is not a percentage of anything.)

#### QUICK CHECK

**Why does "plot before you correlate" matter if you already have  $r$ ?**

- ☐ It doesn't;  $r$  is sufficient
- ✓ **Very different relationships (curves, outlier artefacts) can share the same  $r$  — the scatterplot reveals which you have**
- ☐ Plots change the value of  $r$
- ☐ Journals require plots

**Why:** Anscombe's quartet: identical  $r$ , wildly different stories.  $r$  summarises a shape it cannot see; you must look.

#### EXERCISE

Using the `lsoa` tibble: draw the scatterplot with a linear trend line (`geom_smooth(method = "lm")`), compute both Pearson and Spearman correlations, and write a one-sentence conclusion that respects both traps above.

#### SOLUTION · R

```
ggplot(lsoa, aes(imd_score, adm_rate)) +
```

## SOLUTION · R

```
geom_point(colour = "#0e7c86") +
geom_smooth(method = "lm", se = TRUE, colour = "#005EB8") +
labs(x = "IMD score (higher = more deprived)",
     y = "Emergency admission rate per 1,000")
cor(lsoa$imd_score, lsoa$adm_rate) # ~0.8
cor(lsoa$imd_score, lsoa$adm_rate, method = "spearman")
# "At LSOA level, higher deprivation is strongly associated with higher
# emergency admission rates; this area-level association does not by
# itself show that deprived individuals are those admitted, nor that
# deprivation causes admission."
```

## KEY POINTS

- Pearson = linear, outlier-sensitive; Spearman = ranks, robust; always scatterplot first.
- Correlation  $\neq$  causation: say "associated with".
- Ecological fallacy: area-level findings license only area-level claims.

## MODULE 6 · STATISTICS &amp; EPIDEMIOLOGY METHODS · LESSON 75 OF 105

## 75. Linear regression: modelling a continuous outcome

Correlation says *\*how strongly\**; **regression** says *\*how much\** — "each additional IMD point is associated with 1.9 more admissions per 1,000" is a sentence commissioners can act on. Linear regression fits the best straight line through the data and hands you its equation.

## LM: THE WORKHORSE

```
model <- lm(adm_rate ~ imd_score, data = lsoa)
summary(model)
```

The output's essentials: under **Coefficients**, the **(Intercept)** (predicted rate when IMD = 0 — often outside the data, mathematically necessary, substantively meaningless) and the **slope** for **imd\_score** — the change in outcome per one-unit change in predictor. Each has a standard error, and **Multiple R-squared** reports the share of outcome variation the model explains ( $R^2 = r^2$  with one predictor).

The **broom** package turns that printout into a tidy tibble — far better for reports:

## TIDY MODEL OUTPUT

```
library(broom)
tidy(model, conf.int = TRUE)
#   term          estimate  conf.low conf.high ...
# (Intercept)    60.2      52.9      67.5
# imd_score       1.91      1.68      2.14
glance(model)    # R², n, and friends in one row
```

Reading: rate = 60.2 + 1.91 × IMD, and we're 95%-confident the true slope lies between about 1.7 and 2.1 — comfortably away from zero, so the association isn't plausibly noise.

**Multiple regression** adds predictors, and this is the real prize — each coefficient becomes the association *\*holding the others constant\**:

## ADJUSTING FOR A SECOND VARIABLE

```
# lm(adm_rate ~ imd_score + pct_over_65, data = lsoa2)
```

If the IMD slope shrinks once age is included, part of the raw deprivation gradient was really an age-structure story — regression is how "adjusted for age" happens outside of standardisation.

Honesty checklist before believing any lm: the relationship should look reasonably straight (scatterplot again); residuals shouldn't fan out or curve (`plot(model, which = 1)`); outliers shouldn't own the slope (refit without the extreme point; if the answer changes materially, say so); and predictions outside the observed x-range are extrapolation — the line knows nothing out there. And the ecological fallacy still applies: an area-level model licenses area-level sentences.

**QUICK CHECK**

**In `tidy(model)`, `imd_score` has estimate 1.91, `conf.low` 1.68, `conf.high` 2.14. The plain-English reading?**

- ☐ IMD explains 191% of admissions
- ✓ **Each 1-point rise in IMD score is associated with ~1.9 more admissions per 1,000 (95% CI 1.7 to 2.1)**
- ☐ Admissions rise by 1.91% per year
- ☐ The intercept is 1.91

**Why:** The slope is change-in-outcome per unit-of-predictor, with the CI expressing precision. Getting this sentence right is half of applied statistics.

**QUICK CHECK**

**Adding `pct_over_65` to the model halves the `imd_score` coefficient. Best interpretation?**

- ☐ The model is broken
- ✓ **Part of the crude deprivation association was accounted for by age structure; the remaining slope is the age-adjusted association**
- ☐ Age causes deprivation
- ☐  $R^2$  must have fallen

**Why:** Coefficients in multiple regression are adjusted for co-included variables. Shrinkage on adjustment = the adjuster explained some of the story.

**EXERCISE**

Fit the model on `lsoa`, extract a tidy coefficient table with CIs, and use the model to predict the admission rate for an LSOA with IMD score 40 (`predict()` with `newdata`). State one caveat about that prediction.

**SOLUTION · R**

```
model <- lm(adm_rate ~ imd_score, data = lsoa)
tidy(model, conf.int = TRUE)
predict(model, newdata = tibble(imd_score = 40))
# ≈ 60.2 + 1.91*40 ≈ 137 per 1,000
# Caveat: a point prediction from an area-level model with scatter around
# the line ( $R^2 < 1$ ); real LSOAs at IMD 40 vary widely around ~137, and the
# model says nothing about individuals.
```

**KEY POINTS**

- `lm(outcome ~ predictor)`; slope = change in outcome per unit predictor; `broom::tidy` for clean output.
- Multiple regression = adjusted associations; coefficient shifts reveal confounding.
- Check linearity, residuals, outlier influence; never extrapolate quietly.

## MODULE 6 · STATISTICS &amp; EPIDEMIOLOGY METHODS · LESSON 76 OF 105

## 76. Logistic regression and odds ratios

Health outcomes are usually yes/no — readmitted or not, vaccinated or not, died or survived. A straight line can't model a probability (it would cheerfully predict 130% or –20%), so we use **logistic regression**, which models the \*log-odds\* of the outcome. Its coefficients, exponentiated, become the **odds ratios** that fill every epidemiology paper you'll ever read.

**GLM WITH FAMILY = BINOMIAL**

```
set.seed(9)
readm <- tibble(
  age65 = rbinom(500, 1, 0.45),
  imd_low = rbinom(500, 1, 0.4), # most-deprived flag
  readmit = rbinom(500, 1, plogis(-2 + 0.9*age65 + 0.55*imd_low))
)
model <- glm(readmit ~ age65 + imd_low,
  data = readm, family = binomial)
library(broom)
tidy(model, exponentiate = TRUE, conf.int = TRUE)
#   term      estimate conf.low conf.high
# (Intercept)  0.13      0.09    0.18    <- baseline ODDS, not a ratio
# age65        2.42      1.57    3.77
# imd_low       1.69      1.10    2.61
```

`exponentiate = TRUE` is the magic switch: raw coefficients are log-odds (unreadable); `exp()` turns them into odds ratios. Reading the table: patients 65+ have **2.4 times the odds** of readmission compared with under-65s, \*adjusted for deprivation\*; the most deprived have 1.7 times the odds, adjusted for age. OR = 1 means no association — so a CI crossing 1 (e.g. 0.9–1.4) means the data can't rule out "nothing".

Precision of language, because it's routinely fumbled:

- **Odds** =  $p/(1-p)$ : an event with probability 0.25 has odds of 1-to-3 (0.33). Odds are not probabilities.
- An odds \*ratio\* compares two groups' odds. "2.4 times the odds" is correct; "2.4 times as likely" is not (though for rare outcomes — roughly <10% — the OR approximates the risk ratio, which is why the sloppy phrasing survives).
- Adjusted ORs answer "holding the other variables in this model constant" — no more, no less. Unmeasured confounders remain unmeasured.

Continuous predictors work too — `glm(readmit ~ age, ...)` gives the OR \*per year of age\*; per-decade is often friendlier (`I(age/10)`).

This model is the analytic backbone of case-control studies, risk-factor analyses and adjusted service comparisons; recognising `tidy(m, exponentiate = TRUE)` output means you can now read the results tables of most observational papers.



**QUICK CHECK**

**An adjusted OR for smoking is 1.8 (95% CI 1.2-2.7) for delayed discharge. Correct sentence?**

- ☐ Smokers are 80% likely to be delayed
- ✓ **Smokers have about 1.8 times the odds of delayed discharge, adjusted for the model's other variables; the CI excludes 1, so an association is supported**
- ☐ Smoking causes 1.8 delays
- ☐ 1.8% of delays are due to smoking

**Why:** Odds ratio  $\neq$  probability  $\neq$  percentage of cases. And "adjusted" earns exactly one clause: conditional on covariates in THIS model.

**QUICK CHECK**

**An OR's 95% CI runs 0.85 to 1.60. The takeaway?**

- ☐ The exposure is protective
- ☐ The exposure is harmful
- ✓ **The data are consistent with anything from a modest decrease to a moderate increase — no clear association demonstrated**
- ☐ The model failed to converge

**Why:** CIs crossing 1 (the null for ratios) mean uncertainty spans "less", "same" and "more". Report the interval, not a verdict.

**EXERCISE**

Fit the readmission model above but add an interpretation task: produce the tidy OR table, then write the one-sentence result for imd\_low suitable for a report, including the CI and the word "adjusted".

**SOLUTION · R**

```
model <- glm(readmit ~ age65 + imd_low, data = readm, family = binomial)
tidy(model, exponentiate = TRUE, conf.int = TRUE)
# "After adjusting for age group, patients living in the most deprived
# areas had 1.7 times the odds of 30-day readmission compared with other
# patients (OR 1.69, 95% CI 1.10 to 2.61)."
```

**KEY POINTS**

- Binary outcome → glm(..., family = binomial); exponentiate coefficients → odds ratios.
- OR = ratio of odds, not of probabilities; CI crossing 1 = no demonstrated association.
- "Adjusted for" means: for the variables in this model, only.

**MODULE 6 · STATISTICS & EPIDEMIOLOGY METHODS · LESSON 77 OF 105****77. SPC charts: Making Data Count**

Every metric wiggles. The monthly ritual of "admissions rose 6% — investigate!" followed next month by "fell 5% — celebrate!" is organisational noise-chasing, and NHS England's **Making Data Count** programme exists to end it. Its tool is the **statistical process control (SPC) chart**: your metric over time, with a centre line (the mean) and **control limits** at  $\pm 3$  standard deviations marking the range that pure routine variation should occupy.

The core distinction: **common-cause variation** (the ordinary jitter of a stable process — inside the limits, no pattern) versus **special-cause variation** (a signal that something real changed). Only special cause deserves investigation; reacting to common cause is management by lottery.

Signals, per the NHS rules: a point outside a control limit; a **shift** — 6 or more consecutive points on the same side of the centre line; a **trend** — 6 or more consecutively rising or falling; and 2 of 3 points hugging a limit (outer third). No signal = the process is doing what it always does, however dramatic this month's wiggle feels.

#### NHSRLOTTHEDOTS: THE COMMUNITY STANDARD

```
library(tidyverse)
# install.packages("NHSRplotthedots") # NHS-R community package
library(NHSRplotthedots)
set.seed(21)
monthly <- tibble(
  month = seq(as.Date("2024-04-01"), by = "month", length.out = 24),
  falls = c(rpois(15, 30), rpois(9, 41)) # a real step-up at month 16
)
ptd_spc(monthly,
  value_field = falls,
  date_field = month,
  improvement_direction = "decrease")
```

`ptd_spc()` draws the chart with Making Data Count's standard colouring: grey points for common cause, orange/blue highlighting for special cause in the concerning/improving direction. `improvement_direction` tells it which way is good (falls: down). The example's built-in step change gets flagged as a shift — the chart finds in seconds what a table of 24 numbers hides.

Practice notes: you need ~12+ points before limits mean much (fewer = plot the run chart and wait); when a process *\*genuinely\** changes (a new pathway, a merged service), recalculate limits from the change point rather than blending two eras; and never put targets on as if they were control limits — a target is an aspiration, limits are what the process actually delivers. The chart's most valuable sentence is often "this process will keep producing ~30–45 falls a month until the process itself changes" — capability, not commentary.

This is also your defence in meetings: "this month is within control limits" is the polite, evidenced version of "that's noise, and chasing it wastes a ward manager's afternoon."

#### QUICK CHECK

**A metric falls 8% from last month but sits well inside its control limits with no rule triggered. The SPC-literate response?**

- ☐ Launch an investigation — 8% is a lot
- ✓ **Note it as common-cause variation; no action from one wiggle inside the limits**
- ☐ Recalculate the limits immediately
- ☐ Delete the point as an outlier

**Why:** Two-point comparisons are how noise gets promoted to news. The limits define what "normal wiggle" is for *THIS* process; inside them, this month is unremarkable.

**QUICK CHECK**

**Seven consecutive months sit below the centre line, all inside the limits. This is...**

- ☐ Nothing — no point breached a limit
- ✓ **A shift: a special-cause signal that the process level has likely changed**
- ☐ Proof the target was met
- ☐ A reason to widen the limits

**Why:** *Runs rules catch sustained change that never breaches  $\pm 3\sigma$ . Six-plus one side of the centre is astronomically unlikely by chance — investigate (and if it's a real new level, re-baseline).*

**EXERCISE**

The falls data above shows a shift. Suppose investigation confirms a genuine process change (new frailty ward opened month 16). Describe — in code comments or prose — the two things the SPC chart should now do, and re-run `ptd_spc` using its `rebase` argument (`ptd_rebase`) to implement the first.

**SOLUTION · R**

```
# 1) Recalculate limits from the change point, so the chart describes the
#    NEW process rather than blending two different eras:
ptd_spc(monthly,
  value_field = falls,
  date_field  = month,
  improvement_direction = "decrease",
  rebase = ptd_rebase(as.Date("2025-07-01")))

# 2) Annotate the change (caption/vertical line) so readers connect the
#    signal to its cause — the chart shows THAT it changed; the note says WHY.
```

**KEY POINTS**

- Common cause = routine wiggle (no action); special cause = signal (limit breach, 6+ shift, 6+ trend).
- `NHSRplotthedots::ptd_spc` implements Making Data Count house style; state `improvement_direction`.
- Re-baseline at confirmed process changes; targets are not control limits.

**MODULE 6 · STATISTICS & EPIDEMIOLOGY METHODS · LESSON 78 OF 105**

## 78. Funnel plots: comparing organisations fairly

Rank ten GP practices by any percentage and someone finishes last — but with 40 patients, one practice's "worst" may be pure chance while a giant practice's small deviation is a genuine signal. **Funnel plots** are the fair way to compare units of wildly different sizes, and they're standard furniture in NHS and UKHSA benchmarking.

The construction: x-axis = size (denominator: list size, admissions, births); y-axis = the indicator (a proportion or rate); a horizontal line at the overall average; and **control limits** that funnel inward as size grows — wide for small units (chance moves them a lot), tight for large ones (it can't). Limits at  $\pm 2SD$  (~95%) and  $\pm 3SD$  (~99.8%) are conventional.

**BUILDING ONE**

```
library(tidyverse)
set.seed(5)
practices <- tibble(
```

## BUILDING ONE

```
practice = sprintf("P810%02d", 1:12),
n         = c(2100, 3400, 5600, 1200, 8200, 4400,
              2900, 6100, 950, 7300, 3800, 5100)
) |>
  mutate(uptake = rbinom(12, n, 0.62) / n)      # true rate 62% everywhere
overall <- weighted.mean(practices$uptake, practices$n)
limits <- tibble(n = seq(500, 9000, 50)) |>
  mutate(se      = sqrt(overall * (1 - overall) / n),
         lo95    = overall - 1.96 * se, hi95 = overall + 1.96 * se,
         lo998   = overall - 3.09 * se, hi998 = overall + 3.09 * se)
ggplot() +
  geom_line(data = limits, aes(n, lo95), linetype = "dashed", colour = "grey55") +
  geom_line(data = limits, aes(n, hi95), linetype = "dashed", colour = "grey55") +
  geom_line(data = limits, aes(n, lo998), colour = "grey40") +
  geom_line(data = limits, aes(n, hi998), colour = "grey40") +
  geom_hline(yintercept = overall, colour = "#005EB8") +
  geom_point(data = practices, aes(n, uptake), colour = "#0e7c86", size = 2.5) +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(title = "Vaccination uptake by practice",
       x = "Eligible patients", y = "Uptake",
       caption = "Dashed 95% and solid 99.8% limits around county average")
```

Reading it: points **inside** the funnel are consistent with the average — differences are what chance produces at that size, and ranking them is astrology. Points **outside** (especially beyond 99.8%) are genuinely different — worth understanding, which is not the same as worth blaming: outliers can reflect case mix, coding practice, data quality or population differences before anything about performance. The funnel starts conversations; it doesn't end them.

Every point in this simulation came from an identical 62% true rate — yet the small practices scatter dramatically. That's the whole argument against league tables in one picture.

Real-world refinements: indicators often need risk adjustment or age-standardisation *\*before\** the funnel (compare like with like); count/rate indicators use Poisson-based limits (the **PHEindicatormethods** package computes both flavours properly); and **overdispersion** — when far more than ~5% of points fall outside 95% limits, unit-level differences beyond chance exist system-wide, and methods that widen limits accordingly (as UKHSA's Fingertips does) stop the plot crying wolf.

## QUICK CHECK

**A 950-patient practice sits 6 points below average, inside the 95% funnel. A 8,200-patient practice sits 3 points below, outside 99.8%. Who warrants attention?**

- ☐ The small practice — bigger gap
- ✓ **The large practice — its smaller gap is statistically incompatible with chance at that size; the small one is within expected variation**
- ☐ Both equally
- ☐ Neither; averages are all that matter

**Why:** The funnel exists precisely because gap size alone misleads: precision scales with  $n$ . Distance-in-SEs, not distance-in-points, is the currency.

**QUICK CHECK**

**A practice sits above the 99.8% limit on an unadjusted indicator. Before "performance", rule out...**

✓ **Case mix, population age structure, coding and data-quality differences**

- ☐ Nothing; the statistics are conclusive
- ☐ The colour of the point
- ☐ Whether the practice complained before

**Why:** *Outside-the-funnel* = "really different", not "really worse at the job". Standardise or risk-adjust, check the data, then talk performance.

**EXERCISE**

Add labels to the plot for any practice outside the 95% limits (join practices to the limit formula: flag when  $|\text{uptake} - \text{overall}| > 1.96 * \sqrt{\text{overall} * (1 - \text{overall}) / n}$ ), using `ggrepel` or `geom_text`.

**SOLUTION · R**

```
flagged <- practices |>
  mutate(se = sqrt(overall * (1 - overall) / n),
         outside95 = abs(uptake - overall) > 1.96 * se)
# add to the ggplot:
# geom_text(data = filter(flagged, outside95),
#           aes(n, uptake, label = practice),
#           vjust = -1, size = 3)
# In this simulation, most runs flag ~0-1 practices – consistent with
# a common true rate plus chance, which is exactly the lesson.
```

**KEY POINTS**

- Funnel plots: indicator vs denominator size; limits narrow as n grows; inside = chance-compatible.
- Small units swing wildly by chance — league tables punish smallness, funnels don't.
- Outside the funnel starts an inquiry (case mix, coding, data) — not a verdict; use PHEindicator methods for proper rate funnels.

**MODULE 6 · STATISTICS & EPIDEMIOLOGY METHODS · LESSON 79 OF 105**

## 79. Screening and tests: sensitivity, specificity and the PPV trap

Screening programmes, diagnostic pathways, surveillance case definitions — all rest on four numbers from one 2×2 table, and on one counterintuitive fact every public health analyst must be able to explain to a room.

The table crosses test result against true status:

|               | Disease present     | Disease absent      |
|---------------|---------------------|---------------------|
| Test positive | True positive (TP)  | False positive (FP) |
| Test negative | False negative (FN) | True negative (TN)  |

- **Sensitivity** =  $TP / (TP + FN)$ : of people *with* the disease, the share the test catches. A property of the test.

- **Specificity** =  $TN / (TN + FP)$ : of people \*without\* it, the share correctly cleared. Also a property of the test.
- **PPV** (positive predictive value) =  $TP / (TP + FP)$ : of people who \*test positive\*, the share who actually have it. **Not** a fixed property of the test — it depends on how common the disease is.
- **NPV** =  $TN / (TN + FN)$ : of negatives, the share truly clear.

The trap, worked honestly. A good test — 90% sensitive, 95% specific — screens 10,000 people for a condition with **1% prevalence** (100 cases):

#### THE BASE-RATE CALCULATION

```
prev <- 0.01; sens <- 0.90; spec <- 0.95; N <- 10000
tp <- N * prev * sens           # 90 true positives
fn <- N * prev * (1 - sens)     # 10 missed
fp <- N * (1 - prev) * (1 - spec) # 495 false positives!
tn <- N * (1 - prev) * spec     # 9405
ppv <- tp / (tp + fp)           # 0.154 — 15%
npv <- tn / (tn + fn)           # 0.999
```

A positive result from this genuinely good test is wrong **85% of the time** — because the 5% false-positive rate applies to 9,900 healthy people, swamping the 90 real cases. Raise prevalence to 20% (a symptomatic clinic population) and PPV jumps past 80% with the \*same test\*. This is why screening programmes confirm positives with second-stage tests, why "test everyone for everything" backfires, and why the UK National Screening Committee weighs harms of false positives as seriously as benefits of detection.

The sensitivity/specificity **trade-off** is a dial, not a fact of nature: loosen a case definition (or a test threshold) and sensitivity rises while specificity falls. Surveillance during an outbreak often chooses sensitive definitions (miss nothing, tolerate false alarms); confirmation for a harmful treatment chooses specific ones. State the choice; don't let it be an accident.

For analysts, the same table evaluates \*your\* artefacts: a data-quality flag, a frequent-attender prediction, a record-linkage rule — each has a sensitivity, a specificity, and a PPV that collapses when the thing you're hunting is rare. "The algorithm is 95% accurate" is meaningless until you ask: accurate at what prevalence, and which errors does it make?

#### QUICK CHECK

**Same test (90% sens, 95% spec) used in a population where prevalence is 0.1% instead of 1%. PPV roughly...**

- ☐ Stays 15% — it's the same test
- ✓ **Falls to under 2% — false positives from the huge healthy majority swamp the few true cases**
- ☐ Rises, because the disease is rarer
- ☐ Becomes 90%

**Why:** PPV rides prevalence. At 1-in-1,000: ~9 TPs vs ~500 FPs →  $PPV \approx 1.8\%$ . The rarer the target, the less a positive means — the core argument against indiscriminate screening.

**QUICK CHECK**

**An outbreak team broadens the case definition "so we don't miss anyone". The predictable statistical consequence?**

- ☐ Higher specificity, lower sensitivity
- ✓ **Higher sensitivity, lower specificity — more false positives to investigate**
- ☐ Both improve
- ☐ PPV becomes 100%

**Why:** Casting a wider net catches more true cases AND more bystanders. Sometimes right for surveillance — but the false-positive workload is a real cost to plan for.

**EXERCISE**

A frequent-attender prediction model flags patients: of 200 who truly became frequent attenders, it flagged 150; of 9,800 who didn't, it flagged 490. Compute sensitivity, specificity and PPV, and write the sentence a service manager needs to hear about acting on flags.

**SOLUTION · R**

```
tp <- 150; fn <- 50; fp <- 490; tn <- 9310
sens <- tp / (tp + fn)      # 0.75
spec <- tn / (tn + fp)      # 0.95
ppv <- tp / (tp + fp)       # 0.234
# "The model catches 75% of future frequent attenders, but only about
# 1 in 4 flagged patients will actually become one — any intervention
# offered to flagged patients must be cheap and harmless enough to give
# to three 'wrong' patients for every right one."
```

**KEY POINTS**

- Sensitivity/specificity describe the test; PPV describes a positive result and depends on prevalence.
- Rare conditions → even excellent tests yield mostly false positives; hence confirmatory stages.
- The same 2x2 evaluates models, flags and linkage rules — always ask "at what prevalence?"

**MODULE 7**

# Module 7 — Pipelines, Git & reproducibility

**In this module**

- 80** Connecting R to databases: DBI and odbc
- 81** dbplyr: dplyr that writes SQL for you
- 82** Parameterised queries: the injection lesson
- 83** Project structure and {here}: work that travels
- 84** Git: version control for analysts
- 85** GitHub: sharing, review and the audit trail

**86** RAP: Reproducible Analytical Pipelines

**87** Scheduling and testing: analysis that runs itself (safely)



## MODULE 7 · PIPELINES, GIT &amp; REPRODUCIBILITY · LESSON 80 OF 105

## 80. Connecting R to databases: DBI and odbc

Until now you've imagined CSV exports ferrying data from SQL to R. Professionals skip the ferry: R talks to the warehouse **directly**, so the query, the analysis and the report are one unbroken, re-runnable chain. Two packages do it: **DBI** (the universal steering wheel) and **odbc** (the connector for the ODBC drivers that NHS SQL Server estates run on).

### THE CANONICAL CONNECTION

```
library(DBI)
library(odbc)
con <- dbConnect(odbc(),
  Driver   = "ODBC Driver 18 for SQL Server",
  Server   = "trust-dw.example.nhs.uk",
  Database = "PublicHealth",
  Trusted_Connection = "yes",      # your Windows login
  TrustServerCertificate = "yes") # common on internal estates
```

`Trusted_Connection = "yes"` uses your network identity — **no password in the script**, which is exactly how it should be. If a system genuinely needs credentials, they live in environment variables or the OS keyring (the **keyring** package), never in code: scripts get shared, emailed and committed; secrets in them leak. Treat a password in a script as an incident, not a style choice.

Once connected, the verbs are few:

### TALK TO THE DATABASE

```
dbListTables(con)                # what's there?
adm <- dbGetQuery(con, "
  SELECT admission_id, patient_id, admission_date, specialty
  FROM admissions
  WHERE admission_date >= '2025-04-01'
")                                # runs SQL, returns a tibble-like df
dbDisconnect(con)                # always, when done
```

Everything from Modules 2–3 pours through that `dbGetQuery()` string — CTEs, window functions, the lot — and lands as a data frame ready for dplyr and ggplot. The division of labour from Lesson 40's world holds: let the database filter, join and aggregate millions of rows; bring back the thousands you'll actually analyse.

One-time driver setup per machine:

### WINDOWS

Usually already present on NHS machines (check: search "ODBC Data Sources (64-bit)" → Drivers tab). If  
 ↳ missing, install "Microsoft ODBC Driver 18 for SQL Server" from Microsoft's site – or via IT self-s  
 ↳ ervice on managed laptops.

### MACOS

```
brew install unixodbc
brew tap microsoft/mssql-release https://github.com/Microsoft/homebrew-mssql-release
brew install msodbcsql18
```

### LINUX

```
sudo apt install unixodbc unixodbc-dev
# then Microsoft's msodbcsql18 package per their apt instructions for your distro
```

Diagnostics, in order, when connection fails: does the driver name in your code *exactly* match one listed by `odbc::odbcListDrivers()`? Can you reach the server at all (VPN? on-site network?)? Do you have permission on that database (the DBA grants it; ask)? Error messages here are verbose but honest — read the first line.

**YOU ARE ON LIVE RAILS NOW**

A connection is real. SELECT freely, but everything from Lesson 41 applies double: create only in your sandbox, and treat DELETE/UPDATE/DROP as requiring a colleague's eyes. The playground forgives; production remembers.

**QUICK CHECK**

**Why is `Trusted_Connection = "yes"` preferred over `Uid/Pwd` arguments in the connection string?**

- ☐ It's faster
- ✓ **It authenticates with your network identity, so no credential ever appears in the script**
- ☐ It bypasses permissions
- ☐ It works without a driver

**Why:** *Scripts travel — passwords in them travel too. Integrated auth (or keyring/env vars when unavoidable) keeps secrets out of code, which is baseline governance.*

**QUICK CHECK**

**`dbGetQuery(con, sql)` returns...**

- ☐ A live pointer that updates as data changes
- ✓ **A regular data frame: the result copied into R's memory at execution time**
- ☐ A CSV on disk
- ☐ Nothing until you call `collect()`

**Why:** *`dbGetQuery` executes and pulls the full result immediately — a snapshot. (Lazy, stays-in-database references are the NEXT lesson's `dbplyr` trick.)*

**EXERCISE**

Write the connection-and-extract script for a monthly job: connect with integrated auth, pull emergency admissions for the latest complete month into `emerg`, print how many rows arrived (a sanity check that belongs in every extract), and disconnect. Use a clearly-labelled date placeholder.

**SOLUTION · R**

```
library(DBI); library(odbc)
con <- dbConnect(odbc(),
  Driver = "ODBC Driver 18 for SQL Server",
  Server = "trust-dw.example.nhs.uk",
  Database = "PublicHealth",
  Trusted_Connection = "yes")
emerg <- dbGetQuery(con, "
  SELECT admission_id, patient_id, admission_date, specialty
  FROM admissions
  WHERE admission_method = 'Emergency'
  AND admission_date >= '2026-06-01'
  AND admission_date < '2026-07-01'
")
message(nrow(emerg), " rows extracted") # eyeball vs expectation
dbDisconnect(con)
```

## KEY POINTS

- DBI + odbc connect R to SQL Server; integrated auth keeps credentials out of scripts.
- dbGetQuery runs any SQL you can write and returns a data frame; disconnect when done.
- Heavy lifting in the database, analysis in R; sandbox rules apply on live systems.

## MODULE 7 • PIPELINES, GIT &amp; REPRODUCIBILITY • LESSON 81 OF 105

## 81. dbplyr: dplyr that writes SQL for you

Here's a happy secret: the dplyr you learned in Module 4 can run *\*inside the database\**. **dbplyr** translates dplyr pipelines into SQL, ships them to the server, and only pulls results when you say so — one syntax, two engines.

## LAZY TABLES

```
library(dplyr)
library(dbplyr)
adm_db <- tbl(con, "admissions")      # a REFERENCE, not a download
summary_q <- adm_db |>
  filter(admission_method == "Emergency",
         admission_date >= "2025-04-01") |>
  count(specialty, sort = TRUE)
summary_q                             # runs on the server, previews first rows
show_query(summary_q)                 # reveals the generated SQL
```

`tbl()` creates a **lazy table**: nothing moves. Each verb adds to a query plan; printing runs it server-side and shows a preview (note the row count displays as ?? — it genuinely hasn't counted). The pipeline above becomes `SELECT specialty, COUNT(*) ... GROUP BY ... ORDER BY ...` — check with `show_query()`, which is also a wonderful SQL tutor: write dplyr, read the SQL it makes.

The verb that matters most is the one that ends laziness:

## COLLECT(): THE MOMENT OF TRANSFER

```
local_summary <- summary_q |> collect() # NOW rows travel into R memory
```

The professional pattern is thus: **filter and aggregate lazily, collect small**. A million-row table grouped to 40 specialty counts should cross the network as 40 rows, not a million. Collecting a whole raw table to "work on it locally" recreates the CSV-ferry problem with extra steps.

What translates? Most of your toolkit — filter, select, mutate, group\_by, summarise, joins between `tbl()`s, many functions (`year()`, `paste0`, `if_else`) map to SQL equivalents. What doesn't: arbitrary R functions (your own `age_at()`, model fitting, `ggplot`) — those need real data, so they come *\*after\** `collect()`. The error "no translation for function X" means exactly that: do X post-collect.

Mixed workflows are the everyday reality:

## LAZY HEAVY LIFTING, LOCAL FINESSE

```
plot_df <- tbl(con, "admissions") |>
  filter(admission_date >= "2024-04-01") |>
  count(month = substr(admission_date, 1, 7)) |> # translates to SQL
  collect() |> # small result crosses
  mutate(month = as.Date(paste0(month, "-01"))) # R-only finishing
# ggplot(plot_df, aes(month, n)) + geom_line()
```

**QUICK CHECK**

**adm\_db <- tbl(con, "admissions") followed by filter() and count() has so far transferred how much data to your laptop?**

- ☐ The whole table
- ☐ The filtered rows
- ✓ **Essentially none — lazy tables build a query; data moves only at collect() (or a preview when printed)**
- ☐ One page of rows per verb

**Why:** Laziness is the feature: computation goes TO the data. The network bill arrives only when collect() does.

**QUICK CHECK**

**Your pipeline errors: "no known translation for my\_age\_function()". The fix?**

- ☐ Rewrite the function in C
- ✓ **Move that step after collect() — custom R functions can't run inside the database**
- ☐ Wrap it in show\_query()
- ☐ Use = instead of <-

**Why:** dbplyr can only translate what SQL can express. Aggregate lazily, collect the small result, then apply bespoke R locally.

**EXERCISE**

Using lazy tables on con: count completed admissions per specialty for FY 2025/26 with mean length of stay (julianday isn't SQL Server — use datediff(day, admission\_date, discharge\_date) via sql() or rely on dbplyr's difftime translation; simplest portable route: compute los in the database with sql("DATEDIFF(day, admission\_date, discharge\_date)")), show the generated SQL, then collect.

**SOLUTION · R**

```
q <- tbl(con, "admissions") |>
  filter(!is.na(discharge_date),
         admission_date >= "2025-04-01",
         admission_date < "2026-04-01") |>
  mutate(los = sql("DATEDIFF(day, admission_date, discharge_date)")) |>
  group_by(specialty) |>
  summarise(n = n(), mean_los = mean(los, na.rm = TRUE)) |>
  arrange(desc(n))
show_query(q)      # inspect the T-SQL it wrote
result <- collect(q)
```

**KEY POINTS**

- tbl(con, "table") = lazy reference; verbs build SQL; show\_query() reveals it.
- collect() is the only data transfer — filter/aggregate first, collect small.
- Untranslatable R (own functions, models, plots) belongs after collect().

## MODULE 7 · PIPELINES, GIT &amp; REPRODUCIBILITY · LESSON 82 OF 105

## 82. Parameterised queries: the injection lesson

Sooner or later you'll build SQL from user input — a practice code typed into a Shiny box, a month passed to a report. The obvious way is string-gluing, and the obvious way is a security hole famous enough to have a cartoon (xkcd's "Little Bobby Tables"):

**NEVER BUILD SQL LIKE THIS**

```
practice <- input_from_somewhere      # imagine: "P81004"
sql <- paste0("SELECT * FROM admissions WHERE practice_code = '",
              practice, "'")
dbGetQuery(con, sql)
```

If that input is ever `P81004'; DROP TABLE admissions;--`, your polite `SELECT` becomes two statements, the second catastrophic. That's **SQL injection** — for decades the top web vulnerability class, and the mechanism is exactly this: data allowed to rewrite code. Even without malice, an innocent apostrophe (O'Brien in a surname filter) shatters the string and crashes the query. Same bug, friendlier symptom.

The fix is structural, not vigilance: **parameterised queries** keep SQL and values in separate channels. The database receives the query shape with placeholders, then the values — which can *never* be executed, only compared:

**THE SAFE PATTERN**

```
sql <- "SELECT admission_id, admission_date, specialty
      FROM admissions
      WHERE practice_code = ?
      AND admission_date >= ?"
dbGetQuery(con, sql, params = list("P81004", "2025-04-01"))
```

`?` marks each slot (SQL Server via `odbc` uses `?`; some backends use `$1` — DBI handles the dialect). Apostrophes, semicolons, comment markers — all arrive as inert text. The **glue** package's `glue_sql()` offers the same safety with readable named templates:

**GLUE\_SQL FOR READABLE TEMPLATES**

```
library(glue)
q <- glue_sql("
  SELECT * FROM admissions
  WHERE practice_code = {practice}
  AND admission_date >= {start_date}
", practice = "P81004", start_date = "2025-04-01", .con = con)
dbGetQuery(con, q)
```

(`glue_sql`, not plain `glue` — the `_sql` variant quotes and escapes for the connection.) One legitimate wrinkle: placeholders carry *values*, not identifiers — you can't parameterise a table or column name. When a table name must vary, validate it against a hard-coded allow-list first (`match.arg()` is perfect), then interpolate. Choose from known-good; never trust free text into structure.

The habit generalises beyond security: parameterised queries are also how Lesson 66-style reports take a month or an organisation as input — the same three lines rerun for any practice, injection-proof and typo-proof at once.

**QUICK CHECK**

**Why does a parameterised query stop injection where careful string escaping tends to fail?**

- ☐ Parameters are encrypted
- ✓ **SQL and data travel separately by design — a value cannot become code, so there's nothing to escape imperfectly**
- ☐ The database blocks apostrophes
- ☐ It runs in a sandbox

**Why:** *Escaping is a fight you must win every time; separation removes the fight. The value channel is inert by construction.*

**QUICK CHECK**

**A user-supplied string chooses which TABLE to query. Correct handling?**

- ☐ paste0 it in — tables can't be injected
- ☐ Parameterise it with ?
- ✓ **Validate against a fixed allow-list of known table names, then interpolate the matched value**
- ☐ Refuse ever to vary a table

**Why:** *Placeholders can't hold identifiers, so the defence shifts: user input SELECTS from your list; it never BECOMES the name. match.arg(input, c("admissions", "vaccinations")) is the idiom.*

**EXERCISE**

Write a safe function `monthly_extract(con, specialty, month_start, month_end)` returning that specialty's admissions between two dates, using `params`. Bonus: reject unknown specialties with `match.arg` against a small allow-list.

**SOLUTION · R**

```
monthly_extract <- function(con, specialty, month_start, month_end) {
  specialty <- match.arg(specialty,
    c("Respiratory", "Cardiology", "General Medicine",
      "Elderly Care", "Gastroenterology"))
  dbGetQuery(con, "
    SELECT admission_id, patient_id, admission_date
    FROM admissions
    WHERE specialty = ?
      AND admission_date >= ?
      AND admission_date < ?",
    params = list(specialty, month_start, month_end))
}
# monthly_extract(con, "Respiratory", "2026-06-01", "2026-07-01")
```

**KEY POINTS**

- Never paste user input into SQL; one apostrophe breaks it, one payload owns it.
- `params = list(...)` / `glue_sql` separate code from data — injection becomes impossible, not just unlikely.
- Identifiers can't be parameters: allow-list them; values always ride placeholders.

## MODULE 7 · PIPELINES, GIT &amp; REPRODUCIBILITY · LESSON 83 OF 105

## 83. Project structure and {here}: work that travels

Open a six-month-old folder containing `analysis_FINAL_v3_ACTUAL.R`, `data(2).csv` and `Untitled1.R`, and you meet the enemy: past-you. Structure is the cheap defence, and one layout serves almost every analysis:

### THE STANDARD ANALYSIS PROJECT

```
readmissions-2026/
├─ readmissions-2026.Rproj  ← open THIS, always
├─ README.md              ← what, why, how to run, who to ask
├─ data/                  ← raw inputs, READ-ONLY, never edited
├─ R/                     ← functions used by multiple scripts
├─ scripts/               ← numbered pipeline steps
│   ├─ 01_extract.R
│   ├─ 02_clean.R
│   ├─ 03_analyse.R
│   └─ 04_report.qmd
├─ outputs/               ← everything generated; deletable & rebuildable
└─ docs/                  ← specs, definitions, sign-offs
```

The numbered scripts encode run order — a stranger (or a scheduler) knows 01 precedes 02 without asking. The deeper contract: `outputs/` is **disposable by design**. Delete it, run 01→04, and everything returns identically. If a chart exists that no script can regenerate, that chart is a rumour.

Paths are where projects break in transit, and the villain is the absolute path:

`C:/Users/priya/Documents/...` works for exactly one human on one machine. `setwd()` is the same trap with extra steps — it hard-codes *\*your\** geography into shared code. The cure is the **here** package:

### HERE(): PATHS THAT WORK EVERYWHERE

```
library(here)
adm <- read_csv(here("data", "admissions.csv"))
ggsave(here("outputs", "figures", "trend.png"), p)
```

`here()` finds the project root (the `.Rproj` marks it) and builds paths from there — identical code on Windows, Mac, Linux, a colleague's laptop, or the server that runs it nightly at 3am. Combined with Lesson 42's clean-session settings, you get the professional test of reproducibility: **fresh R session, Run All, zero errors** — on someone else's machine.

The `README.md` deserves five honest minutes: purpose in one paragraph, data sources and their sensitivity, the run order, known caveats, and an owner's name. Six months on, that file is the difference between refresh-in-an-hour and archaeology-for-a-week. The same applies to naming inside the project: `02_clean.R` and `rate_by_lsoa.csv` explain themselves; dates in output filenames (`summary_2026-07.csv`) stop last month's numbers impersonating this month's.

### THE 20-MINUTE REFACTOR

Take any past piece of work and re-house it in this layout with `here()` paths. Then the acid test: fresh session, Run All. The errors you hit are exactly the hidden dependencies this lesson exists to kill.

**QUICK CHECK**

**A script beginning `setwd("C:/Users/asha/projects/flu")` fails on every other machine. The structural fix?**

- ☐ Everyone creates that folder path
- ✓ **An RStudio Project plus `here()` — paths resolve from the project root wherever it lives**
- ☐ Comment out the `setwd`
- ☐ Use shorter paths

**Why:** Portability comes from relative-to-root paths, not from cloning one analyst's disk layout. `.Rproj` + `here()` is the standard.

**QUICK CHECK**

**Why must outputs/ be entirely regenerable from scripts?**

- ☐ To save disk space
- ✓ **It proves the analysis is real: every number and figure has executable provenance — and refreshes become trivial**
- ☐ Because Git ignores it
- ☐ Outputs are unimportant

**Why:** "Delete outputs, rerun, identical results" is reproducibility made operational. Anything that can't be rebuilt can't be trusted or updated.

**EXERCISE**

Sketch (as a text tree) the project structure for a quarterly vaccination-uptake report that extracts from SQL, standardises rates in R, and renders a Quarto report — then write the `here()`-based read and save lines used in `02_clean.R`.

**SOLUTION · TEXT**

```
vax-uptake-quarterly/
├─ vax-uptake-quarterly.Rproj
├─ README.md
├─ data/                # any fixed lookups (ESP weights etc.)
├─ R/standardise.R      # shared rate functions
├─ scripts/
│   ├─ 01_extract.R      # DBI pull → data/raw_extract.csv
│   ├─ 02_clean.R
│   ├─ 03_standardise.R
│   └─ 04_report.qmd
└─ outputs/
```

**SOLUTION · R**

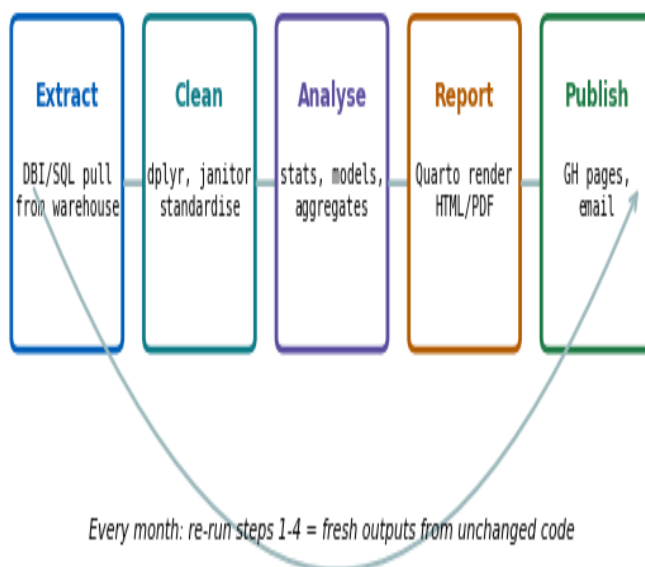
```
# in 02_clean.R
library(here)
raw  <- read_csv(here("data", "raw_extract.csv"))
clean <- raw |> ...          # cleaning steps
write_csv(clean, here("outputs", "clean_uptake.csv"))
```

**KEY POINTS**

- One layout: `data/` (read-only), `scripts/` (numbered), `outputs/` (regenerable), `README` (five honest minutes).
- `here()` + `.Rproj` replace `setwd()` and absolute paths — code runs anywhere.
- The test: fresh session, Run All, on another machine.



## Reproducible Analytical Pipeline (RAP)



### MODULE 7 · PIPELINES, GIT & REPRODUCIBILITY · LESSON 84 OF 105

## 84. Git: version control for analysts

`report_v2_final_AMENDED_use_this.R` is version control by filename, and it fails exactly when it matters.

**Git** — the real thing — records snapshots of your project so you can see what changed, when, why, and by whom, and return to any point. It is standard equipment in NHS analytics job adverts, and the backbone of the RAP lesson ahead.

Install once:

#### WINDOWS

```

winget install --id Git.Git
(Or download from git-scm.com. On managed NHS laptops: IT self-service catalogue.)
Then in Git Bash or PowerShell:
git config --global user.name "Your Name"
git config --global user.email "you@example.nhs.uk"
  
```

#### MACOS

```

xcode-select --install # if you did Lesson 46's setup, Git is already there
git config --global user.name "Your Name"
git config --global user.email "you@example.nhs.uk"
  
```

#### LINUX

```

sudo apt install git
git config --global user.name "Your Name"
git config --global user.email "you@example.nhs.uk"
  
```

The mental model is three zones: your **working folder** (files as they are), the **staging area** (changes you've selected for the next snapshot), and the **repository** (the permanent history). Four commands cover 90% of solo use:

#### THE DAILY LOOP

```
git init          # once per project: start tracking
git status        # what's changed? (run constantly)
git add scripts/02_clean.R # stage chosen changes ( "." for all)
git commit -m "Exclude test patients from cohort"
git log --oneline  # the story so far
```

A **commit** is a labelled snapshot of the whole project. Commit small and often — one logical change per commit — and write messages that state *\*why\**: "Fix FY boundary so 31 March lands in prior year" will save a future investigator an afternoon; "changes" will not. RStudio's Git pane does all of this with checkboxes if the terminal isn't your happy place; the concepts are identical.

Two guardrails specific to health work. First, `.gitignore`: a file listing what Git must never track — and in the NHS that means *\*data\**:

#### .GITIGNORE FOR A HEALTH ANALYTICS REPO

```
data/
outputs/
*.csv
*.xlsx
.Rhistory
.Rdata
```

Git histories are forever and get copied around; a patient-level extract committed once is exceptionally hard to truly delete. **Code in Git, data in the warehouse** — the repository holds the recipe, never the ingredients. Second, look before you leap back: `git diff` shows exactly what changed since the last commit, and `git restore file.R` discards uncommitted changes to a file — the "undo" that makes fearless experimentation rational.

#### QUICK CHECK

**The single most important entry in a health-analytics .gitignore?**

- ☐ .Rhistory
- ✓ **Data files/folders — patient-level data must never enter the repository's permanent history**
- ☐ README.md
- ☐ The .Rproj file

**Why:** Git never forgets, and repos travel. Ignoring data/ and spreadsheet/CSV patterns keeps IG intact by construction. Recipe in Git; ingredients stay home.

#### QUICK CHECK

**Which commit message earns its keep?**

- ☐ "stuff"
- ☐ "final version"
- ✓ **"Use ONS 2025 mid-year estimates as rate denominators"**
- ☐ "asdfgh"

**Why:** Messages are letters to future investigators (usually you). State the why/what; the diff already shows the how.

**EXERCISE**

Write the terminal session that takes the vax-uptake project under version control: initialise, create a .gitignore protecting data and outputs, stage everything, and make a well-messaged first commit. Then show the command to view what changed after you later edit a script.

**SOLUTION · TEXT**

```
cd vax-uptake-quarterly
git init
printf "data/\noutputs/\n*.csv\n*.xlsx\n.Rhistory\n.RData\n" > .gitignore
git add .
git status                # confirm data/ is NOT listed
git commit -m "Initial pipeline: extract, clean, standardise, report"
# ... later, after editing 02_clean.R ...
git diff scripts/02_clean.R  # exact line changes since last commit
```

**KEY POINTS**

- Commit = labelled snapshot; small commits, messages that say why.
- status → add → commit → log is the daily loop; diff/restore make experiments safe.
- .gitignore data and outputs: Git holds code only — always, in health work.

**MODULE 7 · PIPELINES, GIT & REPRODUCIBILITY · LESSON 85 OF 105****85. GitHub: sharing, review and the audit trail**

Git on your laptop is a diary; **GitHub** (or an NHS-hosted GitLab/Azure DevOps — same ideas) makes it a shared, backed-up, reviewable record. NHS-R, UKHSA and a growing list of trusts and DHSC teams publish real pipelines there; your future employer will likely ask for your profile.

Connect local to remote:

**FIRST PUSH**

```
# after creating an empty repo on github.com:
git remote add origin https://github.com/you/vax-uptake-quarterly.git
git push -u origin main          # upload history; -u remembers the pairing
# thereafter just: git push (send) / git pull (fetch colleagues' work)
```

(Authentication today means a Personal Access Token or SSH key, not your password — GitHub walks you through it once.)

The collaboration engine is the **branch + pull request** loop, and it's worth learning even solo because it's how teams actually operate:

**THE FEATURE-BRANCH LOOP**

```
git switch -c fix-fy-boundary    # new branch: a parallel line of history
# ...edit, commit as usual...
git push -u origin fix-fy-boundary
# then on GitHub: open a Pull Request
```

A **pull request (PR)** says "here are my changes — review before they join `main`." A colleague reads the diff, comments line-by-line ("should this be `<` rather than `<=`?"), you push fixes, they approve, it merges. For analytical code this is **QA with an audit trail**: six months later, the PR shows what changed, who checked it, and what they debated — governance evidence that a folder of emailed scripts can never produce. Keep `main` as the always-works branch; experiments live on branches until reviewed.

Etiquette that makes review work: small PRs (one purpose, reviewable in ten minutes), a description saying what and why plus how you tested it, and reviews that critique code, never people. Reviewing others' PRs, incidentally, is the fastest reading-skills course an early-career analyst can take.

Round out the habit with the platform's furniture: **Issues** as the to-do/bug list wired to commits ("Fixes #12" in a commit message closes the ticket), the **README** rendering as your repo's front page, and — because mistakes happen — the rule that a secret or dataset pushed even briefly counts as exposed: rotate the credential and treat removal as damage limitation, not erasure. Public repo or private, the .gitignore discipline from last lesson is doing load-bearing work.

#### QUICK CHECK

**For an NHS analytics team, the pull request's chief value is...**

- ☐ Prettier diffs
- ✓ **Documented peer review: every change to the pipeline carries evidence of who checked it and why it was made**
- ☐ Faster code execution
- ☐ Automatic bug fixing

**Why:** PRs turn QA from a favour into a record. When an indicator is challenged, "reviewed and approved, discussion attached" is an answer; "Dave glanced at it" is not.

#### QUICK CHECK

**You accidentally pushed a file containing a database password, then deleted it in the next commit. Status?**

- ☐ Safe — it's gone
- ✓ **Compromised — history retains it; rotate the credential now and scrub history as secondary cleanup**
- ☐ Safe if the repo is private
- ☐ GitHub deletes secrets automatically

**Why:** Deletion adds a new snapshot; the old one persists (and forks/caches may hold it). Credentials that touch a remote are burned: rotate first, tidy second.

#### EXERCISE

Describe the full loop — commands plus GitHub actions — for safely changing the suppression threshold in a shared team pipeline from <5 to <8, with a colleague's review, referencing issue #23.

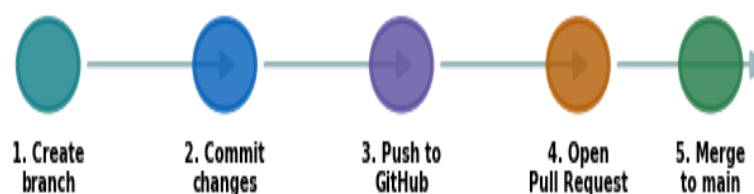
#### SOLUTION · TEXT

```
git switch main && git pull          # start current
git switch -c raise-suppression-threshold
# edit R/safe_outputs.R; update tests/docs
git add R/safe_outputs.R
git commit -m "Raise small-number suppression to <8 (fixes #23)"
git push -u origin raise-suppression-threshold
# On GitHub: open PR "Raise suppression threshold to <8"
#   - description: why (new IG guidance), what changed, tests re-run
#   - request review from colleague
# Colleague comments → you push fixes → approval → Merge
git switch main && git pull          # everyone's main now has it
```

**KEY POINTS**

- push/pull sync local Git with a shared remote; main stays always-working.
- Branch → PR → review → merge = QA with a permanent audit trail.
- Pushed secrets/data are exposed forever: rotate credentials; prevention beats scrubbing.

## Git & GitHub workflow



**Audit trail: Every change has a commit message + review**

*Code is traced; nothing merges without peer eyes; rollback is instant.*

### MODULE 7 • PIPELINES, GIT & REPRODUCIBILITY • LESSON 86 OF 105

## 86. RAP: Reproducible Analytical Pipelines

Everything in this module has been converging on one idea with an official name. A **Reproducible Analytical Pipeline (RAP)** is analysis built so that a competent stranger can regenerate every published number from code alone — no manual steps, no mystery spreadsheets, no "then Sandra copies column C". Born in the Government Statistical Service, championed across the NHS (NHS Digital's RAP community, the Goldacre Review's "RAP should be the norm"), it is less a technology than a standard of evidence.

Why organisations care: the monthly report that takes three days of copy-paste becomes a ten-minute re-render; the "how was this figure derived?" audit question gets answered by pointing at a script; the analyst who wins the lottery stops being a single point of failure; and errors, when found, are fixed once in code rather than hunted through seventeen spreadsheets.

The commonly-cited **baseline RAP** ingredients — all of which you now own:

| Ingredient                                                              | You learned it            |
|-------------------------------------------------------------------------|---------------------------|
| Open-source language (R/Python), not point-and-click                    | Modules 4-6               |
| Version control with meaningful history                                 | Git, previous lessons     |
| Peer review of code                                                     | Pull requests             |
| Data pipelines, not manual steps (extract→clean→analyse→report as code) | DBI, dplyr, Quarto        |
| No manual output editing — reports generated whole                      | Quarto, gt                |
| Documentation a stranger can follow                                     | README, project structure |

Silver and gold levels add packaging functions properly, automated testing (next lesson), and continuous integration. But the mindset upgrade is the real content: **the pipeline is the product**. A one-off analysis answers today's question; a pipeline answers it every month forever, survives your promotion, and can be inspected by anyone with a licence to look.

Honest words about the friction: RAP is slower the *\*first\** time — writing `01_extract.R` takes longer than clicking export. The payback arrives at refresh two and compounds monthly; "we publish this quarterly" is the strongest possible RAP business case. Legacy processes get converted incrementally (automate the extract this quarter, the report next), and the pragmatic minimum for any new work is simply: *\*could I delete outputs/ and rebuild it with one command?\** If yes, you're RAP-shaped already.

Where to see it done: NHS England's RAP community of practice publishes guidance and real examples on GitHub; UKHSA and DHSC repos show production pipelines; the Goldacre Review (2022) is the strategic argument, and quoting it in interviews rarely hurts.

#### QUICK CHECK

##### The defining test of a RAP is...

- ☐ It uses R
- ✓ **A competent stranger can reproduce every published output from the code and documentation alone**
- ☐ It runs in under a minute
- ☐ It has no bugs

**Why:** Language, speed and perfection are incidental; independent reproducibility is the standard. R+Git+PRs are the usual means to that end.

#### QUICK CHECK

##### The strongest early candidate for RAP conversion in a team's portfolio?

- ☐ A one-off ministerial question answered last year
- ✓ **The quarterly report rebuilt by hand every three months**
- ☐ A colleague's personal scratch analysis
- ☐ A dataset nobody uses

**Why:** RAP costs up front and pays per repetition. Recurring products amortise the investment fastest — and remove the most copy-paste risk.

**EXERCISE**

Audit this described process against the baseline RAP table and list what passes/fails, then name the single highest-value fix: "Analyst exports SUS data via a GUI each month, pastes into master.xlsx, updates pivot tables, copies charts into Word, emails v7 to the team; last year's analyst left no notes."

**SOLUTION · TEXT**

Fails: open-source pipeline (GUI export + Excel), no version control (emailed v7), no code review, manual outputs (paste into Word), no documentation (no notes). Passes: essentially nothing.

Highest-value first fix: script the extract (01\_extract.R via DBI) — it anchors the chain in code, kills the riskiest manual step, and every later stage (clean → analyse → Quarto report) can then attach to it incrementally. Add git init the same afternoon.

**KEY POINTS**

- RAP = published numbers regenerable from code by a stranger; the pipeline is the product.
- Baseline: open language, Git, peer review, no manual steps, generated outputs, docs.
- Convert recurring work first; incremental adoption is expected and fine.

**MODULE 7 · PIPELINES, GIT & REPRODUCIBILITY · LESSON 87 OF 105****87. Scheduling and testing: analysis that runs itself (safely)**

The finished pipeline still needs a finger on the Run button — until it doesn't. Automation is the last mile of RAP, and it has two halves: making the run happen, and making sure a *\*bad\** run can't slip through quietly.

**Scheduling.** First make the pipeline runnable in one non-interactive command — a `run_all.R` that sources 01→04 (or `quarto render`). Then hand it to the OS:

**WINDOWS**

```
Task Scheduler (built in): Create Task → Trigger: Monthly, day 1, 07:00 →
Action: Program = "C:\Program Files\R\R-4.5.1\bin\Rscript.exe"
      Arguments = "run_all.R"
      Start in = C:\path\to\project
Tick "Run whether user is logged on or not". Test via right-click → Run.
```

**MACOS**

```
crontab -e # then add:
0 7 1 * * cd /Users/you/project && /usr/local/bin/Rscript run_all.R >> logs/run.log 2>&1
(launchd .plist files are the native alternative; cron is simpler to start.)
```

**LINUX**

```
crontab -e # minute hour day month weekday
0 7 1 * * cd /home/you/project && Rscript run_all.R >> logs/run.log 2>&1
# server estates may offer cron, systemd timers, or Airflow — same idea.
```

Non-negotiables for unattended runs: **absolute certainty about paths** (the scheduler's working directory isn't your terminal's — hence `here()` and the explicit `cd/Start-in`), **logging** (append stdout/stderr to a dated log; a silent 3am failure discovered at the 9am board is the classic automation injury), and **credentials** via integrated auth or env vars, never freshly typed.

**Testing** is the other half, in two flavours. *\*Code tests\** check your functions once and forever — the **testthat** package:

#### TESTS/TESTTHAT/TEST-AGE.R

```
library(testthat)
test_that("age_at handles the birthday boundary", {
  expect_equal(age_at("1960-03-01", "2026-02-28"), 65)
  expect_equal(age_at("1960-03-01", "2026-03-01"), 66)
})
test_that("financial_year assigns 31 March correctly", {
  expect_equal(financial_year(as.Date("2026-03-31")), "2025/26")
  expect_equal(financial_year(as.Date("2026-04-01")), "2026/27")
})
```

`testthat::test_dir("tests")` runs the lot; every bug you ever fix should leave a test behind as a tombstone, so it can't return. *\*Data validation\** checks each run's inputs and outputs against expectations — because the code being right doesn't make this month's feed right:

#### ASSERTIONS THAT STOP BAD RUNS LOUDLY

```
stopifnot(
  nrow(adm) > 1000,                # feed didn't arrive empty
  !any(duplicated(adm$admission_id)), # key still unique
  all(adm$admission_date <= Sys.Date()), # no future admissions
  all(rates$denominator >= rates$numerator) # sanity of construction
)
```

(The **pointblank** package dresses this up with reports and thresholds; `stopifnot` is the honest core.) The design principle: **fail loud, fail early** — a pipeline that halts with "duplicate admission\_ids: 214" at 07:02 and emails the log is a guardian; one that publishes a wrong dashboard politely is a liability. Wire the assertions between every stage, and have the final stage refuse to overwrite yesterday's good output when today's checks fail.

That's the module complete: connection, translation, safety, structure, history, review, standard, automation. Your analysis now runs without you — which, paradoxically, is what makes you valuable.

#### QUICK CHECK

**A scheduled pipeline broke at 3am; the team learned when a director questioned the dashboard. The missing engineering?**

- ☐ A faster server
- ✓ **Logging plus loud failure: assertions halting the run and surfacing the error (log/email) before outputs publish**
- ☐ More RAM
- ☐ Running it manually instead

**Why:** *Automation without observability is risk-transfer to your readers. Halt on failed checks, write logs, never overwrite good outputs with bad.*



**QUICK CHECK**

**Where should the lesson of a just-fixed bug permanently live?**

- ☐ In the analyst's memory
- ✓ **As a testthat case reproducing it — so the suite fails if it ever returns**
- ☐ In an email thread
- ☐ In the commit message only

**Why:** Tests are institutional memory that executes. The FY-boundary bug fixed today, encoded as `expect_equal`, can never quietly reoffend.

**EXERCISE**

Write the pre-publication assertion block for the vaccination pipeline covering: at least 10,000 rows extracted; no duplicate (patient\_id, vaccine, dose\_number); all date\_given values within 2020-01-01..today; every uptake proportion in [0, 1]; and each practice's denominator at least its numerator.

**SOLUTION · R**

```
stopifnot(
  nrow(vax) >= 10000,
  !any(duplicated(vax[, c("patient_id", "vaccine", "dose_number")])),
  all(vax$date_given >= as.Date("2020-01-01")),
  all(vax$date_given <= Sys.Date()),
  all(uptake$uptake >= 0 & uptake$uptake <= 1),
  all(uptake$denominator >= uptake$numerator)
)
# Place after cleaning, before render; schedule wraps the whole script,
# so any failure halts the run and lands in the log — loudly.
```

**KEY POINTS**

- One-command `run_all` + OS scheduler (Task Scheduler / cron / launchd); log everything, mind paths.
- `testthat` guards functions forever; each fixed bug becomes a test.
- `stopifnot/pointblank` validate each run's data; fail loud and early, never publish past a failed check.

**MODULE 8**

# Module 8 — Working with AI

**In this module**

- 88** How LLMs actually work — and why they confidently invent things
- 89** Prompting for SQL: context is everything
- 90** Prompting for R and debugging with AI
- 91** AI code review: the adversarial second pair of eyes
- 92** Governance: what never goes in the box
- 93** AI for documentation, explanation and learning

- 94** Failure modes: a field guide to confident wrongness
- 95** Judgement: the analyst's half of the partnership

## MODULE 8 • WORKING WITH AI • LESSON 88 OF 105

## 88. How LLMs actually work — and why they confidently invent things

You'll use AI assistants throughout your career, so this module teaches them as what they are: power tools with a distinctive failure mode. That starts with an honest model of the machine.

A large language model (LLM) — the engine behind Claude, ChatGPT, Copilot and colleagues — is a system trained on vast amounts of text to do one thing extraordinarily well: **predict plausible continuations**. Ask a question and it generates, token by token, the response most consistent with the patterns it learned. Because human knowledge is written down in patterns, this produces genuinely useful behaviour: it has absorbed more SQL, more R, more epidemiology documentation than any human ever will, and it can remix that fluently for your specific situation.

But note what's absent from that description: a database of facts, a concept of truth, and any awareness of *\*not knowing\**. When you ask about the boundary between mastered territory and gaps, the machine does the same thing in both cases — produces the most plausible-sounding continuation. Plausible-and-true and plausible-and-false feel identical from the inside. The result is **hallucination**: fabricated function arguments, invented ICD-10 codes, confidently-cited papers that don't exist, an R package that *\*should\** exist but doesn't. The fluency never wavers, which is exactly what makes it dangerous — wrongness arrives wearing the same confident prose as rightness.

Consequences worth internalising before you write a single prompt:

- **Confidence is not evidence.** The model's tone carries zero information about correctness. Calibrate on verification, never on how sure it sounds.
- **Errors cluster at the edges:** niche packages, recent changes (training data has a cutoff), UK-specific details amid US-dominated training text, and precise identifiers (codes, dosages, section numbers) where "plausible" and "correct" diverge sharpest. A model may nail the shape of an age-standardisation function and fumble the exact ESP2013 weights.
- **It mirrors your framing.** Ask "why does eating late cause weight gain?" and it tends to answer inside your premise. Leading questions get led answers — ask neutrally when you actually want the truth.
- **Code has a home advantage:** generated code can be *\*run\**. Execution is a truth-test essays never get, which is why coding is among the highest-value, most verifiable uses — provided you actually run and check it, the discipline the rest of this module builds.

None of this is a reason for refusal any more than Wikipedia's editability was. It defines the contract: **AI drafts, you verify, you remain accountable.** The analyst who signs the report is you; "the AI said so" has precisely the professional standing of "some bloke in the pub said so".

### QUICK CHECK

**An LLM cites "Fenton & Marsh (2021), Journal of Public Health Analytics" for a claimed statistic. Correct posture?**

- ☐ Trust it — citations mean it checked
- ✓ **Treat it as unverified until you find the paper yourself; plausible-looking citations are a classic hallucination**
- ☐ Trust it if the journal sounds real
- ☐ Ask the model to confirm it's real

**Why:** The model generates citation-shaped text by the same mechanism as everything else. Asking it to self-confirm just samples the same process twice. Look it up.

**QUICK CHECK****Why is code generation a comparatively SAFE use of LLMs?**

- ☐ Models are trained mostly on code
- ✓ **Code can be executed and tested — you get an objective external check that prose claims never receive**
- ☐ Code cannot contain errors
- ☐ Syntax highlighting reveals mistakes

**Why:** Verifiability, not infallibility, is the safety. A wrong function fails a test; a wrong "fact" just sits there sounding right. This asymmetry shapes the whole module.

**EXERCISE**

Classify each task as lower-risk or higher-risk for AI assistance, with one line of reasoning: (a) "explain what this LEFT JOIN does", (b) "what is the exact ICD-10 code for viral pneumonia?", (c) "draft a first-pass R function to band ages", (d) "what were the 2025 changes to UKHSA notification regulations?"

**SOLUTION · TEXT**

- (a) Lower — explanation of code you can read alongside; errors are visible.
- (b) Higher — precise identifier where plausible ≠ correct; verify in the official ICD-10 classification, never trust recall.
- (c) Lower — a draft you will run and test; execution is the check.
- (d) Higher — recent, UK-specific, possibly post-training-cutoff; go to the primary source (legislation/UKHSA guidance), use AI only to summarise text YOU supply.

**KEY POINTS**

- LLMs predict plausible text; they contain no truth-check and no sense of not-knowing.
- Hallucination clusters at edges: niche, recent, UK-specific, exact identifiers; confidence ≠ correctness.
- Contract: AI drafts, you verify, you sign. Code is safest because execution tests it.

**MODULE 8 · WORKING WITH AI · LESSON 89 OF 105****89. Prompting for SQL: context is everything**

Ask an AI to "write a query for readmissions" and you'll get syntactically perfect SQL for tables you don't have, against a readmission definition you didn't choose. The model can't see your database. **Everything it needs must travel in the prompt** — and learning what that is turns AI from a toy into a colleague.

The anatomy of a working SQL prompt:

**A PROMPT THAT EARNS ITS ANSWER**

```
You are helping with SQL for health analytics (SQLite dialect).
Schema (fictional training data, no real patients):
  admissions(admission_id INTEGER PK, patient_id, admission_date TEXT
    ISO-8601, discharge_date TEXT nullable = still in hospital,
    admission_method 'Emergency'/'Elective', specialty)
  patients(patient_id INTEGER PK, sex 'F'/'M', birth_date TEXT,
    practice_code)
Task: emergency admission COUNT per specialty for financial year
2025/26 (2025-04-01 to 2026-03-31 inclusive), completed spells only,
```

**A PROMPT THAT EARNS ITS ANSWER**

```
sorted busiest first.
Constraints: half-open date logic (>= start, < next FY start);
exclude NULL discharge_date; plain ANSI-ish SQL, no vendor functions.
Return: the query, then one sentence on any assumption you made.
```

Each block is doing load-bearing work. **Dialect** stops it reaching for `GETDATE()` in SQLite (or `strftime` on SQL Server — your Lesson 40 knowledge, now a prompt ingredient). **Schema with types and quirks** ("nullable = still in") prevents invented columns and wrong NULL handling. **Precise definitions** — you specified the FY boundary rather than letting the model pick one; date logic is exactly where plausible-but-wrong hides. **Stated constraints** encode your house rules. And asking it to **surface assumptions** converts silent guesses into visible, checkable ones.

Then the non-negotiable step: **run it, and interrogate the result like you would a junior colleague's work**. Row count sane? Do the specialties match a simple `SELECT DISTINCT specialty`? Does a hand-checked month agree? Lesson 39's data-quality reflexes apply to AI output *\*more\**, not less, because the errors are fluent.

Iteration beats mega-prompts. Get the core query working, then refine in turns: "now add a column for the percentage of each specialty's total", "now restrict to patients 65+ at admission — age from `birth_date`, exact, not year subtraction". Each turn is small enough to verify before building on it. And when a query *\*fails\**, paste the exact error plus the query back — error-message debugging is among the model's strongest events, because millions of identical stack-trace conversations trained it.

Two prompts for learning rather than producing: "explain this query line by line, then tell me what breaks if `discharge_date` is NULL" (comprehension check on inherited code), and "here's my query and my intent — what edge cases would make it silently wrong?" (the model as adversarial reviewer; you'll meet this properly in the code-review lesson).

**THE SCHEMA-CONTEXT HABIT**

Keep a sanitised schema snippet (table names, columns, types, quirks — never data) in a text file. Pasting it into any SQL conversation is ten seconds that eliminates the entire invented-column class of failure.

**QUICK CHECK**

**The single highest-value ingredient missing from "write me a query for flu admissions by month"?**

- ☐ Politeness
- ✓ **The schema — table and column names, types, and quirks the model cannot otherwise know**
- ☐ A request for comments
- ☐ The output format

**Why:** Without schema, every table and column is a guess in disguise. *Dialect*, definitions and constraints follow — but schema is the difference between your database and a fictional one.

**QUICK CHECK**

**AI returns a plausible query using admissions.age\_at\_admission — a column that doesn't exist. This most likely happened because...**

- ☐ The model read your database and misremembered
- ✓ **You didn't supply the schema, so it generated a column that commonly exists in similar tables**
- ☐ SQLite renamed the column
- ☐ The query is fine; the column is hidden

**Why:** *Hallucinated columns are the signature failure of schema-free prompts: the model predicts what a table LIKE yours typically has. Feed the real structure and this class of error largely vanishes.*

**EXERCISE**

Write the full prompt (schema included) asking for: patients with 3+ emergency admissions in FY 2025/26, returning patient\_id and the count, using the playground's real tables. Then list the two checks you'd run on the returned query before trusting it.

**SOLUTION · TEXT**

Dialect: SQLite. Fictional training data.  
 Schema: admissions(admission\_id PK, patient\_id, admission\_date TEXT ISO, discharge\_date TEXT nullable, admission\_method 'Emergency'/'Elective', specialty)  
 Task: patient\_id and number of emergency admissions for patients with 3 or more emergency admissions between 2025-04-01 (inclusive) and 2026-04-01 (exclusive). Sort by count descending.  
 Constraints: GROUP BY + HAVING, half-open dates, no vendor functions.  
 Return the query plus any assumptions.  
 Checks before trusting:  
 1. Run it; pick one returned patient\_id and list their raw emergency admissions with a simple WHERE query — is the count right and are all dates inside the FY window?  
 2. Sanity-scan the logic: HAVING COUNT(\*) >= 3 present? Filter on admission\_method = 'Emergency' BEFORE grouping? Date logic half-open?

**KEY POINTS**

- Prompts must carry what the model can't see: dialect, schema + quirks, exact definitions, constraints.
- Ask for stated assumptions; iterate in small verified steps; paste exact errors when debugging.
- Interrogate AI SQL like a junior's work: run, count, spot-check, cross-check.

**MODULE 8 · WORKING WITH AI · LESSON 90 OF 105****90. Prompting for R and debugging with AI**

R prompting shares the SQL rules — context, precision, iteration — with different context to carry: not schema, but **data shape and tidyverse dialect**.

The data-shape trick is `dput()` on a tiny sample, which serialises real structure into pasteable text:

**GIVE THE MODEL YOUR ACTUAL STRUCTURE**

```
dput(head(adm, 4))
# structure(list(admission_id = c(101, 102, 103, 104),
#   sex = c("F", "M", "F", "F"), age = c(67, 45, 82, 34), ...
```

Paste that (fictional/sanitised data only — the governance lesson is next) and the model stops guessing your column names and types. Alternatively `glimpse()` output works for structure-without-values. Add your dialect preferences explicitly — "tidyverse style, native pipe `|>`, no loops where a vectorised solution exists" — or you'll receive base-R answers in one reply and `data.table` in the next.

**Debugging is the killer app.** The workflow that works:

#### THE DEBUGGING PROMPT PATTERN

```
[1] What I'm trying to do (one sentence of intent)
[2] Minimal code that reproduces the problem
[3] The COMPLETE error or warning, verbatim
[4] What I expected vs what happened
[5] glimpse()/dput() of the data involved
```

Intent matters because the commonest R disasters aren't errors at all — they're *\*wrong results\**: the join that fanned out (Lesson 51's trap), the `mean()` silently NA, the filter that dropped NAs you meant to keep. "Here's my code and output — I expected 5,210 rows and got 8,914" hands the model the discrepancy, which is where diagnosis lives. For honest-to-goodness errors, paste them whole: `object of length 0, unexpected symbol, could not find function` each have signature causes the model pattern-matches instantly (the last one usually means a missing `library()` call — it'll tell you).

Verification discipline, R-flavoured: run suggested code on your data *\*in a fresh object\** (never overwrite your working data with an untested fix); check `nrow()` before/after; spot-check a few rows manually; and beware the model's fondness for **inventing arguments and packages** — `read_csv(guess_encoding = TRUE)` looks plausible and doesn't exist; a recommended package may be real, abandoned, or imaginary. `?function_name` settles arguments in five seconds; a CRAN search settles packages.

Two upgrade prompts: "rewrite this working-but-ugly pipeline for clarity, tidyverse style, same behaviour — explain each change" (refactoring with a diff you can judge), and "add error handling: what happens to this function when the input has zero rows / all NAs / a missing column?" (the model enumerates edge cases faster than you'll brainstorm them; you decide which matter).

#### FRESH-OBJECT DISCIPLINE

`adm2 <- suggested_fix(adm)`, compare, THEN promote. The thirty seconds this costs has saved more analyses than any other habit in this module. Untested AI code overwriting your only in-memory copy of an hour's cleaning is a self-inflicted outage.

#### QUICK CHECK

**An AI-suggested fix uses `arrange(.by_group = TRUE, desc(n))` and you're unsure `.by_group` is real. Fastest reliable check?**

- ☐ Ask the AI if it's sure
- ✓ **?arrange in the console — the official documentation lists the actual arguments**
- ☐ Try random alternatives
- ☐ Assume it's fine; it sounds plausible

**Why:** *Invented-but-plausible arguments are a signature R hallucination. Documentation is authoritative and one keystroke away; the model re-asserting itself is not evidence. (This one happens to be real — but you check.)*

**QUICK CHECK**

**Your pipeline runs clean but returns 8,914 rows where you expected ~5,200. The most productive AI prompt includes...**

- ☐ Just the code
- ✓ **Code + the expected-vs-actual row counts + glimpse() of both input tables — wrong-result bugs live in the discrepancy**
- ☐ "Why is R broken?"
- ☐ The code with column names redacted for safety

**Why:** No-error bugs need the model to see what you EXPECTED. Counts plus structure usually lets it spot the fan-out join or NA-dropping filter immediately. (Sanitised structure, not patient data.)

**EXERCISE**

You have a join producing duplicates. Compose the debugging prompt using the pattern above (invent minimal reproducing code with two tiny tribbles where the lookup table secretly has a duplicate key), and state what a GOOD answer should identify.

**SOLUTION · TEXT**

Intent: one row per admission with practice name attached.

Code:

```
adm <- tribble(~admission_id, ~practice_code, 1, "P1", 2, "P2")
lkp <- tribble(~practice_code, ~practice_name,
               "P1", "Riverside", "P2", "Hilltop", "P2", "Hilltop Branch")
left_join(adm, lkp, by = "practice_code")
```

Expected: 2 rows. Got: 3.

Structure: [glimpse output of both]

A good answer identifies: lkp is not unique on practice\_code (P2 twice), so the join fans out; fixes = deduplicate/choose one row per key in lkp (distinct or a rule for which branch), and cite checking count(lkp, practice\_code) |> filter(n > 1) before joining — plus dplyr's relationship = "many-to-one" argument to make the expectation enforced.

**KEY POINTS**

- Carry context: dput()/glimpse() of sanitised data, dialect preferences, intent.
- Debug with the 5-part pattern; wrong-results bugs need expected-vs-actual, not just code.
- Verify: fresh objects, row counts, ?docs for invented arguments, CRAN for invented packages.

**MODULE 8 · WORKING WITH AI · LESSON 91 OF 105****91. AI code review: the adversarial second pair of eyes**

So far AI has written code; the higher-leverage habit points it the other way. **Review** is where models shine: reading is cheaper than writing, your code is right there to check its claims against, and — unlike your kind colleague — the machine has infinite patience and no feelings to spare. But the value depends entirely on how you ask.

"Is this code okay?" begets "Looks good! Here are some minor suggestions \*". Models are agreeable by disposition; vague review prompts harvest that agreeableness. The fix is **adversarial framing with specific targets**:

**A REVIEW PROMPT WITH TEETH**

Act as a sceptical senior analyst reviewing this R for a published NHS report. Do NOT be polite. Find reasons the OUTPUT could be WRONG.



**A REVIEW PROMPT WITH TEETH**

Check specifically:

1. Joins — any that could fan out or silently drop rows?
2. NA handling — where do missing values change a result invisibly?
3. Date logic — FY boundaries, half-open vs closed intervals, timezone?
4. Aggregation — counting rows where it should count distinct people?
5. Filters — do exclusions match the stated definition in the comment?

Definition it must implement: emergency readmission within 30 days,  
index = first emergency admission per patient in FY, readmission  
counted from DISCHARGE date.

[code pasted here]

Report: numbered findings, each with severity (wrong-answer / fragile / style), the line involved, and a one-line test that would expose it.

The ingredients: a **role with permission to be harsh**; a **checklist drawn from the actual failure taxonomy** of health analytics (you built that taxonomy in Modules 2–4 — fan-out, NULLs, boundaries, rows-vs-people); the **intended definition**, so the model reviews against your spec rather than guessing one; and **findings-with-tests**, because "line 14 counts admissions not patients; test: compare against COUNT(DISTINCT patient\_id)" is actionable where "consider reviewing your aggregation" is fog.

Then triage like a reviewer of the reviewer: findings split into *\*true positives\** (fix, and add the exposing test to your test suite — Lesson 87's tombstone habit), *\*false positives\** (the model misread; fine, ignore — but if it misread, would a human? maybe a comment is due), and *\*style opinions\** (adopt or don't; consistency with your team beats the model's aesthetics). Expect false alarms; an adversarial reviewer who never cries wolf isn't trying.

Where this fits your real QA: AI review is a **pre-filter, not a replacement** for the human peer review your PR process requires (Lesson 85) — it catches the mechanical 60% cheaply so your human reviewer spends attention on the judgement calls: is the definition right, is the comparison fair, should this be published at all. Those questions stay human. And run the review *\*before\** the PR, not instead of it; arriving pre-reviewed is a courtesy to colleagues and makes their genuine findings pop.

One more move for inherited code: "explain what this script actually does, step by step, then list any behaviour that would surprise someone reading only the comments." Mismatches between code and comments are where legacy landmines live.

**QUICK CHECK****Why does "review this code" underperform the adversarial checklist prompt?**

- ☐ Models can't review code
- ✓ **Models default to agreeable; without a mandate to find faults and specific failure classes to hunt, they produce compliments plus trivia**
- ☐ Checklists slow the model down
- ☐ Politeness is required by the API

**Why:** You get what you frame for. Permission-to-be-harsh plus your domain's actual bug taxonomy (joins, NAs, dates, grain) converts a cheerleader into an auditor.

**QUICK CHECK**

**The AI reviewer flags a genuine bug: your "patients admitted" metric counts admissions. After fixing the code, the durable next step is...**

- ☐ Thank the model
- ✓ **Encode the exposing check as a permanent test (e.g. distinct-patient count comparison) so the bug class can't silently return**
- ☐ Delete the old commit history
- ☐ Re-run the same review weekly

**Why:** A finding is transient; a test is institutional memory (Lesson 87). The review's real product is your strengthened test suite.

**EXERCISE**

Your colleague inherited a 200-line script with the comment "excludes day cases" but nobody trusts it. Write the review prompt, and name the two categories you expect findings to fall into that you'd treat differently.

**SOLUTION · TEXT**

Prompt:

"Act as a sceptical reviewer. This R script claims to exclude day cases (LOS = 0) before computing specialty LOS averages. (1) Explain step by step what it ACTUALLY does. (2) List every point where behaviour could diverge from the stated claim – NA discharge dates, LOS calculated before vs after exclusion, joins altering row counts, filters on the wrong table. (3) For each, give severity and a one-line test. [script pasted]"

Treat differently:

- Wrong-answer findings (e.g. NAs make LOS NA, so day cases with missing discharge survive the filter): verify by running the proposed test, fix, keep the test.
- Style/clarity findings (naming, pipe formatting): batch for later or adopt via team conventions – never let them crowd out correctness.

**KEY POINTS**

- Frame review adversarially: harsh role, domain-specific checklist, the intended definition, findings-with-tests.
- Triage findings: fix + keep the test / dismiss false alarms / treat style as optional.
- AI review pre-filters; human peer review still owns judgement — arrive at PRs pre-reviewed.

**MODULE 8 · WORKING WITH AI · LESSON 92 OF 105****92. Governance: what never goes in the box**

This is the lesson to over-learn, because one bad paste can end a career and breach thousands of patients' trust at once. The tools change; the rule doesn't:

**Never put person-identifiable or sensitive data into a public AI tool. Ever.**

Typing into a consumer chatbot is **disclosure to a third party** — legally and practically like emailing the data to a company you have no agreement with. Depending on the service, prompts may be retained, reviewed by humans, or used in training. Public health work is built on data given under legal duties and public trust; UK GDPR, the DPA 2018, the common-law duty of confidentiality and your Caldicott training (Lesson 7) all reach this keyboard moment. "I was just asking for help with

a script" mitigates nothing if the script's test data was real patients.

What counts as unsafe to paste is broader than names:

| Category                     | Examples                                                       |
|------------------------------|----------------------------------------------------------------|
| Direct identifiers           | Names, NHS numbers, addresses, DOBs, postcodes                 |
| Quasi-identifying detail     | Rare diagnosis + small area + age; "the 94-year-old in ward 3" |
| Small-number outputs         | The table with a cell of 2 you're trying to suppress           |
| Business-sensitive           | Unpublished performance, incident details, contract terms      |
| Credentials & infrastructure | Passwords, connection strings, server names, internal URLs     |

The **sanitise-then-ask** workflow keeps AI useful anyway, because almost every technical question survives having its data removed:

#### SAME HELP, ZERO DISCLOSURE

UNSAFE: "Why does this fail? `filter(df, nhs_number == '943 476 5919')`  
— Mrs Ellen Mabey, DOB 12/3/1941, HIV clinic list..."

SAFE: "Why might `filter(df, id_col == 'a value I know exists')`  
return zero rows when the value is visibly in the data?  
`id_col` is character; the source was Excel."  
(Answer arrives just the same: trailing whitespace/encoding —  
and no human being was described to a server.)

Structure travels, values don't: column names (if non-sensitive), types, `glimpse()` shapes, **invented** example rows that mimic the problem. If a bug genuinely needs data to reproduce, *\*fabricate\** a minimal dataset that shows the same behaviour — a skill this course has quietly drilled via every `tribble()` example.

Know your **tool tiers**, because "AI" is not one risk level: public consumer tools (assume everything leaves; PID never); enterprise/approved deployments — some NHS organisations provide sanctioned AI with contracts, data-protection assessments and retention controls (their rules define what's permitted; often more, never everything); and local models running on trust infrastructure (data stays inside; still governed by your IG framework). The operative question is never "is AI allowed?" but **"which tool, under which agreement, for which data?"** — and your organisation's AI/IG policy answers it. If no policy exists yet, the public-tool rule above is the floor, and asking IG *\*before\** a novel use is the move that ages well.

If a paste happens — autocomplete betrayal, wrong window, a moment's autopilot — it's a **data incident**: report it through your organisation's incident route immediately, same as a misdirected email. Fast honest reporting is protective, for patients first and, genuinely, for you; discovery-months-later is the version that destroys trust.

#### QUICK CHECK

**A colleague pastes ten real rows — NHS numbers included — into a public chatbot "just to fix a pivot\_wider error". The correct framing of what occurred?**

- ☐ Fine: debugging is a legitimate purpose
- ✓ **A disclosure of identifiable data to a third party — a reportable data incident regardless of intent**
- ☐ Fine if the chat is deleted afterwards
- ☐ A problem only if the model was training that day

**Why:** Purpose doesn't transform disclosure into non-disclosure, and deletion buttons don't undo transmission. Ten fabricated rows would have fixed the same bug.

**QUICK CHECK**

**Your trust provides an approved enterprise AI tool with a signed data-protection agreement. This means...**

- ☐ Anything may now be pasted
- ✓ **Its governance rules define what's permitted — approved ≠ unlimited; the IG framework still applies**
- ☐ It's identical to the public tool
- ☐ IG approval is no longer needed for new uses

**Why:** Sanctioned tools widen the safe envelope under explicit conditions; they don't abolish the envelope. Read the policy; novel uses still go through IG.

**EXERCISE**

Rewrite this unsafe prompt as a safe one that would get equally useful help: "My query returns the wrong readmission count for patient 4859223601 (John Okafor, DOB 03/05/1958, discharged 12 Jan from Barchester General cardiology after MI). Here are his 6 admission rows: [real data]. Why is the 30-day window missing the February one?"

**SOLUTION · TEXT**

Safe version:

```
"SQLite. 30-day readmission logic seems to miss a qualifying event.
Schema: admissions(admission_id, patient_id, admission_date TEXT ISO,
discharge_date TEXT ISO, admission_method). Fabricated repro:
INSERT INTO t VALUES (1, 99, '2026-01-05', '2026-01-12', 'Emergency');
INSERT INTO t VALUES (2, 99, '2026-02-10', '2026-02-14', 'Emergency');
My rule: readmission if next admission_date is within 30 days of prior
DISCHARGE date. My query: [query]. It returns 0 readmissions for
patient 99; I expect 1 (Feb 10 is 29 days after Jan 12). Why?"
(Likely culprit the model can now find: comparing to admission_date
instead of discharge_date, or an off-by-one/exclusive boundary —
findable entirely without any real person in the prompt.)
```

**KEY POINTS**

- PID, quasi-identifiers, small cells, credentials: never into public AI — pasting is disclosure.
- Sanitise-then-ask: structure and fabricated repro data get the same help with zero risk.
- Tool tiers differ (public / enterprise-approved / local); your IG policy governs; accidental pastes are incidents — report fast.

**MODULE 8 · WORKING WITH AI · LESSON 93 OF 105****93. AI for documentation, explanation and learning**

The unglamorous uses are the reliably profitable ones. Documentation, explanation and translation are tasks where AI's weaknesses matter least — the source material is *in the prompt*, so the model summarises what's in front of it rather than recalling facts from the void — and where analyst time haemorrhages quietly.

**Documenting your own code.** Paste a working script; ask for a README section, roxygen-style function comments, or a plain-English "what this pipeline does" for the analyst who inherits it:

**PROMPTS THAT PAY RENT**

```
"Write concise comments for this function: one line of purpose, one
```

**PROMPTS THAT PAY RENT**

```
per argument, one on the return value. Don't restate the code –
explain intent and any non-obvious choices."
"Draft the README 'How to run' section for this project structure:
[paste tree]. Audience: an analyst new to the team, R installed,
no context."
"Summarise what this 300-line inherited script does as: purpose,
inputs, outputs, step list, and anything surprising. Flag any step
whose purpose you CAN'T infer – don't guess."
```

That last clause — *\*flag, don't guess\** — is the safety catch on all summarisation prompts. You still read the draft against the code (it's a draft, and you sign it), but reviewing accurate-ish prose beats composing from a blank page by a factor your calendar will notice.

**Explaining upward and outward.** Analysts translate constantly: methods for a board paper, caveats for a press office, a funnel plot for a practice manager. AI drafts these translations well *\*when you supply the substance\**:

**AUDIENCE TRANSLATION**

```
"Rewrite for a non-technical Trust board audience, ~120 words,
no jargon: 'Rates were directly age-standardised to ESP2013;
Barchester's apparent excess attenuated after adjustment,
suggesting age structure confounds the crude comparison.'
Keep the caveat. Do not add claims I haven't made."
```

"Do not add claims" earns its place: models pad, and padding in a board paper becomes a commitment someone must later defend. The reverse direction also works — "explain this epidemiology paper's methods section like I'm a new analyst who knows R but not survival analysis" turns dense reading into scaffolded reading (verify against the paper; it's in your hand).

**Learning itself.** As a tutor, an LLM offers what documentation can't: infinitely patient reformulation. Stuck on window functions? "Explain PARTITION BY with a 6-row worked example, then give me three exercises with hidden answers, then critique my attempt." The **generation effect** from Lesson 11 still rules — you learn by attempting, so have it quiz you rather than lecture you. And keep the mnemonic honest: AI explanations of *\*concepts\** are usually sound (concepts are heavily represented in training text); AI recall of *\*specifics\** (this year's fingertips indicator IDs, exact regulation wording) is where you verify against the source. Concepts: trust-but-sample. Specifics: always check.

The meta-skill is noticing which of your weekly hours are draft-shaped. First-pass commit messages, tidy meeting notes into actions, an outline for a QA checklist, alt-text for twelve charts — none are the analysis, all consume the afternoon. Delegate drafts; keep judgement; sign only what you've read.

**QUICK CHECK**

**Why is "summarise this script I've pasted" much safer than "what does the ukhsa\_notifications package do"?**

- ☐ It isn't; both hallucinate equally
- ✓ **The first grounds the model in supplied text; the second relies on recall, where plausible invention thrives**
- ☐ Scripts are shorter
- ☐ Packages are copyrighted

**Why:** *Grounded summarisation constrains the model to material in-context. Recall questions sample training memory — exactly where confident fabrication lives. Structure your asks to be grounded whenever possible.*

**QUICK CHECK**

**In the board-paper translation prompt, "do not add claims I haven't made" defends against...**

- ☐ Rudeness
- ✓ **Padding that introduces unsupported statements you'd then own in a governance meeting**
- ☐ Long words
- ☐ British spelling errors

**Why:** Models elaborate helpfully; in official documents, elaboration = new commitments. Fidelity constraints keep the draft inside what your analysis actually showed.

**EXERCISE**

You must produce alt text for 12 report charts (accessibility requirement, Lesson 61). Write the prompt that gets useful drafts, including the inputs you'd supply and the check you'd perform.

**SOLUTION · TEXT**

Prompt: "Draft alt text (max ~25 words each) for these charts. For each I give: chart type, variables, and the key takeaway. Alt text should convey the takeaway, not just describe axes. Do not invent numbers – use only figures I provide.

1. Line chart; weekly flu admissions Oct-Mar; peak of 214 in w/c 5 Jan.
2. Funnel plot; MMR uptake by practice vs list size; 2 practices below 99.8% limit.
- [...etc]"

Check: read each draft against its actual chart – numbers match, takeaway faithful, nothing invented. (Grounded inputs make this fast; you supplied every fact, so verification is comparison, not research.)

**KEY POINTS**

- Grounded tasks (summarise/translate/document what's pasted) are AI's reliability sweet spot.
- Guard rails in the prompt: flag-don't-guess, don't-add-claims, use-only-my-numbers.
- Delegate the draft-shaped hours; verify concepts by sampling, specifics always; you sign what ships.

**MODULE 8 · WORKING WITH AI · LESSON 94 OF 105****94. Failure modes: a field guide to confident wrongness**

You've met hallucination in principle; this lesson is the bestiary — the specific ways AI assistance goes wrong in analytical work, each with its tell and its counter. Pattern recognition here is what separates "uses AI safely" from "got lucky so far".

**The invented specific.** Function arguments, package names, ICD-10 codes, indicator IDs, regulation clauses — anywhere the answer is a precise token, plausibility and truth part company. *\*Tell:\** the detail is load-bearing and you can't recall seeing it elsewhere. *\*Counter:\** primary source — ?func, CRAN, the official classification. Five seconds each.

**The obsolete answer.** Training data has a cutoff; the model doesn't reliably know what changed after it — new package versions, reorganised institutions, updated guidance. It answers from a world that may have moved. *\*Tell:\** questions containing "current", "latest", or anything post-dating the tool's knowledge. *\*Counter:\** live sources for live facts; use AI to interpret documents you fetch, not to

be the document.

**The premise swallow.** Ask "why does X cause Y?" and the model explains — even when X doesn't cause Y. It optimises for answering your question, not auditing it. \*Tell:\* you embedded a claim in the question. \*Counter:\* ask neutrally ("what's the evidence on X and Y?") or explicitly invite challenge ("first: is the premise sound?").

**The sycophancy slide.** Push back — "are you sure? I think it's the other way" — and models often fold, apologising and adopting \*your\* error. Agreement under pressure is not evidence; it's the disposition again. \*Tell:\* it reversed because you frowned, not because either of you produced a new fact. \*Counter:\* resolve disagreements with authoritative sources or a decisive test, never with a second round of vibes. (And notice the mirror-image trap: it folding doesn't mean you were right, either.)

**The 90% solution.** Generated code that runs, produces sensible-looking output, and is subtly wrong — the closed interval that double-counts boundary days, `n()` where `n_distinct()` was meant, a `left_join` that should have been inner. Nothing errors; the number is just wrong. \*Tell:\* none visible — that's the point. \*Counter:\* everything Modules 2–7 built: hand-check a case, reconcile totals, test boundary dates, review the definition line by line. This failure mode is why those habits exist.

**The confident tone, everywhere.** Not a separate failure but the amplifier on all of them: fluency is constant while accuracy varies, so your calibration must come from verification frequency, never from prose style.

Assemble the counters and you get the **verification ladder** — match the check to the stakes:

| Stakes                           | Verification owed                                                             |
|----------------------------------|-------------------------------------------------------------------------------|
| Throwaway exploration            | Run it; eyeball the result                                                    |
| Analysis you'll rely on          | Run + spot-check cases + reconcile counts against a simpler query             |
| Feeds a decision or publication  | All the above + tests in the suite + independent human review                 |
| Clinical/legal/regulatory detail | Primary source verification, full stop — AI drafts nothing load-bearing alone |

The economics stay excellent: a 30-second draft plus five minutes of laddered checking still demolishes an hour of unaided work. The failure isn't using AI; it's charging the checking time to nobody.

#### QUICK CHECK

**You challenge the AI's (correct) answer and it apologises and switches to your (wrong) one. The lesson?**

- ☐ The model was wrong initially
- ✓ **Model agreement tracks social pressure, not truth — settle disputed facts with sources or tests, not with whoever pushed last**
- ☐ Apologies indicate reliability
- ☐ Asking twice improves accuracy

**Why:** Sycophancy means capitulation carries no information. The tie-breaker is external: documentation, a run, a primary source.



**QUICK CHECK**

**Which failure mode is MOST dangerous in NHS analytical work, and why?**

- ☐ The invented package — it errors immediately
- ✓ **The 90% solution — it runs, looks right, and ships a wrong number with no visible symptom**
- ☐ The obsolete answer — dates give it away
- ☐ The premise swallow — rereading the question reveals it

**Why:** Loud failures self-announce. The silent almost-right query survives every glance and dies only under spot-checks, reconciliation and review — which is why those are non-negotiable for anything that feeds decisions.

**EXERCISE**

Diagnose the failure mode in each vignette and name the counter: (a) AI: "use `fingertipsR::fetch_indicator_v2()`" — no such function; (b) you ask "why did the 2025 data-flow rules ban practice-level extracts?" and get a fluent explanation, but no such ban exists; (c) an AI-drafted query for "admissions in June" uses `BETWEEN '2026-06-01' AND '2026-06-30'` on a datetime column; (d) AI names the "current" UKHSA chief statistician from two years ago.

**SOLUTION · TEXT**

- (a) Invented specific → check package docs/CRAN; the real call is `fingertips_data()` — 5-second lookup.
- (b) Premise swallow → the question asserted the ban; ask neutrally or demand a source; verify in actual guidance before repeating it.
- (c) 90% solution → runs fine, silently drops June 30 admissions with times after midnight; counter = boundary test rows + half-open dates (`>= '2026-06-01' AND < '2026-07-01'`) per Lesson 21.
- (d) Obsolete answer → post-cutoff fact; check the live UKHSA site; AI interprets sources, it isn't one.

**KEY POINTS**

- Name the modes: invented specific, obsolete answer, premise swallow, sycophancy, 90% solution — fluent tone amplifies all.
- Disagreements resolve by source or test, never by who pressed harder.
- Scale verification to stakes (the ladder); checking time is part of the job, budget it.

**MODULE 8 · WORKING WITH AI · LESSON 95 OF 105**

## 95. Judgement: the analyst's half of the partnership

Close the module by naming what you bring, because using AI well requires knowing precisely what it cannot do — not as comfort, but as a job description.

**Question formulation.** Before any query exists, someone turned "the board is worried about winter" into "emergency respiratory admissions among over-65s, by week and LA, versus the previous two winters, age-standardised". That translation — from concern to answerable question — draws on context no prompt contains: what the board can act on, what data actually exists, what was tried last year, which definition the medical director will accept. Module 1's question-first discipline is now your scarcest skill. AI answers questions; deciding \*which question\* is you.

**Definition authority.** "Readmission", "frequent attender", "vaccinated", "excess" — every metric in this course hid a contestable choice, and you learned to make those choices deliberately (30 days



from \*discharge\*; \*emergency\* only; dose 2 by which birthday?). AI will happily implement any definition and invent one if you don't supply it. The signature on the definition is yours, and defending it in the governance meeting cannot be delegated.

**Plausibility sense.** By now a rate of 840 per 1,000, a mean LOS of 45 days, or a practice with 200% uptake should produce a physical sensation. That reflex — compiled from denominators, seasonality, data-quality scars and knowing that Kingsbay is small so its rates swing — is your fastest error detector and the one no tool replicates, because it lives in accumulated contact with \*your\* system's data.

**Ethical and contextual weight.** Should small numbers be shown to this audience? Is comparing these two trusts fair given case mix? Will publishing this map stigmatise an estate? Does the request even serve patients? These aren't technical questions with technical answers; they're judgement calls made by someone accountable, inside relationships and obligations a language model doesn't have. Lesson 7's Caldicott principles and Lesson 67's disclosure control were always judgement wearing procedure's clothes.

**Accountability itself.** When the number is challenged — and good numbers get challenged — someone stands behind it: explains the method, owns the caveat, corrects the record if needed. That someone has a name and a professional reputation, and the tools they used are as relevant to the audience as the brand of their calculator.

So the working partnership, stated plainly: **you own questions, definitions, verification, context and the signature; AI accelerates drafting, translation, recall-shaped labour and the hunt for your blind spots.** Analysts who thrive treat the model as a tireless junior with brilliant recall, no judgement and no stake — delegating accordingly, checking accordingly, and spending the reclaimed hours on exactly the human work above. Analysts who struggle either refuse the tool (and lose the hours) or abdicate to it (and eventually ship the 90% solution with their name on it).

The rest of this course — careers, then capstones — assumes the partnership. The capstones in Module 10 will explicitly ask you to work both ways: some tasks AI-assisted with documented verification, some unaided, so you can feel where each mode earns its keep. That felt sense, more than any rule, is what "AI-literate analyst" means on a CV.

#### QUICK CHECK

**A director asks for "something on winter pressures". The step AI fundamentally cannot do for you is...**

- ☐ Writing the SQL once specified
- ✓ **Converting the vague concern into the specific, answerable, actionable question — using organisational context, data knowledge and knowing what the audience can act on**
- ☐ Formatting the chart
- ☐ Summarising the final report

**Why:** Question formulation runs on context outside any prompt: politics, history, data availability, actionability. It's the analyst's first and least automatable act.

**QUICK CHECK****The healthiest description of an AI-literate analyst's stance?**

- ☐ Trust the tool; it's smarter than us
- ☐ Refuse the tool; it hallucinates
- ✓ **Delegate drafts and recall to it, keep questions, definitions, verification and accountability — like managing a brilliant junior with no judgement**
- ☐ Use it only for emails

**Why:** Both extremes lose: refusal forfeits the hours, abdication ships unowned errors. The professional posture is managed delegation with retained judgement.

**EXERCISE**

For each, write one sentence assigning it to YOU or AI-with-your-verification, with the reason: (a) choosing whether readmissions count from admission or discharge date; (b) drafting the Quarto prose around finished tables; (c) deciding whether a practice-level funnel plot is fair to publish given two tiny practices; (d) generating five candidate edge-case tests for your FY function; (e) signing off the final report.

**SOLUTION · TEXT**

- (a) YOU — a definition with clinical/policy meaning; AI implements whichever you choose but cannot own the choice.
- (b) AI draft, your verification — grounded prose around numbers you supply; you check fidelity and sign.
- (c) YOU — fairness, disclosure and stigma are contextual ethical calls; Lesson 78's caveats inform it, but the judgement is accountable.
- (d) AI generate, you select — enumeration is its strength; deciding which cases matter (31 March!) is yours.
- (e) YOU — always. The signature is the one deliverable that never delegates.

**KEY POINTS**

- Your monopoly: question formulation, definitions, plausibility sense, ethics/context, accountability.
- AI's lane: drafts, translation, recall-shaped work, adversarial enumeration — under your verification.
- Manage it like a brilliant junior with no judgement; the capstones will make you practise both modes.

**MODULE 9**

# Module 9 — Career & professional practice

**In this module**

- 96** The NHS data landscape: what flows where

- 97** Open data: ONS, UKHSA and the art of the free denominator
- 98** Communities, competencies and the shape of the profession
- 99** Building a portfolio that proves it
- 100** Getting hired: applications, interviews and the technical test
- 101** Your 12-week plan: from course to career

## MODULE 9 • CAREER &amp; PROFESSIONAL PRACTICE • LESSON 96 OF 105

## 96. The NHS data landscape: what flows where

Interviews and first weeks in post reward one thing disproportionately: knowing **what the national datasets are** and roughly how data moves. Module 1 sketched this; here's the working map you should be able to reproduce on a whiteboard.

The spine of it: care happens → providers record it in local systems → standardised extracts flow to national collections → analysts query local warehouses and national tools built on them.

| Dataset                           | What it covers                                                            | Analyst's angle                                                    |
|-----------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------|
| SUS (Secondary Uses Service)      | Hospital activity submitted by providers: admitted care, outpatients, A&E | The raw feed; payment-grade; what your trust warehouse ingests     |
| HES (Hospital Episode Statistics) | Cleaned, curated version of hospital data for analysis                    | Research/benchmarking-grade national comparisons                   |
| ECDS                              | Emergency Care Data Set — richer A&E detail (chief complaint, acuity)     | Urgent-care analysis; injury surveillance                          |
| MHSDS                             | Mental Health Services Data Set                                           | Community & inpatient mental health flows                          |
| CSDS                              | Community Services Data Set                                               | Health visiting, community nursing, therapies                      |
| SGSS                              | UKHSA's lab surveillance system                                           | Infectious disease: organisms, specimens, notifications            |
| ODS                               | Organisation Data Service                                                 | The reference list of NHS orgs & codes (your practice_code's home) |

Two structural ideas make all of these legible. First, **episodes vs spells** (Lesson 36): admitted-care data arrives as consultant episodes that analysts assemble into hospital spells — half of all "why is my count different?" mysteries live here. Second, **the terminology stack** (Lesson 6): SNOMED CT for recording care, ICD-10 for classifying diagnoses, OPCS-4 for procedures — plus the identifiers (NHS number for people, ODS codes for organisations, LSOA for places) that make joining possible.

The umpire for all of it is the **NHS Data Model and Dictionary** (via [digital.nhs.uk](https://digital.nhs.uk)) — the official definition of every field: what "admission method" code 21 means, how "finished consultant episode" is defined. Professional reflex: when two systems disagree or a field is ambiguous, the Dictionary is where you check before you argue. Knowing it exists and using it puts you ahead of a surprising share of the field.

Depth is not required on day one — nobody memorises MHSDS tables for fun. The interview-ready standard is: name the major collections, say what each covers in a sentence, explain episode/spell and the coding stack, and know where definitions live. That's this lesson, retold in your own words.

### QUICK CHECK

**Your trust's admission counts differ from a published HES figure. Before suspecting an error, the FIRST structural explanation to consider?**

- ☐ Someone hacked the warehouse
- ✓ **Definitional differences — episodes vs spells, different inclusion rules, different time basis; check definitions in the Data Dictionary**
- ☐ HES is always wrong
- ☐ Your SQL is fine so the data must be

**Why:** National curated datasets and local extracts differ by construction: cleaning rules, episode/spell grain, provider vs residence basis. Reconcile definitions first; most "discrepancies" dissolve there.

**QUICK CHECK**

**A colleague asks what ODS is. The one-sentence answer:**

- ☐ A dashboard for outbreak data
- ✓ **The Organisation Data Service — the national reference list of NHS organisations and their codes**
- ☐ The old name for SNOMED
- ☐ A type of SQL join

**Why:** ODS codes (like your GP practice\_code) are the organisational identifiers that make cross-system joining possible — reference data, unglamorous and essential.

**EXERCISE**

Whiteboard drill: without looking back, write the one-line description of SUS, ECDS, SGSS, and the Data Dictionary — then check yourself against the table and note which you fumbled for revision.

**SOLUTION - TEXT**

SUS: the national submission route for hospital activity data (admissions, outpatients, A&E) — payment-grade raw feed.  
 ECDS: the emergency care dataset — richer A&E detail like chief complaint and acuity.  
 SGSS: UKHSA's laboratory surveillance system for infectious disease results and notifications.  
 Data Dictionary: the official definition of every NHS data field — where you settle "what does this code actually mean".  
 (Fumbled any? That one goes on tomorrow's revision list.)

**KEY POINTS**

- Care → provider systems → national collections (SUS/ECDS/MHSDS/CSDS/SGSS) → warehouses & curated sets (HES).
- Episodes vs spells and the SNOMED/ICD-10/OPCS-4 stack explain most cross-system confusion.
- The NHS Data Model and Dictionary is the umpire; checking it is a professional reflex.

**MODULE 9 • CAREER & PROFESSIONAL PRACTICE • LESSON 97 OF 105**

## 97. Open data: ONS, UKHSA and the art of the free denominator

A huge share of public health questions can be answered — or at least benchmarked — without touching a single patient record, because the UK publishes extraordinary open data. Knowing this landscape makes you productive before your smartcard arrives, keeps your portfolio legal, and supplies the denominators everything in Module 6 depended on.

**ONS (ons.gov.uk)** is the census and statistics mothership: mid-year **population estimates** by age/sex/geography (your rate denominators, Lesson 37), deaths and mortality series, life expectancy, census tables, and the **Open Geography Portal** boundaries you mapped with in Lesson 63. Population estimates by LSOA × age band × sex is the single most reused file in public health analytics — learn its shape.

**UKHSA and DHSC** publish surveillance and indicator data, with the crown jewel being **Fingertips** — hundreds of public health indicators (smoking, obesity, immunisation, screening, mortality) computed consistently for every LA, ICB and practice, with trends, confidence intervals, and comparisons built in. Two career-relevant facts: decision-makers already use it, so quoting it fluently earns trust; and it

has an R package:

#### FINGERTIPSR: INDICATORS STRAIGHT INTO A TIBBLE

```
# install.packages("fingertipsR")
library(fingertipsR)
# discover what exists
profs <- profiles()           # indicator profiles
inds  <- indicators()         # every indicator + ID
# pull one indicator for all local authorities
smoking <- fingertips_data(IndicatorID = 92443,
                          AreaTypeID = 502) # counties/UAs
glimpse(smoking)
```

(IDs change and grow — the discovery functions are the durable skill, not memorised numbers.) A Fingertips pull piped into your Lesson 58-65 charting skills is a legitimate, shareable analysis of \*real\* data with zero governance risk — portfolio gold, as the portfolio lesson will insist.

**NHS England statistics** (statistics under [england.nhs.uk](https://www.england.nhs.uk)) round out the set: waiting times, A&E performance, workforce, vaccination programmes — the operational numbers newspapers quote, downloadable as spreadsheets your Module 4 skills can tame.

Working habits for open data: automate the download in your script (a `download.file()` or direct `read_csv(url)` beats "I clicked and saved it somewhere" for reproducibility — Lesson 86); record the retrieval date and exact source in the README; expect format drift (this year's spreadsheet adds a header row — your pipeline's assertions from Lesson 87 will catch it); and read the accompanying methodology notes, because open data has definitions too — an ONS "death involving X" and "death due to X" differ, and the difference has burned real headlines.

#### QUICK CHECK

**You need age-band denominators for LA-level rates. The canonical open source is...**

- ☐ Your trust's PAS
- ✓ **ONS mid-year population estimates**
- ☐ Fingertips comments section
- ☐ The GP practice website

**Why:** *ONS mid-year estimates by geography/age/sex are THE standard denominator for population rates — free, documented, and what published indicators themselves use.*

#### QUICK CHECK

**The durable skill with fingertipsR is...**

- ☐ Memorising indicator ID 92443
- ✓ **Using profiles()/indicators() to discover current indicators, then fingertips\_data() to pull them reproducibly in-script**
- ☐ Screenshotting the website
- ☐ Editing the package source

**Why:** *IDs and indicator lists evolve; discovery-then-pull inside a script is reproducible against a moving catalogue. Click-and-save is neither.*

#### EXERCISE

Sketch the script skeleton for a reproducible open-data pull: fetch an indicator via `fingertipsR` for all LAs in a region, record provenance, and assert basic sanity before saving to outputs/.

**SOLUTION · R**

```
library(fingertipsR); library(tidyverse); library(here)
pulled_at <- Sys.time()
dat <- fingertips_data(IndicatorID = 92443, AreaTypeID = 502) |>
  filter(ParentName == "South East region")      # adjust to taste
stopifnot(nrow(dat) > 0,
  all(c("AreaName", "Value", "Timeperiod") %in% names(dat)))
write_csv(dat, here("outputs", "smoking_by_la.csv"))
writeLines(c(paste("Indicator 92443 pulled", pulled_at),
  "Source: Fingertips via fingertipsR"),
  here("outputs", "smoking_by_la_PROVENANCE.txt"))
```

**KEY POINTS**

- ONS = populations, deaths, boundaries; Fingertips = computed PH indicators (fingertipsR for R); NHSE = operational stats.
- Open data + your R skills = legal, shareable, portfolio-ready analysis.
- Script the download, record provenance, assert sanity, read the methodology notes.

**MODULE 9 · CAREER & PROFESSIONAL PRACTICE · LESSON 98 OF 105**

## 98. Communities, competencies and the shape of the profession

Health analytics has an unusually generous professional ecosystem — free, welcoming, and disproportionately valuable to newcomers. Plugging in early is a career accelerant; here's the circuit diagram.

**NHS-R Community** ([nhsrcommunity.com](https://nhsrcommunity.com), [github.com/nhs-r-community](https://github.com/nhs-r-community)) is the beating heart of R in the NHS: free training, webinars, an annual conference, active discussion, and open-source packages you've already met (NHSRplotthedots from Lesson 77). Everything is volunteer-built and beginner-tolerant; lurking is fine, and answering one question a month later is how members are made. A sibling **NHS Python community** exists for when your second language calls.

**AphA — the Association of Professional Healthcare Analysts** — is the professional body: branch events, mentoring, and a **competency framework** that names what analysts at each level should know. Read the framework even if you never join: it converts "get better at analysis" into a checklist of specific, assessable skills — and mapping this course against it (you'll find substantial coverage) tells you precisely what to learn next. Related: the government **DDaT capability framework** describes analytical roles across the civil service, useful for UKHSA/DHSC-flavoured careers.

**The Goldacre Review** (2022, *\*Better, Broader, Safer\**) is the strategy document worth actually reading: it argued for RAP as the norm, open code, proper analyst careers and training in the NHS — and it's why several things in this course (Git, RAP, publishing code) moved from "nice" to "expected". Citing it accurately in an interview signals you understand where the profession is heading. **AnalystX / FutureNHS** hosts cross-system communities and resources on the NHS collaboration platform once you have an NHS email.

Conduct in these spaces is a skill of its own: ask questions the way Lesson 90 taught you to prompt (context, minimal example, what you tried — humans appreciate good prompts too); share fictional-data examples, never work data (Lesson 92's rules apply to Slack as much as to chatbots); and contribute at whatever scale fits — a typo fix to an NHS-R package README is a genuine open-source contribution with your name on it, and precisely the kind of thing the portfolio lesson monetises.

The realistic promise: none of this replaces competence, but community is where competence gets noticed — jobs are advertised, collaborators found, and reputations quietly built by people who show up helpfully over time.

**QUICK CHECK**

**The AphA competency framework's practical use for a self-taught analyst is...**

- ☐ It's required reading for legal reasons
- ✓ **Turning vague ambition into a specific skills checklist you can assess yourself against and plan learning from**
- ☐ It contains SQL tutorials
- ☐ Membership guarantees interviews

**Why:** Frameworks operationalise "get better": named skills at named levels. Self-assessment against one produces your actual curriculum — and interview-ready language about your development.

**QUICK CHECK**

**Why does the Goldacre Review matter in a Band 5 interview?**

- ☐ It's a trick question; it doesn't
- ✓ **It's the strategic case for RAP, open code and analyst careers — showing you know it signals you understand the profession's direction, not just its tools**
- ☐ Quoting it is mandatory
- ☐ It lists approved software

**Why:** "Better, Broader, Safer" reframed NHS analytics policy. Connecting your Git/RAP habits to it turns "I know commands" into "I understand why this matters here".

**EXERCISE**

Draft your 90-day community plan: one thing to join, one thing to attend, one thing to contribute, each with a concrete first action you could take this week.

**SOLUTION · TEXT**

Join: NHS-R Community — action: subscribe to the newsletter and join the Slack/discussion space today; read, don't post, for week one.  
 Attend: one NHS-R webinar this month — action: check the events page now and calendar the next beginner-friendly session.  
 Contribute: one small open-source touch within 90 days — action: while studying, note any unclear README sentence in an NHS-R package; when found, open an issue or a one-line PR fixing it.  
 (Small, named, dated — plans without first actions are moods.)

**KEY POINTS**

- NHS-R = free training/community/packages; AphA = professional body + competency framework; Goldacre = the strategy to cite.
- Ask well, share only fictional data, contribute at any scale.
- Community converts competence into visibility; show up helpfully, repeatedly.



## MODULE 9 • CAREER &amp; PROFESSIONAL PRACTICE • LESSON 99 OF 105

## 99. Building a portfolio that proves it

"Self-taught" is a claim; a portfolio is evidence. For career-changers and graduates without NHS experience, a small public body of work is the single strongest differentiator — it lets an interviewer *\*watch you think\** before they've met you. The good news: everything required is already in your hands.

**The rules of admissible evidence.** Only open data (Lesson 97) or fictional data (this course's playground, or data you synthesise). Never anything from a workplace, however anonymised you believe it is — the portfolio that breaches IG proves the opposite of what you intended. State the data source and its licence in every README.

**What three good projects look like** (three finished beats ten abandoned):

- *\*An open-data analysis with a question.\** Not "I explored smoking data" but "Has the deprivation gap in smoking prevalence narrowed since 2015?" — Fingertips pull, tidy pipeline, two or three publication-quality charts (Lesson 61 styling), honest caveats. Question-first structure (Lesson 8) is itself being assessed.
- *\*A capstone from Module 10\*, upgraded to full RAP dress:* project structure, `here()`, Git history with meaningful commits, README with how-to-run, tests. The commit log is quietly examined — it shows working rhythm, not just results.
- *\*One small tool or contribution\*:* a documented function (your FY converter), a Quarto template, or that NHS-R issue/PR from last lesson. Signals you build for others, not just for marks.

**Presentation multiplies substance.** Each repo: a README that opens with the question and a headline chart (recruiters give you ninety seconds; spend them wisely), reproducible structure, and *\*rendered output\** — a Quarto HTML via GitHub Pages lets non-cloners see the goods. Pin your best three on your GitHub profile; add a profile README with two sentences about who you are and what you're learning. Code style counts: consistent naming, comments that explain *\*why\** — reviewers read three files, then extrapolate.

**The AI disclosure question**, answered with Module 8's confidence: it's increasingly normal to note "AI-assisted drafting; all logic verified by tests X and manual reconciliation Y" in a README. Handled that way, it reads as professional maturity — you're demonstrating the workflow employers actually want, verification included. What reads badly is unexplained AI boilerplate you can't discuss in interview; never ship code you couldn't whiteboard.

Finally, aim the portfolio: an NHS trust panel lights up at SPC charts and RAP structure; a UKHSA-flavoured panel at surveillance epi curves and standardised rates. One shared core, one project angled at the door you're actually knocking on.

### QUICK CHECK

**The strongest single artefact in an entry-level health-analytics portfolio is...**

- ☐ A list of completed courses
- ✓ **A finished, question-driven analysis on open/fictional data with reproducible code, clear README and honest caveats**
- ☐ Screenshots of dashboards from a previous job
- ☐ 15 half-started repositories

**Why:** *Finished + reproducible + question-first = watchable thinking. Workplace screenshots are an IG breach in a trench coat; course lists are claims, not evidence.*

**QUICK CHECK****Best handling of AI assistance in portfolio work?**

- ☐ Hide it — admitting it looks weak
- ✓ **Disclose it briefly alongside the verification you performed; be ready to explain any line in interview**
- ☐ Use AI for everything and say so proudly with no checks
- ☐ Avoid AI entirely so the question can't arise

**Why:** The employable skill IS the verified-AI workflow. Disclosure + evidence of checking demonstrates it; concealment risks the one interview question you can't survive: "walk me through this function you didn't write."

**EXERCISE**

Write the opening block of the README for your best project — question, data+licence, one-line method, headline finding, how to run — in under 120 words.

**SOLUTION · TEXT**

```
# Has the deprivation gap in adult smoking narrowed? (England, 2015-2024)
Question: Is the absolute gap in smoking prevalence between the most
and least deprived decile LAs closing?
Data: Fingertips indicator (smoking prevalence, adults), pulled via
fingertipsR on 2026-07-03. Open Government Licence.
Method: tidyverse pipeline; decile-grouped trends; gap series with CIs.
Headline: the gap narrowed ~2.9 percentage points but remains ~9pp;
progress stalled after 2021 (see chart below).
Run: open .Rproj, `renv::restore()`, `quarto render analysis.qmd`.
Outputs regenerate into /outputs. AI-assisted drafting; logic verified
via tests/ and manual spot-checks documented in NOTES.md.
```

**KEY POINTS**

- Portfolio = evidence: 3 finished, question-driven, reproducible projects on open/fictional data only.
- README with question + headline chart; rendered HTML; meaningful commit history; pinned repos.
- Disclose AI use with its verification; never ship code you can't explain aloud.

**MODULE 9 · CAREER & PROFESSIONAL PRACTICE · LESSON 100 OF 105****100. Getting hired: applications, interviews and the technical test**

The market context, plainly: NHS and UKHSA analytical posts are advertised constantly (trusts, ICBs, local authorities, national bodies), entry typically at **Band 4-5** (analyst/officer) with **Band 6** for demonstrated experience — and this course's content maps squarely onto Band 5 person specifications: SQL, R, communication, IG awareness, reproducibility. Roles hide under many titles: performance analyst, intelligence analyst, public health information officer, data officer — search by skill words, not titles.

**The application** in the NHS world is usually a *\*supporting statement\** scored against the person specification, criterion by criterion. The technique is unglamorous and decisive: mirror their structure. For each essential criterion, one short paragraph: name the criterion, evidence it with a specific

example (your capstones and portfolio now exist precisely for this), state the result. "Experience of SQL" ⇒ "Built a 30-day readmission pipeline in SQL (CTEs, window functions) on a simulated hospital dataset; validated against hand-checked cases; code at [github.com/...](https://github.com/...)". Generic enthusiasm scores zero; addressed evidence scores.

**Interviews** blend values-based questions ("tell us about a time you handled conflicting priorities") with technical probes. Prepare STAR stories (Situation-Task-Action-Result) from your projects — yes, self-directed projects count as evidence of planning, problem-solving and delivery. Technical probes at this level are rarely trick questions; they're the concepts this course drilled, asked plainly:

| They ask                                                        | You have                                                          |
|-----------------------------------------------------------------|-------------------------------------------------------------------|
| "Difference between INNER and LEFT JOIN?"                       | Lesson 25-26, plus the anti-join party trick                      |
| "How would you check data quality in a new extract?"            | Lesson 39's checklist, recited with love                          |
| "A director wants patient-level data emailed — what do you do?" | Lessons 7 & 92: purpose, minimisation, proper channels, IG advice |
| "This month's figure jumped 15% — your response?"               | Lesson 77: control limits before conclusions                      |
| "How do you make analysis reproducible?"                        | Module 7, ideally with the word RAP and a Goldacre nod            |

When you don't know, say so \*and show the finding reflex\*: "I haven't used that dataset, but I'd start with the Data Dictionary definition and a data-quality profile" beats bluffing by a mile — they're hiring your approach, not your cache.

**Technical tests** — increasingly common — are usually a small dataset plus questions (write a query, spot the issue, chart a trend), sometimes live, sometimes take-home. Your edge is process made visible: comment assumptions, check row counts aloud, state definitional choices ("I'll treat FY as April–March; flagging that 31 March boundary"), sanity-check the result. Examiners reward the analyst who catches their planted duplicate more than the one with the fanciest window function. If AI tools are permitted, use them exactly as Module 8 taught — and narrate the verification; if not permitted, you've practised unaided throughout this course by design.

Ask \*them\* questions that reveal you know the terrain: "Is analysis version-controlled — is there a RAP ambition here?", "What does QA look like before numbers reach the board?", "R, Python, or both on the team?" The answers tell you about the job; the questions tell them about you.

#### QUICK CHECK

##### An NHS supporting statement is strongest when it...

- ☐ Expresses passion for healthcare at length
- ✓ **Addresses each person-specification criterion explicitly with a specific evidenced example and outcome**
- ☐ Lists every technology ever touched
- ☐ Stays under 100 words to respect their time

**Why:** Statements are scored against the spec. Mirroring it with concrete evidence (your portfolio projects) is how points are awarded; passion without evidence scores like weather.

**QUICK CHECK**

**In a technical test you meet an unfamiliar dataset quirk. The response examiners reward:**

- ☐ Silence until solved perfectly
- ✓ **Narrated process: state the assumption, check the definition/data, flag the uncertainty, proceed transparently**
- ☐ Confident bluffing
- ☐ Refusing the question as unfair

**Why:** Tests audition your working method. Visible checking, stated assumptions and honest flags ARE the assessed skill — the planted data-quality trap is usually the real question.

**EXERCISE**

Draft your STAR answer ( $\leq 150$  words) to "Tell us about a time you found and fixed an error in analysis" using a genuine moment from this course (everyone has one — a fan-out join, an FY boundary, a NULL surprise).

**SOLUTION · TEXT**

S: Building my readmissions capstone, my monthly counts looked ~40% too high versus a simple total.

T: Find the discrepancy before trusting any output.

A: I reconciled grain first (Lesson 39 habit): `COUNT(*)` vs `COUNT(DISTINCT admission_id)` showed duplication after joining a practice lookup. `count()` on the lookup key revealed duplicate `practice_code` rows — a fan-out join. I deduplicated the lookup, added an assertion (row count unchanged by the join) so it can't silently recur, and documented the fix in the commit message.

R: Counts reconciled exactly; the assertion later caught the same issue when I refreshed the lookup. I now check key uniqueness before every join — cheap insurance, permanently adopted.

**KEY POINTS**

- Search by skills not titles; Band 4-5 entry; supporting statements mirror the person spec with evidenced examples.
- Interviews test course-core concepts + values; STAR stories from your own projects; "here's how I'd find out" beats bluffing.
- Technical tests score visible process: assumptions, checks, definitions, sanity — narrate them.

**MODULE 9 · CAREER & PROFESSIONAL PRACTICE · LESSON 101 OF 105****101. Your 12-week plan: from course to career**

A course ends; a practice continues. This closing lesson turns everything behind you into a schedule you can actually keep — because the analysts who land roles are rarely the ones who studied hardest for a fortnight, but the ones who compounded modest effort for months.

**The honest audit first.** You cannot schedule what you haven't measured. Spend one session self-assessing against three checklists: this course's module map (which lessons would you fail if quizzed cold today?), the AphA competency framework (Lesson 98), and three real job adverts at your target band (their essential criteria are the market's syllabus). The gaps — not your favourite topics — set the plan. Expect the classics: window functions rusty, standardisation formula fuzzy, Git branching theoretical.

**The 12-week shape** — adjust ruthlessly to your life, keep the proportions:

| Weeks | Focus                                                                                                    | Deliverable                              |
|-------|----------------------------------------------------------------------------------------------------------|------------------------------------------|
| 1-2   | Consolidate: rework weak-spot lessons; daily playground reps; rebuild spaced-repetition deck (Lesson 11) | Self-audit doc; 20-card deck in rotation |
| 3-5   | Capstone A + B properly (Module 10), full RAP dress, Git throughout                                      | Two pinned portfolio repos               |
| 6-7   | Capstone C or D + fingertipsR open-data project of YOUR question                                         | Third repo with rendered Quarto output   |
| 8     | Portfolio polish week: READMEs, Pages, profile; community contribution (Lesson 98)                       | Reviewable-by-a-stranger portfolio       |
| 9-10  | Applications: statement per real advert, mirrored to spec; mock technical test under time                | 3+ tailored applications sent            |
| 11-12 | Interview prep: STAR bank, concept drills aloud, questions for them; keep applying                       | Interview-ready; pipeline running        |

**The weekly rhythm that survives real life:** three focused sessions beat daily heroics — two build sessions (project work, where learning consolidates) and one maintenance session (spaced-repetition cards, one community touch, one job-board scan). Protect the build sessions on your calendar like shifts. The generation-effect rule from Lesson 11 still governs: reading about window functions is anaesthetic; writing one from a blank playground is the rep that counts.

**Plateaus and dips are features.** Around week 4-6 the novelty is gone and the finish line isn't visible — this is where most self-teaching quietly dies. The countermeasures are structural, not motivational: deliverables with dates (a repo shipped is progress you can see), a study partner or community check-in (borrowed accountability), and permission to shrink a bad week to one session rather than zero. Momentum, not intensity.

**And afterwards** — because week 12 is a waypoint: the field moves (new packages, new datasets, the AI tooling of Module 8 evolving fastest of all), so the professional habit is a steady trickle — an NHS-R webinar a month, release notes skimmed, one new technique tried per quarter on a toy problem before it's needed on a real one. You now possess the rarer thing than knowledge: a proven method for acquiring it. That method — question first, evidence over vibes, verify everything, ship reproducibly — is the actual qualification this course awards.

Go build something. The playground's still open.

#### QUICK CHECK

**Week 5, motivation gone, plan slipping. The evidence-based rescue is...**

- ☐ Wait for motivation to return
- ✓ **Structural fixes: shrink to a minimum viable session, anchor to a dated deliverable, and add external accountability (partner/community)**
- ☐ Restart the course from Lesson 1
- ☐ Double the hours to catch up

**Why:** Motivation follows momentum, not vice versa. Smaller-but-nonzero sessions, visible deliverables and borrowed accountability are what get plans through the trough — heroic doubling produces week-6 burnout.

**QUICK CHECK****The self-audit ranks topics by...**

- ☐ How enjoyable they are to revise
- ✓ **Gap severity against the module map, competency framework and real job adverts**
- ☐ Alphabetical order
- ☐ Whatever the last lesson covered

**Why:** Plans built on favourite topics feel productive and change nothing. The market's syllabus (adverts) and honest cold-quiz failures define where the next hour actually pays.

**EXERCISE**

Write YOUR week-1 plan right now: three calendar-ready sessions (day, duration, exact task), one deliverable for Friday, and the single lesson number you most need to rework first.

**SOLUTION - TEXT**

(Model – replace with yours:)

Mon 19:00-20:30 – Rework Lesson 31-33 (window functions): re-do all three exercises from blank playground, no peeking until attempted.

Wed 19:00-20:30 – Capstone A, part 1: project skeleton, git init, extract query drafted and hand-validated on 3 patients.

Sat 10:00-11:00 – Maintenance: 20 spaced-rep cards (make 5 new from this week's stumbles), skim NHS-R newsletter, scan 5 job adverts, save 1 person spec to audit against.

Friday deliverable: capstone repo exists on GitHub with README stub and first meaningful commit.

Rework-first lesson: (your honest answer – mine was 32.)

**KEY POINTS**

- Audit gaps against module map + competency framework + real adverts; gaps set the plan.
- 12 weeks: consolidate → capstones → portfolio → applications → interviews; 3 protected sessions/week.
- Survive the trough structurally (deliverables, accountability, minimum viable sessions); after week 12, trickle-learn forever.

## Capstone A: 30-day emergency readmissions



Key: exclude deaths without readmission (no opportunity to return)

### MODULE 10

## Module 10 — Capstone projects

### In this module

- 102** Capstone A: the readmissions pipeline (SQL)
- 103** Capstone B: vaccination equity (SQL extract + R analysis)
- 104** Capstone C: the weekly surveillance report (Quarto)
- 105** Capstone D: the mini-RAP, both hands (and the rubric)

## MODULE 10 · CAPSTONE PROJECTS · LESSON 102 OF 105

## 102. Capstone A: the readmissions pipeline (SQL)

Time to graduate from exercises to *\*work\**. Capstone A is a pure-SQL build, done entirely in the playground, producing the analysis every acute trust runs: **emergency readmissions within 30 days**. You built the pieces in Module 3; now you assemble them, alone, to a specification — because real briefs arrive as prose, not as half-written queries.

**The brief** (as a service manager would send it):

**COMMISSION**

"For financial year 2025/26, what is our 30-day emergency readmission rate? I need the headline rate, the monthly trend, and which specialties drive it. And please make sure people who died aren't counted as 'not readmitted' — the last report got grief for that."

**Definitions you must implement** (and in real life, would confirm in writing): an *\*index admission\** is any completed emergency admission discharged in FY 2025/26; a *\*readmission\** is a subsequent **emergency** admission whose admission\_date falls within 30 days **after the index discharge\_date** (1–30 days; a same-day return counts, an admission on day 31 doesn't); each index admission is assessed independently (an admission can be both a readmission and a later index); *\*deaths\**: exclude index admissions where the patient died within 30 days of discharge without readmission — they had no opportunity to return (the deaths table is your friend); report the rate as readmitted-index / eligible-index.

**Build order** (the professional decomposition — resist writing one giant query first):

- Stage 1 — cohort CTE: completed FY 2025/26 emergency discharges with admission\_id, patient\_id, discharge\_date. *\*Check:\** row count; a few hand-verified members.
- Stage 2 — readmission flag: for each index row, does ANY qualifying later admission exist? (Correlated EXISTS, or a self-join + window — your choice; Lesson 33's gap logic and Lesson 38's pattern both work.) *\*Check:\** pick two flagged and two unflagged patients; verify by eye against their raw admissions.
- Stage 3 — mortality exclusion: LEFT JOIN deaths; exclude the died-within-30-days-unreadmitted. *\*Check:\** how many rows did this remove? Plausible?
- Stage 4 — outputs: headline rate; rate by discharge month (spine the months!); rate by specialty of the index admission, sorted, with an n column so small denominators are visible.

**STARTER SCAFFOLD (FINISH IT YOURSELF FIRST!)**

```
WITH cohort AS (
  SELECT admission_id, patient_id, discharge_date, specialty
  FROM admissions
  WHERE admission_method = 'Emergency'
        AND discharge_date >= '2025-04-01'
        AND discharge_date < '2026-04-01'
)
SELECT COUNT(*) AS index_admissions FROM cohort;
```

**Quality bar** (this is the marked rubric — self-assess honestly): correct half-open date logic everywhere; readmission window measured from *\*discharge\**, emergency-only, 1–30 days inclusive; deaths handled per spec; every stage checked with a written-down verification (comment them into your SQL); monthly output has no missing months; specialty output includes denominators. Stretch goals: exclude index admissions in the final 30 days of the FY *\*or\** justify why you didn't (the follow-up-window problem — real reports must choose); compute day-to-readmission distribution (LAG/julianday) and its median.



**QUICK CHECK**

**A patient's index discharge is 2026-03-20; they're readmitted (emergency) on 2026-04-05. Under this brief's definitions, the index admission is...**

- ☐ Not in the cohort — the readmission is outside the FY
- ✓ **In the cohort and counted as readmitted — the INDEX discharge sets cohort membership; the 30-day window may cross the FY boundary**
- ☐ Excluded as a death
- ☐ Counted twice

**Why:** Cohort membership keys on index discharge in-FY; the outcome window follows the patient wherever it lands. (The stretch goal asks whether late-March indexes get a FAIR window — a different, deliberate decision.)

**QUICK CHECK**

**Why must the died-without-readmission group be EXCLUDED rather than counted as "not readmitted"?**

- ☐ Small numbers rules
- ✓ **They had no opportunity to be readmitted — counting them as successes deflates the rate and misrepresents the pathway**
- ☐ Deaths are confidential
- ☐ SQL cannot join to deaths

**Why:** Denominators must contain only those at risk of the outcome. Immortal-to-the-outcome members flatter the metric — exactly the grief the manager remembers.

**EXERCISE**

Complete Stage 2: extend the scaffold so each cohort row gains readmitted = 1/0 using a correlated EXISTS against admissions (emergency, admission\_date > discharge\_date, within 30 days via julianday). Then produce the headline rate.

**SOLUTION · SQL**

```
WITH cohort AS (
  SELECT admission_id, patient_id, discharge_date
  FROM admissions
  WHERE admission_method = 'Emergency'
    AND discharge_date >= '2025-04-01'
    AND discharge_date < '2026-04-01'
),
flagged AS (
  SELECT c.*,
  EXISTS (
    SELECT 1 FROM admissions r
    WHERE r.patient_id = c.patient_id
      AND r.admission_method = 'Emergency'
      AND r.admission_date > c.discharge_date
      AND julianday(r.admission_date) - julianday(c.discharge_date) <= 30
  ) AS readmitted
  FROM cohort c
)
SELECT COUNT(*) AS eligible,
  SUM(readmitted) AS readmitted,
  ROUND(100.0 * SUM(readmitted) / COUNT(*), 1) AS rate_pct
FROM flagged;
-- (Stage 3 next: LEFT JOIN deaths and exclude died<=30d & readmitted=0.)
```

**KEY POINTS**

- Real briefs are prose: extract definitions, confirm them, then build in checked stages.
- Cohort → flag → exclusions → outputs, with a verification comment at every stage.
- Denominators = those at risk; windows from discharge; half-open dates; show your n.

**MODULE 10 · CAPSTONE PROJECTS · LESSON 103 OF 105**

## 103. Capstone B: vaccination equity (SQL extract + R analysis)

Capstone B is the full relay: **SQL extracts, R analyses** — the everyday shape of professional work — applied to an equity question that public health teams genuinely run: *\*is MMR uptake lower in more deprived areas, and which practices need support?\**

**The brief:** "For children turning 2 during FY 2025/26 (i.e. born 2023-04-01 to 2024-03-31), what proportion had received **2+ MMR doses by their 2nd birthday**? Show the deprivation gradient (IMD decile of the child's LSOA), and give me a fair practice-level comparison — last time someone just ranked percentages and the small practices were furious."

**Part 1 — SQL (playground).** One tidy extract, child-level:

**THE EXTRACT TO BUILD ON**

```
SELECT p.patient_id,
       p.birth_date,
       date(p.birth_date, '+2 years')      AS second_birthday,
       p.practice_code,
       li.imd_decile,
       COALESCE(v.mmr_doses_by_2, 0)      AS mmr_doses_by_2
FROM patients p
JOIN lsoa_imd li ON li.lsoa_code = p.lsoa_code
LEFT JOIN (
  SELECT vx.patient_id, COUNT(*) AS mmr_doses_by_2
  FROM vaccinations vx
  JOIN patients p2 ON p2.patient_id = vx.patient_id
  WHERE vx.vaccine = 'MMR'
        AND vx.date_given < date(p2.birth_date, '+2 years')
  GROUP BY vx.patient_id
) v ON v.patient_id = p.patient_id
WHERE p.birth_date >= '2023-04-01'
      AND p.birth_date < '2024-04-01';
```

Study the two Module 2/3 patterns doing the heavy lifting: the **LEFT JOIN to a pre-aggregated subquery** (so zero-dose children survive — an inner join here silently deletes the very children the question is about), and the **\*\*derived subquery computing doses \*before each child's own\* second birthday\*\*** (a per-row deadline, not a fixed date). Verify like a professional: total children plausible? Any imd\_decile NULLs (unmatched LSOAs — decide and document their fate)? Hand-check one child with 2 doses, one with 1, one with 0.

**Part 2 — R.** Export mentally (in real life: `dbGetQuery`; here, imagine the extract as `kids`) and deliver:

- *\*Gradient:\** uptake (`mean(mmr_doses_by_2 >= 2)`) by IMD decile; a clean ordered bar or point chart, Lesson 61-styled, with counts shown. Is the gradient monotone? Say what you see *\*and its limits\** (ecological, Lesson 74 — deciles describe areas, not families).
- *\*Fair practice comparison:\** a **funnel plot** (Lesson 78) of practice uptake vs eligible n, 95%/99.8% limits around the overall proportion — the direct answer to the furious-small-practices problem.

Flag outliers as *\*conversation-starters\**, with the case-mix caveat in the caption.

- *\*Safe outputs\** (Lesson 67): suppress any published cell with 1–4 children; check the decile table and any practice list against secondary-suppression leaks before "sending".

**Quality bar:** zero-dose children demonstrably included (state the check); per-child deadline logic correct (a dose at age 2y+1d must not count — test one); decile chart shows n; funnel limits computed correctly (Lesson 78's formula) and interpreted with restraint; suppression applied and stated; a 150-word manager summary that answers the brief in plain English with one caveat about area-level inference. Stretch: age-standardise... is meaningless here (single cohort) — instead, stretch by adding dose-1-by-1st-birthday as a second metric and commenting on where drop-off concentrates.

#### QUICK CHECK

**Replacing the LEFT JOIN (to the doses subquery) with an INNER JOIN would...**

- ☐ Change nothing; the WHERE clause protects it
- ✓ **Silently drop every zero-dose child, inflating uptake and destroying the equity analysis**
- ☐ Double-count doses
- ☐ Break the date logic only

**Why:** Children with no MMR rows vanish under an inner join — and unvaccinated children are the analytical point. COALESCE(0) + LEFT JOIN is the survival kit; the row-count check proves it worked.

#### QUICK CHECK

**The funnel plot answers the "furious small practices" complaint because...**

- ☐ It hides small practices
- ✓ **Its limits widen for small denominators, so a small practice's low percentage is only flagged if it exceeds what chance produces at that size**
- ☐ It ranks practices more politely
- ☐ Everyone falls inside limits by design

**Why:** League tables punish smallness with noise; funnels encode expected variation-by-size, separating "genuinely different" from "small and unlucky" — Lesson 78, now deployed diplomatically.

#### EXERCISE

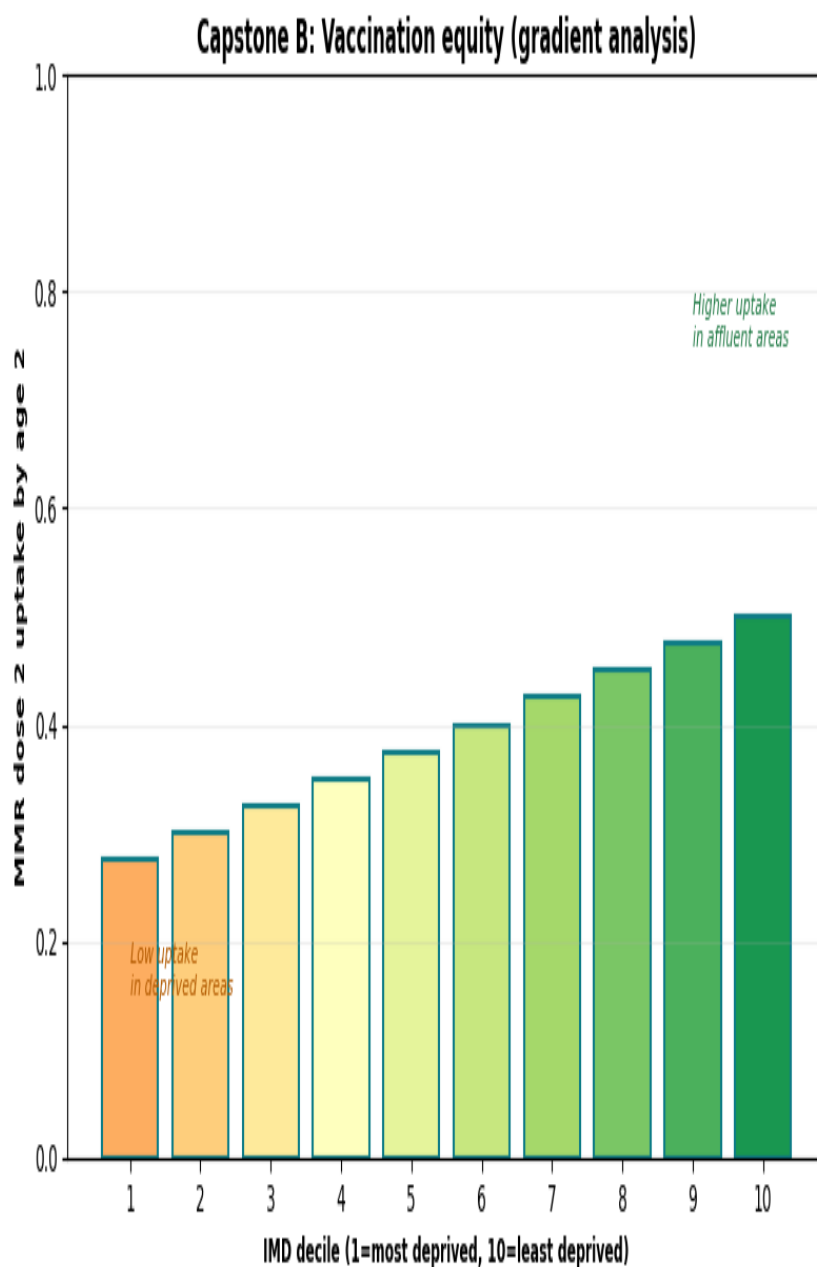
Write the R for the decile gradient table WITH suppression, ready to publish: uptake % by imd\_decile, eligible n, suppressing any decile where eligible n < 5 (unlikely, but the check is the habit), formatted to 1 dp.

#### SOLUTION · R

```
gradient <- kids |>
  group_by(imd_decile) |>
  summarise(eligible = n(),
            uptake = mean(mmr_doses_by_2 >= 2)) |>
  mutate(uptake_pct = round(100 * uptake, 1),
         published = if_else(eligible < 5, "<5 suppressed",
                           paste0(uptake_pct, "% (n=", eligible, ")"))) |>
  arrange(imd_decile)
gradient |> select(imd_decile, published)
# Then: ggplot(gradient, aes(factor(imd_decile), uptake)) + geom_col(...)
# Caption: "Deciles describe areas of residence, not individual families."
```

**KEY POINTS**

- SQL extracts child-level truth (LEFT JOIN + per-row deadline); R turns it into gradient + funnel + safe outputs.
- The zero-dose check and the age-2y-1d dose test ARE the capstone.
- Fair comparison = funnel, not ranking; publish with suppression and an ecological caveat.

**MODULE 10 · CAPSTONE PROJECTS · LESSON 104 OF 105****104. Capstone C: the weekly surveillance report (Quarto)**

Capstone C builds the artefact that runs a thousand Monday mornings: an **automated weekly surveillance report** — the Module 5-7 skills fused into one product whose defining virtue is that *\*next week costs nothing\**.

**The brief:** "A weekly HTML report on respiratory emergency admissions for the winter period: this week's count with a comparison we can trust (not just 'up 12%!'), the season so far against the spine of expected weeks, an age-band breakdown, and a data-quality footer so we stop getting caught by incomplete feeds. It'll go to the system call every Tuesday — non-analysts read it."

**Architecture** (one `.qmd`, parameterised — Lesson 67):

#### SURVEILLANCE\_REPORT.QMD SKELETON

```
---
title: "Weekly respiratory surveillance"
params:
  week_ending: 2026-02-01      # the ONLY thing that changes weekly
format:
  html:
    embed-resources: true
    toc: true
execute: { echo: false, warning: false }
---
```

**The five sections to build**, each mapping to lessons you own:

- **\*Headline\***, computed inline (Lesson 66): admissions in the week ending `params$week_ending` — with the comparison done properly: not last-week's delta but position against the **SPC control limits** of the preceding stable weeks (Lesson 77 / `NHSRplotthedots`). "Within expected variation" is a finding; teach your readers that.
- **\*Season curve\***: weekly counts on a complete date spine (Lesson 32's zero-fill — a missing week must plot as 0 or be flagged, never silently skipped), current week highlighted, incomplete-week shading per Lesson 60's honesty rule.
- **\*Age bands\***: this week vs the season's weekly average, small multiples or a dodged bar (Lessons 59/62); bands from the canonical `CASE/case_when` pattern so they match `la_population`'s.
- **\*Data-quality footer\*** (Lesson 39, automated): rows received vs 4-week average; % missing key fields; latest admission\_date seen (staleness detector); each with a plain-English verdict ("feed appears complete").
- **\*The tripwires\*** (Lesson 87): assertions that **fail the render** if data smells wrong — because a report that publishes garbage confidently is worse than no report:

#### THE RENDER-BLOCKING CHECKS

```
stopifnot(
  max(adm$admission_date) >= params$week_ending - 6, # feed fresh enough
  nrow(this_week) > 0,                               # week exists at all
  mean(is.na(adm$age_band)) < 0.05                   # coding intact
)
```

**Fictional data note:** generate a plausible two-winter weekly series in a setup chunk (`rpois` with a seasonal bump — Lesson 60 showed the trick) so the report runs self-contained; structure it exactly as the playground extract would arrive, so swapping in `dbGetQuery()` later is a one-line change. That separation — *\*data acquisition vs report logic\** — is the design habit that makes reports portable to real warehouses.

**Quality bar:** renders start-to-finish with one command; changing `week_ending` re-targets everything (test two different weeks!); every number in prose is inline code, zero typed statistics; SPC framing used for the headline (no naked two-week deltas); incomplete data visibly flagged; checks demonstrably block a sabotaged render (test by feeding it a truncated dataset — seriously, do this); reads sensibly to a non-analyst (the hardest criterion — have a human try). Stretch: parameterise by LA as well and batch-render three reports with `quarto_render` in a loop (Lesson 67's pattern).

**QUICK CHECK**

**The headline says "within expected variation" instead of "up 12% on last week" because...**

- ☐ It sounds calmer
- ✓ **Two-point comparisons promote noise to news; SPC limits define what this process normally produces, so only genuine signals trigger alarm**
- ☐ Percentages confuse readers
- ☐ Last week's data was missing

**Why:** Lesson 77's entire argument, operationalised weekly. Reports that cry wolf on common-cause wiggle train readers to ignore the real signal when it comes.

**QUICK CHECK**

**Why must the render FAIL (not warn) when the feed looks stale or truncated?**

- ☐ Failures are easier to log
- ✓ **A confidently published wrong report causes decisions; a loudly absent report causes a phone call — the second is vastly cheaper**
- ☐ Quarto cannot show warnings
- ☐ To punish the data team

**Why:** Fail-loud is the automation covenant (Lesson 87). The Tuesday call survives a missing report; trust does not survive a wrong one.

**EXERCISE**

Write the data-quality footer chunk: a small tibble of check / value / verdict for the three tripwire metrics, rendered with gt, verdicts computed not typed.

**SOLUTION · R**

```
dq <- tibble(
  check = c("Rows this week vs 4-wk mean",
            "Missing age band",
            "Latest admission seen"),
  value = c(sprintf("%d vs %.0f", nrow(this_week), mean_4wk),
            sprintf("%.1f%%", 100 * mean(is.na(adm$page_band))),
            as.character(max(adm$admission_date))),
  verdict = c(
    if_else(nrow(this_week) > 0.7 * mean_4wk, "OK", "LOW - investigate"),
    if_else(mean(is.na(adm$page_band)) < 0.05, "OK", "DEGRADED"),
    if_else(max(adm$admission_date) >= params$week_ending - 6,
            "Fresh", "STALE - do not circulate")
  ))
gt(dq) |> tab_header(title = "Data quality this week")
```

**KEY POINTS**

- One parameterised .qmd: headline (SPC-framed), spined season curve, age bands, automated DQ footer.
- Inline numbers everywhere; assertions that block bad renders; incomplete weeks visibly flagged.
- Separate data acquisition from report logic — fictional now, dbGetQuery later, same report.

## MODULE 10 · CAPSTONE PROJECTS · LESSON 105 OF 105

## 105. Capstone D: the mini-RAP, both hands (and the rubric)

The final capstone is the whole job in miniature — **an end-to-end reproducible pipeline built twice-minded**: some components AI-assisted under Module 8 discipline, some deliberately unaided, with a written reflection comparing the modes. This is the repo you pin first (Lesson 99); it demonstrates not just skills but *\*judgement about your own workflow\** — the rarest line on a CV.

**The brief — choose one question** (or invent one this shaped): "Which GP practices have the highest emergency admission **rates** (per 1,000 list size) among their 65+ patients, and is high use associated with area deprivation?" — or a question of your own against the playground/open data, provided it demands: an extract, a rate with a real denominator, a join across  $\geq 3$  tables, one statistical method from Module 6, and a rendered report.

**Required RAP dress** (Lesson 86's baseline, all of it): `.Rproj` + Lesson 83 structure; Git from the first commit with meaningful messages; README a stranger could run from; numbered scripts 01→04 (or targets, if you're feeling gold-tier); `here()` throughout; `testthat` for at least your two riskiest functions; assertions between stages; outputs/ fully regenerable; suppression on anything "published".

**The two-hands protocol** — the capstone's signature. Split components explicitly:

| Unaided (no AI)                 | AI-assisted (Module 8 discipline)                   |
|---------------------------------|-----------------------------------------------------|
| The core extract query          | Adversarial review OF that query (Lesson 91 prompt) |
| Definitions & the analysis plan | First-draft README and roxygen comments             |
| Verification of everything      | Edge-case test generation for your functions        |
| The final interpretation        | Refactoring one ugly-but-working pipeline stage     |

Log as you go (a plain `AILOG.md`): each AI interaction's purpose, what it produced, **what verification you performed, what it got wrong**. Then write the 300-word reflection: where did assistance genuinely accelerate? Where did it mislead or add friction? What would you delegate next time, and what never? Honest specifics ("it invented a `by_group` argument; ?`arrange` caught it in 10 seconds") are worth ten platitudes — this document is interview ammunition of the highest grade.

**The rubric** — score yourself, harshly, /20:

| Criterion           | 0                           | 1                       | 2                                                                 |
|---------------------|-----------------------------|-------------------------|-------------------------------------------------------------------|
| Correctness         | Numbers unverified          | Spot-checked            | Reconciled + boundary-tested, checks documented                   |
| Reproducibility     | Runs on your machine, maybe | Fresh session clean run | Stranger-test passed (structure+README+seeded)                    |
| Definitions         | Implicit                    | Stated                  | Stated, justified, edge cases decided (FY! deaths! zeroes!)       |
| Statistical honesty | Point estimates bare        | Uncertainty shown       | Uncertainty + fairness handled (funnel/CI/standardisation as apt) |
| Safe outputs        | None                        | Suppression applied     | Suppression + secondary-leak check, automated                     |
| Git craft           | One giant commit            | Regular commits         | Small commits, messages explain WHY, tidy history                 |
| Testing             | None                        | Key functions tested    | Tests + inter-stage assertions; a sabotage run fails loudly       |
| Report quality      | Numbers dumped              | Clear to analysts       | Clear to non-analysts; inline stats; styled; alt text             |

| Criterion     | 0                        | 1       | 2                                                  |
|---------------|--------------------------|---------|----------------------------------------------------|
| AI discipline | Undisclosed / unverified | Logged  | Logged + verified + reflection with real specifics |
| Reflection    | Absent                   | Present | Insightful: changes your future workflow, quotably |

16+ = portfolio-lead ready. 12–15 = one focused revision pass. Below 12 = the gaps are your Lesson 101 plan, which is the system working. When you're done: pin it, write the README's headline chart, and go back to Lesson 100 — you now have the evidence that statement needs.

That's the course. One hundred and five lessons ago you'd never written a query; you now build verified, reproducible, honestly-communicated analytical pipelines with a documented AI workflow. The remaining distance to "professional" is repetition and community — both scheduled in Lesson 101. Off you go: Barsetshire's data isn't going to analyse itself.

**QUICK CHECK**

**The AILog + reflection exists primarily because...**

- ☐ Anthropic requires it
- ✓ **Documented, verified, critically-evaluated AI use is itself the employable skill — and the log is the evidence you can discuss under interview pressure**
- ☐ AI output is always wrong
- ☐ It pads the repository

**Why:** "I use AI with a verification discipline, here's the paper trail, here's what it got wrong" is the 2026 differentiator. Undocumented assistance is a liability; documented judgement is a credential.

**QUICK CHECK**

**Rate-not-count is mandatory in the brief because...**

- ☐ Counts are confidential
- ✓ **Practices differ hugely in list size — raw counts would rank size, not need; per-1,000 rates (right denominators!) enable the comparison**
- ☐ Rates are easier to compute
- ☐ The rubric says so

**Why:** Lesson 37's founding principle, now load-bearing: comparisons require denominators. Choosing the RIGHT one (65+ list size, not total) is part of the definitions criterion.

**EXERCISE**

Write your Stage-boundary assertion block for this capstone: after the extract (03 tables joined), before analysis — covering grain, join fan-out, denominator sanity, and rate bounds.

**SOLUTION · R**

```
stopifnot(
  # grain: one row per practice after aggregation
  !any(duplicated(practice_rates$practice_code)),
  # fan-out guard: patients didn't multiply through joins
  nrow(extract) == n_distinct(extract$patient_id),
  # denominators: every practice has a positive 65+ list size
  all(practice_rates$list_65plus > 0),
  # rates: humanly possible (per 1,000, 65+ emergency admissions)
  all(practice_rates$rate_per_1000 >= 0),
  all(practice_rates$rate_per_1000 < 1000)
)
# Each failure names its line: the debugging starts WHERE it fails.
```



**KEY POINTS**

- One question, full RAP dress, split deliberately: unaided core, AI-assisted periphery, everything verified.
- AILog + honest reflection = the credential; the rubric (/20) = your revision plan.
- 105 lessons in: you build verified, reproducible, honest pipelines. Now repeat until fluent.